CEBU INSTITUTE OF TECHNOLOGY UNIVERSITY

COLLEGE OF COMPUTER STUDIES

Software Design Description
for
DASIGConnect

Change History Signature

Version	Date	Author	Changes	Reason (MVP Validation Findings)
1.0	05/16/2026	Chris Daniel Cabatana, Jaylord Bayonas, Lerah Caones, Mark Anton Camoro, Richemmae V. Bigno	Initial document	— (pre-validation)
2.0	08/22/2026	Chris Daniel Cabatana, Jaylord Bayonas, Lerah Caones, Mark Anton Camoro, Richemmae V. Bigno		— (pre-validation)
3.0	10/04/2026	Chris Daniel Cabatana, Jaylord Bayonas, Lerah Caones, Mark Anton Camoro, Richemmae V. Bigno	Re-aligned to SRS v6.0 use-case numbering (UC-1.1–UC-1.10, UC-2.1–UC-2.5, UC-3.1–UC-3.6) and verified against the implemented codebase: three-role model (Contributor, Moderator, Administrator, plus the Admin Owner flag); Cloudflare R2 object storage with presigned uploads and a backend media proxy; Railway backend hosting; Java HttpClient integrations; DB-backed session revocation; network-wide guard-rail switch and DB-level slot-conflict constraint; 15-minute review lock; review-edit diffs and severity; atomic publish claim and Fast-Track async publishing; Gemini/Voyage media enrichment pipeline; System Health, Audit Log, and Watermarking designs; updated API endpoints, components, and data schema (Flyway V1–V122; unified promotion flow, configurable posting window, removal of submission category, Moderator and Administrator review notifications)	— (pre-validation; implementation verification)

Version	Date	Author	Changes	Reason (MVP Validation Findings)
4.0	10/17/2026	Chris Daniel Cabatana, Jaylord Bayonas, Lerah Caones, Mark Anton Camoro, Richemmae V. Bigno	Re-verified against release v3.0.2 code and SRS v7.0, correcting statements the code contradicts: Google Gemini performs image classification (Claude does not); Admin Owner transfer only to an active Administrator; institution uniqueness rules and pending-invitation deactivation block; no lockout email; single-attempt email delivery; 30 notification event types; Review Queue ordering (GR-T1 badge vs sort); media-processing dead letters; token key derived from FACEBOOK_APP_SECRET; System Health probes and formula-safe snapshot export (PR #409). Added Section 4, Planned Design for Validation-Driven Refactoring (RP-01–RP-05), and Appendix — REST API Endpoint Table.	VF-01–VF-05 from the MVP Validation Findings (12 Oct 2026) drive the planned design in Section 4: readiness gate (RP-01), next action and task-event log (RP-02), caption on upload and media-first picker (RP-03), mobile primary-action bar (RP-04), bulk invitations (RP-05).
5.0	10/18/2026	Chris Daniel Cabatana, Jaylord Bayonas, Lerah Caones, Mark Anton Camoro, Richemmae V. Bigno	Implemented the five validation-driven refactoring priorities (RP-01–RP-05) following MVP pilot findings: replaced the composer's blended 0–100 score with a binary 7-of-7 Required Submission Gate and separate Post Quality Tips (RP-01, REQ-124); introduced dashboard role next-action prompt cards, unified plain-language status copy across UI/SSE, and database task telemetry logging in task_events and task_completion_logs (RP-02, REQ-125, REQ-126); automated background AI caption generation upon image upload with inline Use/Edit/Dismiss controls and quality telemetry (RP-03, REQ-320, REQ-321); established full mobile operability at 360px with zero horizontal overflow and notification deep-linking (RP-04, REQ-218); and implemented bulk CSV user invitations with preview validation and batch audit logging (RP-05, REQ-127).	Directly resolved all five MVP pilot findings: eliminated contributor submission ambiguity caused by the blended percentage score (VF-01); increased low unaided task completion and sub-70 SUS through role guidance and explicit status copy (VF-02); raised low AI feature discovery via automatic captioning on upload (VF-03); eliminated mobile review access barriers on 360px phone viewports (VF-04); and scaled user onboarding through bulk batch invitations (VF-05).

Preface

This Software Design Document (SDD) describes the architectural and detailed design of DASIGConnect, a multi-tenant social media content management system developed for the Department of Science and Technology — Accelerated Science and Technology Human Resource Development Program (DOST DASIG). The system is designed to replace ad hoc coordination between member institutions and a centralized Administrator by introducing a structured, self-service content submission and publication workflow integrated with the DASIG Facebook Page.

This document translates the functional and non-functional requirements defined in the Software Requirements Specification (SRS) into concrete design decisions: system architecture, component structure, data models, API contracts, security mechanisms, and AI integration design. Where the SRS defines *what* the system must do, this document defines *how* the system does it.

Intended Audience

This document is intended for the development team responsible for implementing and maintaining DASIGConnect. It assumes familiarity with the SRS and working knowledge of the technologies involved, including Spring Boot, PostgreSQL with pgvector, React, the Facebook Graph API, and the Anthropic, Google Gemini, and Voyage AI APIs. It may also be referenced by technical reviewers and DASIG stakeholders evaluating design decisions during the pilot period.

Relationship to Other Documents

This SDD should be read in conjunction with the SRS, which remains the authoritative reference for system requirements, business rules, guard rails, and non-functional constraints. Where this document references a requirement by its identifier (e.g., GR-H1, BR-ACC-03, UC-1.3), the SRS is the source of record for that requirement's full specification. Any conflict between this document and the SRS shall be resolved in favor of the SRS unless a formal change request has been approved.

Design Conventions

Throughout this document, the term *shall* denotes a design decision that directly implements a mandatory requirement from the SRS. The term *is* or *uses* describes an implementation choice made within the latitude allowed by the requirements. Sequence diagrams follow UML 2.x notation. All API endpoint paths are relative to the application base URL and are prefixed with `/api/v1` unless otherwise noted.

Table of Contents

Preface ***4***

Intended Audience *4*

Relationship to Other Documents *4*

Design Conventions *4*

Table of Contents ***5***

1. Introduction *7*

1.1. Purpose *7*

1.2. Scope *7*

1.3. Definitions and Acronyms *8*

1.4. References *9*

2. Architectural Design *10*

Architectural Overview *10*

Presentation Tier *10*

Application Tier *10*

Data Tier *11*

External Service Integrations *12*

Background Processing *12*

Architectural Block Diagram *14*

Security Architecture Overview *15*

3. Detailed Design *16*

1. Module 1 *16*

1.1. Administrator Account Management *16*

1.2. Institution Management *24*

1.3. Contributor Account Management *31*

1.4. Login & Session Management *38*

1.5. Post Drafting & Templates *47*

1.6. AI & Text Tools Integration *55*

1.7. Media Attachment Rules *61*

1.8. Engagement Helpers *68*

1.9. Draft Submission *74*

1.10. Moderator Account Management *81*

2. Module 2 *87*

2.1. Library Uploads & Albums *87*

2.2. Semantic Search & Filtering *96*

2.3. Media History Tracking *104*

2.4. Admin Approval Workflow *111*

2.5. Automated Watermarking *120*

3. Module 3 *125*

3.1. Master Calendar Visibility *125*

3.2. Automated Facebook Post Publishing *131*

3.3. Automated Reminders & Alerts *138*

3.4. Social Engagement Analytics *144*

3.5. System Health Analytics *151*

3.6. Audit Log Review *157*

4. Planned Design for Validation-Driven Refactoring (RP-01–RP-05) *163*

4.1. RP-01 — Submission gate separated from quality tips (VF-01; REQ-124) *163*
4.2. RP-02 — Next action, plain-language status, task-event log (VF-02; REQ-125, REQ-126) *163*
4.3. RP-03 — Caption on upload, media-first picker, log-based GO3 (VF-03; REQ-320, REQ-321) *163*
4.4. RP-04 — Mobile operability (VF-04; REQ-218) *164*
4.5. RP-05 — Bulk invitations with preview and batch audit (VF-05; REQ-127) *164*
Appendix *165*
Overview *165*
Table Inventory *165*
Master Entity Relationship Diagram *166*
Complete SQL DDL *170*
Appendix — REST API Endpoint Table *179*

1. Introduction

1.1. Purpose

This Software Design Description (SDD) defines the architectural and detailed design of DASIGConnect — a web-based Social Media Content Workflow and Scheduling Management System developed for the DOST Acadême–Science and Innovation Group (DASIG) under DOST Region 7. DASIGConnect centralizes and automates the social media content coordination operations of the DASIG multi-institutional network, specifically targeting its official Facebook page.

This document translates the functional and non-functional requirements specified in the DASIGConnect Software Requirements Specification (SRS) into concrete design decisions. It covers the system architecture, user interface design, front-end component structure, back-end component design, object-oriented models, and data design for all twenty-one use cases across three development modules. It serves as the authoritative technical reference for the development team throughout the entire project lifecycle.

1.2. Scope

This SDD covers the complete design of DASIGConnect across three modules:

Module 1 — User Access & Content Creation

- 1.1 Administrator Account Management (UC-1.1)
- 1.2 Institution Management (UC-1.2)
- 1.3 Contributor Account Management (UC-1.3)
- 1.4 Login & Session Management (UC-1.4)
- 1.5 Post Drafting & Templates (UC-1.5)
- 1.6 AI & Text Tools Integration (UC-1.6)
- 1.7 Media Attachment Rules (UC-1.7)
- 1.8 Engagement Helpers (UC-1.8)
- 1.9 Draft Submission (UC-1.9)
- 1.10 Moderator Account Management (UC-1.10)

Module 2 — Media Library & Approval Pipeline

- 2.1 Library Uploads & Albums (UC-2.1)
- 2.2 Semantic Search & Filtering (UC-2.2)
- 2.3 Media History Tracking (UC-2.3)
- 2.4 Admin Approval Workflow (UC-2.4)
- 2.5 Automated Watermarking (UC-2.5)

Module 3 — Scheduling, Publishing, & Analytics

- 3.1 Master Calendar Visibility (UC-3.1)
- 3.2 Automated Facebook Post Publishing (UC-3.2)
- 3.3 Automated Reminders & Alerts (UC-3.3)
- 3.4 Social Engagement Analytics (UC-3.4)
- 3.5 System Health Analytics (UC-3.5)
- 3.6 Audit Log Review (UC-3.6)

The system is built on a React single-page application front end, a Spring Boot REST API back end, and a PostgreSQL relational database with the pgvector extension hosted on Supabase (database only). Media files and generated derivatives are stored in Cloudflare R2 through an S3-compatible integration: the browser uploads directly to R2 using backend-issued presigned URLs, and media is read through the backend's own media proxy. Five external services are integrated: the Facebook Graph API v25.0 for automated post publishing, Page engagement retrieval, and Messenger notification delivery; the Anthropic Claude API (Claude Haiku 4.5, vision-capable) for AI caption generation, proofreading, template drafting, and image classification; the Google Gemini API for optional structured media metadata enrichment; the Voyage AI API (`voyage-4-lite` text and `voyage-multimodal-3.5` image embeddings, 1024 dimensions) for semantic media search and recommendation; and the Resend email API for notification and account-activation email delivery.

1.3. Definitions and Acronyms

- **Admin Owner** — A governance flag (`users.is_admin`) held by exactly one active Administrator. It is not a separate role. It is required for actions targeting existing Administrator accounts, account erasure, Admin Owner transfer, and connecting a different Facebook Page.
- **ERD** — Entity-Relationship Diagram — a visual representation of the database schema and table relationships.
- **Fast-Track (Live Event)** — A submission mode that skips scheduled-slot selection and publishes asynchronously immediately after approval.
- **Guard Rails** — Network-wide scheduling rules (conflict buffer, minimum lead time, posting window, maximum horizon, per-institution quota, daily cap — defaults ±30 min, 2 h, 8:00 AM–8:00 PM, 30 days, 3, 6), all Administrator-configurable and enforced when the runtime switch `page_settings.guardrails_enforced` is on.
- **JPA** — Java Persistence API — the Java standard specification for object-relational mapping; implemented by Hibernate in Spring Data JPA.
- **JWT** — JSON Web Token — a compact, URL-safe token format used for stateless authentication; implemented using the jjwt library in Spring Boot.
- **ORM** — Object-Relational Mapper — an abstraction layer between application code and the database; DASIGConnect uses Spring Data JPA with Hibernate.
- **pgvector** — A PostgreSQL extension enabling `VECTOR(1024)` column storage and cosine similarity nearest-neighbor search for Voyage AI embeddings.
- **Presigned URL** — A short-lived, signed R2 URL issued by the backend that lets the browser `PUT` file bytes directly to object storage without the backend receiving the file.
- **R2** — Cloudflare R2, the S3-compatible object storage service used for media files and generated watermark derivatives.
- **RBAC** — Role-Based Access Control — a security model in which access is granted based on the user's assigned role (Contributor, Moderator, or Administrator, with a single Administrator additionally holding the Admin Owner flag).
- **REST** — Representational State Transfer — the architectural style used for the Spring Boot back-end API.
- **RLS** — Row-Level Security — a PostgreSQL feature enforcing per-institution data isolation at the database level.
- **SDD** — Software Design Description: this document.
- **Spring Boot** — An open-source Java-based framework used to build the DASIGConnect REST API back end, providing auto-configuration, embedded server, and production-ready features.
- **Spring Data JPA** — A Spring Boot module providing repository abstractions (JpaRepository) over JPA/Hibernate, used as the primary data access layer throughout DASIGConnect.

- **Spring Security** — A Spring Boot module providing authentication, JWT verification, and role-based access control through `@PreAuthorize` method security.
- **SRS** — Software Requirements Specification — the companion requirements document for DASIGConnect.
- **SSE** — Server-Sent Events — an HTTP-based protocol used by DASIGConnect for real-time server-to-client push notifications.
- **UUID** — Universally Unique Identifier — used as primary keys throughout the data model.

1.4. References

- DASIGConnect Software Requirements Specification (SRS), v7.0 — same project, companion document
- DASIGConnect System Flow Document — teammate flow specification document
- DASIGConnect MVP Validation Findings & Refactoring Priorities (12 Oct 2026) — VF-01–VF-05, RP-01–RP-05
- Spring Boot Documentation — https://docs.spring.io/spring-boot/docs/current/reference/html
- Spring Data JPA Documentation — https://docs.spring.io/spring-data/jpa/docs/current/reference/html
- Spring Security Documentation — https://docs.spring.io/spring-security/reference
- Flyway Documentation — https://documentation.red-gate.com/flyway
- Voyage AI Embedding API Documentation — https://docs.voyageai.com
- Anthropic Claude API Documentation — https://docs.anthropic.com
- Google Gemini API Documentation — https://ai.google.dev/gemini-api/docs
- Meta Graph API Reference v25.0 — https://developers.facebook.com/docs/graph-api
- Meta Messenger Platform Documentation — https://developers.facebook.com/docs/messenger-platform
- Cloudflare R2 Documentation — https://developers.cloudflare.com/r2
- Resend API Documentation — https://resend.com/docs
- pgvector GitHub Repository — https://github.com/pgvector/pgvector
- React Documentation — https://react.dev
- TanStack Query Documentation — https://tanstack.com/query
- FullCalendar Documentation — https://fullcalendar.io/docs
- Brooke, J. (1996). SUS: A quick and dirty usability scale. Taylor & Francis.

2. Architectural Design

DASIGConnect follows a three-tier client-server architecture designed around a single guiding principle: routine, high-volume work is handled by Contributors within system-enforced rules and centralized Moderator/Administrator review, while the Administrator's exception-handling role is reserved for situations where human judgment provides genuine value beyond routine review. The architecture enforces this principle through a layered design where each tier has a clearly bounded responsibility.

Architectural Overview

Presentation Tier

The front end is a React 19 + TypeScript single-page application built with Vite and served via Vercel's global CDN. Users access DASIGConnect through a standard web browser with no additional installations required. The SPA handles all routing, component rendering, and state management client-side, communicating with the back end exclusively through RESTful API calls over HTTPS and persistent SSE connections for real-time notifications. Media bytes are uploaded directly to Cloudflare R2 through backend-issued presigned URLs.

Technology	Role
React 19 + TypeScript (Vite)	SPA framework — functional components, hooks, Context API
React Router v7	Client-side routing; access grouped by layout-route gates (`RequireSignedIn`, `RequireReviewer`, `RequireAdmin`)
Tailwind CSS v4 + shadcn/ui + Radix UI	Utility-first styling and accessible UI primitives
Axios	HTTP client; Authorization header managed by `setAuthToken`; response interceptor normalizes the `{error: {code, message}}` envelope and triggers the session-expired modal on 401
TanStack Query (React Query)	Server-state caching, background refetch, and request de-duplication
FullCalendar.js	Master Calendar — week/month views, drag-and-drop rescheduling, color-coded state cards
React Context + custom hooks	Authentication/session state (`useAuthSession`, `useSessionCountdown`, `useLoginLockout`) and toast notifications
EventSource (SSE)	Real-time in-app notifications and read-state sync

Application Tier

The back end is a Spring Boot 4 (Java 21) REST API server deployed on Railway. It handles all business logic, state machine enforcement, guard rail validation, external API orchestration, tenant isolation, and background job scheduling. The API is stateless — session state is carried in the JWT issued at login, while revocation is persisted in the database.

Technology	Role
Spring Boot 4 (Spring Web MVC)	REST API framework — auto-configuration, embedded Tomcat server
Spring Security	JWT authentication filter (`JwtAuthenticationFilter`), RBAC via `@PreAuthorize`, tenant scope resolution (`TenantScopeService`)
Spring Data JPA + Hibernate	ORM layer — JpaRepository interfaces, JPQL queries, native SQL for pgvector and atomic conditional updates

Technology	Role
Flyway	Versioned schema migrations (`db/migration`, V1–V122); Hibernate DDL generation is not used
HikariCP	JDBC connection pool, capped at 5 connections for the Supabase Session Pooler
jjwt	JWT generation, signing (HS256), and verification
Java `HttpClient` (java.net.http)	Synchronous HTTP client for the Facebook Graph API, Anthropic Claude, Google Gemini, Voyage AI, and Resend
AWS SDK for Java v2 (S3)	S3-compatible client for Cloudflare R2 (presigned PUT, download, delete, bucket scan)
Spring `@Scheduled`	Background cron and fixed-delay jobs (publishing, stale detection, token health, embedding reconciliation, retention, and others — see Background Processing)
Spring `@Cacheable`	In-memory caching for the analytics summary (60-second TTL)
Spring Application Events	Decoupled notification dispatch, Fast-Track publishing (`@TransactionalEventListener(AFTER_COMMIT)`), and watermark failure alerts
In-memory rate limiters	Per-user hourly limits for AI caption (30/h), proofreading (30/h), and top-post template generation (10/h); account lockout for login

Data Tier

The database is PostgreSQL hosted on Supabase, with Row-Level Security (RLS) policies enforcing per-institution data isolation at the database level. The pgvector extension enables `VECTOR(1024)` storage and cosine similarity nearest-neighbor search for the Voyage AI embeddings powering semantic search, media recommendation, and album matching. Media files are stored in Cloudflare R2 and served through the backend media proxy.

Technology	Role
PostgreSQL (Supabase)	Primary relational database — users, institutions, submissions, media metadata, notifications, analytics caches, and audit data
pgvector	`VECTOR(1024)` columns in `media_asset_embeddings` (and legacy `media_assets.embedding`) and the `<=>` cosine distance operator
Row-Level Security (RLS)	Database-level tenant isolation and append-only enforcement for `audit_log`
GiST exclusion constraint	Network-wide ±30-minute slot conflict protection on `slot_reservations` (GR-H1), exempting only explicit Administrator overrides
Cloudflare R2	Object storage for uploaded media, institution logos, and transient watermarked derivatives (`generated/watermarked/`)
Backend media proxy	`GET /api/v1/media-files/{objectPath}` (`MediaProxyController`) — stable, unauthenticated read URL stored in `media_assets.storage_url`

External Service Integrations

Service	Provider	Purpose
Facebook Graph API v25.0	Meta	Automated post publishing (two-step photo method + single-call video method), Page token validation and OAuth re-authorization, post/Page engagement retrieval, Messenger notification delivery and link-code webhook
Anthropic Claude API (Claude Haiku 4.5, vision)	Anthropic	AI caption generation (UC-1.6), writing check/proofreading (UC-1.6, UC-2.4), image classification and description (UC-2.1), top-post template drafting (UC-1.5 A11), content idea suggestions (UC-3.3)
Google Gemini API (gemini-3.8-flash, configurable)	Google	Image classification (UC-2.1) into one of 14 categories plus an asset type, factual description, and up to 30 tags, with structured enrichment (scenes, objects, activities, OCR text, event hypotheses) used to improve search and recommendation
Voyage AI API (`voyage-4-lite`, `voyage-multimodal-3.5`)	Voyage AI	1024-dimension semantic (text) and image embeddings for semantic search (UC-2.2), AI-Suggested Media and Album Auto-Match (UC-1.7)
Resend API	Resend	Transactional email — invitations, password reset, notification emails (single attempt per email; no automatic retry)
Cloudflare R2	Cloudflare	S3-compatible media object storage

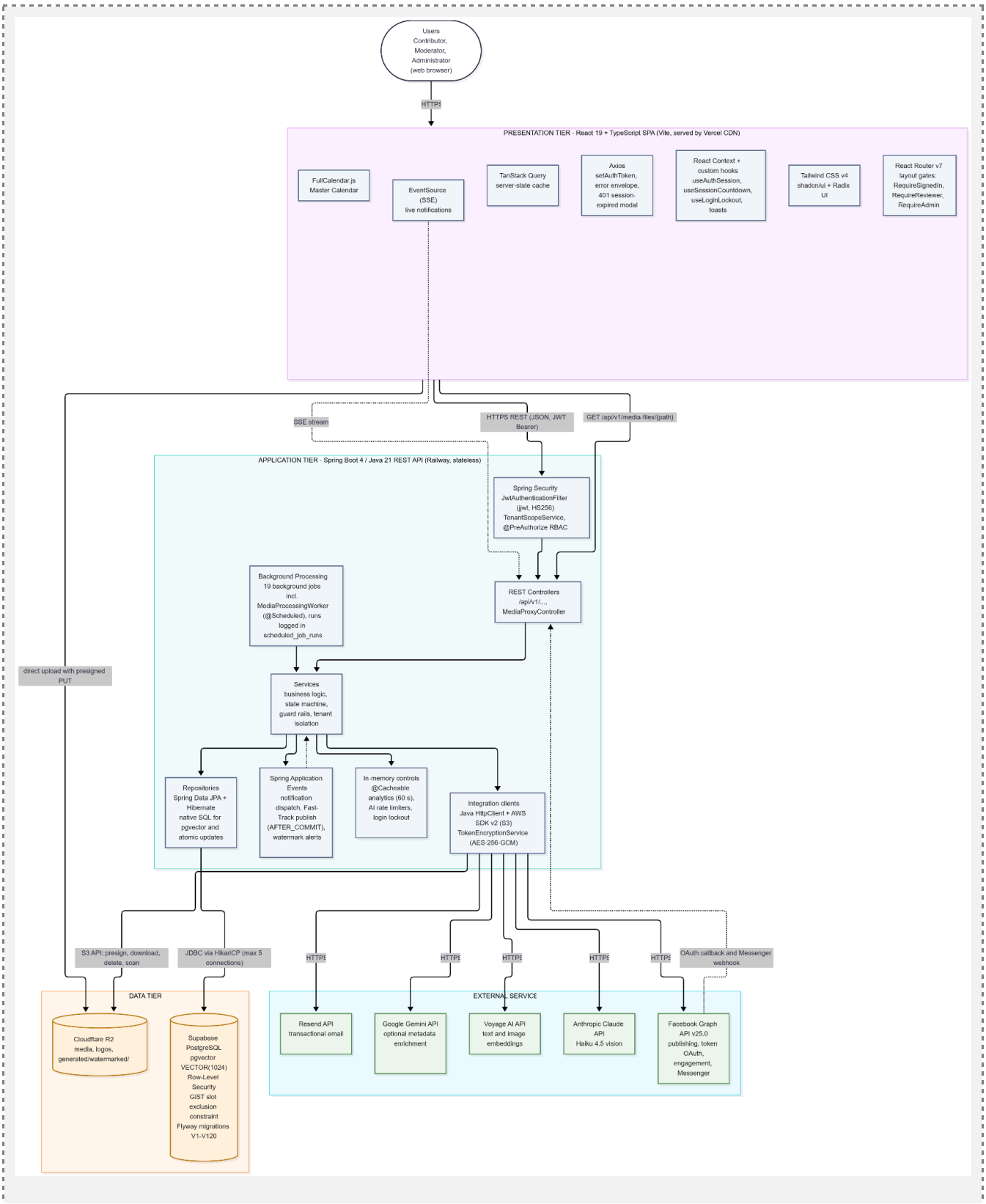
Background Processing

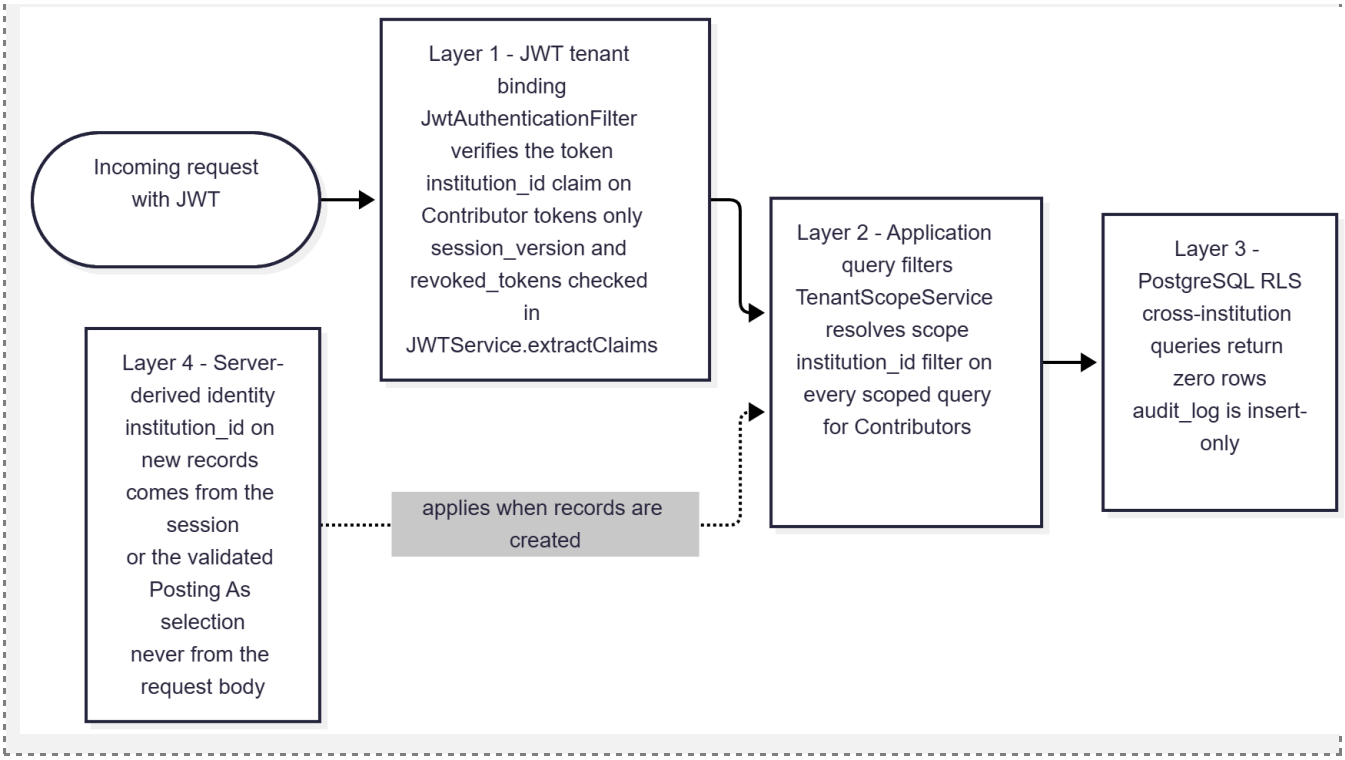
Job	Schedule	Purpose
PublishingSchedulerJob	Every minute	Claims and publishes due `scheduled` / `direct_post_scheduled` submissions within a 5-minute lookback (UC-3.2)
StaleSubmissionDetectorJob	Every 5 minutes	Missed scheduled post → `publish_failed`; never-reviewed post past its slot → `missed_review` (slot released); recovers stuck `publishing` rows including Fast-Track (GR-T9)
TokenPublishingEscalationJob	Every 5 minutes	Retries token-blocked submissions once a usable token exists; escalates at 24h and fails at 48h (UC-3.2 A3)
ReviewLockCleanupJob	Every minute	Expires 15-minute review locks and reverts `in_review` → `pending` (UC-2.4 A4)
ValidationDeadlineNotificationJob	Every 5 minutes	Reviewer deadline alerts for submissions nearing their slot; the GR-T1 priority flag itself is computed when the queue is read
AbandonmentDetectorJob	Every 5 minutes	Clears manual-publish sessions open longer than 2 hours (UC-2.4 A8)
TokenHealthCheckJob	Daily	Validates the active Facebook Page token; expiry and invalid-token alerts (GR-T3, GR-T4)
EmbeddingReconciliationJob	Every 30 seconds (app.media-processing.reconcile-delay-ms)	Re-queues image assets missing current embeddings in bounded batches; jobs that exhaust 5 attempts are dead-lettered and the asset is marked FAILED (GR-T7)

Job	Schedule	Purpose
MediaProcessingWorker	Polls every second	Leases and runs `media_processing_jobs` (classification, enrichment, embeddings, submission media context) with retry and dead-letter handling
SocialEngagementSyncJob	Every 15 minutes	Refreshes cached Facebook post engagement (`submission_engagement_metrics`) (UC-3.4)
InvitationExpiryJob	Hourly	Transitions expired `pending`/`pending_email_undelivered` accounts to `expired` (GR-T8)
StaleDraftSlotReleaseJob	Daily 02:00 UTC	Releases slots held by drafts inactive for 7 days (GR-T2)
MediaAssetRetentionPurgeJob	Daily 02:30 UTC	Purges soft-deleted media after the 30-day retention window
GeneratedWatermarkPurgeJob	Daily 02:45 UTC	Deletes watermarked derivatives older than 3 days (UC-2.5)
EmptyScheduleWarningJob	Weekly (Mon)	Empty-schedule warning with AI content-idea suggestions (T-07)
EmbeddingFailureDigestJob	Weekly (Mon)	Embedding failure digest to Administrators (T-12)
AuditLogRetentionJob	Daily 03:45 UTC	Retention purge of aged audit entries
ScheduledJobRunRetentionJob	Daily 03:15 UTC	Prunes old `scheduled_job_runs` history
RevokedTokenCleanupJob	Hourly	Deletes expired `revoked_tokens` rows

Every tracked job records its runs in `scheduled_job_runs`, which feeds System Health (UC-3.5). Administrators may re-run any tracked job on demand through `ManualJobRunner`. Sixteen jobs are tracked; AuditLogRetentionJob, RevokedTokenCleanupJob, and the MediaProcessingWorker are scheduled but not tracked on System Health.

Architectural Block Diagram





Security Architecture Overview

Tenant isolation is enforced through four complementary layers ensuring no cross-institution data leakage:

1. **JWT-Based Tenant Binding** — `institution_id` is embedded as a verified claim in every Contributor token; Moderator and Administrator tokens carry no `institution_id` (network-wide). The claim is verified and attached to the request context by `JwtAuthenticationFilter` before any controller logic executes. Every token also carries `session_version`; revoked or outdated tokens are rejected inside `JWTService.extractClaims` (backed by `revoked_tokens` and `users.session_version`).
2. **Application Query Filters** — `TenantScopeService` resolves the caller's scope; every institution-scoped query applies the `institution_id` filter for Contributors.
3. **PostgreSQL RLS** — Database-level policies return zero rows for cross-institution queries and make `audit_log` insert-only.
4. **Server-Derived Identity** — `institution_id` on new records is always set from the authenticated session (or, for Moderators/Administrators, from the validated "Posting As" selection), never trusted from the request body.

Additional controls: BCrypt password hashing, a 12-character password policy, 5-attempt/15-minute account lockout, AES-256-GCM encryption of Facebook Page tokens (`TokenEncryptionService`), HMAC-signed stateless OAuth `state`, formula-injection-safe CSV exports, and secrets supplied only through Railway environment variables.

3. Detailed Design

1. Module 1

Module 1 establishes the system foundation: secure user authentication with role-based routing, institution workspace provisioning, invitation-based onboarding for all three roles, and the three-step post composer with integrated AI caption, writing-check, template, media, and scheduling tools.

1.1. Administrator Account Management

1.1.1. User Interface Design

Admin Management Screen (`/admin/admin-management`)

A table listing all Administrator accounts with name, email, status badge (PENDING, PENDING_EMAIL_UNDELIVERED, ACTIVE, INACTIVE, EXPIRED, CANCELLED), and an "Admin Owner" tag on the single owner account. An "Invite Administrator" action is available to any Administrator while the combined Admin cap (active Admins + outstanding Admin invitations + pending promotions ≤ 3) is not reached. Deactivate, Reactivate, Delete, Erase, Demote, and Transfer Admin Owner actions are rendered only for the Admin Owner; peer Administrators see them disabled. Pending Administrator invitations appear in a separate card with Resend and Cancel actions.

Invite Administrator Dialog

An email chip input with a Send Invitation button. On success the dialog closes with a toast and the account appears with PENDING status. A cap violation returns an inline error directing the actor to remove or transfer an existing Administrator first.

Change Role / Promote Dialog

Opened from User Management on an ACTIVE Moderator or Contributor. Proposing a promotion to Administrator creates a pending request; the target's role does not change until they confirm. Demotion of an Administrator (Admin Owner only) is immediate; demotion to Contributor requires selecting an active institution.

Promotion and Owner-Transfer Confirmation Prompts

Shown to the target user on their next session when a pending promotion (72-hour expiry) or Admin Owner transfer (24-hour expiry) exists, with Confirm and Decline actions.

Invitation Accept Page (`/invite?token=…`)

A public route shared by all invitation flows. Validates the token, then displays a password setup form (New Password, Confirm Password) with live per-rule validation of the 12-character password policy. Submit stays disabled until all rules pass.

1.1.2. Front-end Component(s)

AdministratorManagementScreen

- Lists Administrators via `GET /users/admins` and pending Admin invitations via `GET /invitations/pending/admins`. Renders owner-only actions from `User.adminOwner` in the authenticated profile (`GET /me`).
- React functional component / Admin route (`RequireAdmin`)

InvitationComposer / EmailChipsInput

- Controlled form submitting `POST /invitations` with `role = admin` and no institution. Shows inline cap and conflict errors.
- React functional component

ChangeRoleModal / ConfirmDialog

- Submits `PATCH /users/{id}/role` (propose promotion or demote), `PATCH /users/{id}/status` (deactivate/reactivate), `DELETE /users/{id}` (delete), `POST /users/{id}/erase`, `POST /users/{id}/admin-transfer`, and `DELETE /users/{id}/promotion` (cancel proposal).
- React functional components

Promotion / Transfer Confirmation

- Calls `POST /users/promotion/confirm`, `POST /users/promotion/decline`, and `POST /users/admin-transfer/confirm`.
- React functional component rendered in the authenticated shell

InviteScreen

- Public route validating via `GET /invitations/validate?token=…` and activating via `POST /invitations/accept`. Shared across all three roles.
- React functional component

1.1.3. Back-end Component(s)

InvitationController

- `POST /invitations` (Admin/Moderator; role-gated per caller), `GET /invitations/validate`, `POST /invitations/accept`, `POST /invitations/{id}/resend`, `DELETE /invitations/{id}` (cancel), `GET /invitations/pending/admins` (Admin).
- Spring `@RestController`

UserController

- `GET /users/admins`, `PATCH /users/{id}/status`, `PATCH /users/{id}/role`, `DELETE /users/{id}`, `POST /users/{id}/erase`, `POST /users/{id}/admin-transfer`, `POST /users/admin-transfer/confirm`, `POST /users/promotion/confirm|decline`, `DELETE /users/{id}/promotion`.
- Spring `@RestController`

InvitationService

- Generates a 32-byte CSPRNG token (72-hour validity), stores only its SHA-256 hash, reuses PENDING / PENDING_EMAIL_UNDELIVERED / CANCELLED / EXPIRED account rows, rejects ACTIVE and INACTIVE targets (INACTIVE must be reactivated), supersedes older open tokens, and re-checks the Admin cap (`app.admins.max`, default 3) at both invite and accept.
- Spring `@Service`

UserService

- Implements deactivate/reactivate (cap-checked, no exclusions), footprint-aware delete (`USER_REMOVED` anonymized row vs. hard delete), erase (`USER_ANONYMIZED`), propose/confirm/decline/cancel promotion through one unified pending-promotion record (`pending_promotion_role` = admin or moderator; Admin cap re-checked at confirm via `AdminCapPolicy`; sessions revoked), demotion, and Admin Owner transfer (to another ACTIVE Administrator only — a non-Administrator target is rejected with 400; on confirmation the flag moves and both accounts remain Administrators). Self-targeting is rejected for every action. Owner-only checks use `JwtUserDetails.adminOwner()`.
- Spring `@Service`

JWTService

- Account-wide session revocation by incrementing `users.session_version` on deactivate, role change, and promotion confirmation.
- Spring `@Service`

AuditLogService / EmailService / NotificationService

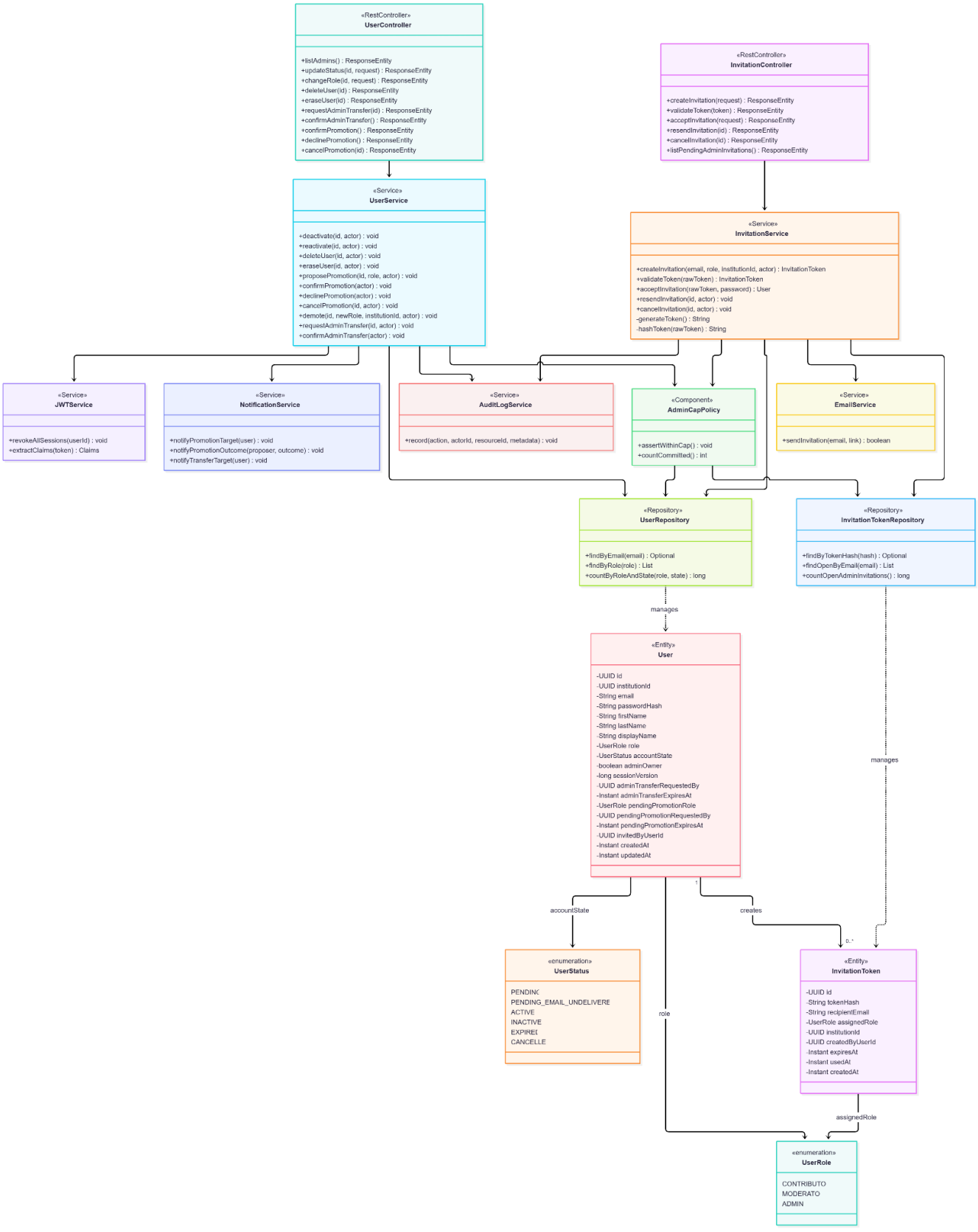
- Records `INVITATION_ACCEPTED`, `USER_STATUS_UPDATED`, `USER_ROLE_CHANGED`, `USER_REMOVED`, `USER_ANONYMIZED`, `ADMIN_TRANSFER_REQUESTED`, `ADMIN_TRANSFER_CONFIRMED`, and `ADMIN_PROMOTION_*` events; dispatches activation email via Resend; notifies promotion targets and proposers.
- Spring `@Service`

UserRepository / InvitationTokenRepository

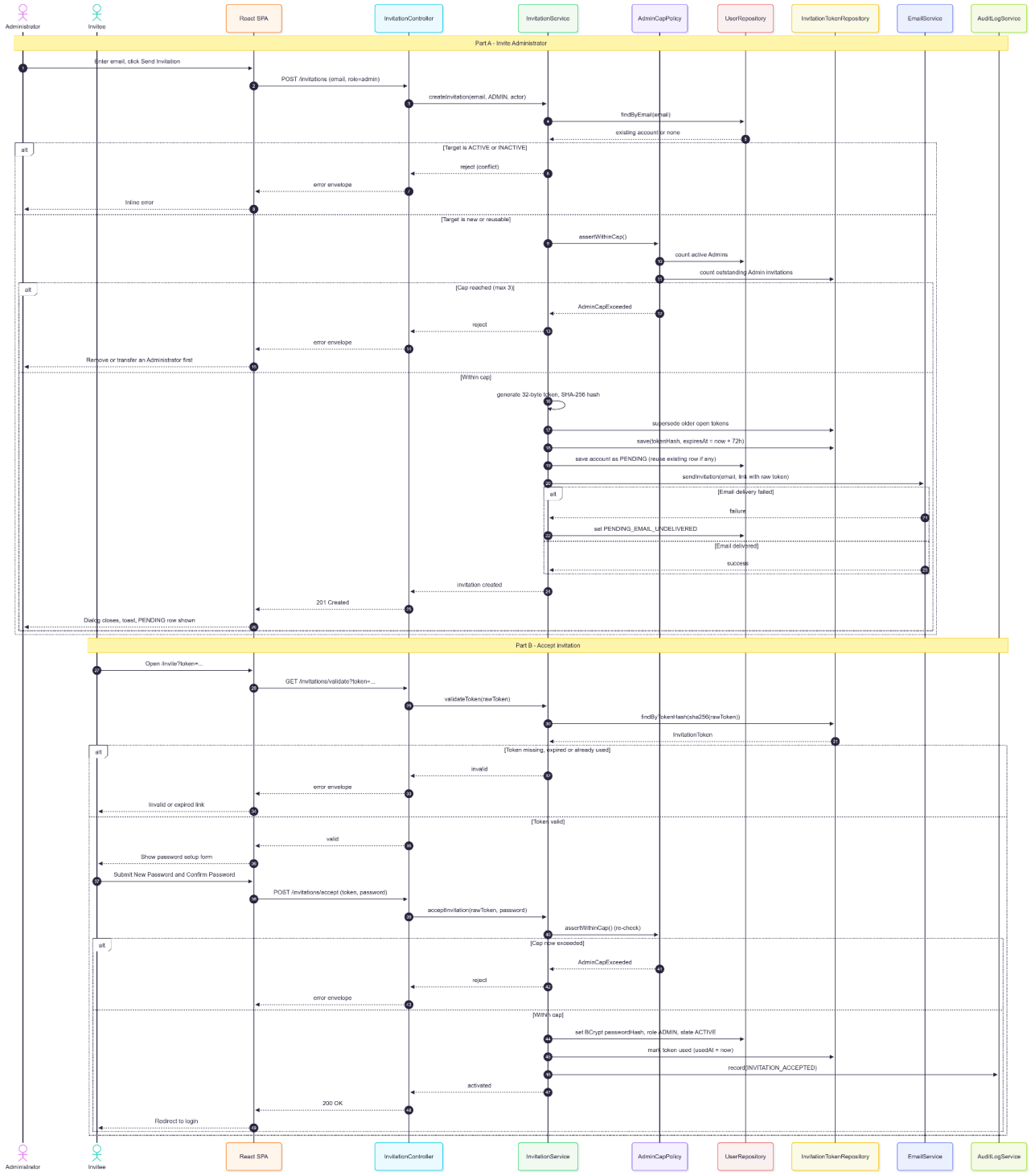
- Spring `@Repository` extending `JpaRepository<User, UUID>` / `JpaRepository<InvitationToken, UUID>`

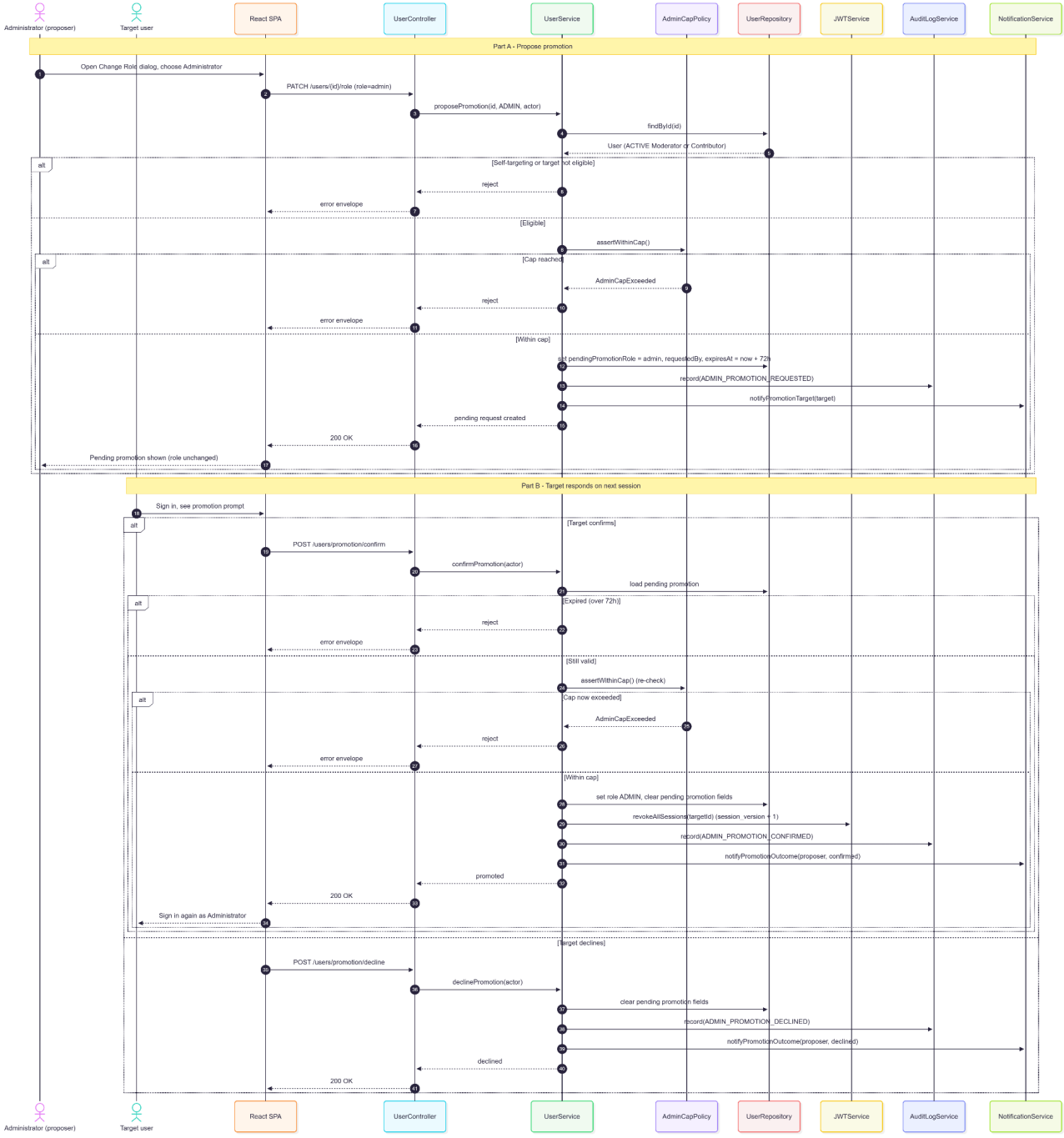
1.1.4. Object-Oriented Component(s)

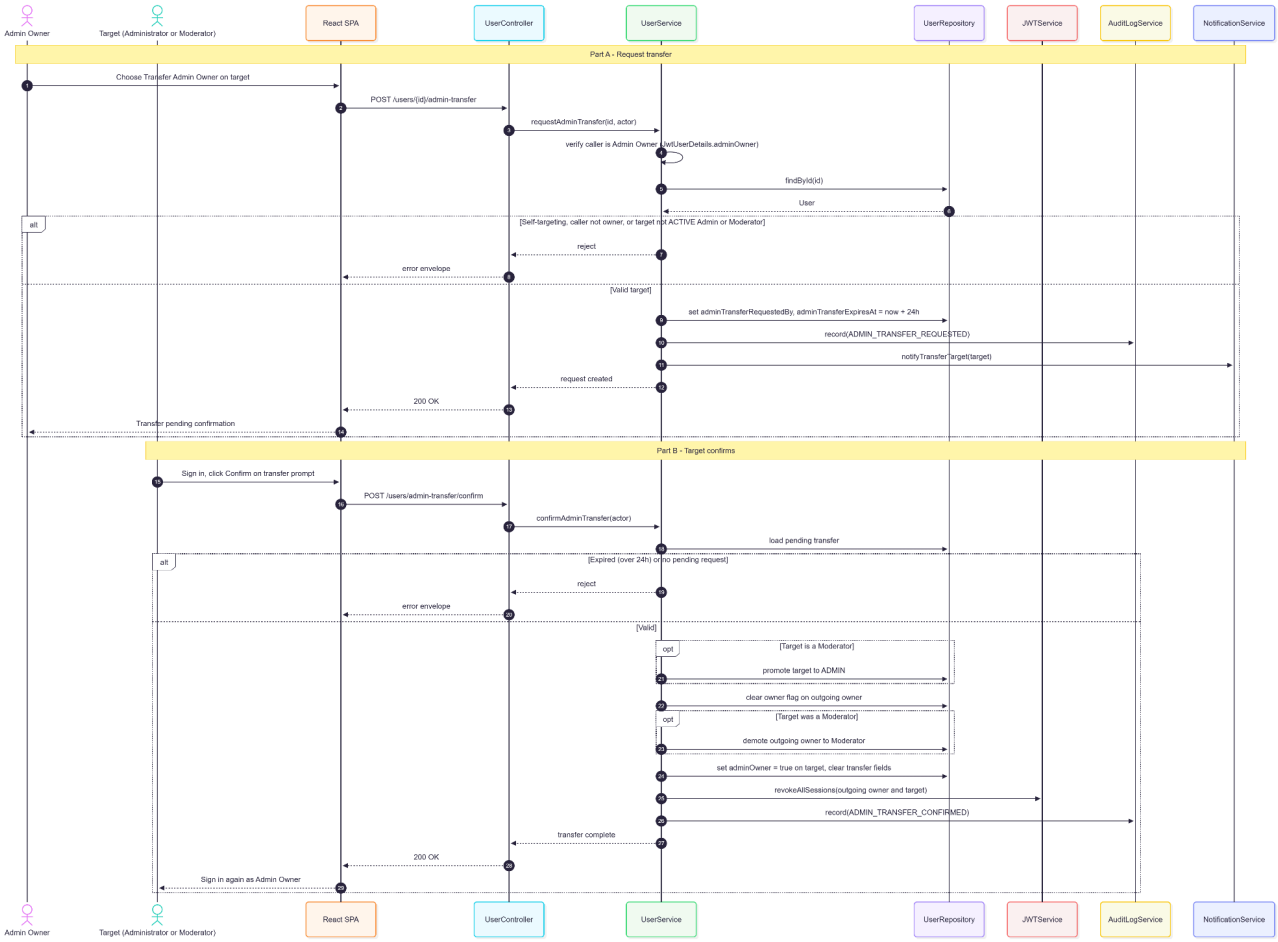
- **Class Diagram**



- **Sequence Diagram**



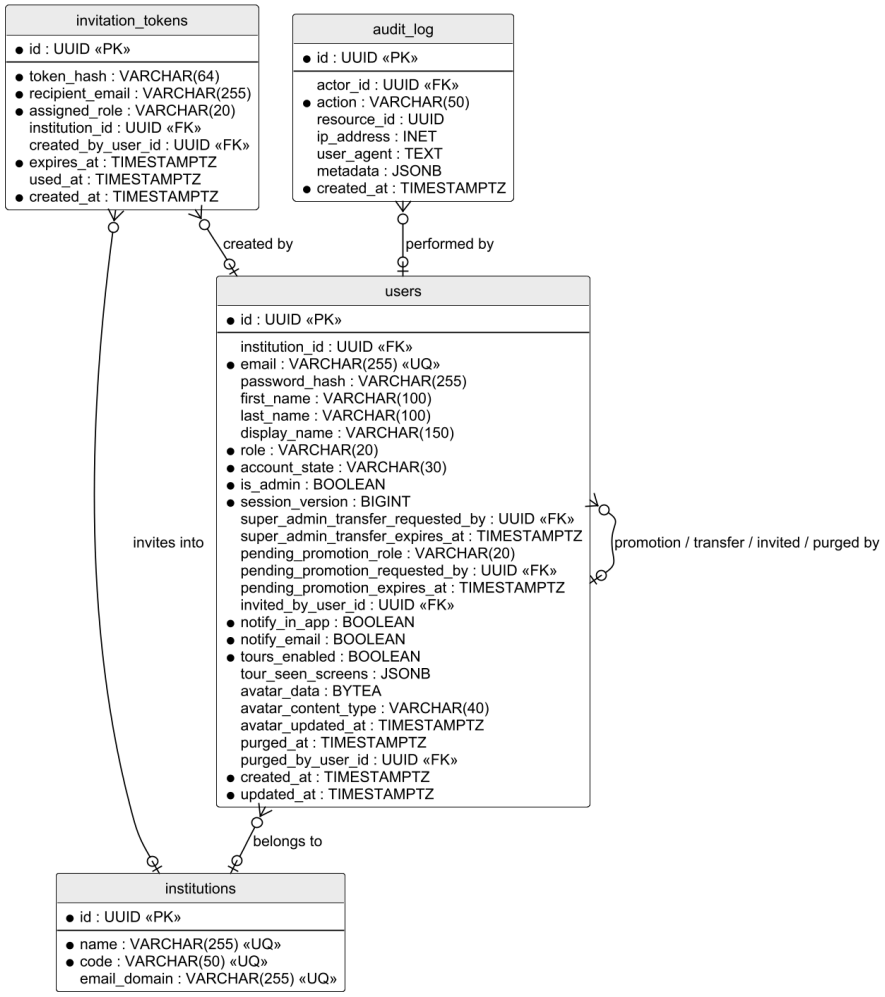

Administrator (proposer)
Target user
React SPA
UserController
UserService
AdminCapPolicy
UserRepository
JWTService
AuditLogService
NotificationService
Part A - Propose promotion
1 Open Change Role dialog, choose Administrator
2 PATCH /users/{id}/role (role=admin)
3 proposePromotion(id, ADMIN, actor)
4 findById(id)
5 User (ACTIVE Moderator or Contributor)
alt
[Self-targeting or target not eligible]
6 reject
7 error envelope
[Eligible]
8 assertWithinCap()
alt
[Cap reached]
9 AdminCapExceeded
10 reject
11 error envelope
[Within cap]
12 set pendingPromotionRole = admin, requestedBy, expiresAt = now + 72h
13 record(ADMIN_PROMOTION_REQUESTED)
14 notifyPromotionTarget(target)
15 pending request created
16 200 OK
17 Pending promotion shown (role unchanged)
Part B - Target responds on next session
18 Sign in, see promotion prompt
alt
[Target confirms]
19 POST /users/promotion/confirm
20 confirmPromotion(actor)
21 load pending promotion
alt
[Expired (over 72h)]
22 reject
23 error envelope
[Still valid]
24 assertWithinCap() (re-check)
alt
[Cap now exceeded]
25 AdminCapExceeded
26 reject
27 error envelope
[Within cap]
28 set role ADMIN, clear pending promotion fields
29 revokeAllSessions(targetId) (session_version + 1)
30 record(ADMIN_PROMOTION_CONFIRMED)
31 notifyPromotionOutcome(proposer, confirmed)
32 promoted
33 200 OK
34 Sign in again as Administrator
[Target declines]
35 POST /users/promotion/decline
36 declinePromotion(actor)
37 clear pending promotion fields
38 record(ADMIN_PROMOTION_DECLINED)
39 notifyPromotionOutcome(proposer, declined)
40 declined
41 200 OK



1.1.5. Data Design

- **Entity Relationship Diagram**

UC-1.1 · ERD — Administrator account management



- **Schema**

```
-- users table (all roles)
CREATE TABLE users (
    id                                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    institution_id                   UUID REFERENCES institutions(id),   -- NULL for moderator/admin
    email                            VARCHAR(255) UNIQUE NOT NULL,
    password_hash                    VARCHAR(255),                       -- NULL until activated
    first_name                       VARCHAR(100),
    last_name                        VARCHAR(100),
    display_name                     VARCHAR(150),
    role                             VARCHAR(20) NOT NULL
                                     CHECK (role IN ('contributor', 'moderator', 'admin')),
    account_state                    VARCHAR(30) NOT NULL DEFAULT 'pending'
                                     CHECK (account_state IN ('pending', 'pending_email_undelivered',
                                            'active', 'expired', 'inactive', 'cancelled')),
    is_admin                         BOOLEAN NOT NULL DEFAULT FALSE,         -- Admin Owner flag
    session_version                  BIGINT NOT NULL DEFAULT 0,              -- account-wide revocation
    super_admin_transfer_requested_by UUID REFERENCES users(id),                          -- pending
Admin Owner transfer
    super_admin_transfer_expires_at  TIMESTAMPTZ,
    pending_promotion_role           VARCHAR(20)                             -- 'admin' | 'moderator'
(V118/V119)
                                     CHECK (pending_promotion_role IN ('admin', 'moderator')),
    pending_promotion_requested_by   UUID REFERENCES users(id),              -- proposing Administrator
```

```
    pending_promotion_expires_at    TIMESTAMPTZ,                            -- 72-hour confirmation window
    invited_by_user_id              UUID REFERENCES users(id),
    notify_in_app                   BOOLEAN NOT NULL DEFAULT TRUE,
    notify_email                    BOOLEAN NOT NULL DEFAULT TRUE,
    tours_enabled                   BOOLEAN NOT NULL DEFAULT TRUE,
    tour_seen_screens               JSONB,
    avatar_data                     BYTEA,
    avatar_content_type             VARCHAR(40),
    avatar_updated_at               TIMESTAMPTZ,
    purged_at                       TIMESTAMPTZ,                            -- anonymization marker
    purged_by_user_id               UUID REFERENCES users(id),
    created_at                      TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at                      TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- Exactly one active Admin Owner at a time (V79)
CREATE UNIQUE INDEX idx_users_single_active_admin_owner ON users ((role))
    WHERE role = 'admin' AND account_state = 'active' AND is_admin = TRUE;   -- exactly one active Admin
Owner

-- Invitation tokens (one-time activation links, all roles)
CREATE TABLE invitation_tokens (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    token_hash      VARCHAR(64) NOT NULL,           -- SHA-256 of raw CSPRNG token
    recipient_email VARCHAR(255) NOT NULL,
    assigned_role   VARCHAR(20) NOT NULL
                       CHECK (assigned_role IN ('contributor', 'moderator', 'admin')),
    institution_id  UUID REFERENCES institutions(id),  -- required only for contributor
    created_by_user_id UUID REFERENCES users(id),           -- scopes Moderator invitation management
    expires_at      TIMESTAMPTZ NOT NULL,           -- 72 hours from generation
    used_at         TIMESTAMPTZ,
    created_at      TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

        Related tables used by this use case:

-- audit_log (defined in UC-3.6)
CREATE TABLE audit_log (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    actor_id        UUID REFERENCES users(id),
    action          VARCHAR(50) NOT NULL,
    resource_id UUID,
    ip_address  INET,
    user_agent  TEXT,
    metadata        JSONB,
    created_at  TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
```

1.2. Institution Management

1.2.1. User Interface Design

Institution Management Screen (`/institution-management`)

A table of registered institutions with Name, Code, Status badge (Active / Inactive), logo, and Contributor count, plus summary counts by status. Administrators see a "Register Institution" button and per-row actions (Edit, Upload Logo, Deactivate/Reactivate, Delete, Reassign Users). Moderators see the same list in read-only form with no action controls. The protected default institution "DASIG Central Visayas" shows Deactivate and Delete disabled with an explanatory tooltip.

Register / Edit Institution Modal

Fields: Institution Name, Institution Code (unique), and Email Domain. Inline errors are shown for a duplicate code or email domain when registering, and for a name or email domain already used by another institution when editing.

Deactivate Confirmation

Blocked while Contributors remain assigned or invitations to the institution are still pending; the dialog directs the Administrator to reassign the Contributors (or resolve the invitations) first.

Permanent Delete Confirmation

Requires typing the institution's code. Blocked for the protected institution, any institution with PENDING/PENDING_EMAIL_UNDELIVERED/ACTIVE Contributors, any submission history, or any non-deleted media.

Reassign User

From User Management, an Administrator moves a Contributor to another ACTIVE institution; historical submissions stay attributed to the original institution.

1.2.2. Front-end Component(s)

InstitutionManagementScreen

- Fetches `GET /institutions` and `GET /institutions/summary-counts`; hosts create/edit/deactivate/reactivate/delete dialogs. Role-aware rendering (Admin vs. Moderator read-only).
- React functional component / Reviewer route (`RequireReviewer`)

Institution Logo Upload

- `PUT /institutions/{id}/logo` (multipart); logos are served by `GET /institutions/{id}/logo`.
- React functional component

Reassign Action (User Management)

- `PATCH /users/{id}/institution`.
- React functional component

1.2.3. Back-end Component(s)

InstitutionController

- `POST /institutions` (Admin), `GET /institutions`, `GET /institutions/{id}`, `GET /institutions/summary-counts`, `GET /institutions/public` (unauthenticated list used by public pages), `PUT /institutions/{id}` (Admin), `PATCH /institutions/{id}/deactivate|reactivate` (Admin), `DELETE /institutions/{id}` (Admin, code confirmation), `PUT/GET /institutions/{id}/logo`, `PATCH /institutions/{id}/ai-media-hybrid-ranking` (Admin). Class-level `@PreAuthorize("hasAnyRole('ADMIN','MODERATOR')")`; write endpoints are Admin-only.
- Spring `@RestController`

InstitutionService

- Validates uniqueness of code and email domain on create, and of name and email domain on edit; enforces protected-institution rules; blocks deactivation while Contributors are assigned or invitations to the institution are pending; performs eligibility-checked permanent deletion (cleans tokens, reservations, page settings, watermark configurations, soft-deleted media and albums, then best-effort R2 purge after commit) and writes `INSTITUTION_DELETED`.
- Spring `@Service`

WorkspaceProvisionerService

- Provisions the institution workspace (default page settings and media library scoping) on creation; RLS policies are defined once in Flyway migrations and key off `institution_id`.
- Spring `@Service`

UserService

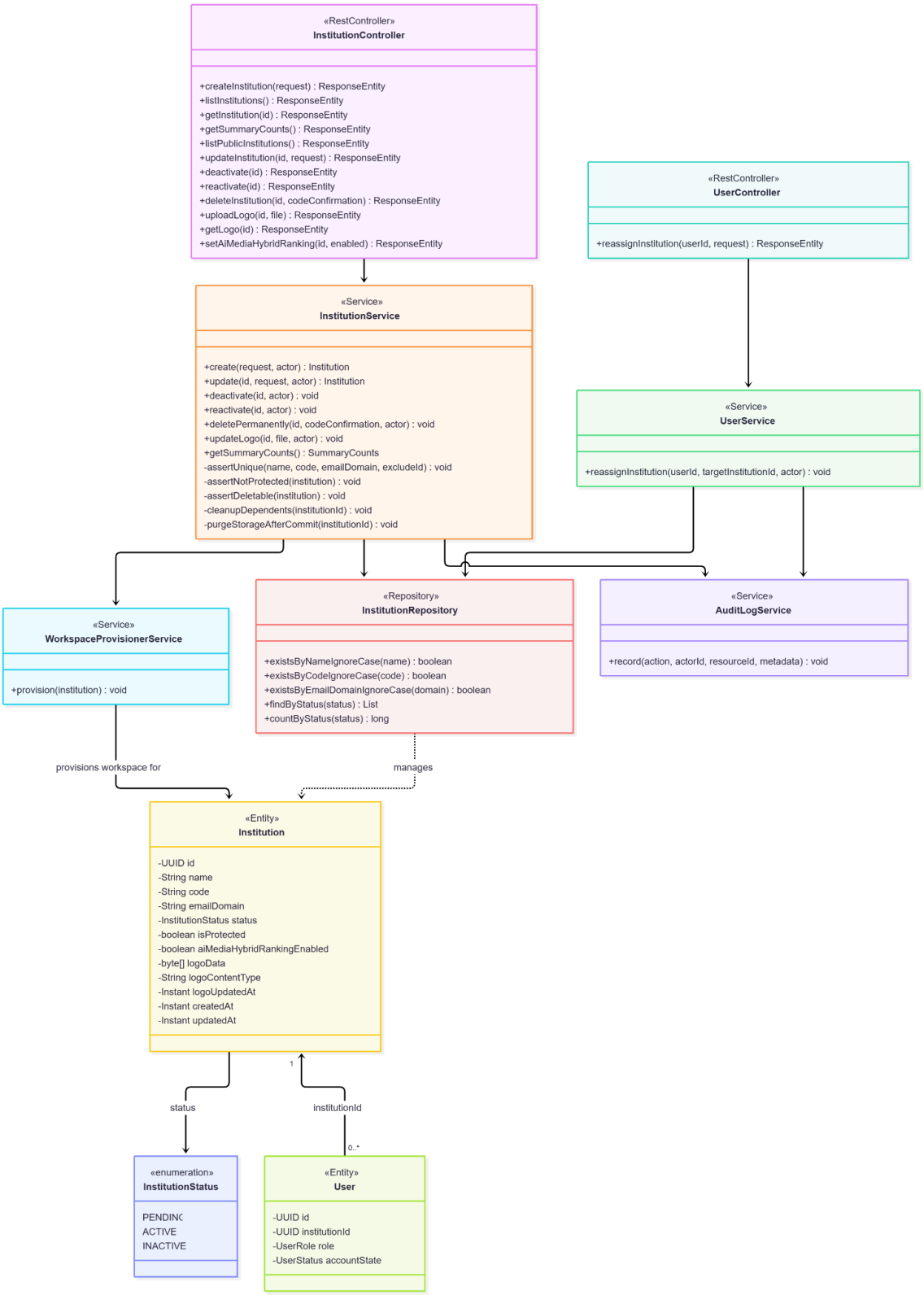
- Reassigns a Contributor's `institution_id` to an ACTIVE target institution.
- Spring `@Service`

InstitutionRepository / AuditLogService

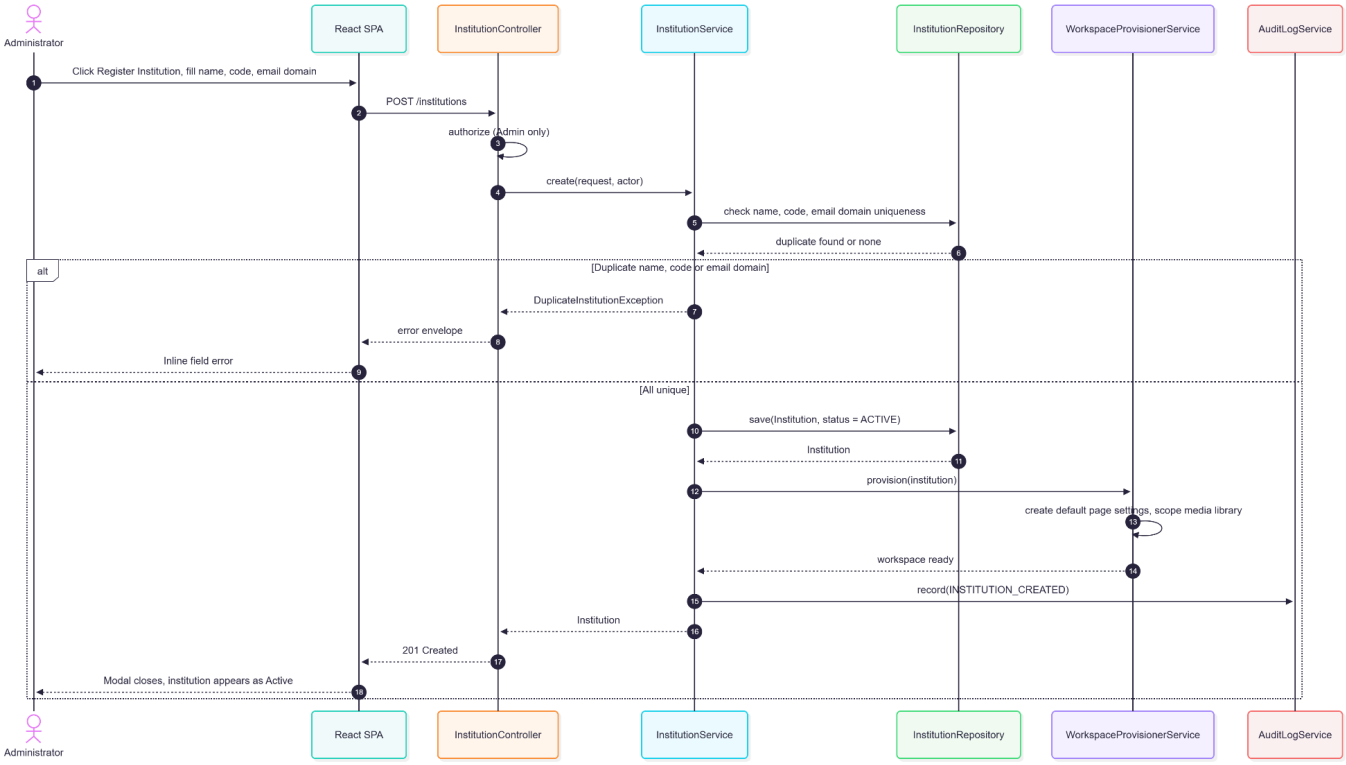
- Spring `@Repository` extending `JpaRepository<Institution, UUID>`; audit entries for create, edit, deactivate, reactivate, delete, and reassignment.

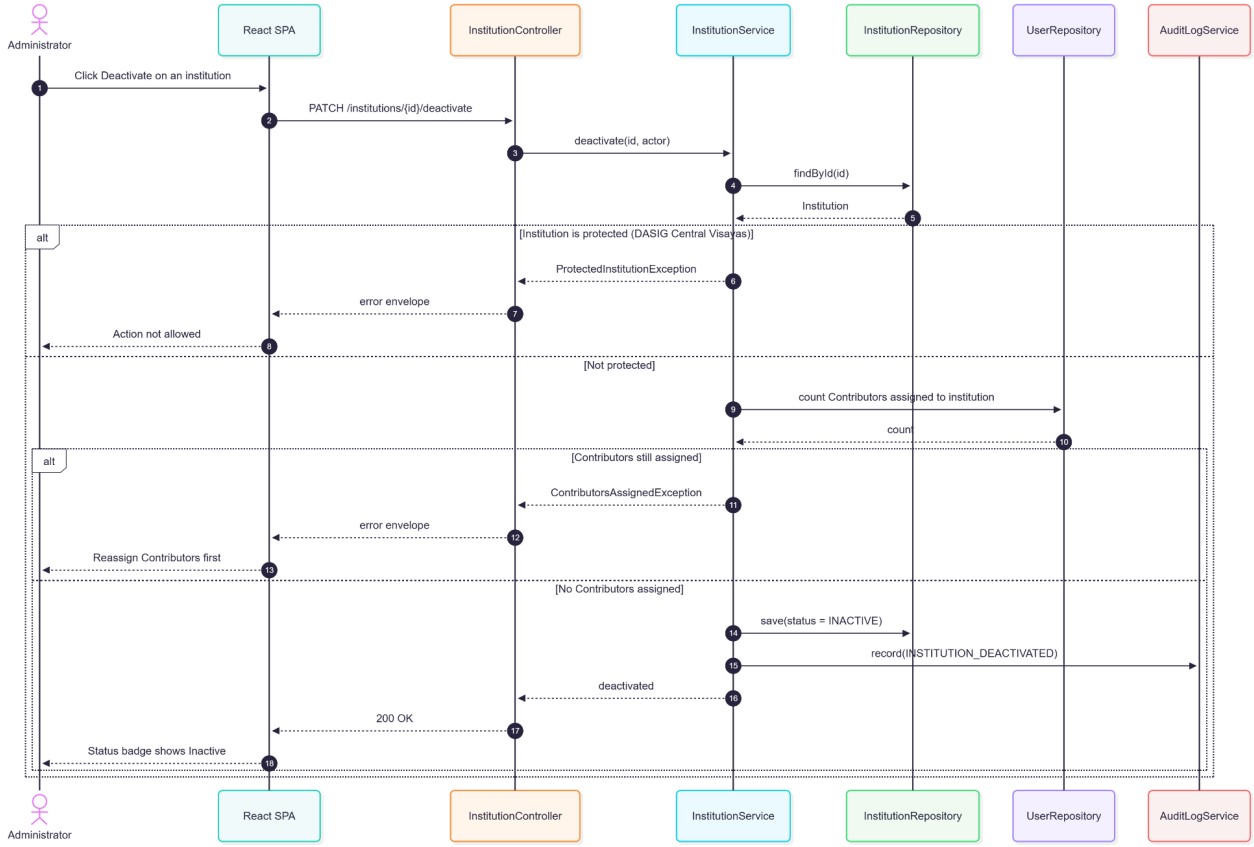
1.2.4. Object-Oriented Component(s)

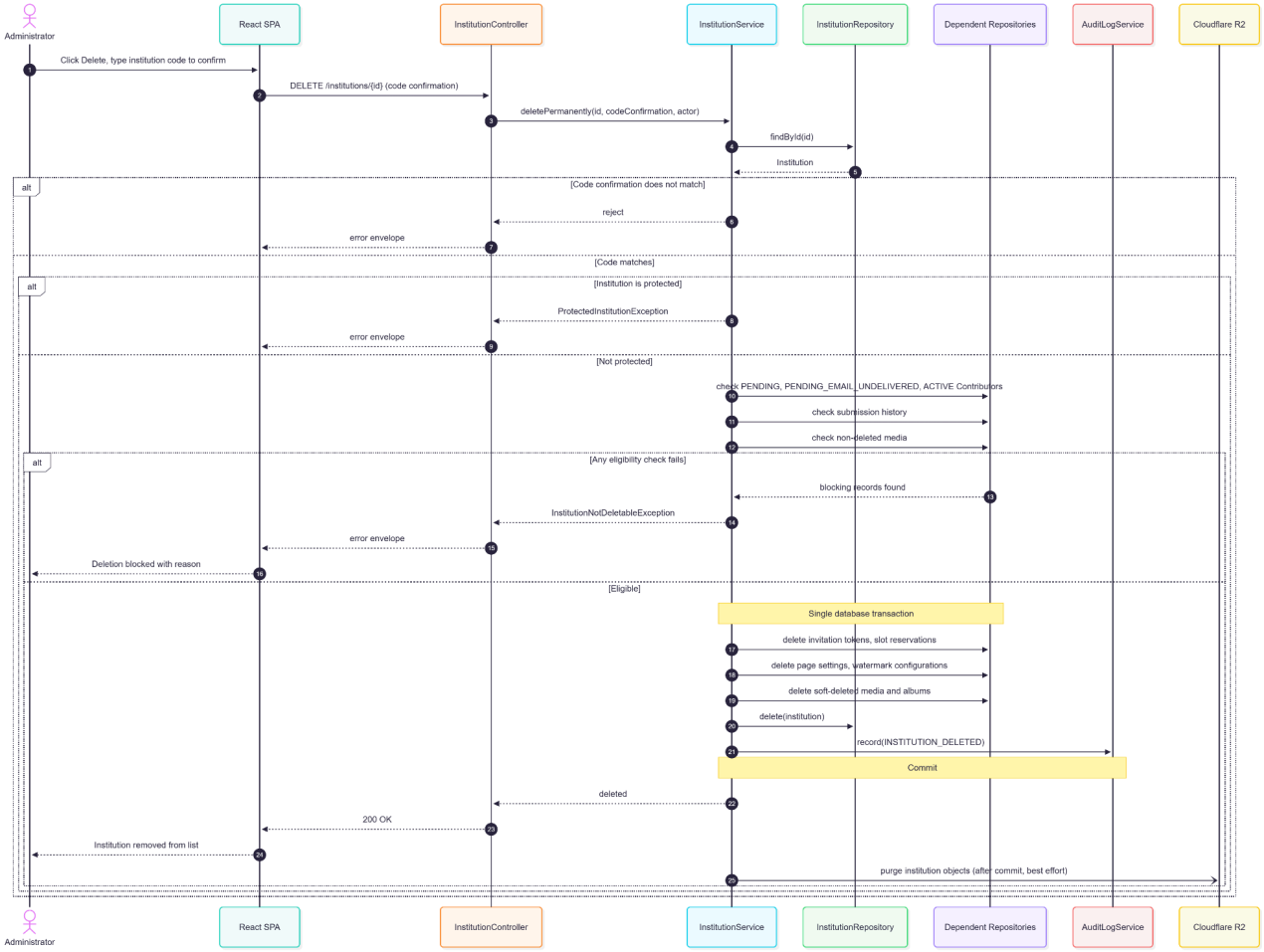
- **Class Diagram**

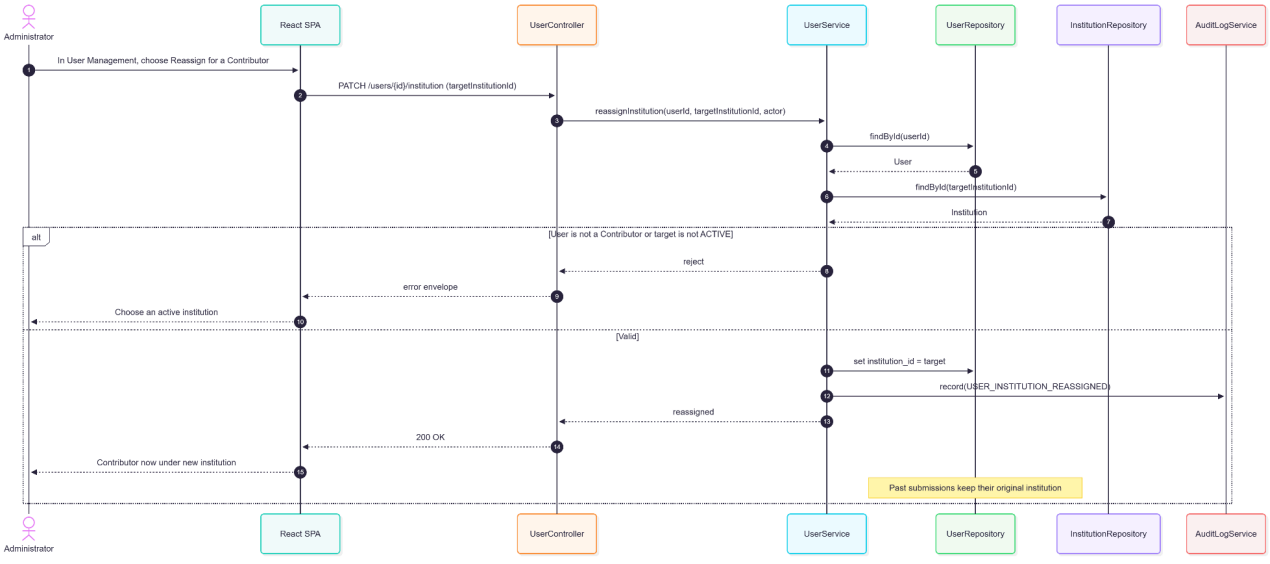


- **Sequence Diagram**



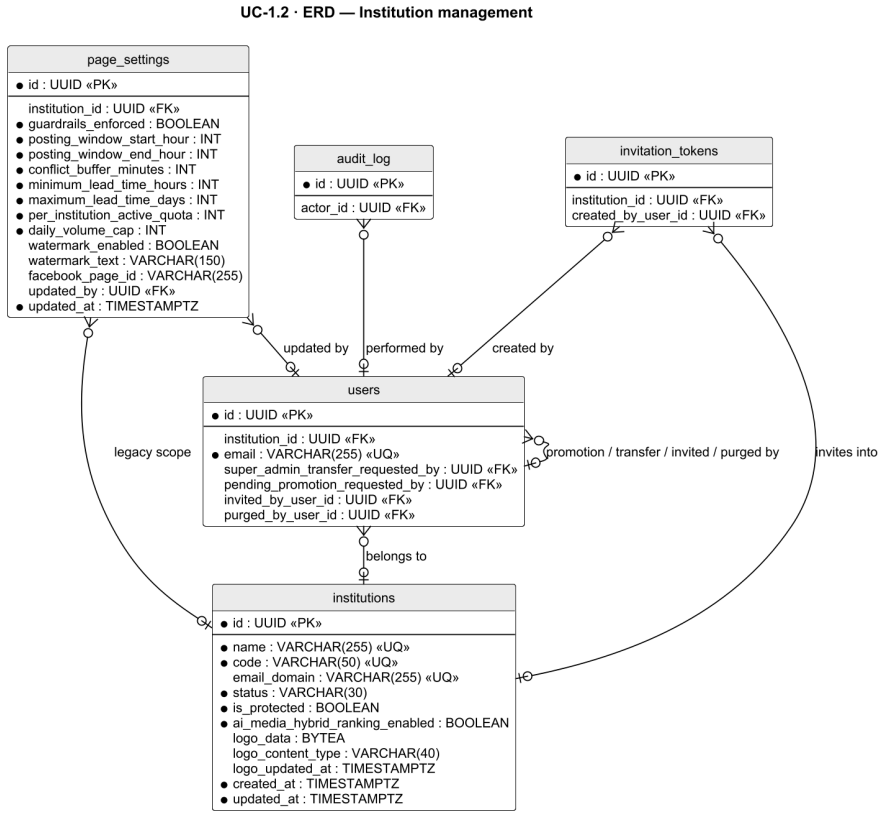



Administrator
React SPA
InstitutionController
InstitutionService
InstitutionRepository
Dependent Repositories
AuditLogService
Cloudflare R2
Click Delete, type institution code to confirm
DELETE /institutions/{id} (code confirmation)
deletePermanently(id, codeConfirmation, actor)
findById(id)
Institution
alt
[Code confirmation does not match]
reject
error envelope
[Code matches]
alt
[Institution is protected]
ProtectedInstitutionException
error envelope
[Not protected]
check PENDING, PENDING_EMAIL_UNDELIVERED, ACTIVE Contributors
check submission history
check non-deleted media
alt
[Any eligibility check fails]
blocking records found
InstitutionNotDeletableException
error envelope
Deletion blocked with reason
[Eligible]
Single database transaction
delete invitation tokens, slot reservations
delete page settings, watermark configurations
delete soft-deleted media and albums
delete(institution)
record(INSTITUTION_DELETED)
Commit
deleted
200 OK
Institution removed from list
purge institution objects (after commit, best effort)


Administrator
React SPA
UserController
UserService
UserRepository
InstitutionRepository
AuditLogService
In User Management, choose Reassign for a Contributor
PATCH /users/{id}/institution (targetInstitutionId)
reassignInstitution(userId, targetInstitutionId, actor)
findById(userId)
User
findById(targetInstitutionId)
Institution
alt
[User is not a Contributor or target is not ACTIVE]
reject
error envelope
Choose an active institution
[Valid]
set institution_id = target
record(USER_INSTITUTION_REASSIGNED)
reassigned
200 OK
Contributor now under new institution
Past submissions keep their original institution

1.2.5. Data Design

- **Entity Relationship Diagram**



- **Schema**

```
CREATE TABLE institutions (
    id                              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    name                            VARCHAR(255) UNIQUE NOT NULL,
    code                            VARCHAR(50)  UNIQUE NOT NULL,   -- e.g. CITU
    email_domain                    VARCHAR(255) UNIQUE,
    status                          VARCHAR(30) NOT NULL DEFAULT 'active'
                                    CHECK (status IN ('pending', 'active', 'inactive')),
    is_protected                    BOOLEAN NOT NULL DEFAULT FALSE,
    ai_media_hybrid_ranking_enabled BOOLEAN NOT NULL DEFAULT FALSE,
    logo_data                       BYTEA,
    logo_content_type               VARCHAR(40),
    logo_updated_at                 TIMESTAMPTZ,
    created_at                      TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at                      TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- Seed (initial deployment)
INSERT INTO institutions (name, code, status, is_protected)
VALUES ('DASIG Central Visayas', 'DASIG-CENTRAL', 'active', TRUE);

-- RLS pattern (applied in Flyway to each institution-scoped table)
ALTER TABLE submissions ENABLE ROW LEVEL SECURITY;
CREATE POLICY institution_isolation ON submissions
    USING (institution_id = current_setting('app.current_institution_id', true)::UUID
           OR current_setting('app.current_role', true) IN ('moderator', 'admin'));

        Related tables used by this use case:

-- page_settings (defined in UC-1.4)
```

```
CREATE TABLE page_settings (
    id                      UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    institution_id          UUID REFERENCES institutions(id),
    guardrails_enforced BOOLEAN NOT NULL DEFAULT TRUE,
    posting_window_start_hour INT NOT NULL DEFAULT 8,       -- GR-H4 window start (Asia/Manila, V116)
    posting_window_end_hour   INT NOT NULL DEFAULT 20,      -- GR-H4 window end (exclusive)
    conflict_buffer_minutes   INT NOT NULL DEFAULT 30,      -- GR-H1 (V121)
    minimum_lead_time_hours   INT NOT NULL DEFAULT 2,       -- GR-H2 (V121)
    maximum_lead_time_days  INT NOT NULL DEFAULT 30,        -- GR-H3 (V121)
    per_institution_active_quota INT NOT NULL DEFAULT 3,  -- GR-S1 (V122)
    daily_volume_cap        INT NOT NULL DEFAULT 6,         -- GR-S2 (V122)
    watermark_enabled   BOOLEAN,
    watermark_text          VARCHAR(150),
    facebook_page_id        VARCHAR(255),
    updated_by              UUID REFERENCES users(id),
    updated_at              TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
```

1.3. Contributor Account Management

1.3.1. User Interface Design

User Invitations Screen and User Management Screen (`/admin/user-management`)

Institution-grouped cards listing Contributor accounts (name, email, status badge, last activity) with a pending-invitations card and a delivery-issues alert for PENDING_EMAIL_UNDELIVERED accounts. "Invite Contributor" opens the invitation composer, where the actor selects an ACTIVE target institution and enters one or more emails. Administrators see Deactivate/Reactivate/Delete/Erase/Change Role actions; Moderators see only Resend, Cancel, and Delete on invitations they personally sent.

Reissue Invitation Confirmation

A confirmation dialog for EXPIRED, CANCELLED, PENDING, or PENDING_EMAIL_UNDELIVERED accounts that issues a fresh 72-hour token and invalidates the previous one.

Invitation Accept Page (`/invite?token=…`) — shared with UC-1.1.

1.3.2. Front-end Component(s)

UserInvitationsScreen / UserManagementScreen

- Fetches `GET /users`, `GET /users/counts`, `GET /invitations/pending`, `GET /invitations/pending/count`. Hosts `InstitutionUsersCard`, `PendingInvitationsCard`, `DeliveryIssuesAlert`, `ActionMenu`.
- React functional component / Reviewer route

InvitationComposer

- Submits `POST /invitations` with `role = contributor` and `institutionId`. A Moderator may only choose the Contributor role.
- React functional component

PendingInvitationsCard

- `POST /invitations/{id}/resend`, `DELETE /invitations/{id}` (cancel), `DELETE /invitations/by-user/{userId}` (Admin).

- React functional component

1.3.3. Back-end Component(s)

InvitationController — shared with UC-1.1; Moderator access to resend/cancel/delete is restricted in `InvitationService` to tokens whose `created_by_user_id` is the caller.

UserController

- `PATCH /users/{id}/status` (Admin), `DELETE /users/{id}` (Admin any; Moderator only for a CANCELLED/EXPIRED Contributor they invited), `POST /users/{id}/erase` (Admin Owner), `PATCH /users/{id}/role` (proposal to Moderator/Administrator).
- Spring `@RestController`

InvitationService

- Rejects invitations to INACTIVE institutions (BR-ACC-01), rejects re-invitation of ACTIVE or INACTIVE accounts, binds the token to the institution (a mismatched token returns a generic invalid-token error).
- Spring `@Service`

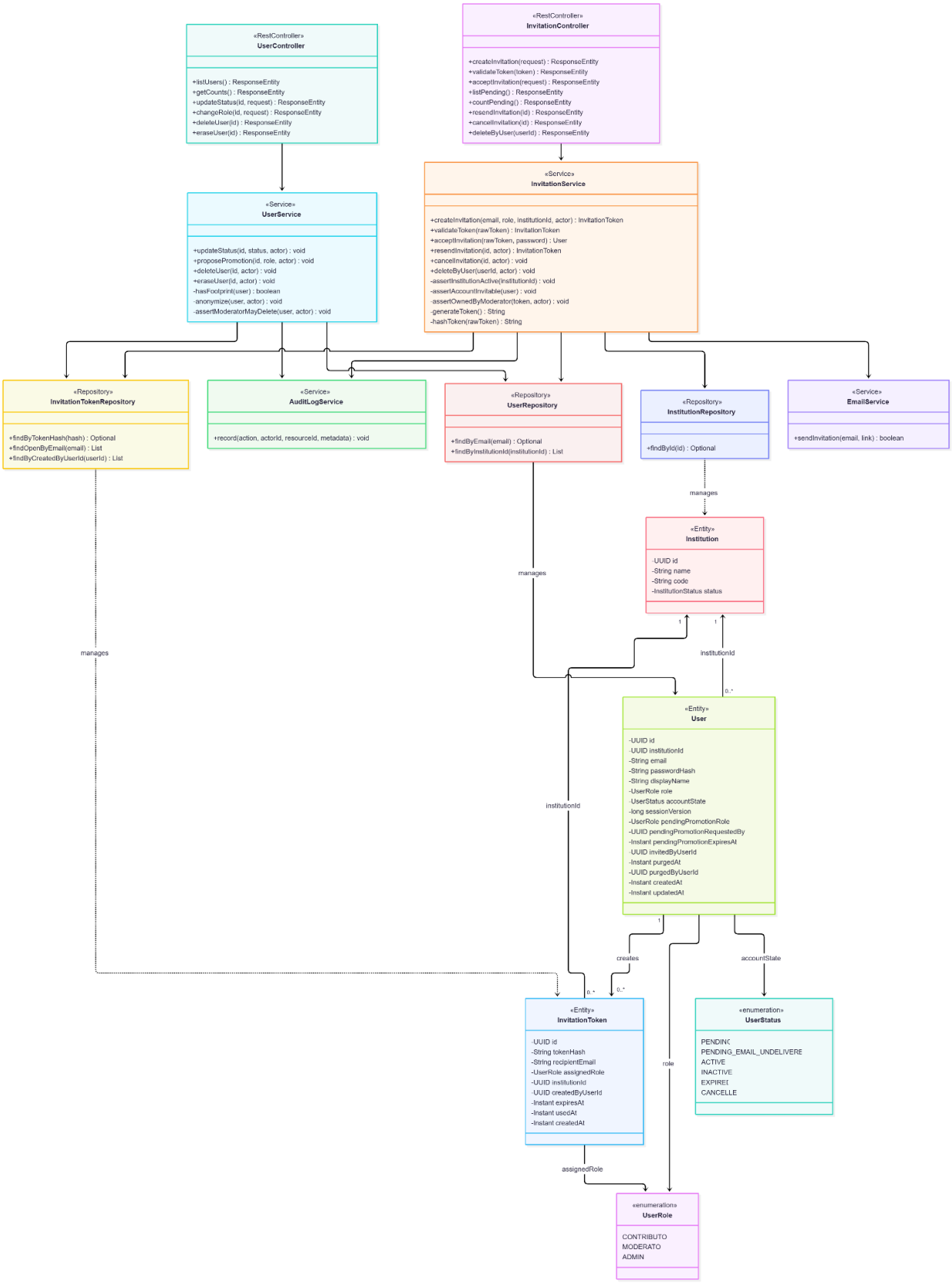
UserService

- Footprint-aware delete: any submission, media upload, validation log, created album, or audit entry keeps the row as an anonymized inactive record (`USER_REMOVED`); only footprint-free accounts are hard-deleted.
- Spring `@Service`

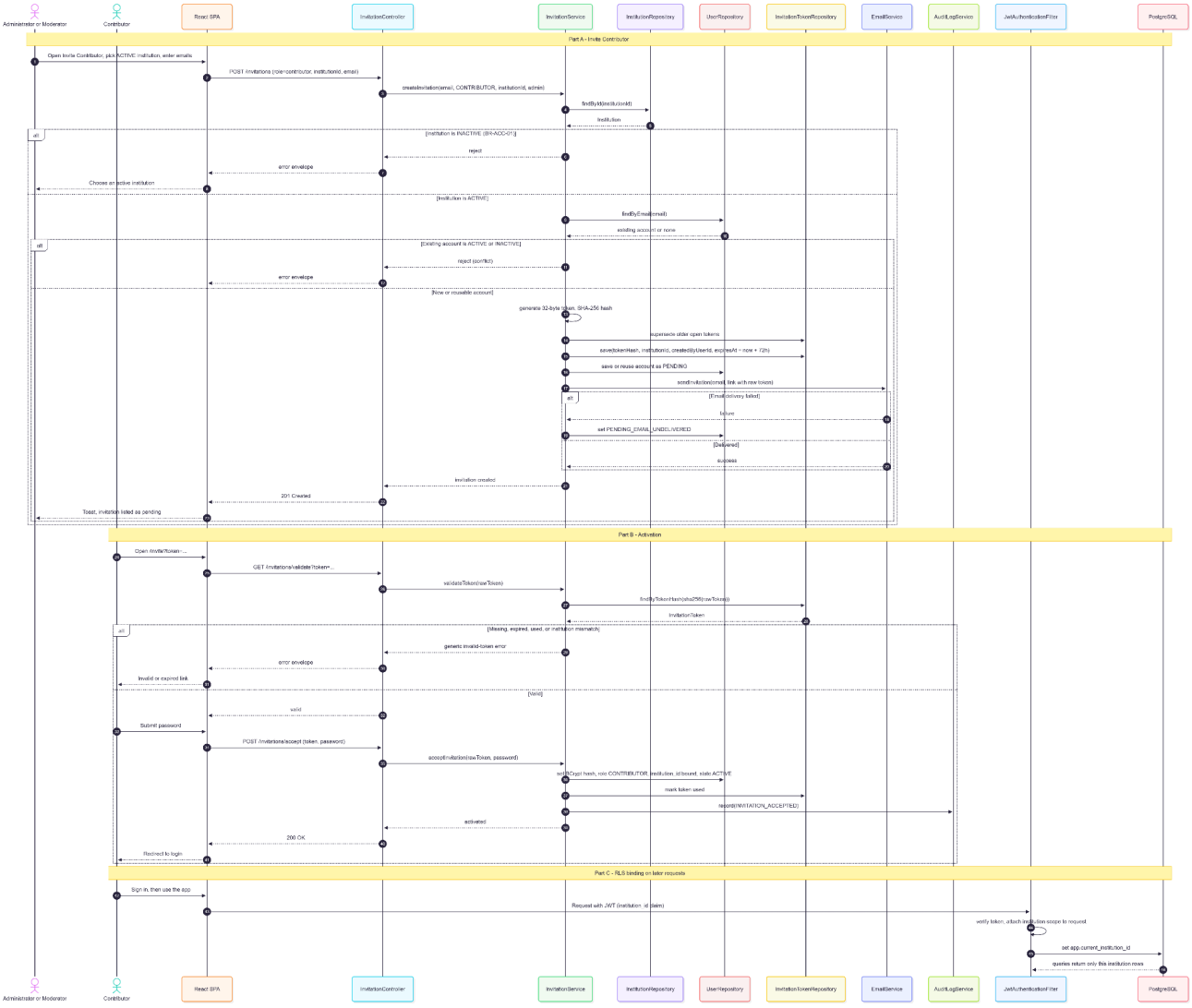
AuditLogService / EmailService — as in UC-1.1.

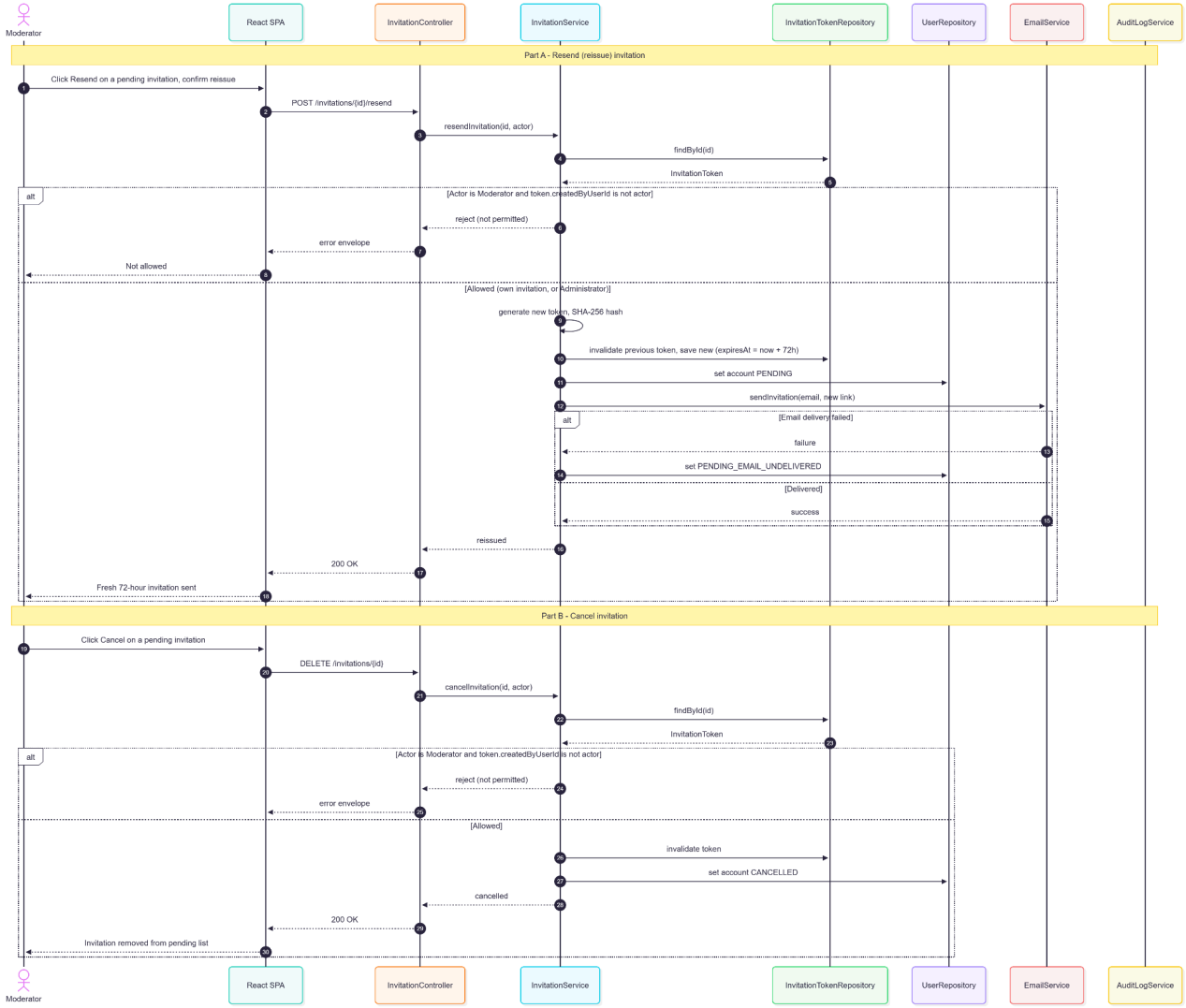
1.3.4. Object-Oriented Component(s)

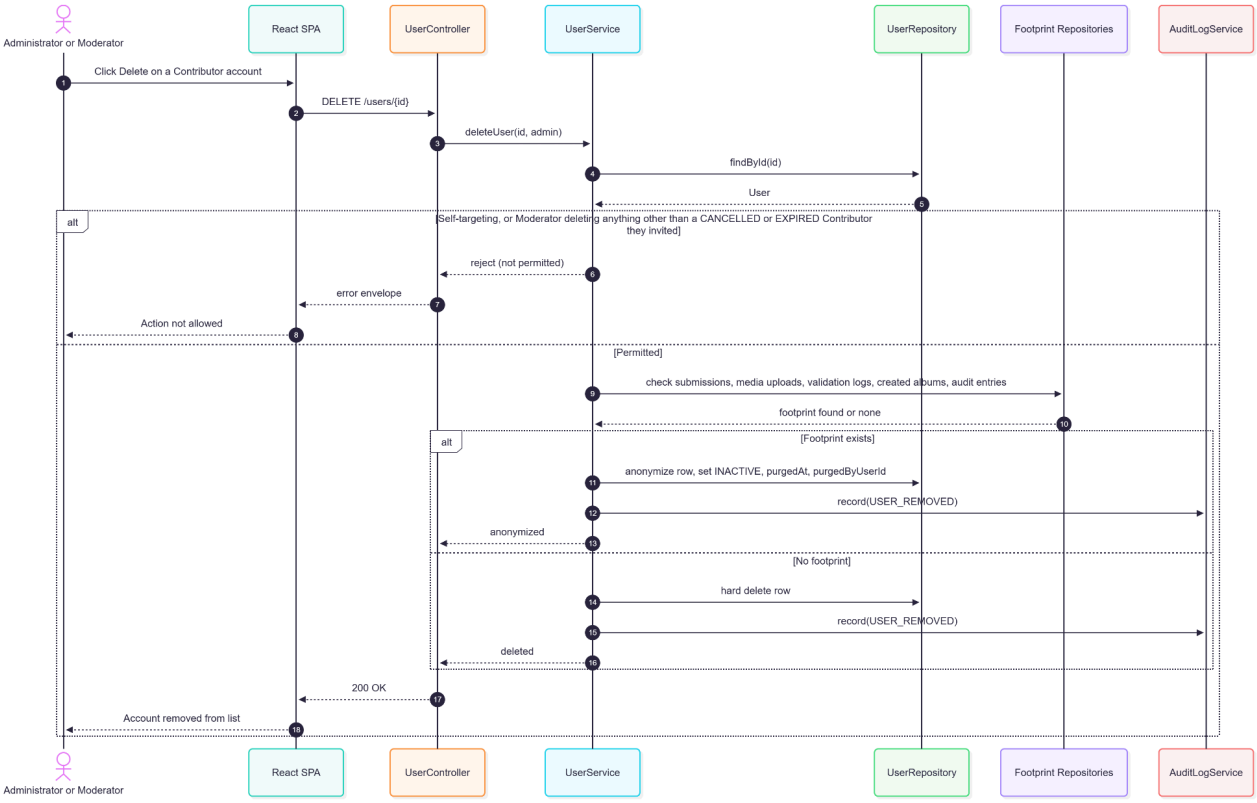
- **Class Diagram**



- **Sequence Diagram**



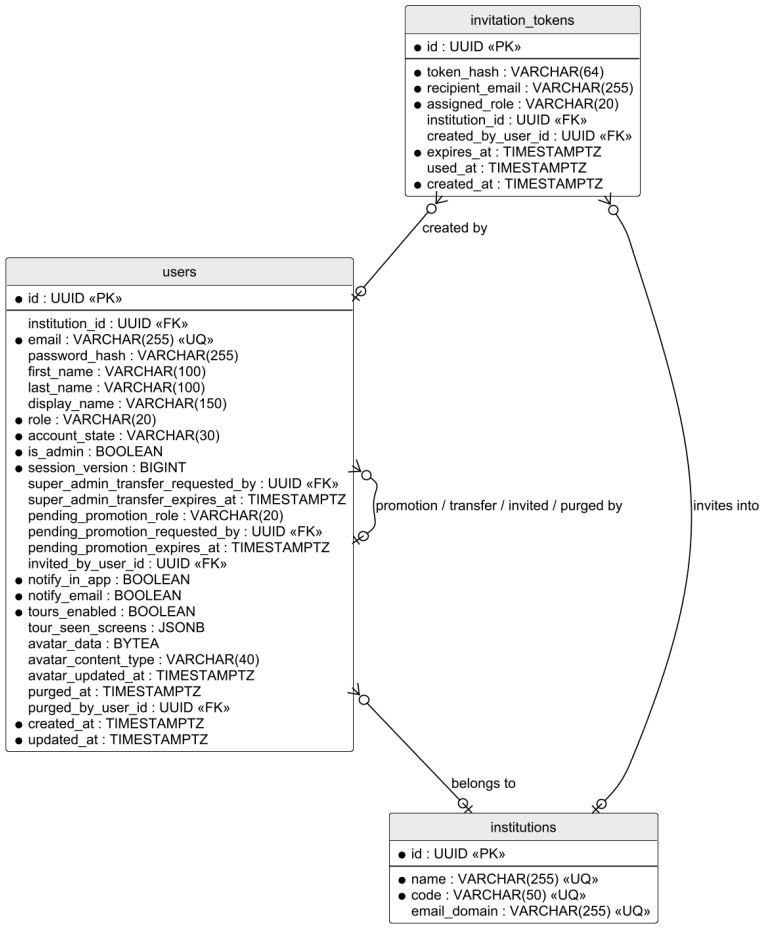

Moderator
React SPA
InvitationController
InvitationService
InvitationTokenRepository
UserRepository
EmailService
AuditLogService
Part A - Resend (reissue) invitation
1 Click Resend on a pending invitation, confirm reissue
2 POST /invitations/{id}/resend
3 resendInvitation(id, actor)
4 findById(id)
5 InvitationToken
alt
[Actor is Moderator and token.createdByUserId is not actor]
6 reject (not permitted)
7 error envelope
8 Not allowed
[Allowed (own invitation, or Administrator)]
9 generate new token, SHA-256 hash
10 invalidate previous token, save new (expiresAt = now + 72h)
11 set account PENDING
12 sendInvitation(email, new link)
alt
[Email delivery failed]
13 failure
14 set PENDING_EMAIL_UNDELIVERED
[Delivered]
15 success
16 reissued
17 200 OK
18 Fresh 72-hour invitation sent
Part B - Cancel invitation
19 Click Cancel on a pending invitation
20 DELETE /invitations/{id}
21 cancelInvitation(id, actor)
22 findById(id)
23 InvitationToken
alt
[Actor is Moderator and token.createdByUserId is not actor]
24 reject (not permitted)
25 error envelope
[Allowed]
26 invalidate token
27 set account CANCELLED
28 cancelled
29 200 OK
30 Invitation removed from pending list



1.3.5. Data Design

- **Entity Relationship Diagram**

UC-1.3 · ERD — Contributor account management



- **Schema**

```
-- users (columns used by Contributor account management)
CREATE TABLE users (
    id                              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    institution_id                  UUID REFERENCES institutions(id),   -- required for contributors
    email                           VARCHAR(255) UNIQUE NOT NULL,
    password_hash                   VARCHAR(255),                       -- NULL until activated
    first_name                      VARCHAR(100),
    last_name                       VARCHAR(100),
    display_name                    VARCHAR(150),
    role                            VARCHAR(20) NOT NULL
                                    CHECK (role IN ('contributor', 'moderator', 'admin')),
    account_state                   VARCHAR(30) NOT NULL DEFAULT 'pending',
    session_version                 BIGINT NOT NULL DEFAULT 0,          -- incremented on deactivation
/ role change
    pending_promotion_role          VARCHAR(20)                         --   proposed   Moderator   or
Administrator promotion
                                    CHECK (pending_promotion_role IN ('admin', 'moderator')),
    pending_promotion_requested_by UUID REFERENCES users(id),
    pending_promotion_expires_at   TIMESTAMPTZ,                    -- 72-hour confirmation window
    invited_by_user_id              UUID REFERENCES users(id),
    purged_at                       TIMESTAMPTZ,                        --    set    when    anonymized
(delete/erase)
    purged_by_user_id               UUID REFERENCES users(id),
    created_at                      TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at                      TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
```

```
CREATE TABLE invitation_tokens (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    token_hash      VARCHAR(64) NOT NULL,           -- SHA-256 of raw token
    recipient_email VARCHAR(255) NOT NULL,
    assigned_role   VARCHAR(20) NOT NULL
                    CHECK (assigned_role IN ('contributor', 'moderator', 'admin')),
    institution_id  UUID REFERENCES institutions(id),  -- required for contributor invitations
    created_by_user_id UUID REFERENCES users(id),           -- scopes Moderator resend/cancel/delete
    expires_at      TIMESTAMPTZ NOT NULL,           -- 72 hours from generation
    used_at         TIMESTAMPTZ,
    created_at      TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_users_email                ON users(email);
CREATE INDEX idx_users_institution          ON users(institution_id);
CREATE INDEX idx_invitation_tokens_hash     ON invitation_tokens(token_hash);
CREATE INDEX idx_invitation_tokens_institution ON invitation_tokens(institution_id);
CREATE INDEX idx_invitation_tokens_created_by ON invitation_tokens(created_by_user_id);
```

1.4. Login & Session Management

1.4.1. User Interface Design

Login Page (`/login`)

A centered card with email, password (show/hide toggle), and a Log In button. Invalid credentials show a generic error. After 5 failed attempts the form is replaced by a lockout banner with the remaining duration (15 minutes). A "Forgot password?" link sits below the form. PENDING, EXPIRED, and INACTIVE accounts receive state-specific messages.

Forgot / Reset Password Pages (`/forgot-password`, `/forgot-password-sent`, `/reset-password`)

An email input with a neutral confirmation regardless of account existence; the reset page enforces the password policy with live per-rule feedback.

Account Settings Screen (`/settings`)

Tabs for Personal Settings (all roles: display name, avatar, password change, notification channel preferences, onboarding-guide toggle and reset, and — for Moderators and Administrators only — Facebook Messenger linking) and Page Settings (Administrators only: guard-rail enforcement switch and the guard-rail thresholds — conflict buffer, minimum and maximum lead time, posting-window hours, per-institution quota, daily cap — Facebook Page token management, Watermark Configuration, UC-2.5).

Link Messenger (Personal Settings)

Generates a 10-minute link code that the user sends to the DASIG Facebook Page in Messenger; the panel then shows the linked state with an Unlink action.

Session Warning Banner

Shown during the final five minutes of the JWT lifetime with "Stay Logged In" (opens the re-login modal) and "Dismiss" (hides the banner only).

Session Expired Modal

Opened at countdown zero or on a session-related 401. The user re-enters email and password; a successful re-login issues a new JWT, reloads `/me`, clears authenticated query caches, and restarts the countdown.

1.4.2. Front-end Component(s)

LoginScreen / ForgotScreen / ForgotSentScreen / ResetPasswordScreen / NoAccountScreen

- Submit `POST /auth/login`, `POST /auth/forgot-password`, `POST /auth/reset-password`.
- React functional components / public routes

useAuthSession / useLoginLockout / useSessionCountdown / usePasswordResetFlow

- Session restore from stored token and `GET /me`, login with lockout feedback, JWT `exp` countdown, session-expired modal, invite activation, and sign-out (`POST /auth/logout`).
- React custom hooks

RouteGates (`RequireSignedIn`, `RequireReviewer`, `RequireAdmin`) / ProtectedRoute

- Layout-route gates in `App.tsx`; unauthorized roles see `ForbiddenPage`.
- React functional components

DashboardShell (navigation)

- Shows Admin Management, User Management, System Health, Audit Log, and Page Settings only to Administrators; Institution Management and Review Queue to Moderators and Administrators; Dashboard, Submissions, Calendar, Media Repository, Notifications, Analytics, and Settings to all roles.
- React layout component

AccountSettingsScreen

- `PATCH /me/settings`, `PATCH /me/tour-preferences`, `POST /auth/change-password`, `PUT /users/{id}/avatar`, Messenger panel (`GET/POST/DELETE /integrations/messenger/connection[/link-code]`), and Page Settings (`GET/PUT /settings/page`).
- React functional component

1.4.3. Back-end Component(s)

AuthController

- `POST /auth/login` (validate credentials and account state, issue an 8-hour JWT), `POST /auth/logout` (revoke this token only), `POST /auth/refresh` (authenticated; not currently called by the frontend).
- Spring `@RestController`

PasswordController / PasswordService / PasswordPolicy

- `POST /auth/forgot-password` (neutral response; 1-hour single-use reset token), `POST /auth/reset-password` (increments `session_version`, invalidating all sessions), `POST`

`/auth/change-password` (requires current password; other sessions unaffected). Policy: ≥12 characters, upper and lower case, digit, special character, no whitespace, no common fragments, no email/name.

- Spring `@RestController` / `@Service`

JWTService

- HS256 tokens embedding user ID, role, `institution_id` (Contributors only), Admin Owner flag, and `session_version`. `extractClaims` rejects tokens present in `revoked_tokens` or older than the user's current `session_version`, so revocation survives restarts and spans instances.
- Spring `@Service`

JwtAuthenticationFilter / TenantScopeService

- Authenticates each request and resolves the tenant scope used by services.
- Spring Security filter / `@Service`

AccountLockoutService

- Tracks failed attempts in account_lockouts; locks for 15 minutes after 5 failures (login returns 401 "Account temporarily locked" with the remaining time; no lockout email is sent).
- Spring `@Service`

MessengerConnectionController / MessengerWebhookController / MessengerConnectionService / MessengerWebhookService

- Issues hashed 10-minute link codes; the Meta webhook (`GET/POST /integrations/messenger/webhook`) verifies a code sent to the Page and binds the sender's Page-scoped ID (`page_scoped_user_id`) to the user. Restricted to Moderators and Administrators.
- Spring `@RestController` / `@Service`

UserController

- `GET /me`, `PATCH /me/settings`, `PATCH /me/tour-preferences`.
- Spring `@RestController`

PageSettingsController / GuardRailSettingsService

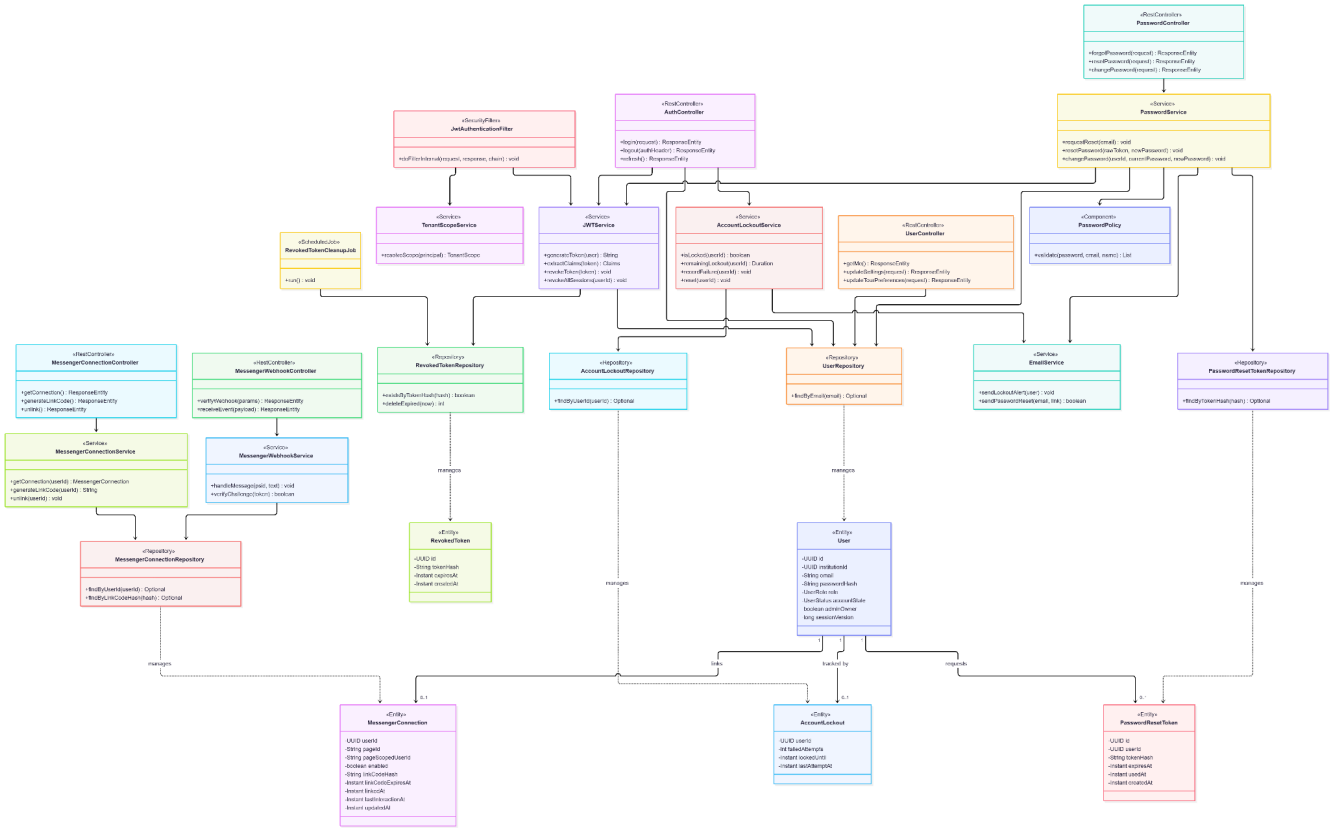
- `GET/PUT /settings/page` (Admin) toggles the network-wide `guardrails_enforced` switch and sets every guard-rail threshold: conflict buffer (default 30 min), minimum lead time (2 h), maximum lead time (30 days), posting window (8–20), per-institution active quota (3), and daily volume cap (6), all read through `GuardRailSettingsService`. Page Settings changes are not written to the audit log.
- Spring `@RestController` / `@Service`

RevokedTokenCleanupJob

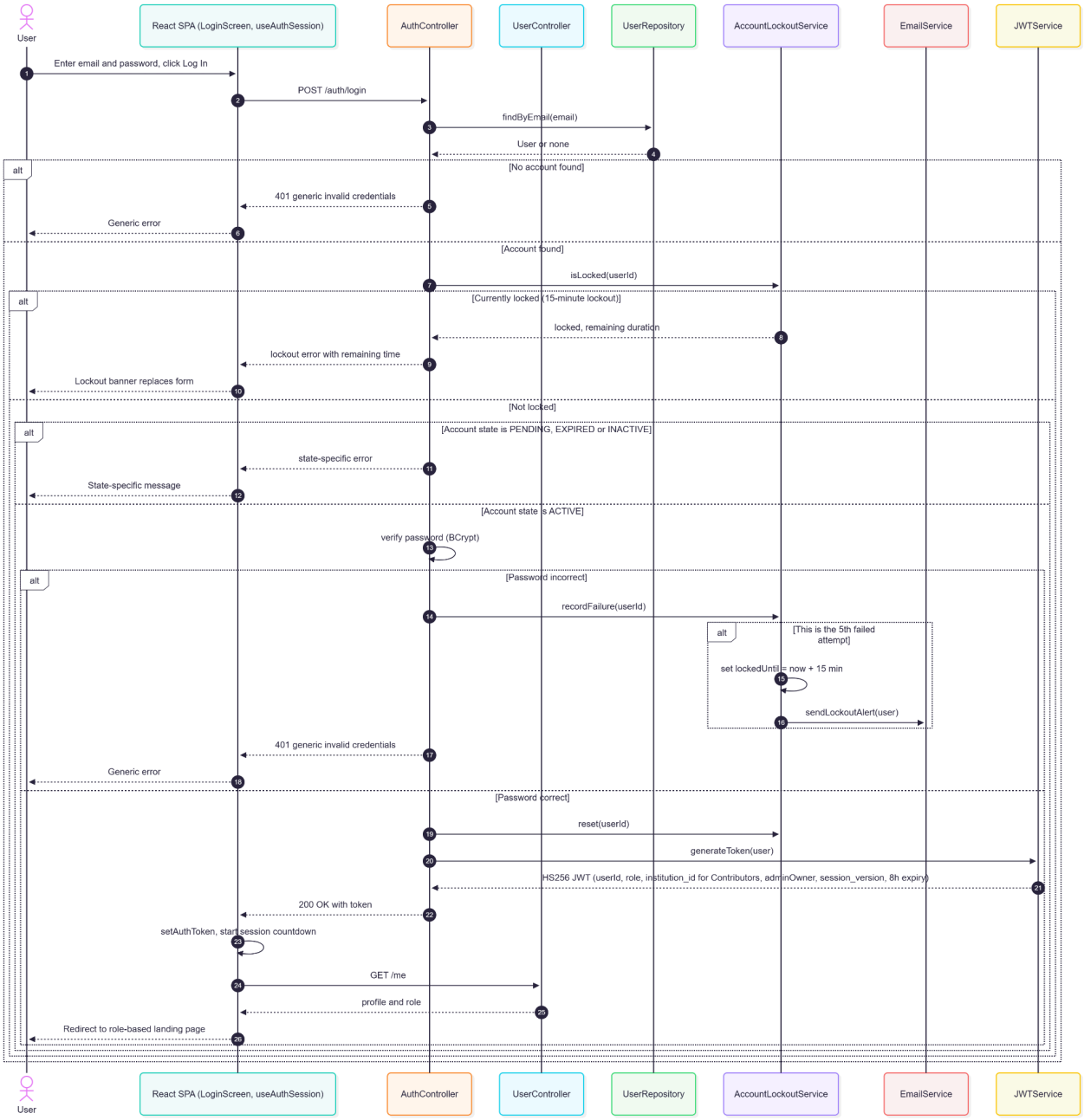
- Hourly removal of expired revoked-token rows.
- Spring `@Scheduled`

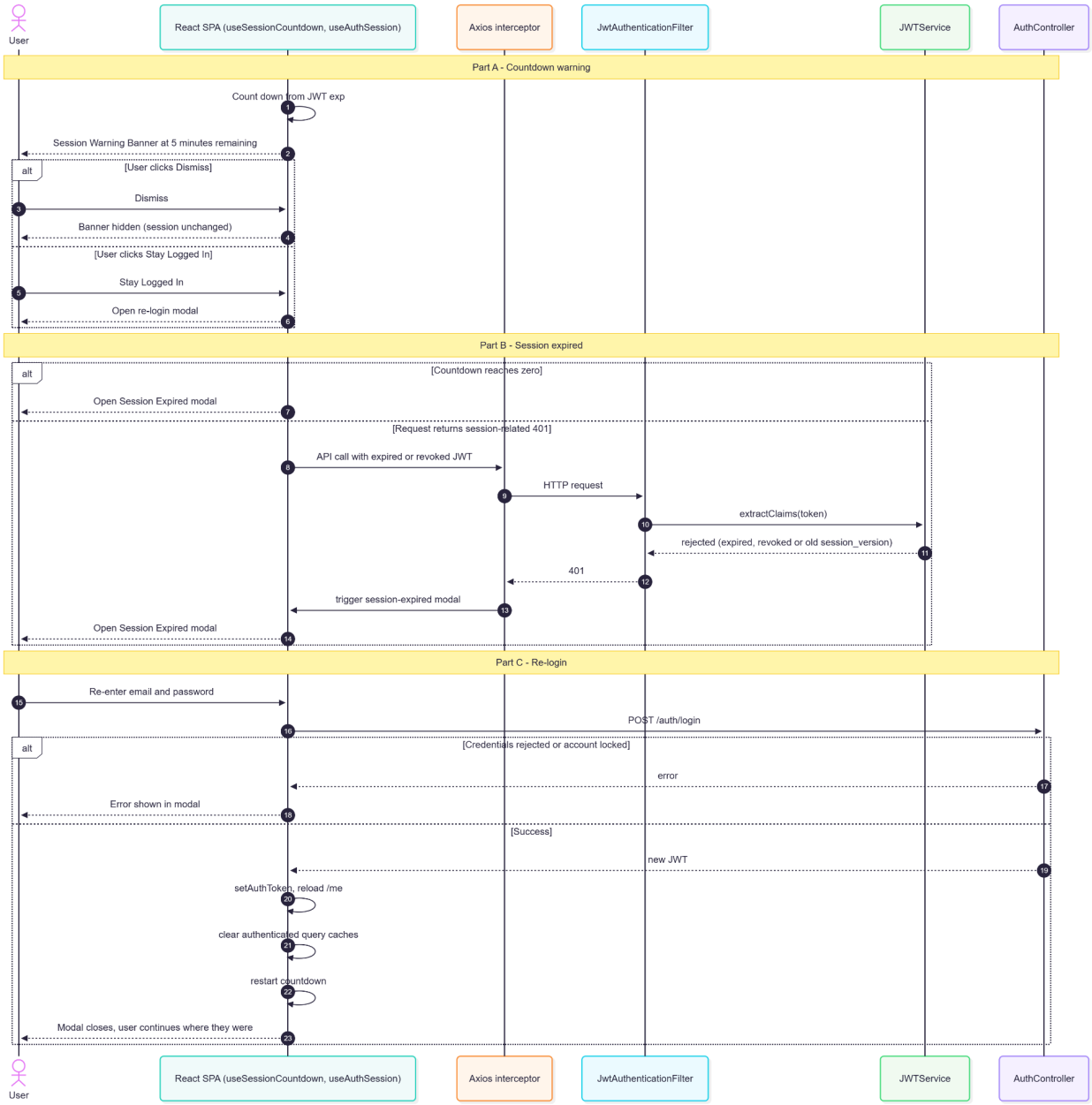
1.4.4. Object-Oriented Component(s)

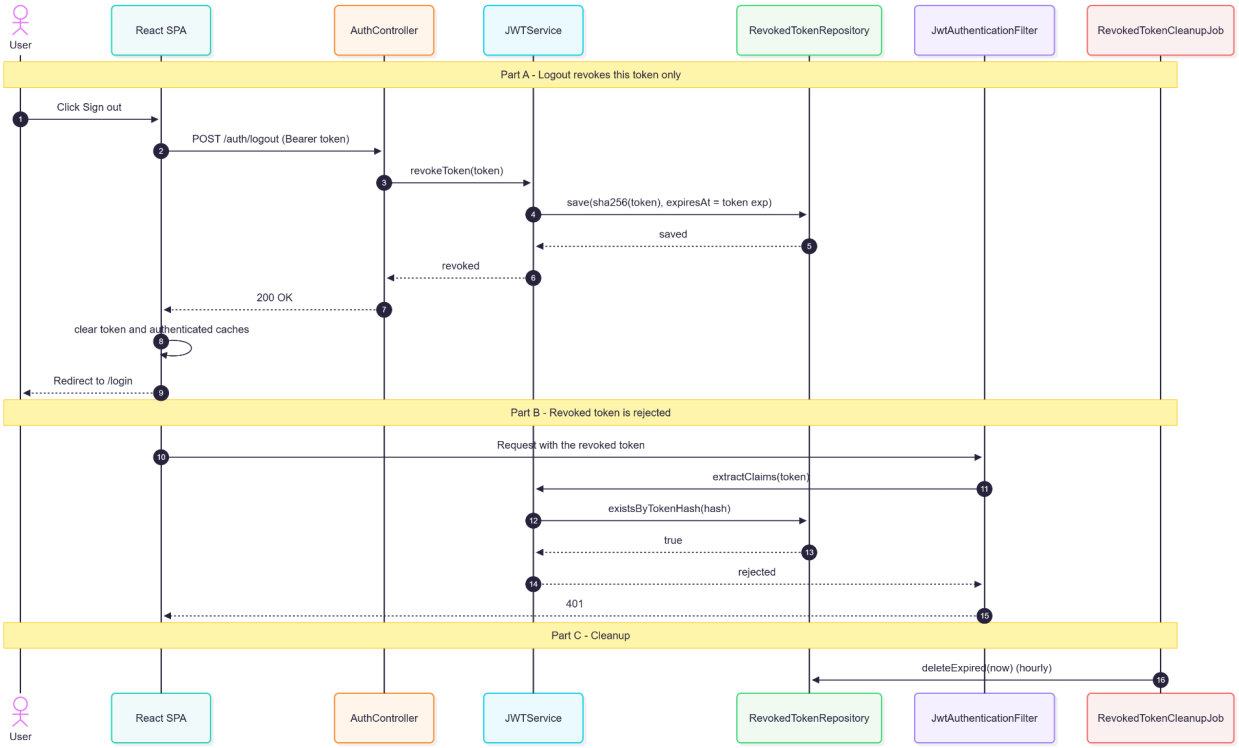
- **Class Diagram**

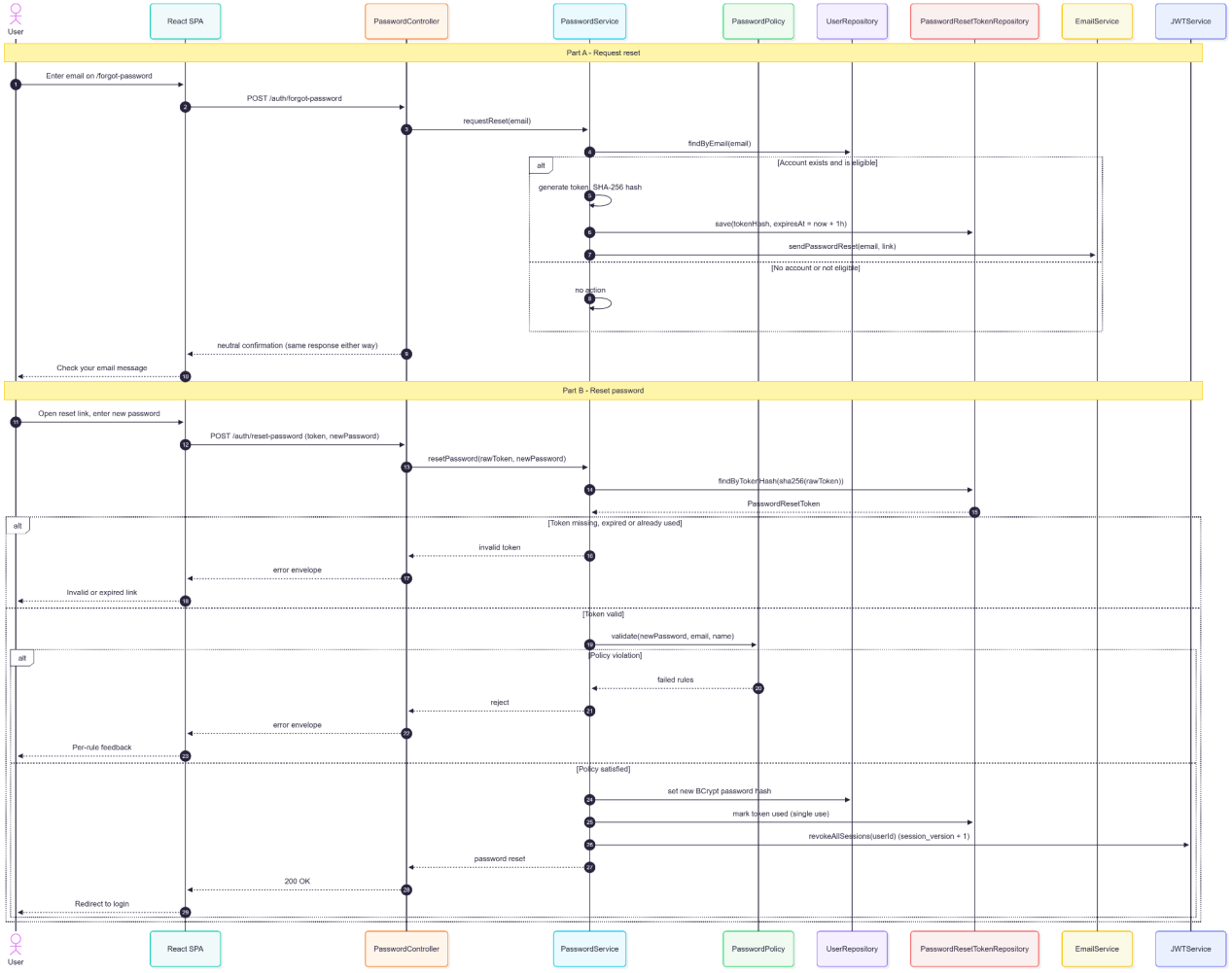


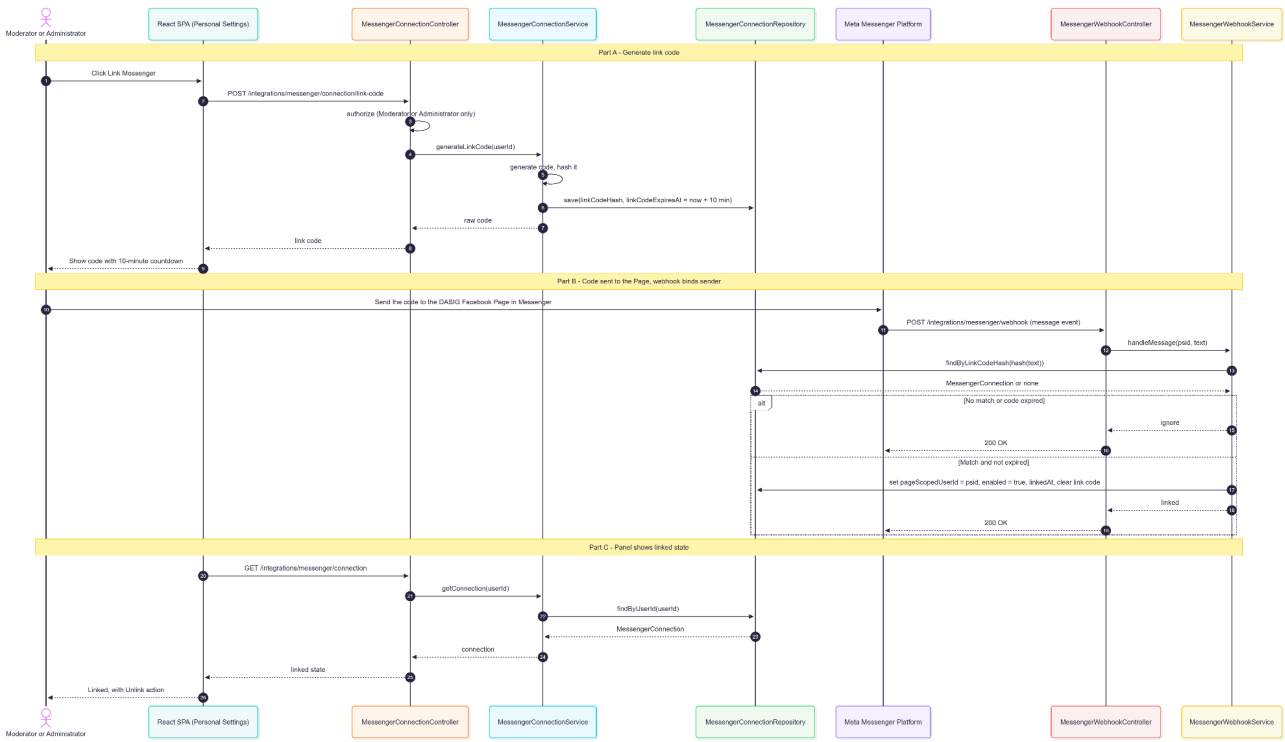
- **Sequence Diagram**


User
React SPA (LoginScreen, useAuthSession)
AuthController
UserController
UserRepository
AccountLockoutService
EmailService
JWTService
Enter email and password, click Log In
POST /auth/login
findByEmail(email)
User or none
alt
[No account found]
401 generic invalid credentials
Generic error
[Account found]
isLocked(userId)
alt
[Currently locked (15-minute lockout)]
locked, remaining duration
lockout error with remaining time
Lockout banner replaces form
[Not locked]
alt
[Account state is PENDING, EXPIRED or INACTIVE]
state-specific error
State-specific message
[Account state is ACTIVE]
verify password (BCrypt)
alt
[Password incorrect]
recordFailure(userId)
alt
[This is the 5th failed attempt]
set lockedUntil = now + 15 min
sendLockoutAlert(user)
401 generic invalid credentials
Generic error
[Password correct]
reset(userId)
generateToken(user)
HS256 JWT (userId, role, institution_id for Contributors, adminOwner, session_version, 8h expiry)
200 OK with token
setAuthToken, start session countdown
GET /me
profile and role
Redirect to role-based landing page
User
React SPA (LoginScreen, useAuthSession)
AuthController
UserController
UserRepository
AccountLockoutService
EmailService
JWTService


User
React SPA (useSessionCountdown, useAuthSession)
Axios interceptor
JwtAuthenticationFilter
JWTService
AuthController
Part A - Countdown warning
Count down from JWT exp
1
Session Warning Banner at 5 minutes remaining
2
alt
[User clicks Dismiss]
Dismiss
3
Banner hidden (session unchanged)
4
[User clicks Stay Logged In]
Stay Logged In
5
Open re-login modal
6
Part B - Session expired
alt
[Countdown reaches zero]
Open Session Expired modal
7
[Request returns session-related 401]
API call with expired or revoked JWT
8
HTTP request
9
extractClaims(token)
10
rejected (expired, revoked or old session_version)
11
401
12
trigger session-expired modal
13
Open Session Expired modal
14
Part C - Re-login
Re-enter email and password
15
POST /auth/login
16
alt
[Credentials rejected or account locked]
error
17
Error shown in modal
18
[Success]
new JWT
19
setAuthToken, reload /me
20
clear authenticated query caches
21
restart countdown
22
Modal closes, user continues where they were
23
User
React SPA (useSessionCountdown, useAuthSession)
Axios interceptor
JwtAuthenticationFilter
JWTService
AuthController

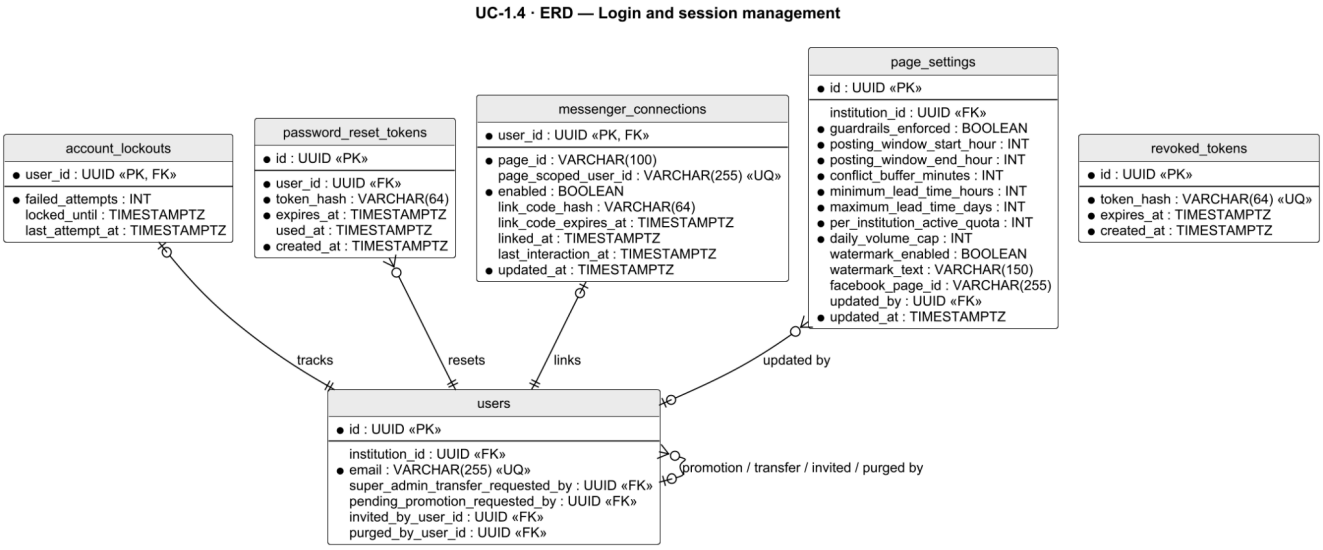

User
React SPA
AuthController
JWTService
RevokedTokenRepository
JwtAuthenticationFilter
RevokedTokenCleanupJob
Part A - Logout revokes this token only
1 Click Sign out
2 POST /auth/logout (Bearer token)
3 revokeToken(token)
4 save(sha256(token), expiresAt = token exp)
5 saved
6 revoked
7 200 OK
8 clear token and authenticated caches
9 Redirect to /login
Part B - Revoked token is rejected
10 Request with the revoked token
11 extractClaims(token)
12 existsByTokenHash(hash)
13 true
14 rejected
15 401
Part C - Cleanup
16 deleteExpired(now) (hourly)
User
React SPA
AuthController
JWTService
RevokedTokenRepository
JwtAuthenticationFilter
RevokedTokenCleanupJob


User
React SPA
PasswordController
PasswordService
PasswordPolicy
UserRepository
PasswordResetTokenRepository
EmailService
JWTService
Part A - Request reset
Enter email on /forgot-password
POST /auth/forgot-password
requestReset(email)
findByEmail(email)
[Account exists and is eligible]
alt
generate token, SHA-256 hash
save(tokenHash, expiresAt = now + 1h)
sendPasswordReset(email, link)
[No account or not eligible]
no action
neutral confirmation (same response either way)
Check your email message
Part B - Reset password
Open reset link, enter new password
POST /auth/reset-password (token, newPassword)
resetPassword(rawToken, newPassword)
findByTokenHash(sha256(rawToken))
PasswordResetToken
[Token missing, expired or already used]
alt
invalid token
error envelope
Invalid or expired link
[Token valid]
validate(newPassword, email, name)
[Policy violation]
alt
failed rules
reject
error envelope
Per-rule feedback
[Policy satisfied]
set new BCrypt password hash
mark token used (single use)
revokeAllSessions(userId) (session_version + 1)
password reset
200 OK
Redirect to login


Moderator or Administrator
React SPA (Personal Settings)
MessengerConnectionController
MessengerConnectionService
MessengerConnectionRepository
Meta Messenger Platform
MessengerWebhookController
MessengerWebhookService
Part A - Generate link code
Click Link Messenger
POST /integrations/messenger/connection/link-code
authorize (Moderator or Administrator only)
generateLinkCode(userId)
generate code, hash it
save(linkCodeHash, linkCodeExpiresAt = now + 10 min)
raw code
link code
Show code with 10-minute countdown
Part B - Code sent to the Page, webhook binds sender
Send the code to the DASIG Facebook Page in Messenger
POST /integrations/messenger/webhook (message event)
handleMessage(psid, text)
findByLinkCodeHash(hash(text))
MessengerConnection or none
alt
[No match or code expired]
ignore
200 OK
[Match and not expired]
set pageScopedUserId = psid, enabled = true, linkedAt, clear link code
linked
200 OK
Part C - Panel shows linked state
GET /integrations/messenger/connection
getConnection(userId)
findByUserId(userId)
MessengerConnection
connection
linked state
Linked, with Unlink action

1.4.5. Data Design

- **Entity Relationship Diagram**



- **Schema**

```
CREATE TABLE account_lockouts (
    user_id         UUID PRIMARY KEY REFERENCES users(id),
    failed_attempts INT NOT NULL DEFAULT 0,
    locked_until    TIMESTAMPTZ,
    last_attempt_at TIMESTAMPTZ
);

CREATE TABLE password_reset_tokens (
    id      UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id UUID NOT NULL REFERENCES users(id),
    token_hash VARCHAR(64) NOT NULL,
    expires_at TIMESTAMPTZ NOT NULL,        -- 1 hour
    used_at TIMESTAMPTZ,
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- Single-token logout revocation (DB-backed, survives restarts)
CREATE TABLE revoked_tokens (
    id      UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    token_hash VARCHAR(64) UNIQUE NOT NULL,
    expires_at TIMESTAMPTZ NOT NULL,
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- Facebook Messenger notification linking (Moderator/Admin)
CREATE TABLE messenger_connections (
    user_id                UUID PRIMARY KEY REFERENCES users(id) ON DELETE CASCADE,
    page_id                VARCHAR(100) NOT NULL,
    page_scoped_user_id  VARCHAR(255) UNIQUE,       -- Messenger PSID once linked
    enabled                BOOLEAN NOT NULL DEFAULT FALSE,
    link_code_hash         VARCHAR(64),
    link_code_expires_at TIMESTAMPTZ,               -- 10 minutes
    linked_at              TIMESTAMPTZ,
    last_interaction_at  TIMESTAMPTZ,
    updated_at             TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- Network-wide page settings (guard-rail switch and thresholds)
CREATE TABLE page_settings (
```

```
    id                      UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    institution_id          UUID REFERENCES institutions(id),   -- legacy, unused
    guardrails_enforced BOOLEAN NOT NULL DEFAULT TRUE,
    posting_window_start_hour INT NOT NULL DEFAULT 8,       -- GR-H4 window start (Asia/Manila, V116)
    posting_window_end_hour   INT NOT NULL DEFAULT 20,      -- GR-H4 window end (exclusive)
    conflict_buffer_minutes   INT NOT NULL DEFAULT 30,      -- GR-H1 (V121)
    minimum_lead_time_hours   INT NOT NULL DEFAULT 2,       -- GR-H2 (V121)
    maximum_lead_time_days  INT NOT NULL DEFAULT 30,        -- GR-H3 (V121)
    per_institution_active_quota INT NOT NULL DEFAULT 3,  -- GR-S1 (V122)
    daily_volume_cap        INT NOT NULL DEFAULT 6,         -- GR-S2 (V122)
    watermark_enabled   BOOLEAN,                            -- legacy, superseded by UC-2.5
    watermark_text          VARCHAR(150),                                -- legacy
    facebook_page_id        VARCHAR(255),                       -- legacy, unused
    updated_by              UUID REFERENCES users(id),
    updated_at              TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
```

1.5. Post Drafting & Templates

1.5.1. User Interface Design

Submissions List (`/submissions`)

A server-paginated list of the actor's submissions with filter tabs: Drafts, Action Needed, Submitted, Published, Rejected, Failed, All. Each card shows the media thumbnail, event title, status badge, and last-updated time. A "New Post" button opens the composer; an optional onboarding guide explains the list.

Composer Wizard (`/submissions/new`, `/submissions/:submissionId`)

A three-step wizard — **Media** → **Post Details** → **Organize & Schedule** — with a step indicator, Next/Back navigation (forward gated by the step's Required items; Back always enabled), and a persistent Save as Draft button. Once a draft exists, changes autosave after a short delay. The desktop layout is two columns (form + Readiness panel); at ≤900 px the Readiness panel opens as a bottom sheet from a status chip (*Required: X of 7* / *Required: 7 of 7* ✓) and the Submit/Save Draft bar is sticky, with **Submit for Approval** promoted as the primary always-visible action across all steps once all 7 required checks pass.

Step 2 — Post Details

"Posting As" institution selector (Moderators and Administrators; defaults to DASIG Central Visayas), Event Title, Event Date, and Caption (3,000-character limit, live counter). The caption action row contains **Suggest Caption**, **Templates**, **Fancy text**, and **Check writing**; an empty caption shows a "Need a head start?" starter. Hashtag pills insert into the caption.

Post Templates Panel

Opened from the Templates button (a right-hand drawer on desktop, a bottom sheet on small screens). Pinned header actions: **Generate from Top Posts** and **Save current caption**. Filters: All / Built-in / My templates. Applying a template requires at least one attached media item, replaces the caption, records the template, and clears current tags.

Readiness Checklist Panel

A live score out of 100 (Required 75%, Recommended 25%) with grade and dial; each row jumps to the relevant step/field. Required: title, date, caption, ≥1 media, file requirements, album, valid schedule (Standard). Recommended: caption length, ≥1 hashtag, ≥1 media tag, ≥1 per-media caption, template used or skipped.

Facebook Preview

A persistent toggle showing an in-page Facebook-style preview of caption, media carousel, and hashtags.

Revision Feedback

A `needs_revision` submission reopens with reviewer remarks parsed into per-field comments (pulsing fields, "mark addressed"); a `rejected` submission shows the reason with "Edit & Resubmit".

1.5.2. Front-end Component(s)

SubmissionListScreen

- `GET /submissions/page` (server pagination, bucket counts), `DELETE /submissions/{id}` (draft/rejected), `POST /submissions/{id}/withdraw`.
- React functional component / Signed-in route

SubmissionScreen

- Wizard state, readiness computation, autosave, dirty-state navigation guard; `GET /submissions/lookups` (institutions, guard-rail switch), `POST /submissions`, `PATCH /submissions/{id}`, `GET /submissions/{id}`.
- React functional component

StepProgress / SharedPrimitives / CalendarDateField / TimePickerField

- Step indicator and form primitives.

Templates Panel (`renderTemplatesPanel`) / TopPostTemplateModal

- `GET /post-templates`, `POST /post-templates`, `DELETE /post-templates/{id}`, `POST /ai/templates/from-top-posts`. Built-in templates are frontend constants.

InPageFacebookPreview / SubmissionReadOnlyView / SubmissionCardMedia

- Preview and read-only rendering.

RevisionFeedbackBanner / RevisionFeedbackModal

- Per-field reviewer remarks.

SpotlightTour

- Contextual, view-only onboarding guides (submission list, composer, Save Draft); preferences persisted via `PATCH /me/tour-preferences`.

(AI caption, Fancy Text, and Check writing components are detailed in UC-1.6; media components in UC-1.7; scheduling components in UC-1.8.)

1.5.3. Back-end Component(s)

SubmissionController

- `GET /submissions/lookups`, `GET /submissions`, `GET /submissions/page`, `POST /submissions` (create draft), `GET /submissions/{id}`, `PATCH /submissions/{id}` (update draft), `DELETE /submissions/{id}` (draft or rejected only).
- Spring `@RestController`

SubmissionService

- Draft lifecycle; server-derived `institution_id` (Contributor) or validated Posting-As institution (Moderator/Admin); caption limit `MAX_CAPTION_CHARS = 3000` (code points; 400 on create/update); submissions and templates no longer carry a category (V120); deletion removes slot reservations, purges orphaned draft uploads, and writes `SUBMISSION_DELETED`.
- Spring `@Service`

PostTemplateController / PostTemplateService

- Personal, owner-scoped templates (`GET`, `POST`, `DELETE /post-templates`).
- Spring `@RestController` / `@Service`

AiTemplateController / TopPostTemplateService

- `POST /ai/templates/from-top-posts`: ranks the Page's last 100 posts by reactions + 2×comments + 3×shares, sends the top 6 (with bottom 3 as contrast) to Claude, returns a draft-only placeholder template; refuses below 8 engaged posts; 10 requests/hour/user.
- Spring `@RestController` / `@Service`

UserController

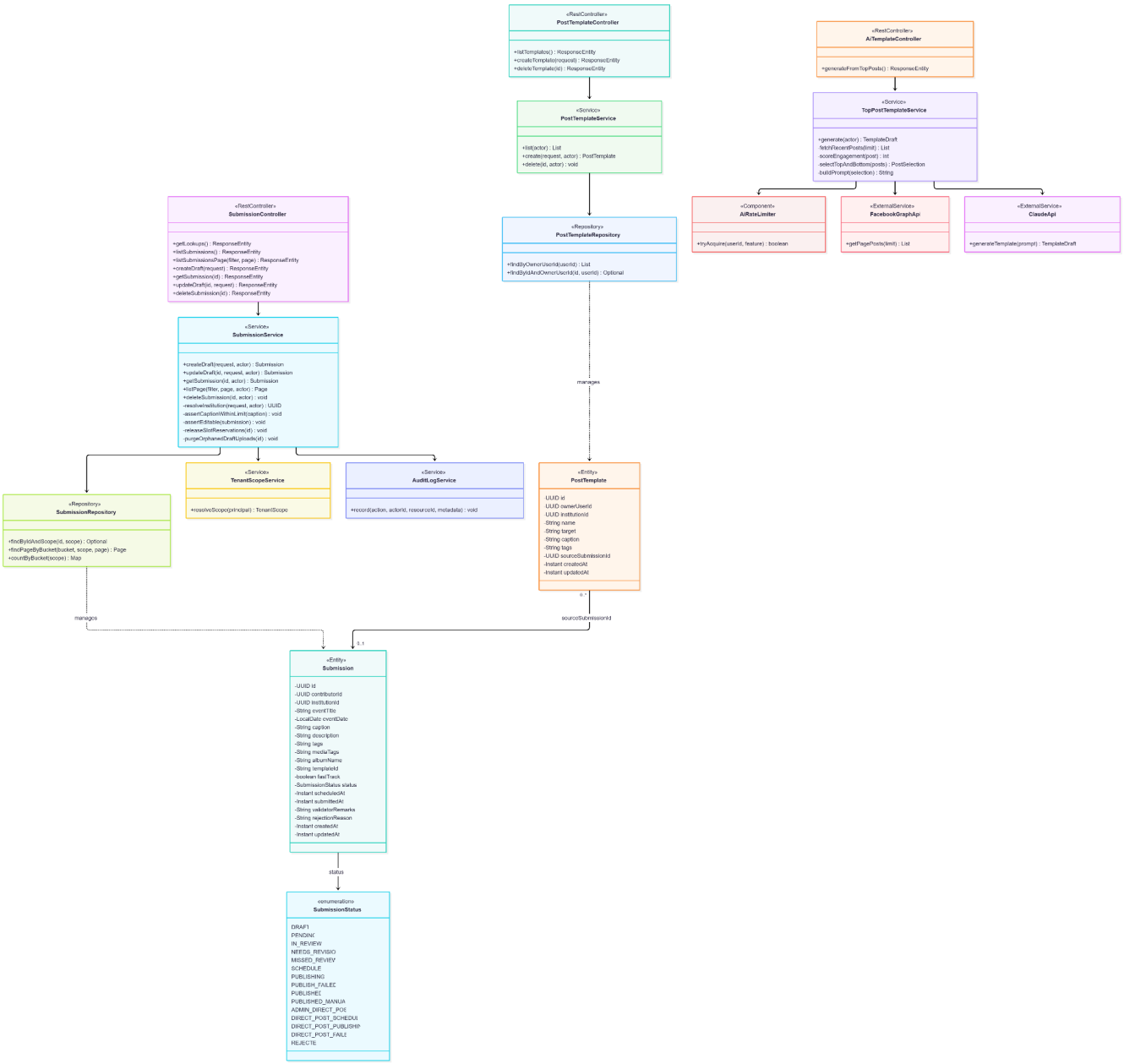
- `PATCH /me/tour-preferences` for onboarding-guide state.

SubmissionRepository / AuditLogService

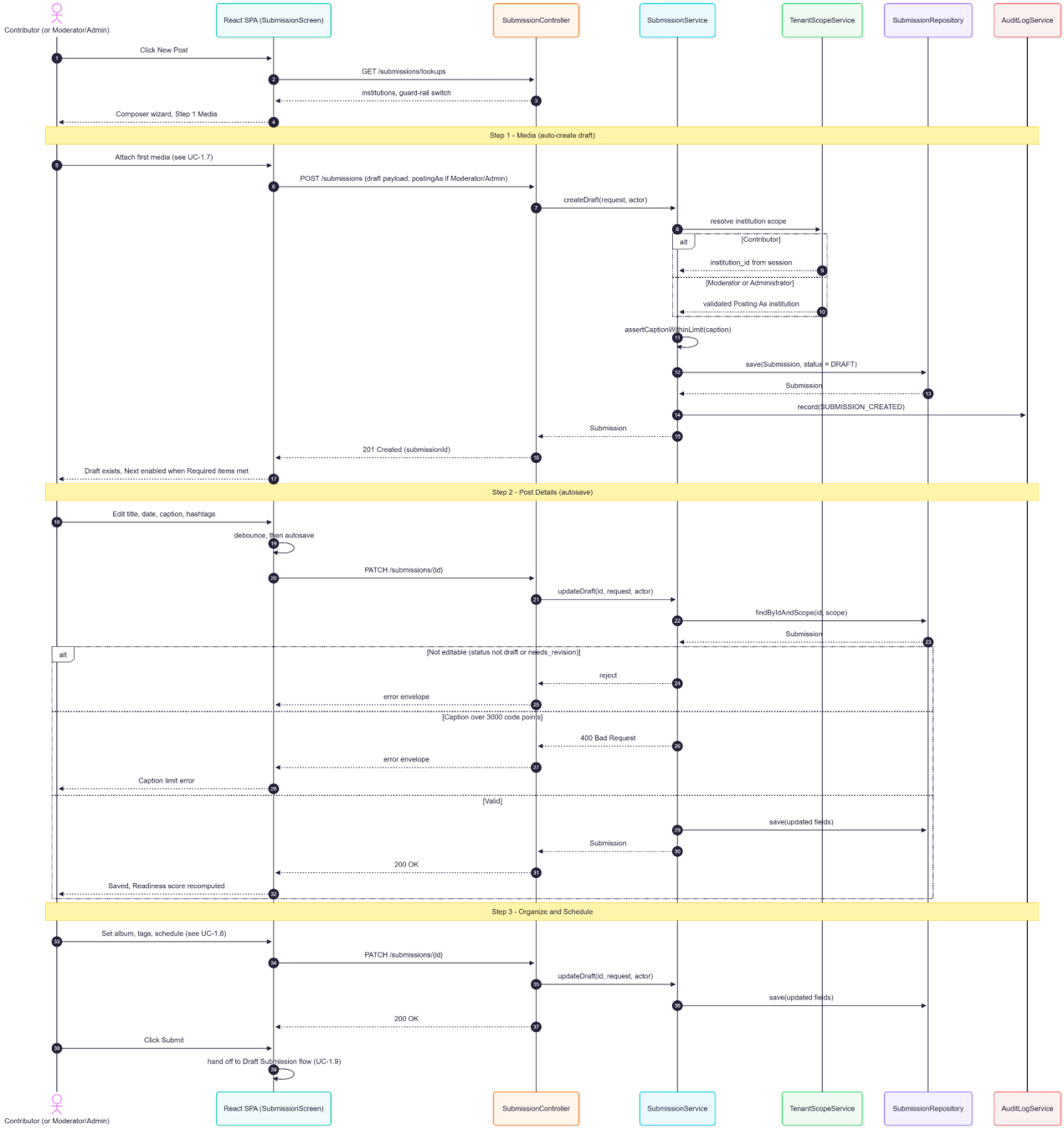
- Spring `@Repository` extending `JpaRepository<Submission, UUID>`; audit entries for creation, deletion, and withdrawal.

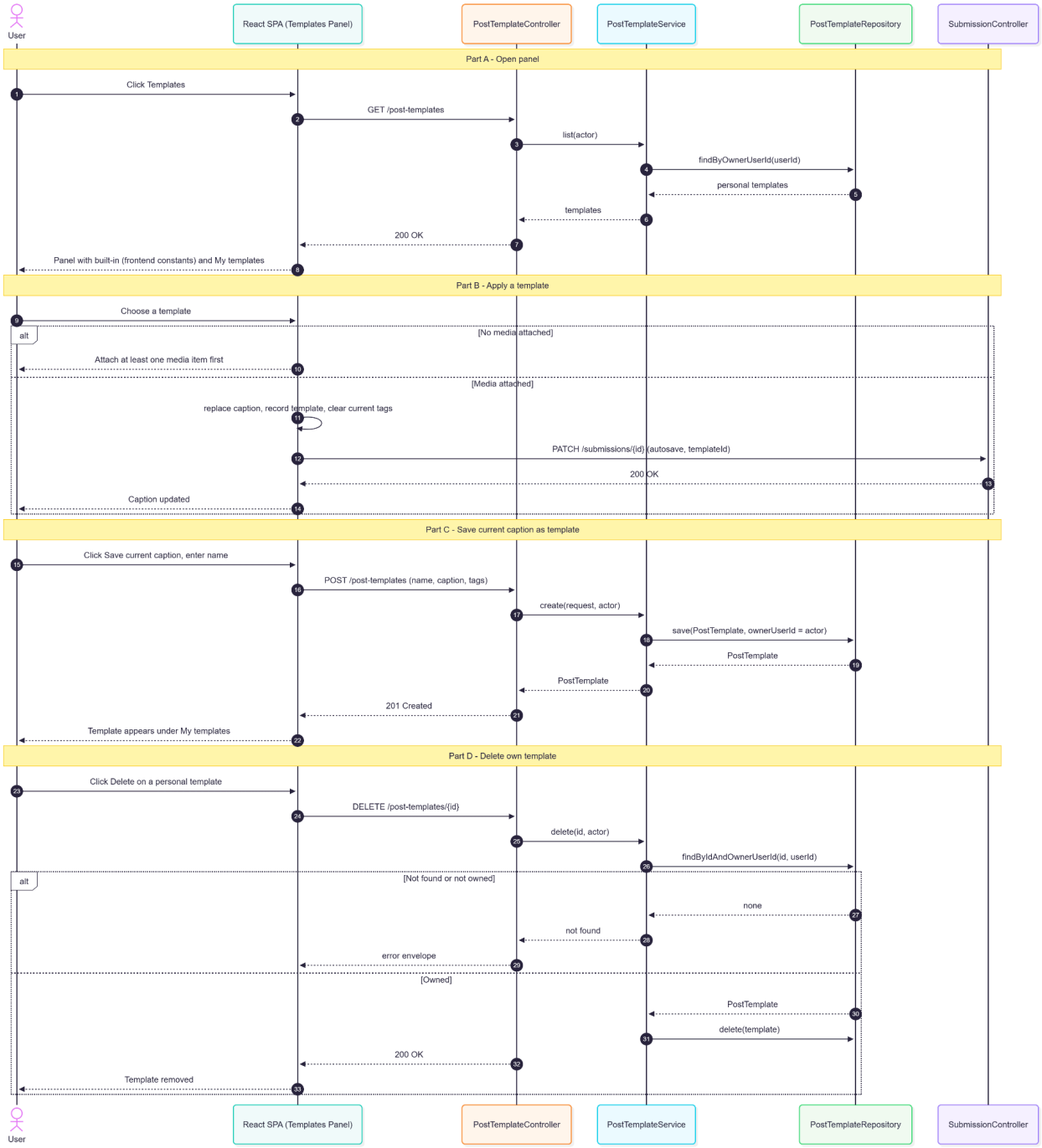
1.5.4. Object-Oriented Component(s)

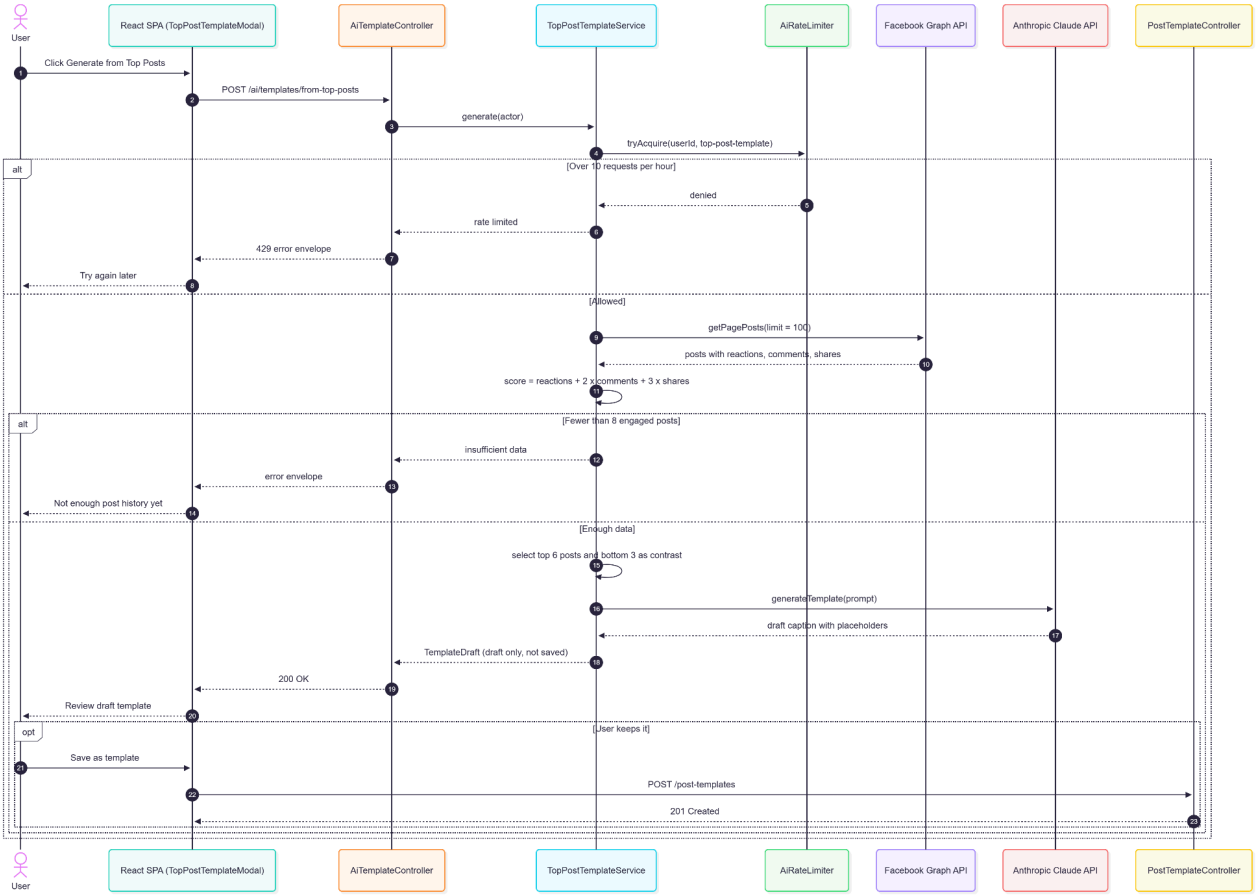
- **Class Diagram**

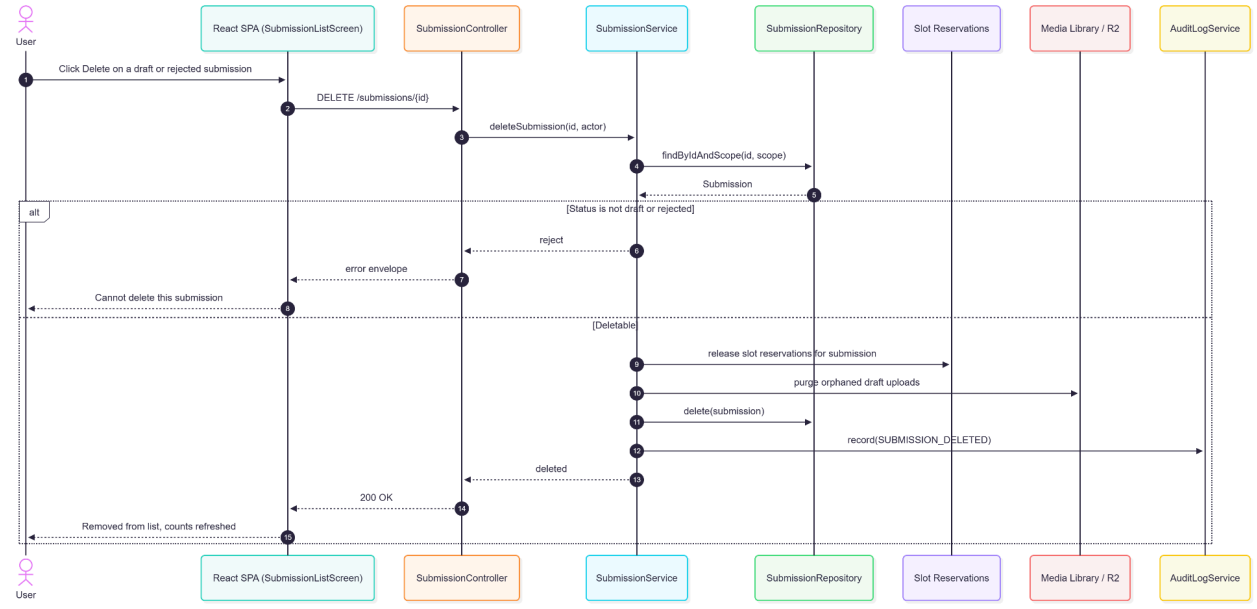


- **Sequence Diagram**


Contributor (or Moderator/Admin)
React SPA (SubmissionScreen)
SubmissionController
SubmissionService
TenantScopeService
SubmissionRepository
AuditLogService
1 Click New Post
2 GET /submissions/lookups
3 institutions, guard-rail switch
4 Composer wizard, Step 1 Media
Step 1 - Media (auto-create draft)
5 Attach first media (see UC-1.7)
6 POST /submissions (draft payload, postingAs if Moderator/Admin)
7 createDraft(request, actor)
8 resolve institution scope
alt
[Contributor]
9 institution_id from session
[Moderator or Administrator]
10 validated Posting As institution
11 assertCaptionWithinLimit(caption)
12 save(Submission, status = DRAFT)
13 Submission
14 record(SUBMISSION_CREATED)
15 Submission
16 201 Created (submissionId)
17 Draft exists, Next enabled when Required items met
Step 2 - Post Details (autosave)
18 Edit title, date, caption, hashtags
19 debounce, then autosave
20 PATCH /submissions/{id}
21 updateDraft(id, request, actor)
22 findByIdAndScope(id, scope)
23 Submission
alt
[Not editable (status not draft or needs_revision)]
24 reject
25 error envelope
[Caption over 3000 code points]
26 400 Bad Request
27 error envelope
28 Caption limit error
[Valid]
29 save(updated fields)
30 Submission
31 200 OK
32 Saved, Readiness score recomputed
Step 3 - Organize and Schedule
33 Set album, tags, schedule (see UC-1.8)
34 PATCH /submissions/{id}
35 updateDraft(id, request, actor)
36 save(updated fields)
37 200 OK
38 Click Submit
39 hand off to Draft Submission flow (UC-1.9)

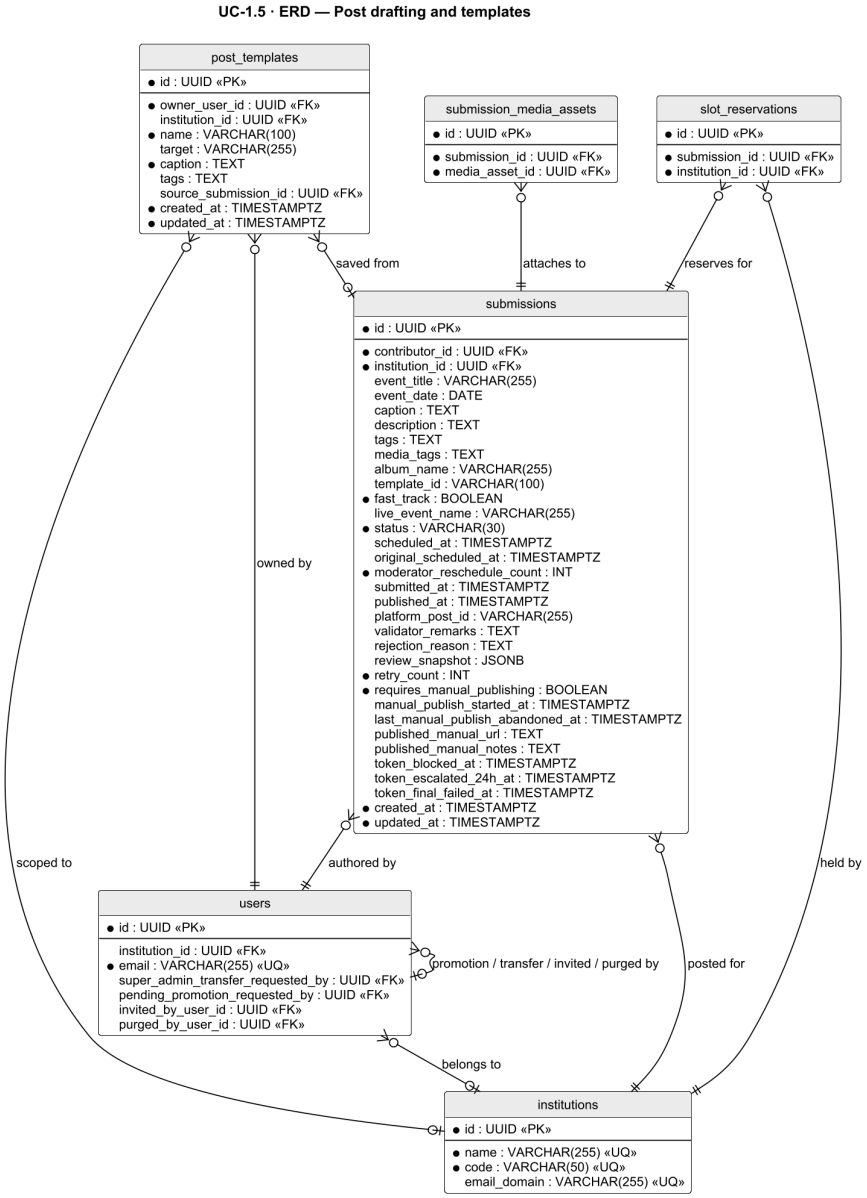

User
React SPA (Templates Panel)
PostTemplateController
PostTemplateService
PostTemplateRepository
SubmissionController
Part A - Open panel
1 Click Templates
2 GET /post-templates
3 list(actor)
4 findByOwnerUserId(userId)
5 personal templates
6 templates
7 200 OK
8 Panel with built-in (frontend constants) and My templates
Part B - Apply a template
9 Choose a template
alt
[No media attached]
10 Attach at least one media item first
[Media attached]
11 replace caption, record template, clear current tags
12 PATCH /submissions/{id} (autosave, templateId)
13 200 OK
14 Caption updated
Part C - Save current caption as template
15 Click Save current caption, enter name
16 POST /post-templates (name, caption, tags)
17 create(request, actor)
18 save(PostTemplate, ownerUserId = actor)
19 PostTemplate
20 PostTemplate
21 201 Created
22 Template appears under My templates
Part D - Delete own template
23 Click Delete on a personal template
24 DELETE /post-templates/{id}
25 delete(id, actor)
26 findByIdAndOwnerUserId(id, userId)
alt
[Not found or not owned]
27 none
28 not found
29 error envelope
[Owned]
30 PostTemplate
31 delete(template)
32 200 OK
33 Template removed
User
React SPA (Templates Panel)
PostTemplateController
PostTemplateService
PostTemplateRepository
SubmissionController


User
React SPA (TopPostTemplateModal)
AiTemplateController
TopPostTemplateService
AiRateLimiter
Facebook Graph API
Anthropic Claude API
PostTemplateController
Click Generate from Top Posts
POST /ai/templates/from-top-posts
generate(actor)
tryAcquire(userId, top-post-template)
alt
[Over 10 requests per hour]
denied
rate limited
429 error envelope
Try again later
[Allowed]
getPagePosts(limit = 100)
posts with reactions, comments, shares
score = reactions + 2 x comments + 3 x shares
alt
[Fewer than 8 engaged posts]
insufficient data
error envelope
Not enough post history yet
[Enough data]
select top 6 posts and bottom 3 as contrast
generateTemplate(prompt)
draft caption with placeholders
TemplateDraft (draft only, not saved)
200 OK
Review draft template
opt
[User keeps it]
Save as template
POST /post-templates
201 Created


User
React SPA (SubmissionListScreen)
SubmissionController
SubmissionService
SubmissionRepository
Slot Reservations
Media Library / R2
AuditLogService
Click Delete on a draft or rejected submission
DELETE /submissions/{id}
deleteSubmission(id, actor)
findByIdAndScope(id, scope)
Submission
alt
[Status is not draft or rejected]
reject
error envelope
Cannot delete this submission
[Deletable]
release slot reservations for submission
purge orphaned draft uploads
delete(submission)
record(SUBMISSION_DELETED)
deleted
200 OK
Removed from list, counts refreshed

1.5.5. Data Design

- **Entity Relationship Diagram**



- **Schema**

```
CREATE TABLE submissions (
    id                          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    contributor_id              UUID NOT NULL REFERENCES users(id),
    institution_id              UUID NOT NULL REFERENCES institutions(id),
    event_title                 VARCHAR(255),
    event_date                  DATE,
    caption                     TEXT,                           -- ≤ 3000 code points
    description                 TEXT,
    tags                        TEXT,                           -- hashtags
    media_tags                  TEXT,
    album_name                  VARCHAR(255),
    template_id                 VARCHAR(100),
    fast_track                  BOOLEAN NOT NULL DEFAULT FALSE,
    live_event_name             VARCHAR(255),
    status                      VARCHAR(30) NOT NULL DEFAULT 'draft'
```

```
                                    CHECK (status IN ('draft', 'pending', 'in_review', 'needs_revision',
                                           'missed_review', 'scheduled', 'publishing', 'publish_failed',
                                           'published', 'published_manual', 'admin_direct_post',
                                           'direct_post_scheduled', 'direct_post_publishing',
                                           'direct_post_failed', 'rejected')),
    scheduled_at                    TIMESTAMPTZ,
    original_scheduled_at            TIMESTAMPTZ,                   -- snapshot at approval (UC-3.1 A1)
    moderator_reschedule_count       INT NOT NULL DEFAULT 0,
    submitted_at                    TIMESTAMPTZ,
    published_at                    TIMESTAMPTZ,
    platform_post_id                VARCHAR(255),
    validator_remarks               TEXT,                          -- revision remarks
    rejection_reason                TEXT,
    review_snapshot                 JSONB,                         -- original values for Restore
original (UC-2.4)
    retry_count                     INT NOT NULL DEFAULT 0,
    requires_manual_publishing       BOOLEAN NOT NULL DEFAULT FALSE,
    manual_publish_started_at        TIMESTAMPTZ,
    last_manual_publish_abandoned_at TIMESTAMPTZ,
    published_manual_url             TEXT,
    published_manual_notes           TEXT,
    token_blocked_at                TIMESTAMPTZ,
    token_escalated_24h_at           TIMESTAMPTZ,
    token_final_failed_at            TIMESTAMPTZ,
    created_at                      TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at                      TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- Personal post templates (built-in templates are frontend constants)
CREATE TABLE post_templates (
    id                      UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    owner_user_id           UUID NOT NULL REFERENCES users(id),
    institution_id          UUID REFERENCES institutions(id),
    name                    VARCHAR(100) NOT NULL,
    target                  VARCHAR(255),
    caption                 TEXT NOT NULL,
    tags                    TEXT,
    source_submission_id UUID REFERENCES submissions(id),
    created_at              TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at              TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
```

1.6. AI & Text Tools Integration

1.6.1. User Interface Design

Suggest Caption (caption action row)

Opens a prompt dialog with an optional instruction field (280-character limit; Generate disabled past it) and a tone selector — Professional (default), Community, Energetic. Generation proceeds when there is an attached image, an existing caption, or a typed prompt. A loading state is shown during the request.

Caption Suggestion

Displays **one** caption in the selected tone with Insert, Regenerate, Edit, and Dismiss actions. Error states: timeout (30 s, retry available), AI unavailable (auto-recovers after ~5 s), and rate-limited (shows reset time, 30 requests/hour).

Fancy Text Panel

A button-opened panel (not triggered by text selection) previewing the selected caption text in Bold Serif, Italic Serif, Bold Sans-Serif, Italic Sans-Serif, and Script, plus Plain (revert). Characters without a Unicode equivalent pass through unchanged.

Check Writing

Disabled when the caption is empty. Returns advisory spelling, grammar, clarity, and meaning-change suggestions, each with Apply and Dismiss. Never blocks editing, saving, or submission. Also available in the reviewer edit form (UC-2.4), where it runs automatically after a caption edit and reports only problems the edit introduced.

1.6.2. Front-end Component(s)

AiCaptionButton / AiCaptionPromptDialog / AiCaptionSuggestion / useAiCaptionAssist

- Submits `POST /ai/caption` (auto-saving the draft first if needed); logs outcomes best-effort to `POST /ai/caption/log` (`use`, `use_then_edited`, `dismiss`, `re_generate`, tone).
- React functional components / hook

FancyTextTool

- Pure client-side Unicode mapping; respects `CAPTION_CHAR_LIMIT = 3000`.
- React functional component

Check Writing panel

- `POST /ai/proofread`; applied suggestions reported to `POST /ai/proofread/applied` for adoption analytics.
- React functional component

1.6.3. Back-end Component(s)

CaptionController

- `POST /ai/caption`, `POST /ai/caption/log`. 30 requests per rolling hour per user (in-memory limiter → `429` with reset headers); 280-character prompt cap (`400`).
- Spring `@RestController`

CaptionGenerationService

- Builds the prompt from up to 4 images, media metadata, event title/date/institution, existing caption, instruction, and tone; detects instruction-vs-draft-refine intent; parses a requested word count (largest number, `MAX_REQUESTED_CAPTION_WORDS = 2000`) and retries up to 3 times to satisfy a requested range. Submission category is not part of the prompt.
- Spring `@Service`

ClaudeVisionClient

- Java `HttpClient` wrapper for the Anthropic Messages API (`claude-haiku-4-5`); base64-encodes images, downscales images > 5 MB in memory (70% → 50% → 35% → 25% → 15%), 30-second timeout (`504`), service errors (`503`). Legacy read fallback for old images on the private Supabase bucket via `DASIG_SUPABASE_SERVICE_ROLE_KEY`.

- Spring `@Component`

ProofreadController / ProofreadService

- `POST /ai/proofread` (30/hour/user, suggestions only), `POST /ai/proofread/applied`.
- Spring `@RestController` / `@Service`

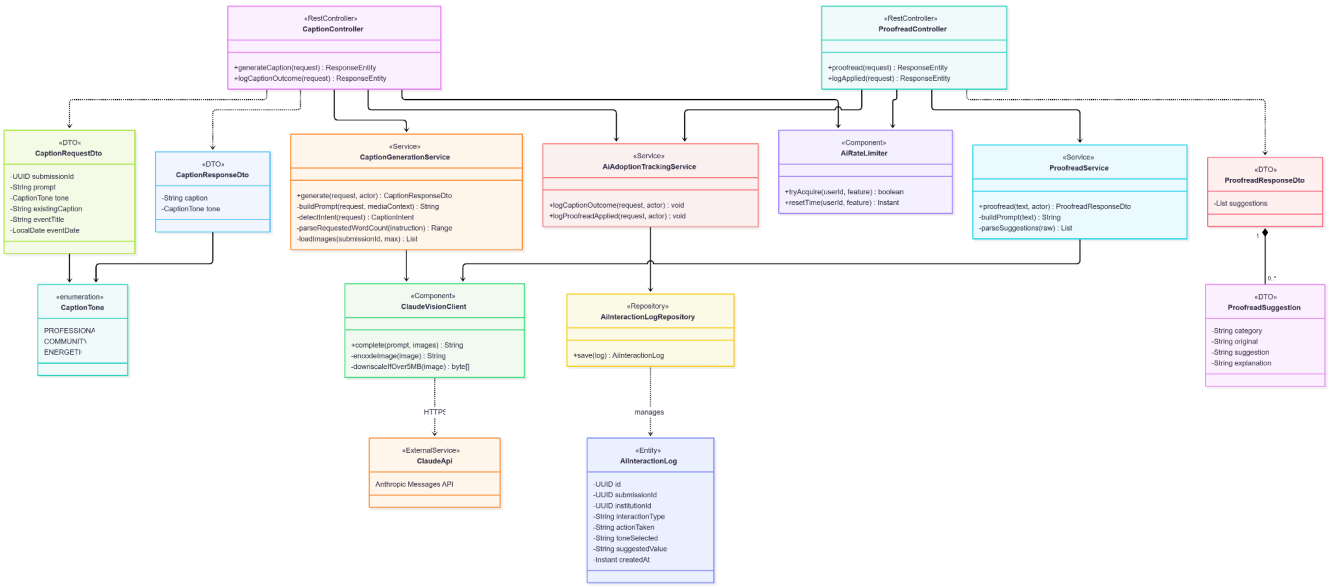
AiAdoptionTrackingService / AiInteractionLogRepository

- Persists caption and proofreading interaction events to `ai_interaction_log` for AI Feature Adoption analytics (UC-3.4).
- Spring `@Service` / `@Repository`

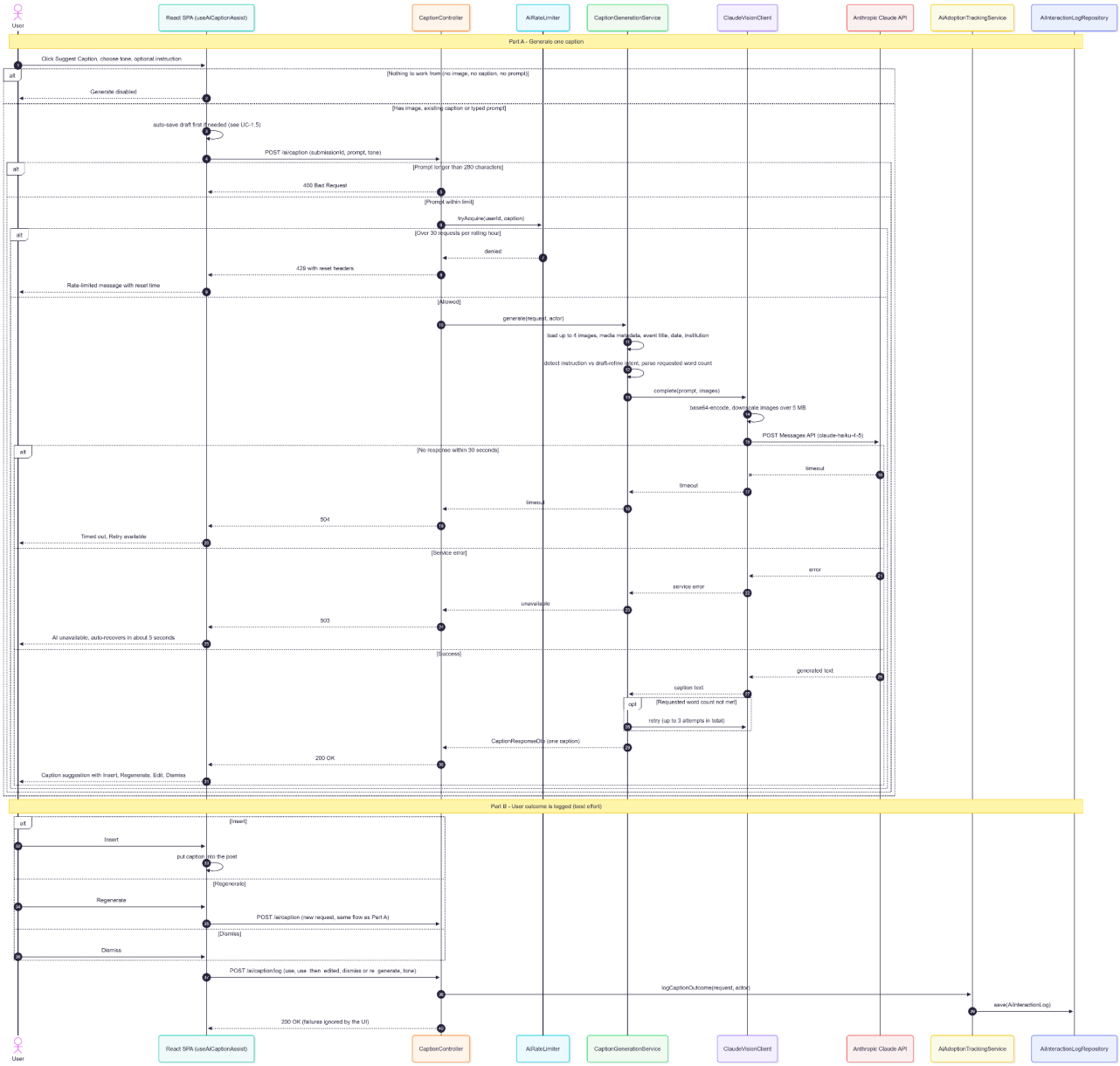
(Fancy Text is a client-side transformation — no back-end component.)

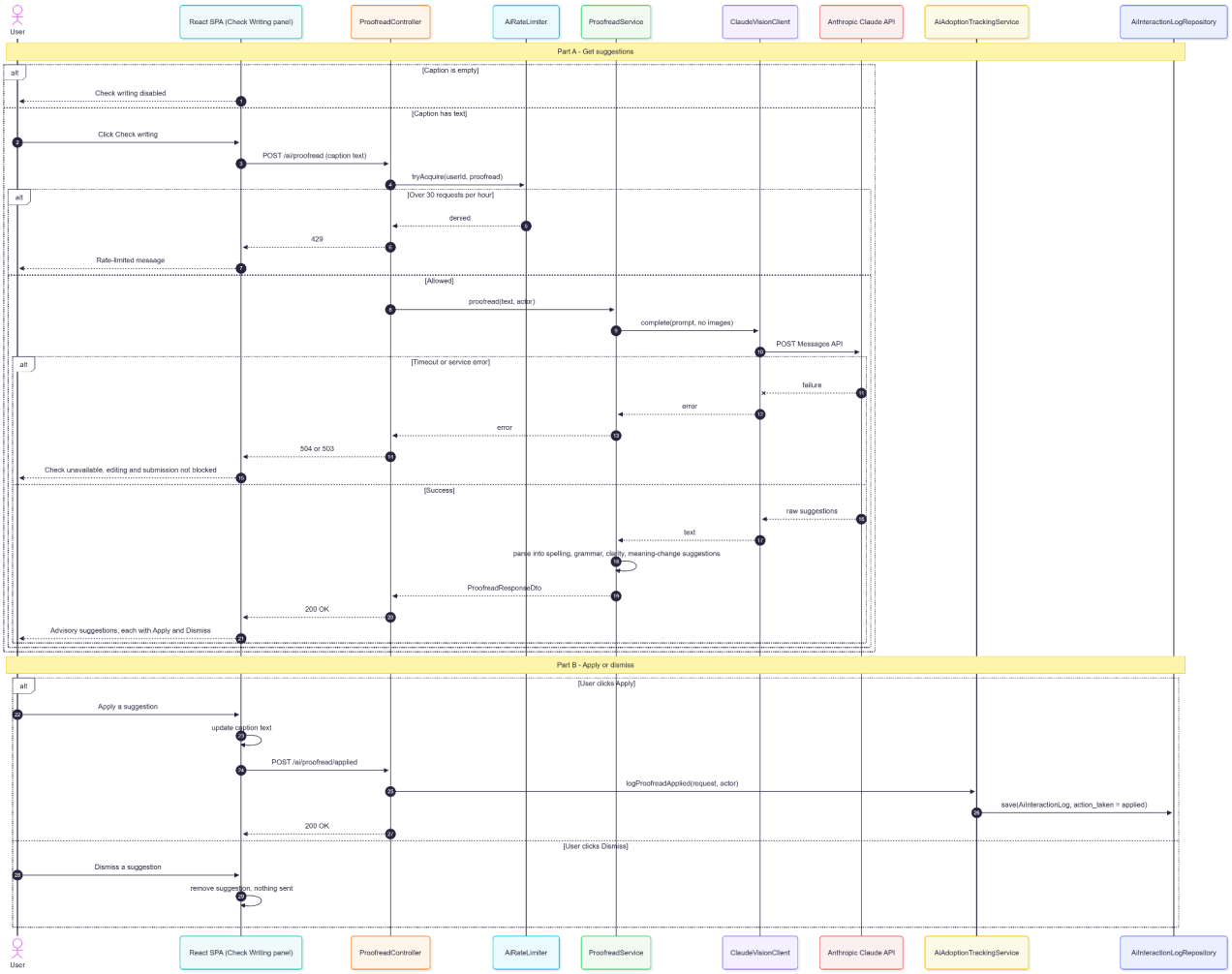
1.6.4. Object-Oriented Component(s)

- **Class Diagram**



- **Sequence Diagram**

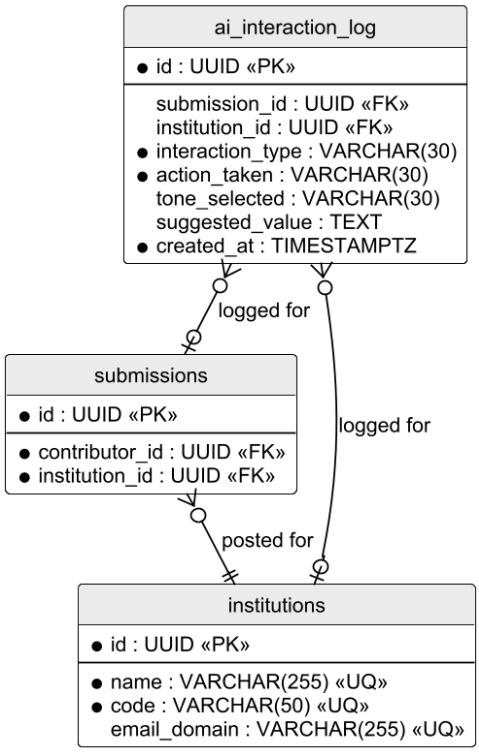

User
React SPA (useAiCaptionAssist)
CaptionController
AiRateLimiter
CaptionGenerationService
ClaudeVisionClient
Anthropic Claude API
AiAdoptionTrackingService
AiInteractionLogRepository
Part A - Generate one caption
Click Suggest Caption, choose tone, optional instruction
alt
[Nothing to work from (no image, no caption, no prompt)]
Generate disabled
[Has image, existing caption or typed prompt]
auto-save draft first if needed (see UC-1.5)
POST /ai/caption (submissionId, prompt, tone)
alt
[Prompt longer than 280 characters]
400 Bad Request
[Prompt within limit]
tryAcquire(userId, caption)
alt
[Over 30 requests per rolling hour]
denied
429 with reset headers
Rate-limited message with reset time
[Allowed]
generate(request, actor)
load up to 4 images, media metadata, event title, date, institution
detect instruction vs draft-refine intent, parse requested word count
complete(prompt, images)
base64-encode, downscale images over 5 MB
POST Messages API (claude-haiku-4-5)
alt
[No response within 30 seconds]
timeout
timeout
timeout
504
Timed out, Retry available
[Service error]
error
service error
unavailable
503
AI unavailable, auto-recovers in about 5 seconds
[Success]
generated text
caption text
opt
[Requested word count not met]
retry (up to 3 attempts in total)
CaptionResponseDto (one caption)
200 OK
Caption suggestion with Insert, Regenerate, Edit, Dismiss
Part B - User outcome is logged (best effort)
alt
[Insert]
Insert
put caption into the post
[Regenerate]
Regenerate
POST /ai/caption (new request, same flow as Part A)
[Dismiss]
Dismiss
POST /ai/caption/log (use, use_then_edited, dismiss or re_generate, tone)
logCaptionOutcome(request, actor)
save(AiInteractionLog)
200 OK (failures ignored by the UI)


User
React SPA (Check Writing panel)
ProofreadController
AiRateLimiter
ProofreadService
ClaudeVisionClient
Anthropic Claude API
AiAdoptionTrackingService
AiInteractionLogRepository
Part A - Get suggestions
alt
[Caption is empty]
Check writing disabled
[Caption has text]
Click Check writing
POST /ai/proofread (caption text)
tryAcquire(userId, proofread)
alt
[Over 30 requests per hour]
denied
429
Rate-limited message
[Allowed]
proofread(text, actor)
complete(prompt, no images)
POST Messages API
alt
[Timeout or service error]
failure
error
error
504 or 503
Check unavailable, editing and submission not blocked
[Success]
raw suggestions
text
parse into spelling, grammar, clarity, meaning-change suggestions
ProofreadResponseDto
200 OK
Advisory suggestions, each with Apply and Dismiss
Part B - Apply or dismiss
alt
[User clicks Apply]
Apply a suggestion
update caption text
POST /ai/proofread/applied
logProofreadApplied(request, actor)
save(AiInteractionLog, action_taken = applied)
200 OK
[User clicks Dismiss]
Dismiss a suggestion
remove suggestion, nothing sent

1.6.5. Data Design

- **Entity Relationship Diagram**

UC-1.6 · ERD — AI and text tools integration



- **Schema**

```
CREATE TABLE ai_interaction_log (
    id               UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    submission_id    UUID REFERENCES submissions(id),
    institution_id   UUID REFERENCES institutions(id),
    interaction_type VARCHAR(30) NOT NULL,   -- caption_suggestion | proofread | media_recommendation |
...
    action_taken     VARCHAR(30) NOT NULL,   -- shown | use | use_then_edited | dismiss | re_generate |
applied | ...
    tone_selected    VARCHAR(30),
    suggested_value  TEXT,
    created_at       TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
```

1.7. Media Attachment Rules

1.7.1. User Interface Design

Step 1 — Media

Device upload (drag-and-drop or picker) and a Media Library tab. Upload cards show thumbnail, progress, and status, with Retry on network failure and a remove action. Media order can be rearranged by drag.

Media Library Tab (composer picker)

Reuses the library browser (UC-2.2) with search and, for Moderators/Administrators, an institution filter defaulting to the submission's institution. Each result has an Attach action.

AI Suggestions Tab

Enabled once event title + caption + media tags reach ≥10 characters. Shows up to 8 ranked suggestions (score ≥ 0.40) with match reasons and blue AI accents; hidden when nothing clears the floor.

Media Caption Modal

Per-asset caption (≤500 characters, live counter) and, for photos, a "Skip watermark for this image" checkbox. A caption indicator appears on the card.

Album Field (Organize & Schedule step)

Select an existing album, type a new name, or click **Auto-Match**: a confident match (≥0.55) auto-fills with a reason badge; an ambiguous result (0.32–0.55) lists up to 3 ranked candidates; otherwise a "no confident match" notice is shown. Media tags are entered alongside.

Mixed-Media Notice

An inline notice when images and video are mixed, explaining that automated publishing will route the post to the Failed tab for manual recovery.

1.7.2. Front-end Component(s)

SubmissionScreen media step

- Requests `POST /submissions/{id}/media/upload-url` (presigned R2 PUT), uploads bytes directly to R2, then registers metadata with `POST /submissions/{id}/media`. Library picks use `POST /submissions/{id}/assets`; removal `DELETE /submissions/{id}/assets/{assetId}`; reorder `PATCH /submissions/{id}/media/order`. Client-side validation of 7 formats and 50 MB.
- React functional component

SimilarMediaPanel (AI Suggestions)

- `POST /ai/submissions/{id}/suggest-media`, `POST /ai/submissions/{id}/suggest-media-status`, `GET /ai/submissions/{id}/similar-media`; logs via `POST /ai/submissions/{id}/log-interaction`.
- React functional component

AlbumCombobox

- Album select/create with `POST /ai/submissions/{id}/suggest-album` and ranked `suggestions`. Shared with UC-2.1.
- React functional component

Media Caption Modal

- Saves per-asset caption and `skipWatermark` with the draft.
- React modal component

1.7.3. Back-end Component(s)

SubmissionController (media endpoints)

- `POST /submissions/{id}/media/upload-url`, `POST /submissions/{id}/media`, `PATCH /submissions/{id}/media/order`, `POST /submissions/{id}/assets`, `DELETE /submissions/{id}/assets/{assetId}`. Enforces the 10-asset limit (BR-SUB-02).
- Spring `@RestController`

MediaStorageService

- AWS S3 SDK against R2: presigned PUT URLs, server-side validation of the declared MIME type, extension, and size (file content is not inspected for magic bytes), object download, delete, and public URL resolution via the backend proxy (`{BACKEND_PUBLIC_BASE_URL}/api/v1/media-files/`), or a custom R2 domain when R2_CUSTOM_DOMAIN_URL is set.
- Spring `@Service`

AIRecommendationController / AIRecommendationService

- `suggestMedia`: Voyage query embedding over event context → pgvector search in the submission institution's library → re-rank by semantic, visual, recency, category, and tag signals → keep score ≥ 0.40, top 8; metadata/tag fallback when embedding retrieval fails. `suggestAlbum`: blends tag overlap with closest-asset visual similarity per root album (max score, not centroid); thresholds 0.55 / 0.32.
- Spring `@RestController` / `@Service`

MediaRetrievalEmbeddingService / SubmissionMediaContextService / MediaProcessingQueueService

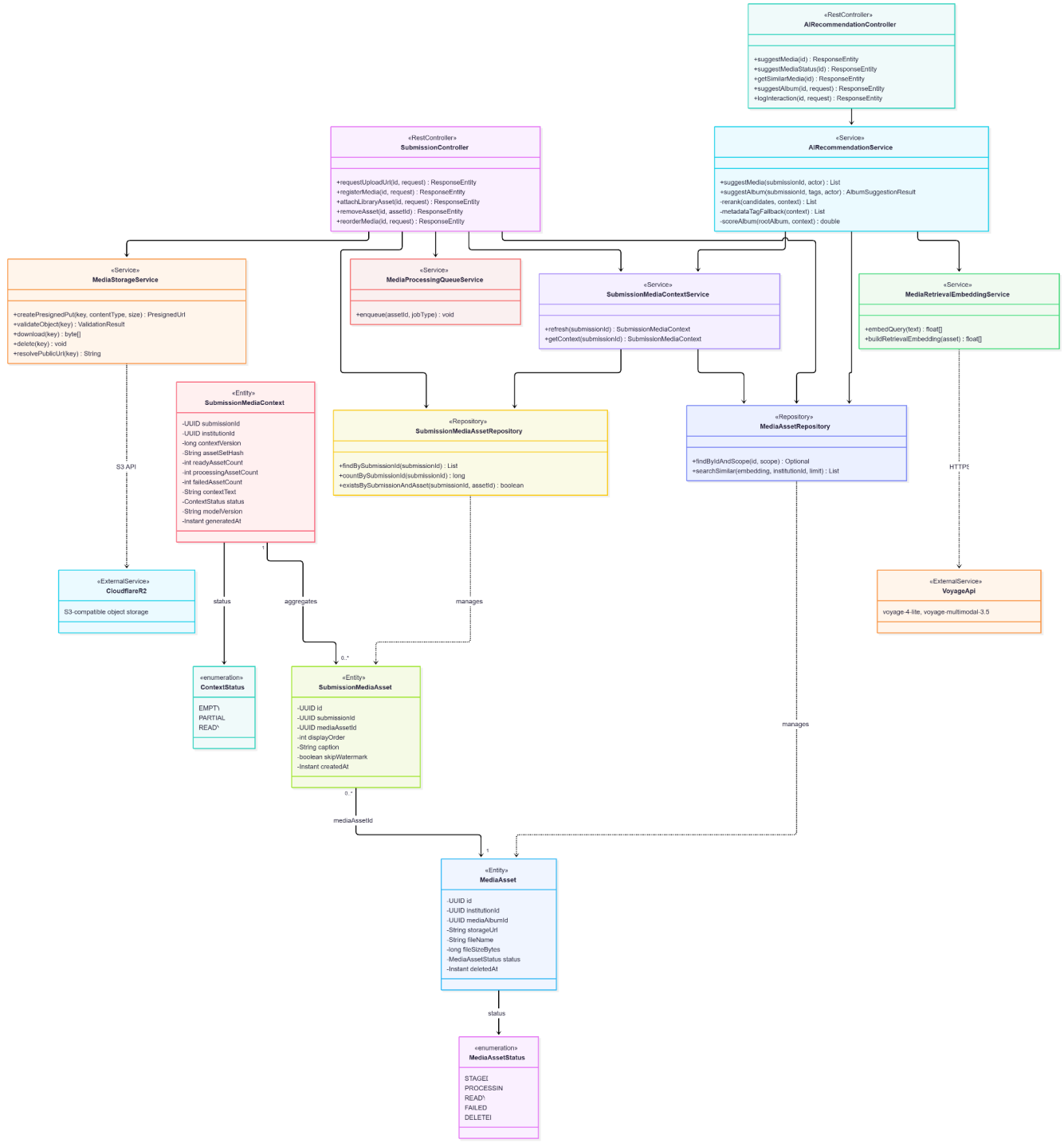
- Builds retrieval embeddings and the aggregated `submission_media_contexts` used for suggestions; queues processing jobs for new uploads.
- Spring `@Service`

SubmissionMediaAssetRepository

- Junction persistence with `display_order`, `caption`, `skip_watermark`.
- Spring `@Repository`

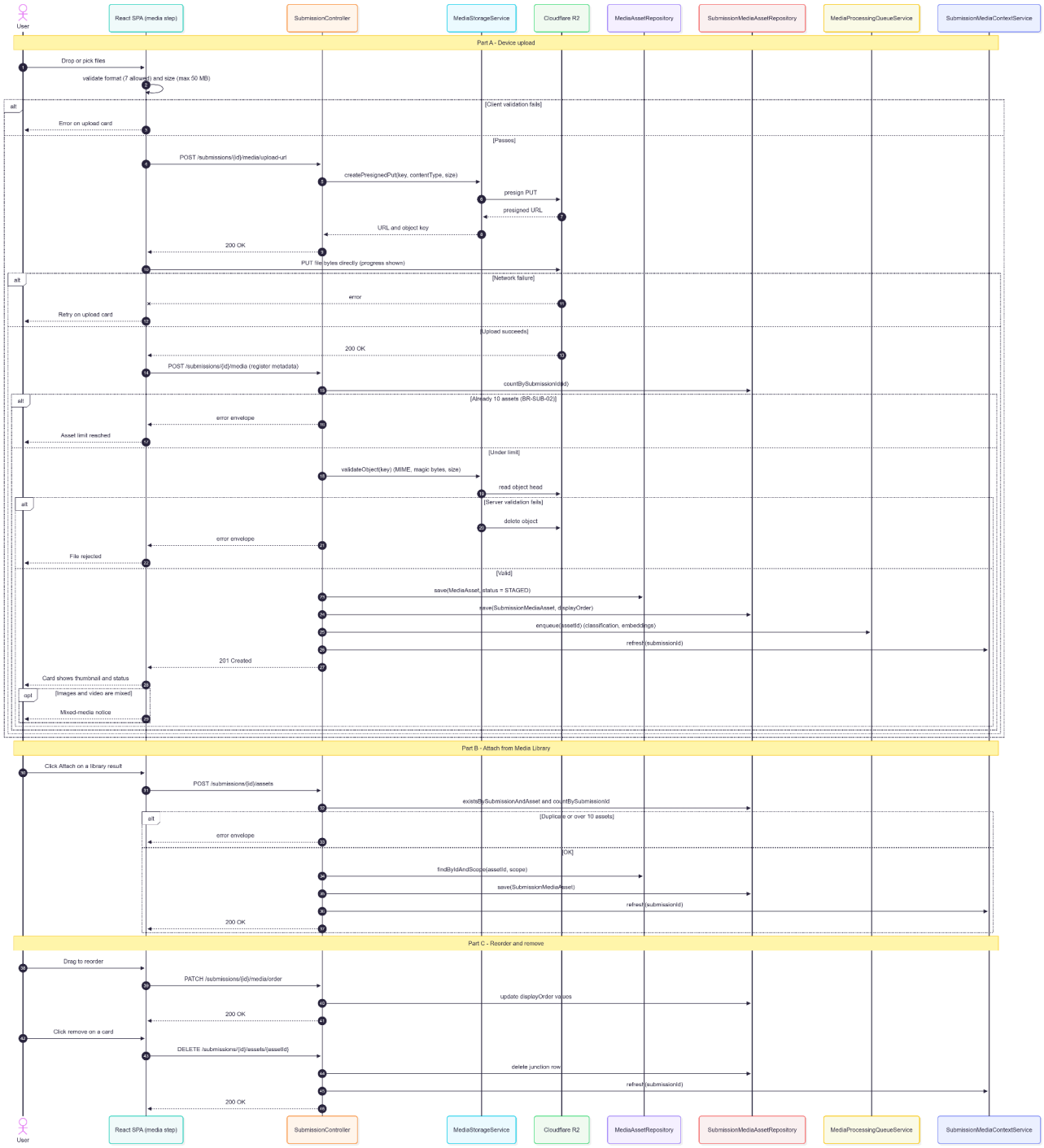
1.7.4. Object-Oriented Component(s)

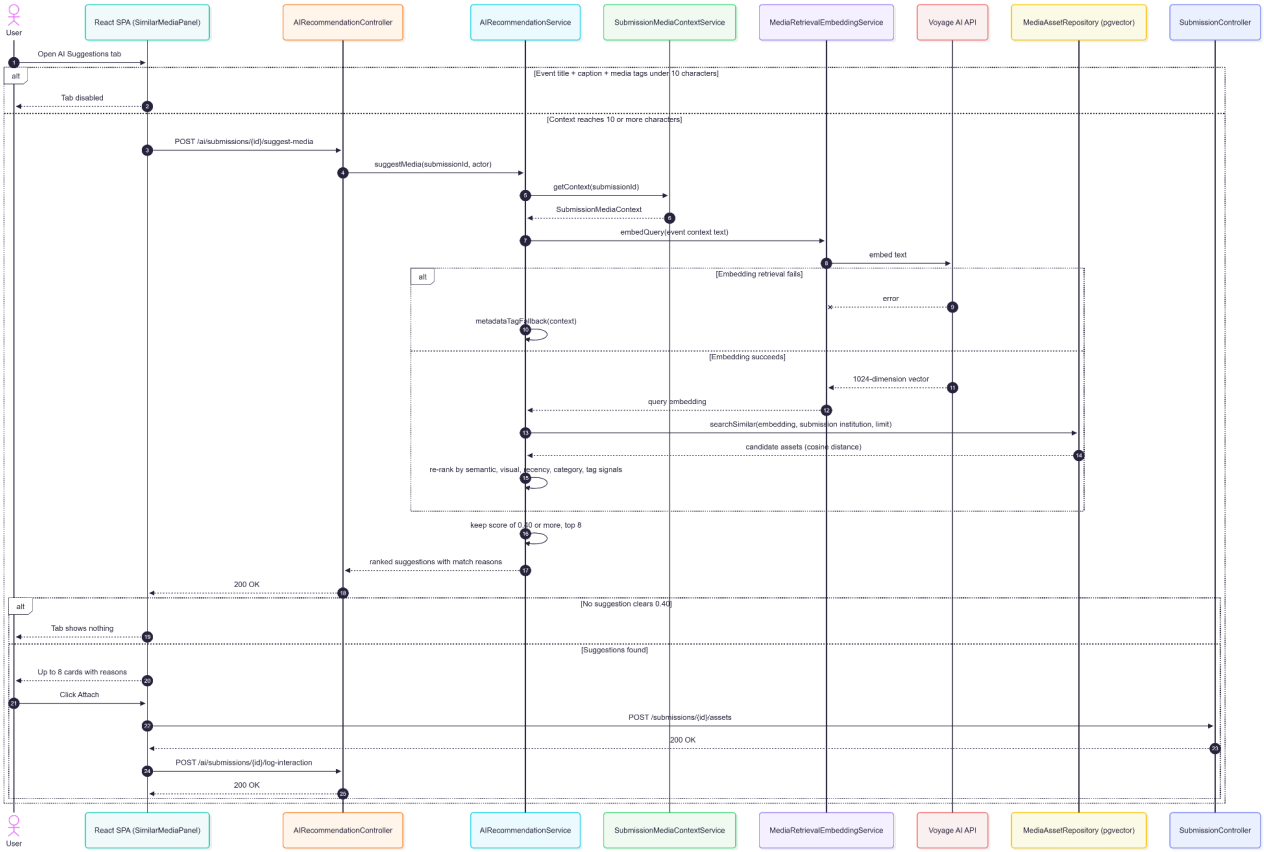
- **Class Diagram**

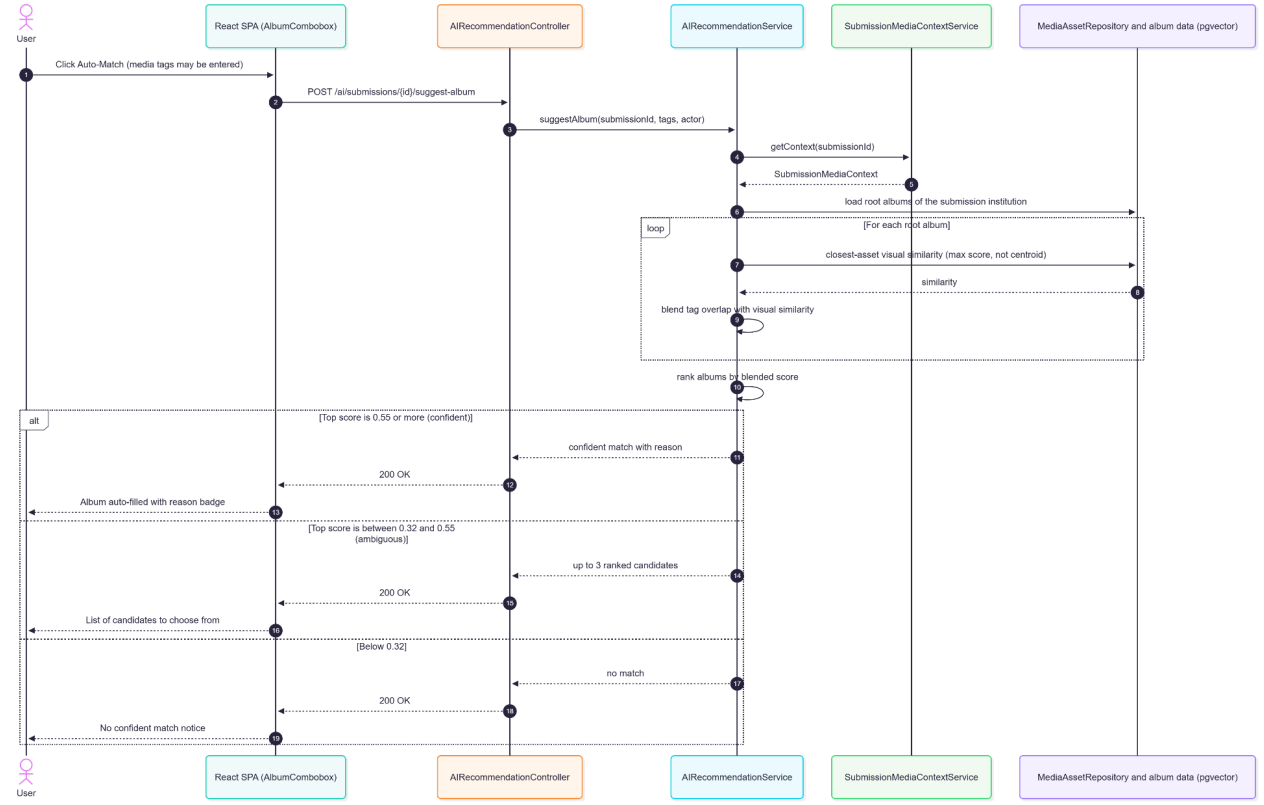


.

- **Sequence Diagram**

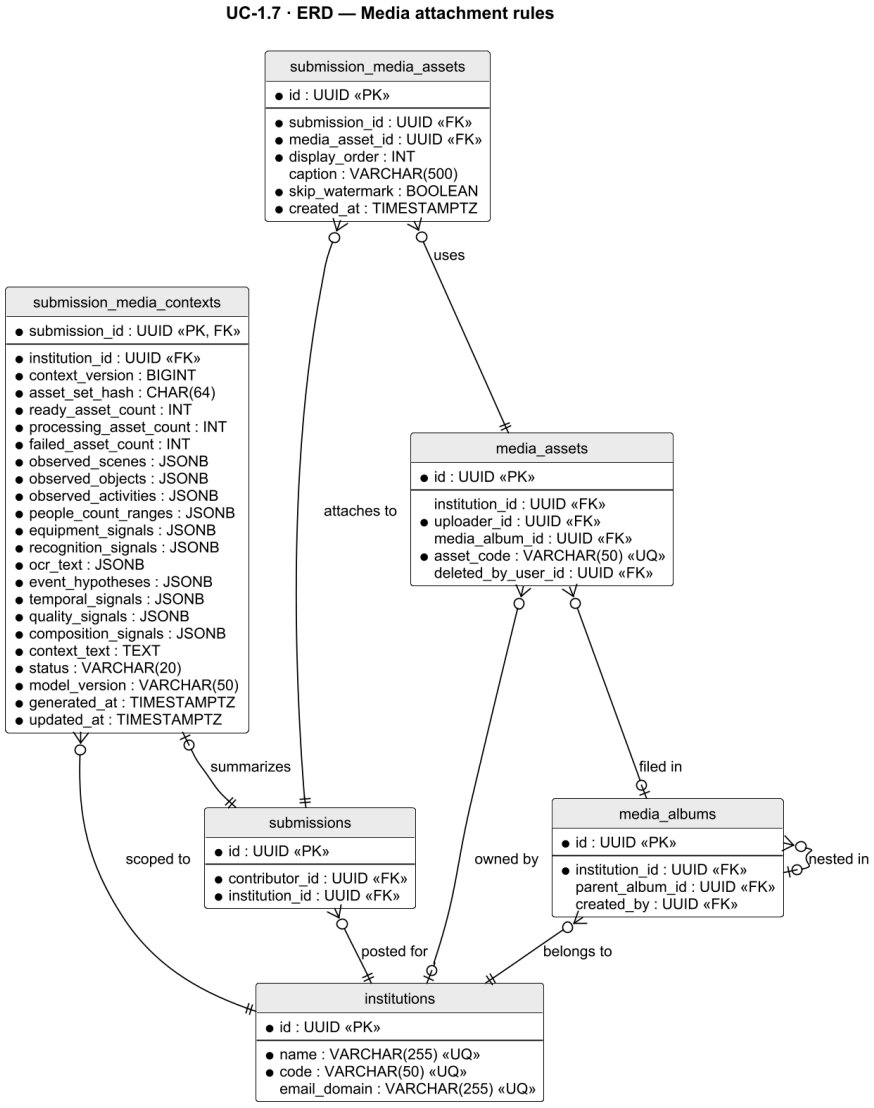

User
React SPA (media step)
SubmissionController
MediaStorageService
Cloudflare R2
MediaAssetRepository
SubmissionMediaAssetRepository
MediaProcessingQueueService
SubmissionMediaContextService
Part A - Device upload
Drop or pick files
validate format (7 allowed) and size (max 50 MB)
alt
[Client validation fails]
Error on upload card
[Passes]
POST /submissions/{id}/media/upload-url
createPresignedPut(key, contentType, size)
presign PUT
presigned URL
URL and object key
200 OK
PUT file bytes directly (progress shown)
alt
[Network failure]
error
Retry on upload card
[Upload succeeds]
200 OK
POST /submissions/{id}/media (register metadata)
countBySubmissionId(id)
alt
[Already 10 assets (BR-SUB-02)]
error envelope
Asset limit reached
[Under limit]
validateObject(key) (MIME, magic bytes, size)
read object head
alt
[Server validation fails]
delete object
error envelope
File rejected
[Valid]
save(MediaAsset, status = STAGED)
save(SubmissionMediaAsset, displayOrder)
enqueue(assetId) (classification, embeddings)
refresh(submissionId)
201 Created
Card shows thumbnail and status
opt
[Images and video are mixed]
Mixed-media notice
Part B - Attach from Media Library
Click Attach on a library result
POST /submissions/{id}/assets
existsBySubmissionAndAsset and countBySubmissionId
alt
[Duplicate or over 10 assets]
error envelope
[OK]
findByIdAndScope(assetId, scope)
save(SubmissionMediaAsset)
refresh(submissionId)
200 OK
Part C - Reorder and remove
Drag to reorder
PATCH /submissions/{id}/media/order
update displayOrder values
200 OK
Click remove on a card
DELETE /submissions/{id}/assets/{assetId}
delete junction row
refresh(submissionId)
200 OK


User
React SPA (SimilarMediaPanel)
AIRecommendationController
AIRecommendationService
SubmissionMediaContextService
MediaRetrievalEmbeddingService
Voyage AI API
MediaAssetRepository (pgvector)
SubmissionController
Open AI Suggestions tab
alt
[Event title + caption + media tags under 10 characters]
Tab disabled
[Context reaches 10 or more characters]
POST /ai/submissions/{id}/suggest-media
suggestMedia(submissionId, actor)
getContext(submissionId)
SubmissionMediaContext
embedQuery(event context text)
embed text
alt
[Embedding retrieval fails]
error
metadataTagFallback(context)
[Embedding succeeds]
1024-dimension vector
query embedding
searchSimilar(embedding, submission institution, limit)
candidate assets (cosine distance)
re-rank by semantic, visual, recency, category, tag signals
keep score of 0.40 or more, top 8
ranked suggestions with match reasons
200 OK
alt
[No suggestion clears 0.40]
Tab shows nothing
[Suggestions found]
Up to 8 cards with reasons
Click Attach
POST /submissions/{id}/assets
200 OK
POST /ai/submissions/{id}/log-interaction
200 OK


User
React SPA (AlbumCombobox)
AIRecommendationController
AIRecommendationService
SubmissionMediaContextService
MediaAssetRepository and album data (pgvector)
Click Auto-Match (media tags may be entered)
POST /ai/submissions/{id}/suggest-album
suggestAlbum(submissionId, tags, actor)
getContext(submissionId)
SubmissionMediaContext
load root albums of the submission institution
loop
[For each root album]
closest-asset visual similarity (max score, not centroid)
similarity
blend tag overlap with visual similarity
rank albums by blended score
alt
[Top score is 0.55 or more (confident)]
confident match with reason
200 OK
Album auto-filled with reason badge
[Top score is between 0.32 and 0.55 (ambiguous)]
up to 3 ranked candidates
200 OK
List of candidates to choose from
[Below 0.32]
no match
200 OK
No confident match notice

1.7.5. Data Design

- **Entity Relationship Diagram**



.

- **Schema**

```
CREATE TABLE submission_media_assets (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    submission_id   UUID NOT NULL REFERENCES submissions(id) ON DELETE CASCADE,
    media_asset_id UUID NOT NULL REFERENCES media_assets(id),
    display_order   INT NOT NULL DEFAULT 0,
    caption         VARCHAR(500),                   -- per-asset caption (published per photo)
    skip_watermark BOOLEAN NOT NULL DEFAULT FALSE,  -- photo-only opt-out (UC-2.5 A5)
    created_at      TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    UNIQUE (submission_id, media_asset_id)
);

-- Aggregated AI context per submission used for suggestions
CREATE TABLE submission_media_contexts (
    submission_id           UUID PRIMARY KEY REFERENCES submissions(id) ON DELETE CASCADE,
    institution_id          UUID NOT NULL REFERENCES institutions(id),
    context_version         BIGINT NOT NULL DEFAULT 1,
    asset_set_hash          CHAR(64) NOT NULL,              -- detects a changed set of attached assets
    ready_asset_count       INT NOT NULL DEFAULT 0,
```

```
    processing_asset_count INT NOT NULL DEFAULT 0,
    failed_asset_count      INT NOT NULL DEFAULT 0,
    observed_scenes         JSONB NOT NULL DEFAULT '[]'::jsonb,
    observed_objects        JSONB NOT NULL DEFAULT '[]'::jsonb,
    observed_activities     JSONB NOT NULL DEFAULT '[]'::jsonb,
    people_count_ranges     JSONB NOT NULL DEFAULT '[]'::jsonb,
    equipment_signals       JSONB NOT NULL DEFAULT '[]'::jsonb,
    recognition_signals     JSONB NOT NULL DEFAULT '[]'::jsonb,
    ocr_text                JSONB NOT NULL DEFAULT '[]'::jsonb,
    event_hypotheses        JSONB NOT NULL DEFAULT '[]'::jsonb,
    temporal_signals        JSONB NOT NULL DEFAULT '{}'::jsonb,
    quality_signals         JSONB NOT NULL DEFAULT '[]'::jsonb,
    composition_signals     JSONB NOT NULL DEFAULT '[]'::jsonb,
    context_text            TEXT NOT NULL DEFAULT '',       -- text used for suggestion queries
    status                  VARCHAR(20) NOT NULL CHECK (status IN ('EMPTY', 'PARTIAL', 'READY')),
    model_version           VARCHAR(50) NOT NULL,
    generated_at            TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at              TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
```

1.8. Engagement Helpers

1.8.1. User Interface Design

Schedule Mode (Organize & Schedule step)

A toggle between **Schedule** (default) and **Live Event (Fast-Track)**. Fast-Track hides all date/time and recommendation UI.

When to Post — Recommendation Tiles

A compact row of up to 3 calendar-style slot tiles under a "When to Post *" heading. The source explanation sits behind an ⓘ toggle; the "why" line shows only for the selected tile; a "Top pick" badge and engagement bars appear only when real Page history (`HISTORICAL`) separates the slots. With insufficient history the panel shows "Best-practice guidance" (Tue/Wed/Thu 6 PM). If the Facebook analytics call fails, the panel is hidden and the picker remains.

Date and Time Picker

Selecting a tile fills the fields; manual entry is always allowed. Validation is debounced; hard guard-rail blocks show as errors, and soft warnings (three-unpublished-post institution quota, six-post daily cap) show inline beneath the fields (`.sub-inline-warning`) only when guard rails are enforced.

1.8.2. Front-end Component(s)

EngagementRecommendationsPanel

- `GET /engagement-recommendations` for the selected institution; skipped for Fast-Track drafts and for Administrator composers without an institution scope.
- React functional component

CalendarDateField / TimePickerField

- Date/time inputs with debounced validation via `POST /submissions/{id}/evaluate-slot` (saved draft) or `POST /guardrails/validate`.
- React functional components

1.8.3. Back-end Component(s)

EngagementRecommendationController / EngagementRecommendationService

- Reads the Page's last 100 posts (reactions + comments + shares) from the Graph API, buckets by weekday and hour within 8 AM–8 PM Asia/Manila, keeps the top 5 buckets when ≥20 samples exist (else default Tue/Wed/Thu 6 PM), walks a 30-day lookahead for the first matching slots, drops hard-blocked candidates, and returns up to 3 slots with soft warnings and a score.
- Spring `@RestController` / `@Service`

GuardRailController / GuardRailService / GuardRailSettingsService

- `POST /guardrails/validate`. Hard rules: network-wide conflict buffer (GR-H1, default ±30 min), minimum lead time (GR-H2, default 2 h), maximum horizon (GR-H3, default 30 days), and posting window (GR-H4, default 8:00 AM–8:00 PM Asia/Manila). Soft: per-institution active quota (GR-S1, default 3 scheduled-but-unpublished posts) and daily volume cap (GR-S2, default 6 per day). Every threshold is Administrator-configurable in Page Settings (V116, V121, V122) and read through `GuardRailSettingsService`. The database exclusion constraint (UC-1.8 schema) stays fixed at 30 minutes regardless of the configured buffer. When the switch is off, a Contributor's slot only needs to be in the future; a schedule change made by a Moderator or Administrator is still validated, and only an Administrator may bypass a hard block with a reason (audited `SCHEDULE_GUARDRAIL_OVERRIDE`). Returns a structured `GuardRailResult` of violations and warnings.
- Spring `@RestController` / `@Service`

SubmissionController / SlotReservationService

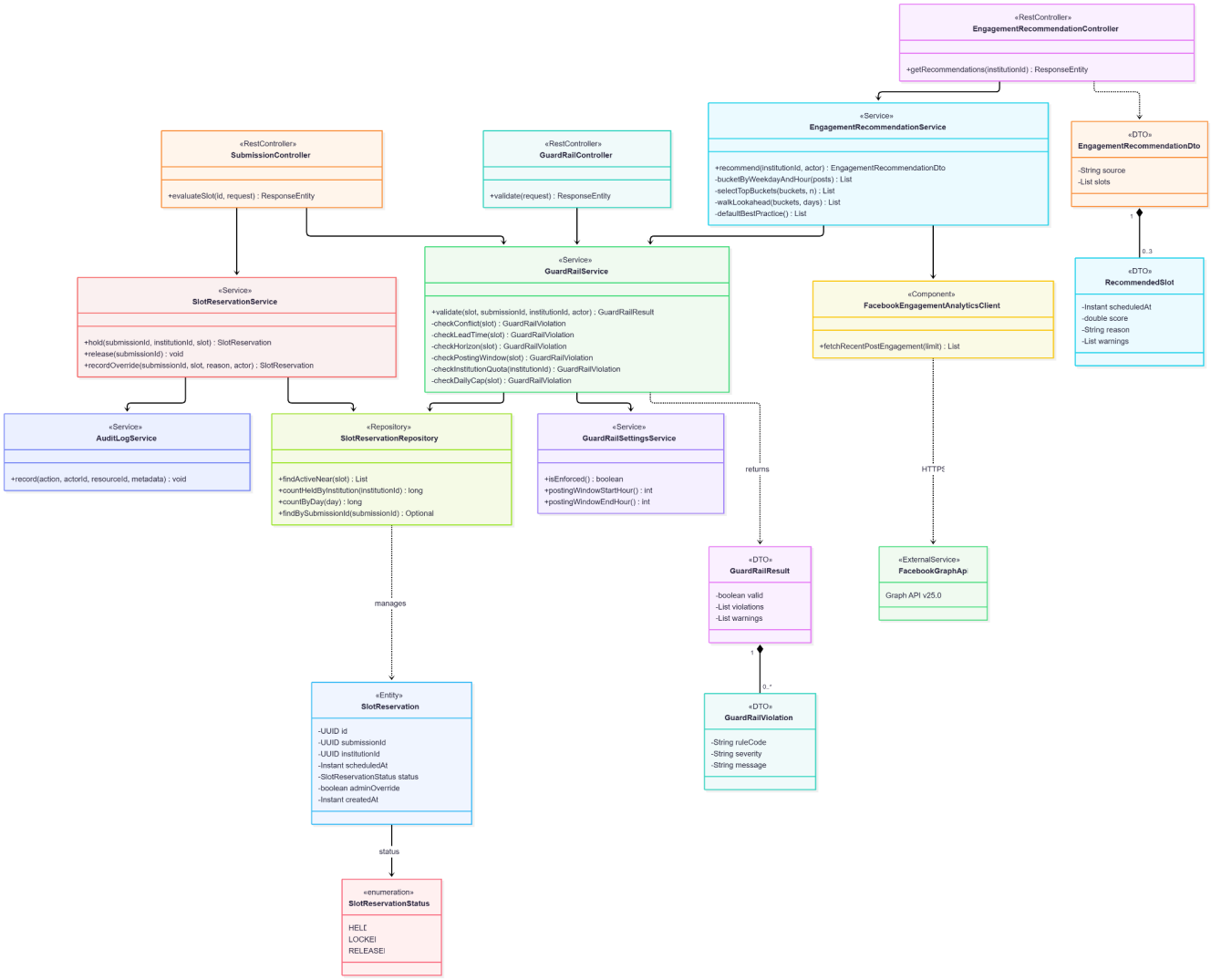
- `POST /submissions/{id}/evaluate-slot`; holds a `held` reservation for Standard drafts (not Fast-Track).
- Spring `@RestController` / `@Service`

FacebookEngagementAnalyticsClient

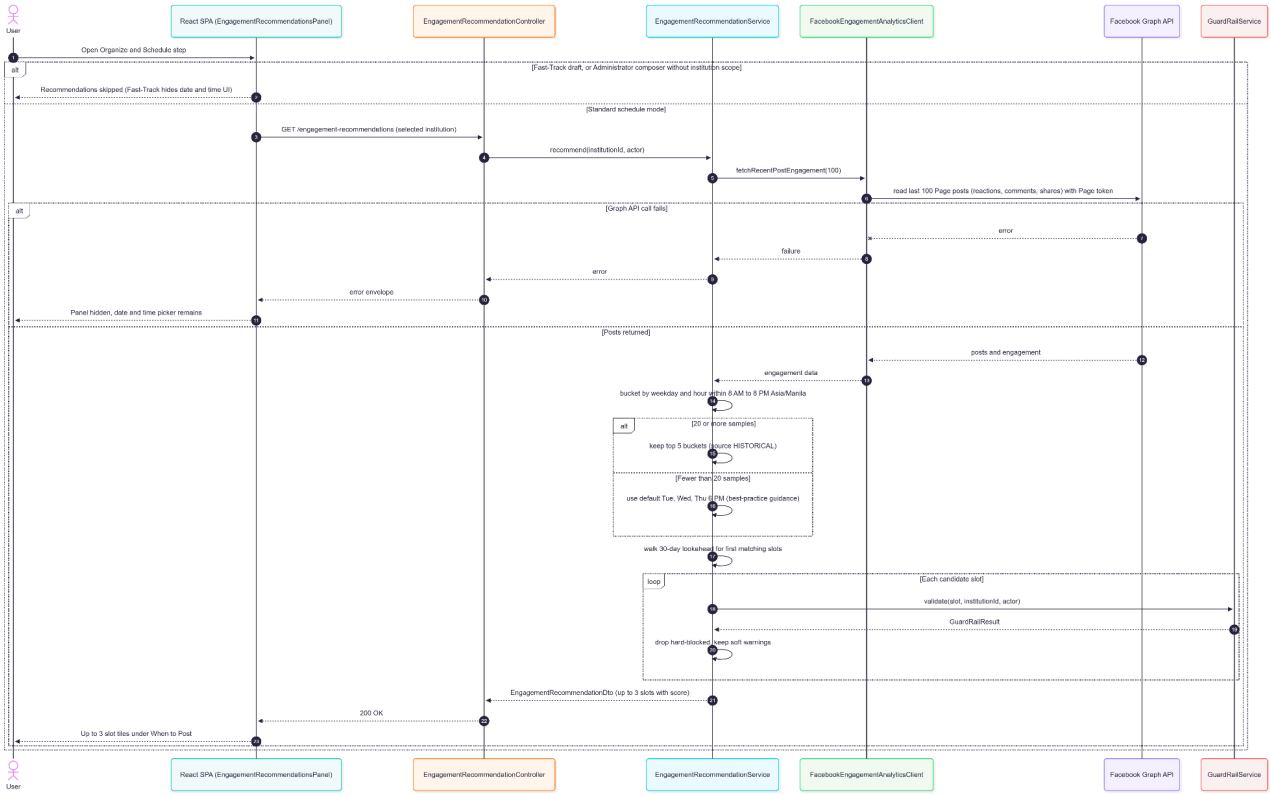
- Graph API engagement reads using the active Page token.
- Spring `@Component`

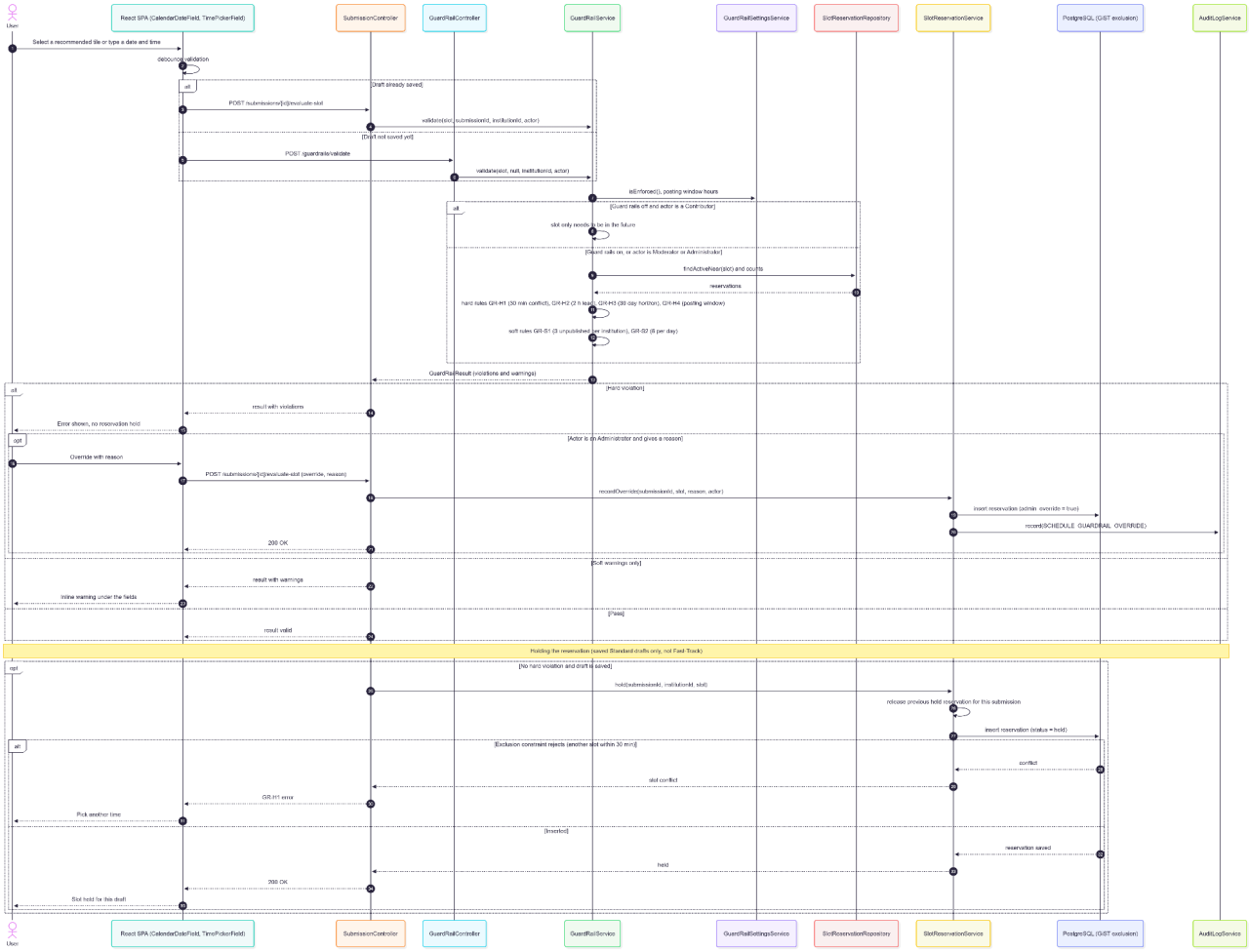
1.8.4. Object-Oriented Component(s)

- **Class Diagram**



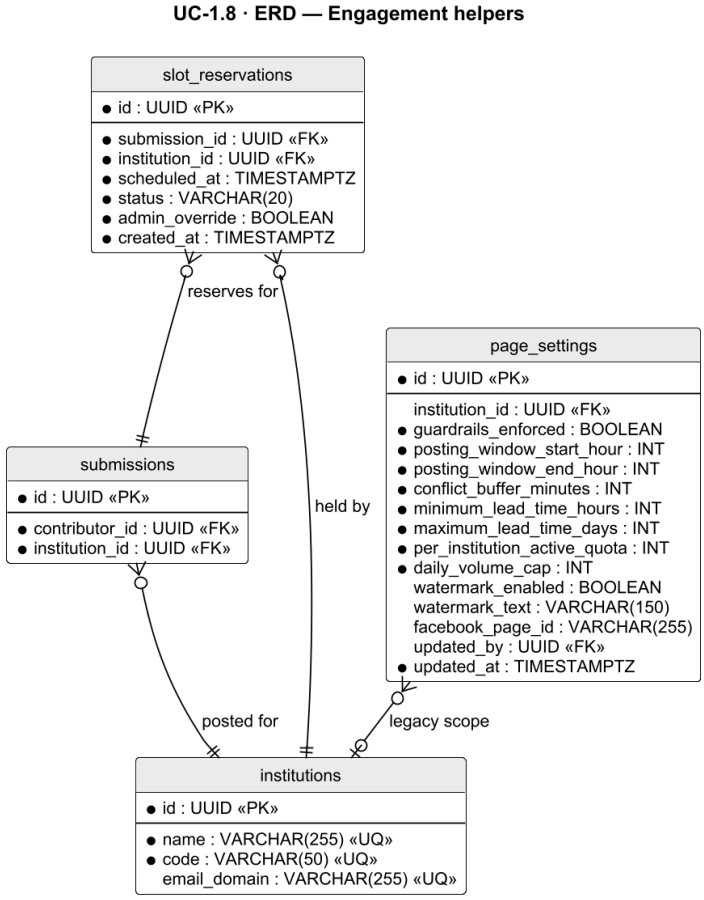
- **Sequence Diagram**


User
React SPA (EngagementRecommendationsPanel)
EngagementRecommendationController
EngagementRecommendationService
FacebookEngagementAnalyticsClient
Facebook Graph API
GuardRailService
Open Organize and Schedule step
alt
[Fast-Track draft, or Administrator composer without institution scope]
Recommendations skipped (Fast-Track hides date and time UI)
[Standard schedule mode]
GET /engagement-recommendations (selected institution)
recommend(institutionId, actor)
fetchRecentPostEngagement(100)
read last 100 Page posts (reactions, comments, shares) with Page token
alt
[Graph API call fails]
error
failure
error
error envelope
Panel hidden, date and time picker remains
[Posts returned]
posts and engagement
engagement data
bucket by weekday and hour within 8 AM to 8 PM Asia/Manila
alt
[20 or more samples]
keep top 5 buckets (source HISTORICAL)
[Fewer than 20 samples]
use default Tue, Wed, Thu 6 PM (best-practice guidance)
walk 30-day lookahead for first matching slots
loop
[Each candidate slot]
validate(slot, institutionId, actor)
GuardRailResult
drop hard-blocked, keep soft warnings
EngagementRecommendationDto (up to 3 slots with score)
200 OK
Up to 3 slot tiles under When to Post


User
React SPA (CalendarDateField, TimePickerField)
SubmissionController
GuardRailController
GuardRailService
GuardRailSettingsService
SlotReservationRepository
SlotReservationService
PostgreSQL (GiST exclusion)
AuditLogService
Select a recommended tile or type a date and time
debounced validation
alt
[Draft already saved]
POST /submissions/{id}/evaluate-slot
validate(slot, submissionId, institutionId, actor)
[Draft not saved yet]
POST /guardrails/validate
validate(slot, null, institutionId, actor)
isEnforced(), posting window hours
[Guard rails off and actor is a Contributor]
alt
slot only needs to be in the future
[Guard rails on, or actor is Moderator or Administrator]
findActiveNear(slot) and counts
reservations
hard rules GR-H1 (30 min conflict), GR-H2 (2 h lead), GR-H3 (30 day horizon), GR-H4 (posting window)
soft rules GR-S1 (3 unpublished per institution), GR-S2 (6 per day)
GuardRailResult (violations and warnings)
alt
[Hard violation]
result with violations
Error shown, no reservation held
opt
[Actor is an Administrator and gives a reason]
Override with reason
POST /submissions/{id}/evaluate-slot (override, reason)
recordOverride(submissionId, slot, reason, actor)
insert reservation (admin_override = true)
record(SCHEDULE_GUARDRAIL_OVERRIDE)
200 OK
[Soft warnings only]
result with warnings
Inline warning under the fields
[Valid]
result valid
Holding the reservation (saved Standard drafts only, not Fast-Track)
opt
[No hard violation and draft is saved]
hold(submissionId, institutionId, slot)
release previous held reservation for this submission
insert reservation (status = held)
alt
[Exclusion constraint rejects (another slot within 30 min)]
conflict
slot conflict
GR-H1 error
Pick another time
[Inserted]
reservation saved
held
200 OK
Slot held for this draft

1.8.5. Data Design

- **Entity Relationship Diagram**



- **Schema**

```
CREATE TABLE slot_reservations (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    submission_id  UUID NOT NULL REFERENCES submissions(id),
    institution_id UUID NOT NULL REFERENCES institutions(id),
    scheduled_at   TIMESTAMPTZ NOT NULL,
    status          VARCHAR(20) NOT NULL DEFAULT 'held'
                    CHECK (status IN ('held', 'locked', 'released')),
    admin_override BOOLEAN NOT NULL DEFAULT FALSE,  -- set only for an Admin hard-block override
    created_at      TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- GR-H1 enforced by the database (V95/V96): no two active, non-override
-- reservations within ±30 minutes of each other, network-wide.
CREATE OR REPLACE FUNCTION epoch_seconds_immutable(ts timestamptz)
RETURNS bigint LANGUAGE sql IMMUTABLE PARALLEL SAFE
AS $$ SELECT(EXTRACT(EPOCHFROMts))::bigint; $$;

ALTER TABLE slot_reservations ADD CONSTRAINT excl_slot_reservations_network_buffer
    EXCLUDE USING gist (
            int8range(epoch_seconds_immutable(scheduled_at),
                  epoch_seconds_immutable(scheduled_at) + 1800, '[]') WITH &&
    ) WHERE (status <> 'released' AND NOT admin_override);
-- Each reservation covers [t, t + 30 min]; two ranges overlap exactly when the
-- slots are within 30 minutes of each other.

        Related tables used by this use case:
```

```
-- page_settings (defined in UC-1.4)
CREATE TABLE page_settings (
    id                      UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    institution_id          UUID REFERENCES institutions(id),
    guardrails_enforced BOOLEAN NOT NULL DEFAULT TRUE,
    posting_window_start_hour INT NOT NULL DEFAULT 8,       -- GR-H4 window start (Asia/Manila, V116)
    posting_window_end_hour   INT NOT NULL DEFAULT 20,      -- GR-H4 window end (exclusive)
    conflict_buffer_minutes   INT NOT NULL DEFAULT 30,      -- GR-H1 (V121)
    minimum_lead_time_hours   INT NOT NULL DEFAULT 2,       -- GR-H2 (V121)
    maximum_lead_time_days  INT NOT NULL DEFAULT 30,        -- GR-H3 (V121)
    per_institution_active_quota INT NOT NULL DEFAULT 3,  -- GR-S1 (V122)
    daily_volume_cap        INT NOT NULL DEFAULT 6,         -- GR-S2 (V122)
    watermark_enabled   BOOLEAN,
    watermark_text          VARCHAR(150),
    facebook_page_id        VARCHAR(255),
    updated_by              UUID REFERENCES users(id),
    updated_at              TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
```

1.9. Draft Submission

1.9.1. User Interface Design

Submit for Approval

The final action in the composer. The frontend re-checks title, date, caption, media, album, and (Standard) schedule; anything missing jumps back to the relevant step with a toast. If only Recommended items remain, a non-blocking confirmation shows the readiness score ("Submit anyway / Go back").

Submission Success State

The composer closes and returns to the Submissions list with the item shown as Pending (or a "Live Event" badge for Fast-Track).

Slot Conflict at Submit

On a `409` slot conflict the confirmation closes and the composer returns to Organize & Schedule with a prompt to pick a new time.

Withdraw Submission

Available on `pending` items with no review lock; confirmation returns the item to Draft.

1.9.2. Front-end Component(s)

SubmissionScreen (submit flow)

- `POST /submissions/{id}/submit`; specific `409` handling redirects to the Schedule step.
- React functional component

SubmissionListScreen (withdraw)

- `POST /submissions/{id}/withdraw` with a confirmation prompt.
- React functional component

1.9.3. Back-end Component(s)

SubmissionController

- `POST /submissions/{id}/submit` (`draft`/`needs_revision`/`rejected` → `pending`), `POST /submissions/{id}/withdraw` (`pending` → `draft`, refused with `409` once a review lock exists).
- Spring `@RestController`

SubmissionService

- `assertContentComplete` (title, date, caption + length, media, album) runs before any state change or media reconciliation; Standard drafts are re-validated against the guard rails (`409` on conflict); Fast-Track skips slot validation. On success: reconciles media (new uploads are bound to the institution, filed into the album resolved from `album_name`, marked PROCESSING; library picks keep their album; all receive the post's media tags), clears a prior rejection reason, sets `submitted_at`, and writes `SUBMISSION_SUBMITTED`. Notification failures are caught so they never roll back the transition.
- Spring `@Service`

NotificationService

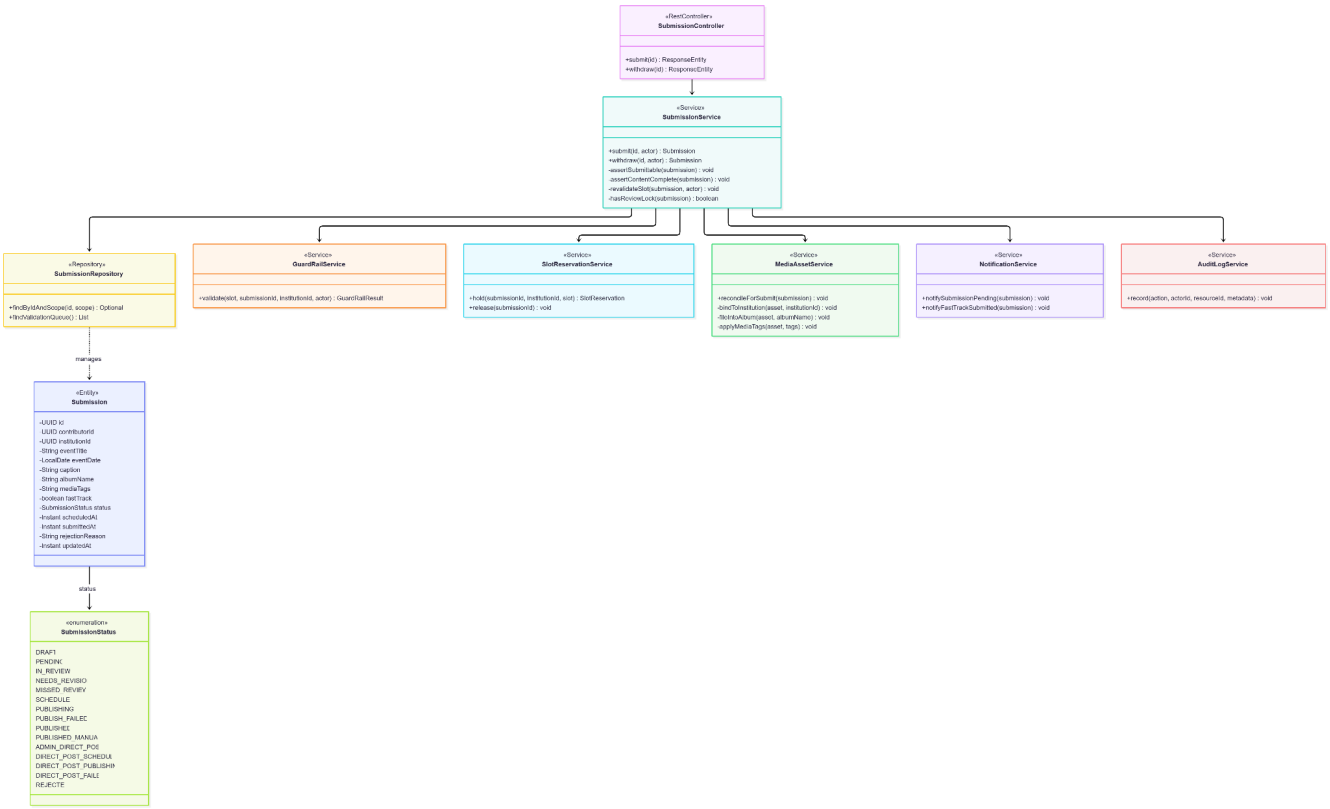
- T-01 (standard): in-app + Messenger to every active Moderator and Administrator. T-11 (Fast-Track): in-app + email + Messenger, urgent, to every active Moderator and Administrator (`NotificationEventListener.reviewers()`).
- Spring `@Service`

SubmissionRepository

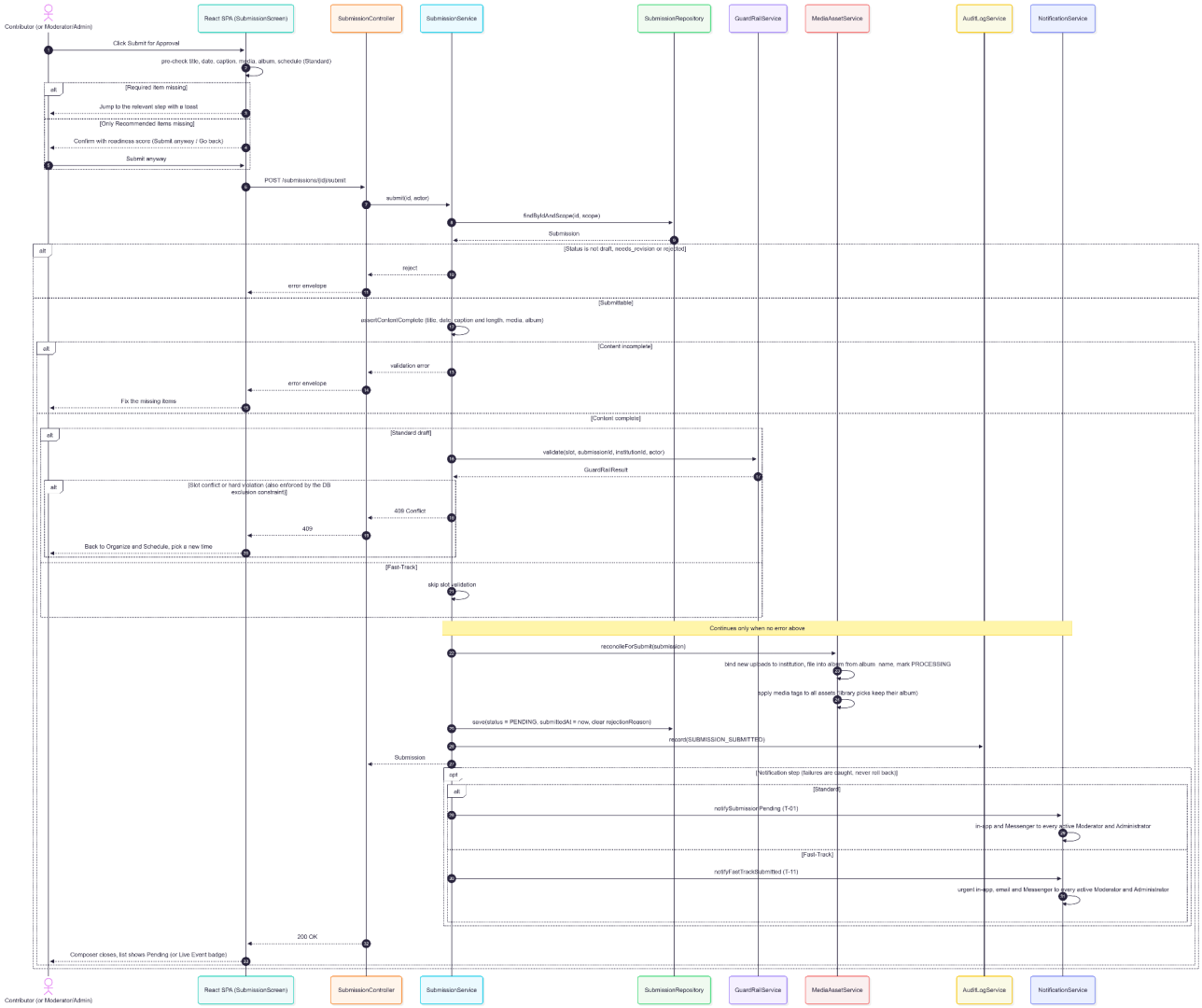
- `findValidationQueue(priorityCutoff)`, behind GET /validation/queue, orders fast_track DESC, then submissions due within 30 minutes (GR-T1), then `scheduled_at ASC NULLS LAST`, `submitted_at ASC. findValidationPage`, behind the paginated GET /validation/queue/page that the Review Queue screen uses, orders fast_track DESC and then the selected sort, without the 30-minute term — there GR-T1 appears only as a badge. SubmissionSummaryDto.priorityFlag marks those due within 30 minutes in both.
- Spring `@Repository`

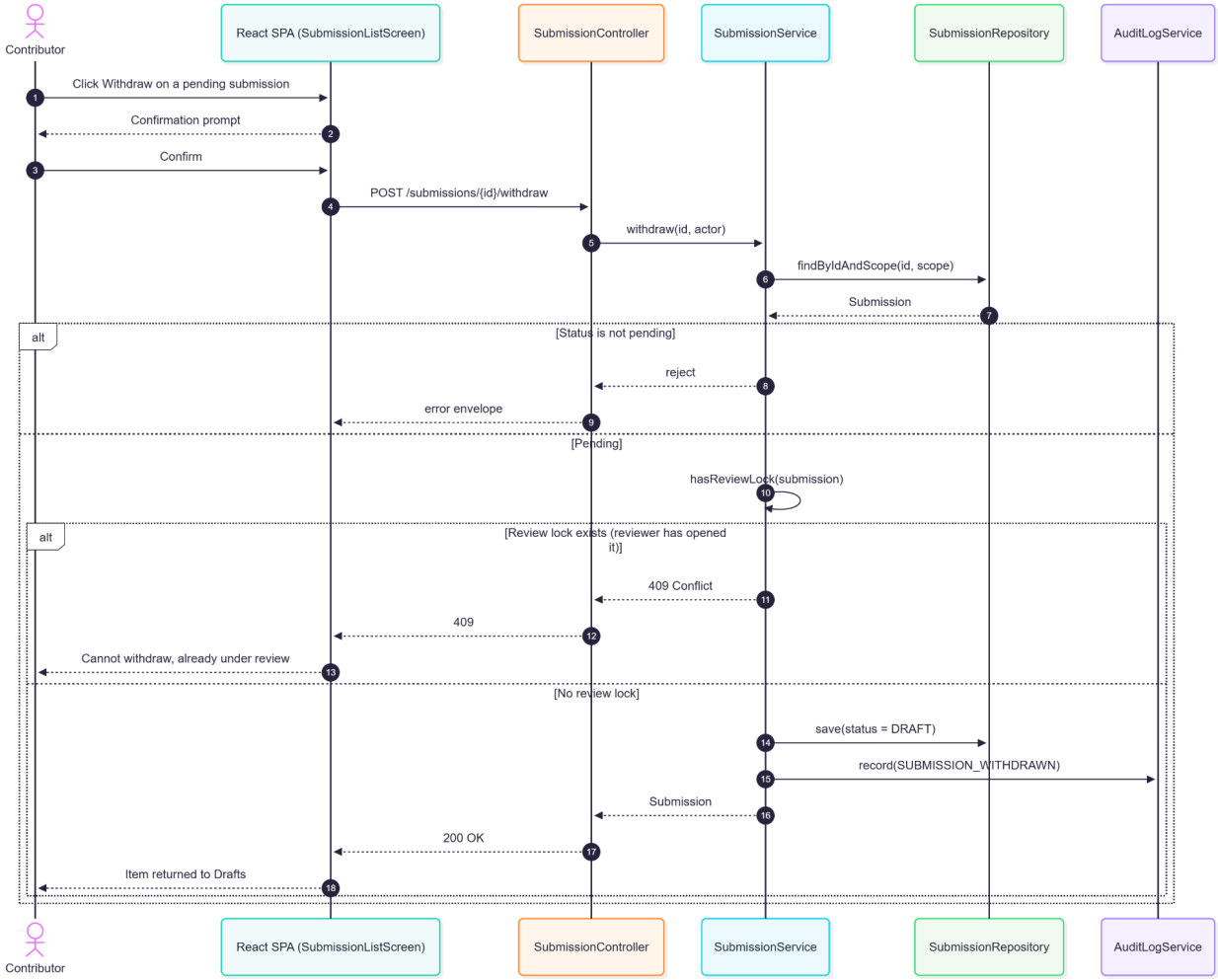
1.9.4. Object-Oriented Component(s)

- **Class Diagram**



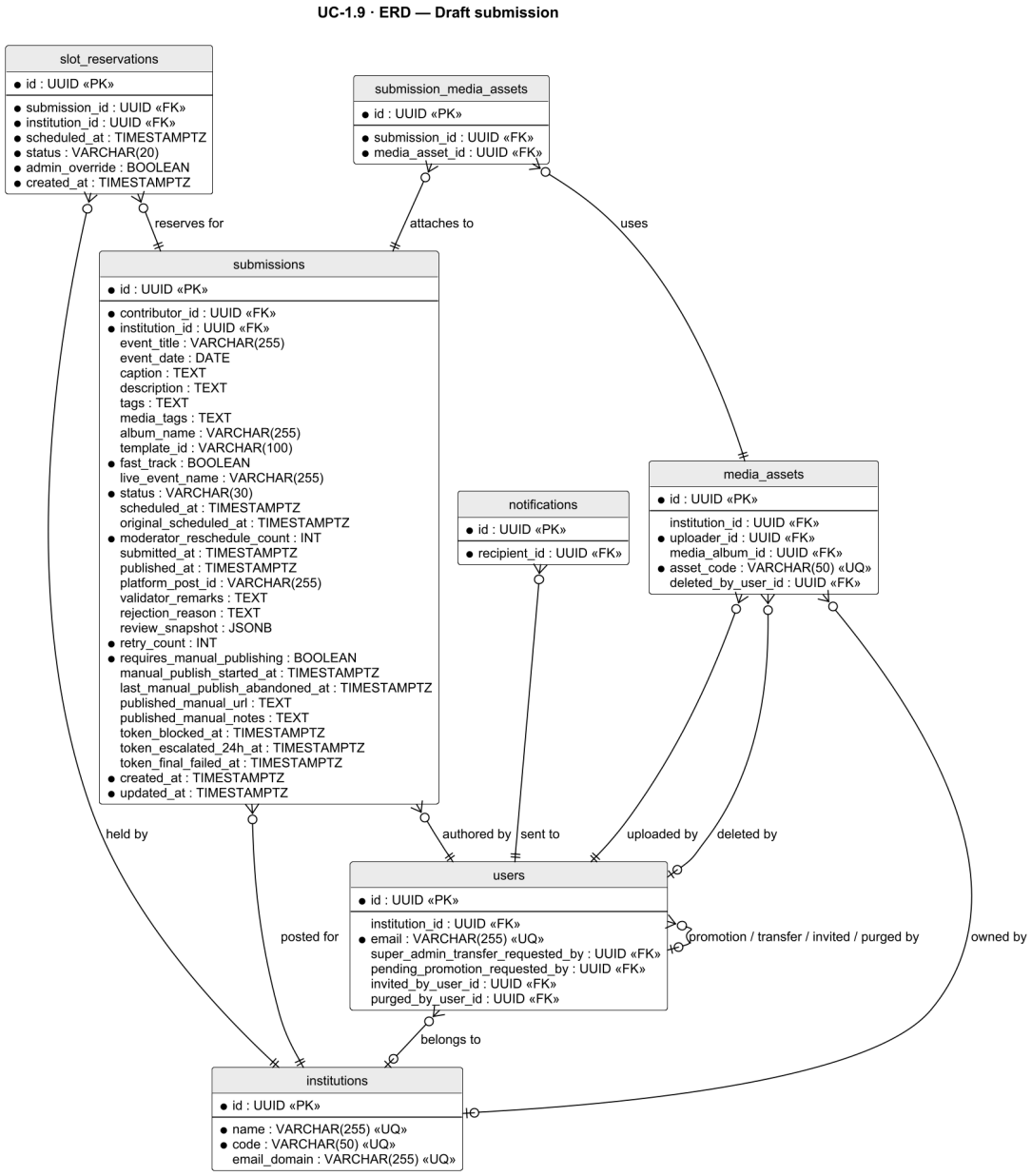
- **Sequence Diagram**


Contributor (or Moderator/Admin)
React SPA (SubmissionScreen)
SubmissionController
SubmissionService
SubmissionRepository
GuardRailService
MediaAssetService
AuditLogService
NotificationService
Click Submit for Approval
pre-check title, date, caption, media, album, schedule (Standard)
alt
[Required item missing]
Jump to the relevant step with a toast
[Only Recommended items missing]
Confirm with readiness score (Submit anyway / Go back)
Submit anyway
POST /submissions/{id}/submit
submit(id, actor)
findByIdAndScope(id, scope)
Submission
[Status is not draft, needs_revision or rejected]
reject
error envelope
[Submittable]
assertContentComplete (title, date, caption and length, media, album)
[Content incomplete]
validation error
error envelope
Fix the missing items
[Content complete]
[Standard draft]
validate(slot, submissionId, institutionId, actor)
GuardRailResult
[Slot conflict or hard violation (also enforced by the DB exclusion constraint)]
409 Conflict
409
Back to Organize and Schedule, pick a new time
[Fast-Track]
skip slot validation
Continues only when no error above
reconcileForSubmit(submission)
bind new uploads to institution, file into album from albums name, mark PROCESSING
apply media tags to all assets (library picks keep their album)
save(status = PENDING, submittedAt = now, clear rejectionReason)
record(SUBMISSION_SUBMITTED)
Submission
opt
[Notification step (failures are caught, never roll back)]
[Standard]
notifySubmissionPending (T-01)
in-app and Messenger to every active Moderator and Administrator
[Fast-Track]
notifyFastTrackSubmitted (T-11)
urgent in-app, email and Messenger to every active Moderator and Administrator
200 OK
Composer closes, list shows Pending (or Live Event badge)


Contributor
React SPA (SubmissionListScreen)
SubmissionController
SubmissionService
SubmissionRepository
AuditLogService
1 Click Withdraw on a pending submission
2 Confirmation prompt
3 Confirm
4 POST /submissions/{id}/withdraw
5 withdraw(id, actor)
6 findByIdAndScope(id, scope)
7 Submission
alt
[Status is not pending]
8 reject
9 error envelope
[Pending]
10 hasReviewLock(submission)
alt
[Review lock exists (reviewer has opened it)]
11 409 Conflict
12 409
13 Cannot withdraw, already under review
[No review lock]
14 save(status = DRAFT)
15 record(SUBMISSION_WITHDRAWN)
16 Submission
17 200 OK
18 Item returned to Drafts
Contributor
React SPA (SubmissionListScreen)
SubmissionController
SubmissionService
SubmissionRepository
AuditLogService

1.9.5. Data Design

- **Entity Relationship Diagram**



- **Schema**

```
-- UC-1.9 operates on submissions (defined in UC-1.5) and slot_reservations (UC-1.8);

-- no new tables. Columns used by draft submission and withdrawal:

CREATE TABLE submissions (

    id                          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    contributor_id              UUID NOT NULL REFERENCES users(id),
    institution_id              UUID NOT NULL REFERENCES institutions(id),
    event_title                 VARCHAR(255),
    event_date                  DATE,
    caption                     TEXT,
    description                 TEXT,
```

```
    tags                            TEXT,
    media_tags                      TEXT,
    album_name                      VARCHAR(255),
    template_id                     VARCHAR(100),
    fast_track                      BOOLEAN NOT NULL DEFAULT FALSE,
    live_event_name                 VARCHAR(255),
    status                          VARCHAR(30) NOT NULL DEFAULT 'draft'
                                    CHECK (status IN ('draft', 'pending', 'in_review', 'needs_revision',
                                           'missed_review', 'scheduled', 'publishing', 'publish_failed',
                                           'published', 'published_manual', 'admin_direct_post',
                                           'direct_post_scheduled', 'direct_post_publishing',
                                           'direct_post_failed', 'rejected')),
    scheduled_at                    TIMESTAMPTZ,
    original_scheduled_at           TIMESTAMPTZ,
    moderator_reschedule_count      INT NOT NULL DEFAULT 0,
    submitted_at                    TIMESTAMPTZ,
    published_at                    TIMESTAMPTZ,
    platform_post_id                VARCHAR(255),
    validator_remarks               TEXT,
    rejection_reason                TEXT,
    review_snapshot                 JSONB,
    retry_count                     INT NOT NULL DEFAULT 0,
    requires_manual_publishing      BOOLEAN NOT NULL DEFAULT FALSE,
    manual_publish_started_at       TIMESTAMPTZ,
    last_manual_publish_abandoned_at TIMESTAMPTZ,
    published_manual_url            TEXT,
    published_manual_notes          TEXT,
    token_blocked_at                TIMESTAMPTZ,
    token_escalated_24h_at          TIMESTAMPTZ,
    token_final_failed_at           TIMESTAMPTZ,
    created_at                      TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at                      TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE TABLE slot_reservations (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    submission_id  UUID NOT NULL REFERENCES submissions(id),
    institution_id UUID NOT NULL REFERENCES institutions(id),
    scheduled_at   TIMESTAMPTZ NOT NULL,
    status          VARCHAR(20) NOT NULL DEFAULT 'held'
                    CHECK (status IN ('held', 'locked', 'released')),
    admin_override BOOLEAN NOT NULL DEFAULT FALSE,
    created_at      TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

        Indexes:

-- Indexes supporting submit-time slot re-validation and the approval queue
CREATE INDEX idx_submissions_status         ON submissions(status);
CREATE INDEX idx_submissions_scheduled_at ON submissions(scheduled_at)
    WHERE status IN ('scheduled', 'publishing', 'pending', 'in_review',
                     'direct_post_scheduled', 'direct_post_publishing');
CREATE INDEX idx_slot_reservations_slot   ON slot_reservations(scheduled_at, institution_id)
    WHERE status != 'released';
-- Concurrent slot conflicts at submit (A4) are also rejected by the slot_reservations
-- exclusion constraint defined in UC-1.8 (GR-H1, ±30 minutes). Queue order
-- (fast_track DESC, due-within-30-minutes first, scheduled_at, submitted_at) is applied by the repository
query.
```

1.10. Moderator Account Management

1.10.1. User Interface Design

Moderator Management (User Management, `/admin/user-management`)

A Moderators card listing network-wide Moderator accounts (no institution) with status badges and pending Moderator invitations. Any active Administrator may Invite, Deactivate, Reactivate, Delete, Resend, and Cancel; Erase is Admin Owner only.

Change Role Dialog

Moves an active account between Contributor and Moderator. Contributor → Moderator clears the institution; Moderator → Contributor requires selecting an active institution. Either direction revokes sessions and releases any review locks the account holds.

1.10.2. Front-end Component(s)

UserManagementScreen / InvitationComposer / PendingInvitationsCard / ChangeRoleModal

- `GET /users/moderators`, `GET /invitations/pending/network`, `POST /invitations` (`role = moderator`), `PATCH /users/{id}/status`, `DELETE /users/{id}`, `POST /users/{id}/erase`, `PATCH /users/{id}/role`, `POST /users/promotion/confirm|decline`, `DELETE /users/{id}/promotion` (one unified promotion flow for both target roles).
- React functional components

1.10.3. Back-end Component(s)

InvitationController / InvitationService — network-scoped Moderator invitations (no institution), same token, reuse, and conflict rules as UC-1.1.

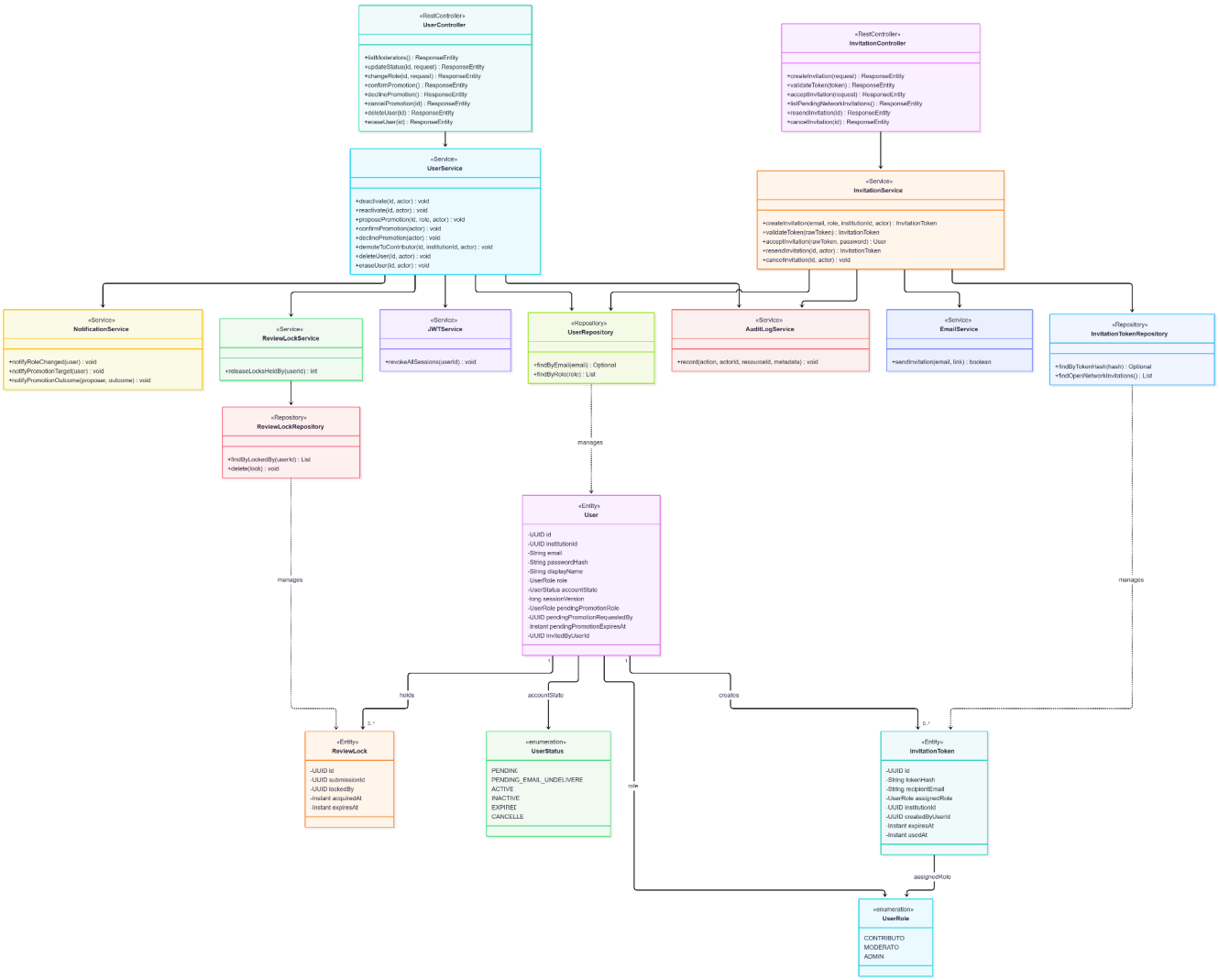
UserController / UserService

- Deactivate/reactivate/delete/erase as in UC-1.1/UC-1.3 (any active Admin; erase Admin Owner only). Role change between Contributor and Moderator: a Contributor → Moderator proposal is stored in the unified `pending_promotion_role/requested_by/expires_at` columns (72 hours) and applied on the target's confirmation; Moderator → Contributor is applied directly. On change: session_version++, review locks released via ReviewLockService, `MODERATOR_PROMOTION_CONFIRMED` (confirmed proposal) or `USER_ROLE_CHANGED` (direct change) written, affected user notified.
- Spring `@RestController` / `@Service`

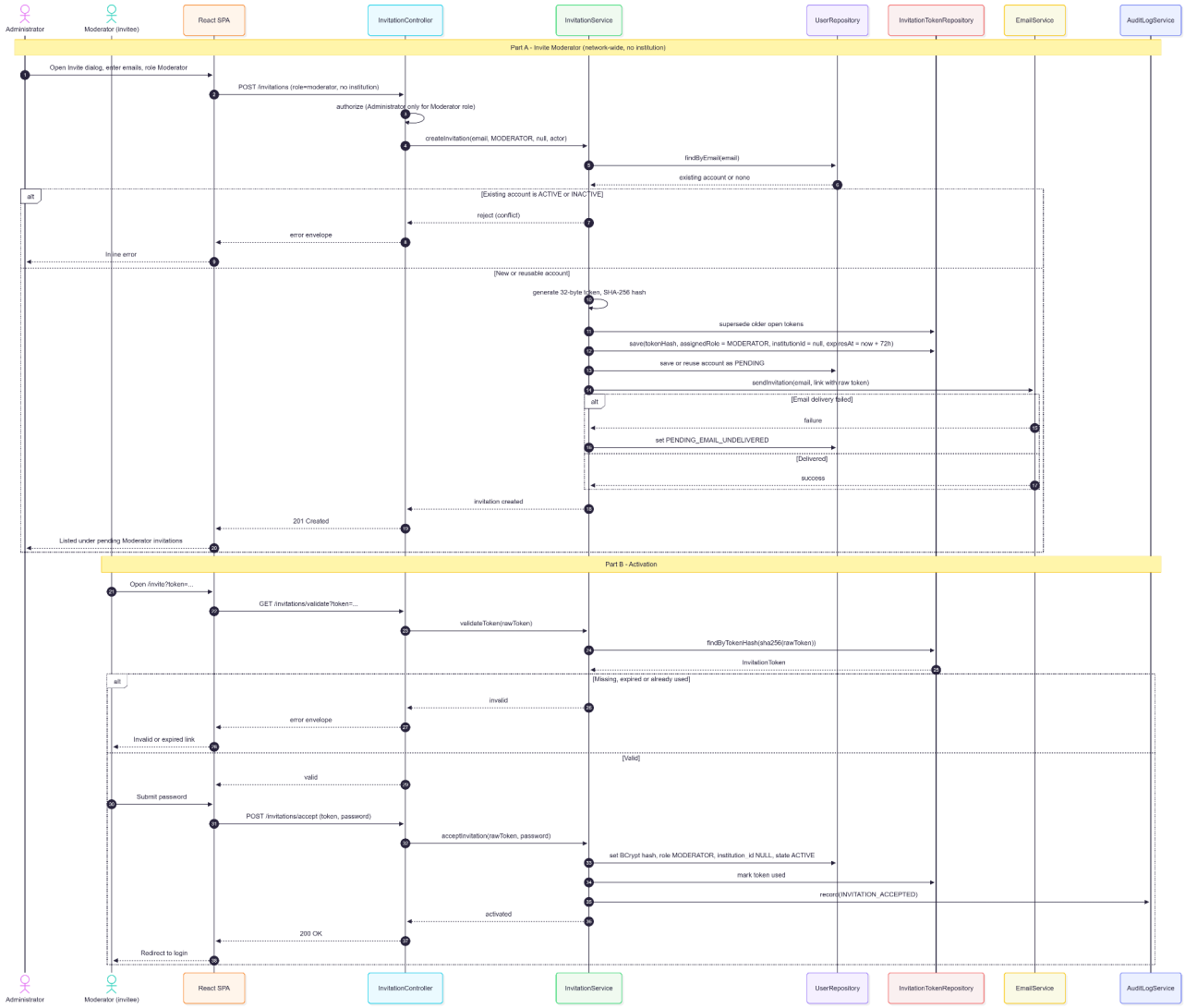
ReviewLockService — releases locks held by a deactivated or role-changed Moderator.

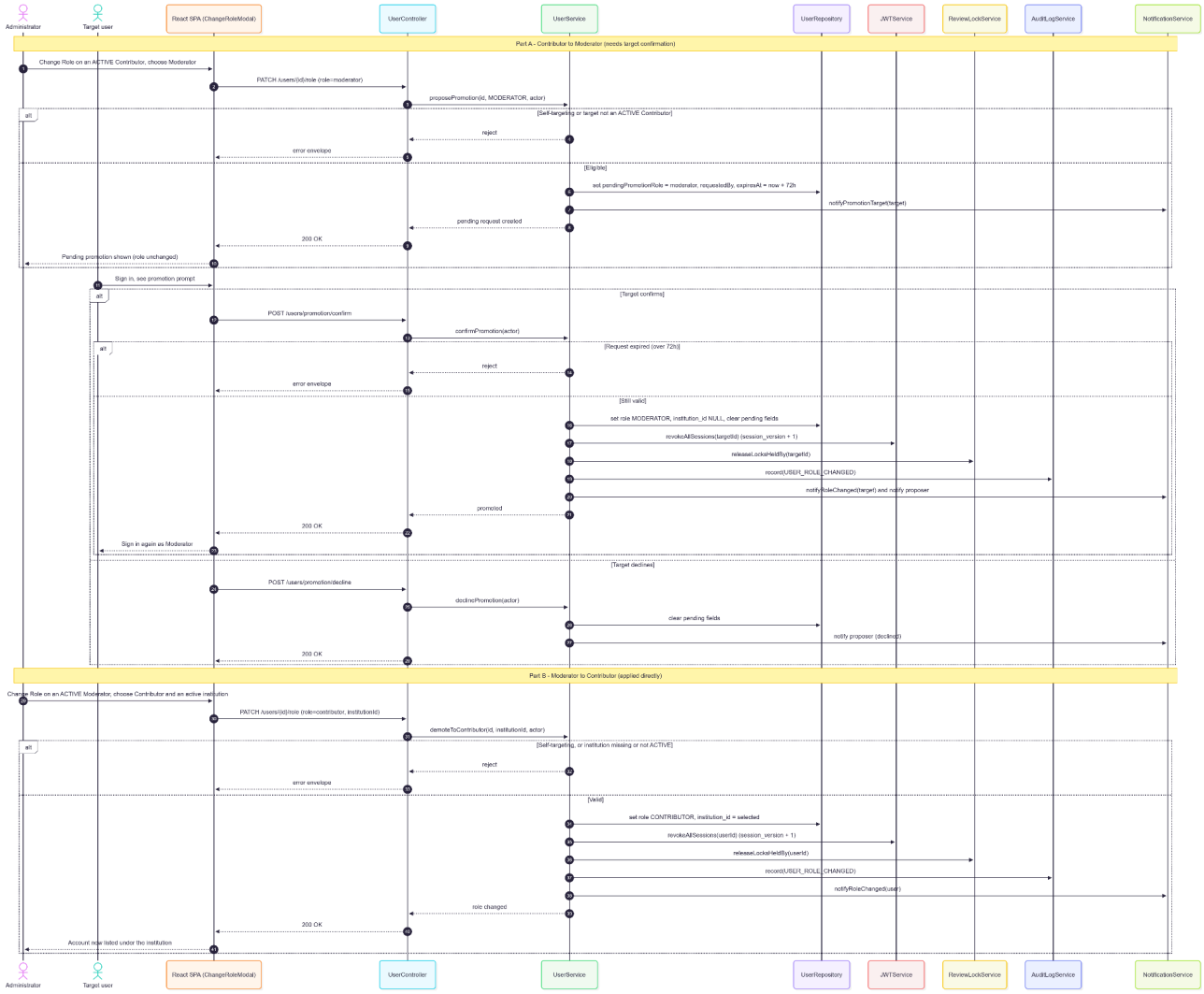
1.10.4. Object-Oriented Component(s)

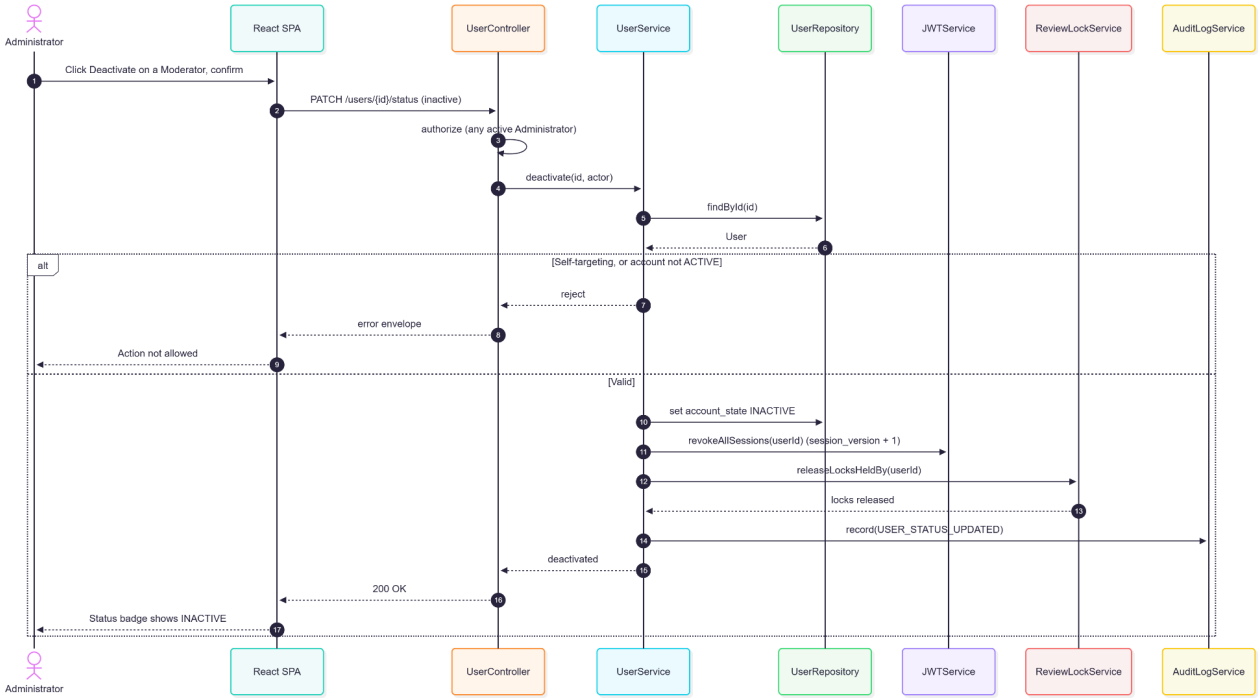
- **Class Diagram**



- **Sequence Diagram**

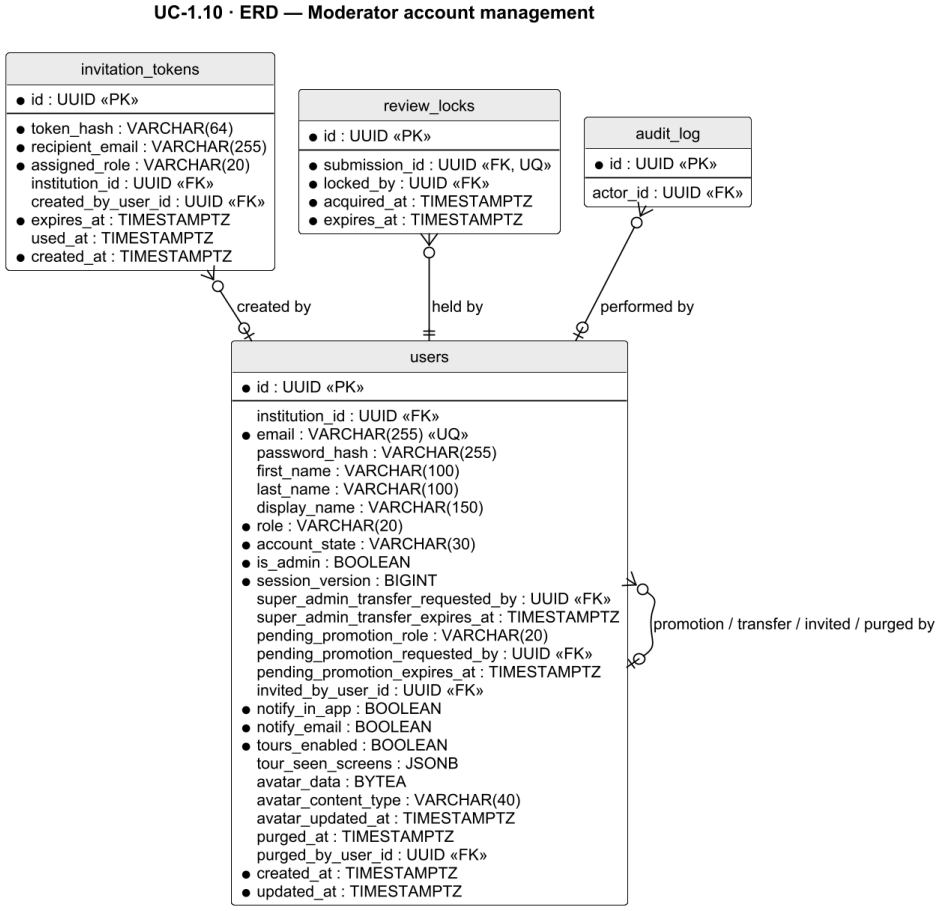

Administrator
Moderator (invitee)
React SPA
InvitationController
InvitationService
UserRepository
InvitationTokenRepository
EmailService
AuditLogService
Part A - Invite Moderator (network-wide, no institution)
1 Open Invite dialog, enter emails, role Moderator
2 POST /invitations (role=moderator, no institution)
3 authorize (Administrator only for Moderator role)
4 createInvitation(email, MODERATOR, null, actor)
5 findByEmail(email)
6 existing account or none
alt
[Existing account is ACTIVE or INACTIVE]
7 reject (conflict)
8 error envelope
9 Inline error
[New or reusable account]
10 generate 32-byte token, SHA-256 hash
11 supersede older open tokens
12 save(tokenHash, assignedRole = MODERATOR, institutionId = null, expiresAt = now + 72h)
13 save or reuse account as PENDING
14 sendInvitation(email, link with raw token)
alt
[Email delivery failed]
15 failure
16 set PENDING_EMAIL_UNDELIVERED
[Delivered]
17 success
18 invitation created
19 201 Created
20 Listed under pending Moderator invitations
Part B - Activation
21 Open /invite?token=...
22 GET /invitations/validate?token=...
23 validateToken(rawToken)
24 findByTokenHash(sha256(rawToken))
25 InvitationToken
alt
[Missing, expired or already used]
26 invalid
27 error envelope
28 Invalid or expired link
[Valid]
29 valid
30 Submit password
31 POST /invitations/accept (token, password)
32 acceptInvitation(rawToken, password)
33 set BCrypt hash, role MODERATOR, institution_id NULL, state ACTIVE
34 mark token used
35 record(INVITATION_ACCEPTED)
36 activated
37 200 OK
38 Redirect to login


Administrator
Target user
React SPA (ChangeRoleModal)
UserController
UserService
UserRepository
JWTService
ReviewLockService
AuditLogService
NotificationService
Part A - Contributor to Moderator (needs target confirmation)
Change Role on an ACTIVE Contributor, choose Moderator
PATCH /users/{id}/role {role=moderator}
proposePromotion(id, MODERATOR, actor)
alt
[Self-targeting or target not an ACTIVE Contributor]
reject
error envelope
[Eligible]
set pendingPromotionRole = moderator, requestedBy, expiresAt = now + 72h
notifyPromotionTarget(target)
pending request created
200 OK
Pending promotion shown (role unchanged)
Sign in, see promotion prompt
alt
[Target confirms]
POST /users/promotion/confirm
confirmPromotion(actor)
alt
[Request expired (over 72h)]
reject
error envelope
[Still valid]
set role MODERATOR, institution_id NULL, clear pending fields
revokeAllSessions(targetId) (session_version + 1)
releaseLocksHeldBy(targetId)
record(USER_ROLE_CHANGED)
notifyRoleChanged(target) and notify proposer
promoted
200 OK
Sign in again as Moderator
[Target declines]
POST /users/promotion/decline
declinePromotion(actor)
clear pending fields
notify proposer (declined)
200 OK
Part B - Moderator to Contributor (applied directly)
Change Role on an ACTIVE Moderator, choose Contributor and an active institution
PATCH /users/{id}/role {role=contributor, institutionId}
demoteToContributor(id, institutionId, actor)
alt
[Self-targeting, or institution missing or not ACTIVE]
reject
error envelope
[Valid]
set role CONTRIBUTOR, institution_id = selected
revokeAllSessions(userId) (session_version + 1)
releaseLocksHeldBy(userId)
record(USER_ROLE_CHANGED)
notifyRoleChanged(user)
role changed
200 OK
Account now listed under the institution



1.10.5. Data Design

- **Entity Relationship Diagram**



- **Schema**

```
        -- UC-1.10 reuses users and invitation_tokens (UC-1.1) and review_locks (UC-2.4);
        -- no new tables. Moderator accounts use role = 'moderator' with institution_id IS NULL
        -- (network-wide).


-- users (columns used by Moderator account management)
CREATE TABLE users (
    id                              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    institution_id                  UUID REFERENCES institutions(id),   -- NULL for moderators
    email                           VARCHAR(255) UNIQUE NOT NULL,
    password_hash                   VARCHAR(255),
    display_name                    VARCHAR(150),
    role                            VARCHAR(20) NOT NULL
                                    CHECK (role IN ('contributor', 'moderator', 'admin')),
    account_state                   VARCHAR(30) NOT NULL DEFAULT 'pending',
    session_version                 BIGINT NOT NULL DEFAULT 0,              -- incremented on deactivation
/ role change
    pending_promotion_role          VARCHAR(20)                             -- 'moderator' for Contributor
→ Moderator
                                    CHECK (pending_promotion_role IN ('admin', 'moderator')),
    pending_promotion_requested_by  UUID REFERENCES users(id),
    pending_promotion_expires_at    TIMESTAMPTZ,                            -- 72-hour confirmation window
    super_admin_transfer_requested_by UUID REFERENCES users(id),                            --    Moderator
as Admin Owner transfer target
    super_admin_transfer_expires_at   TIMESTAMPTZ,
    invited_by_user_id              UUID REFERENCES users(id),
    purged_at                       TIMESTAMPTZ,
    purged_by_user_id               UUID REFERENCES users(id),
    created_at                      TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at                      TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- Moderator invitations carry no institution
CREATE TABLE invitation_tokens (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    token_hash      VARCHAR(64) NOT NULL,
    recipient_email VARCHAR(255) NOT NULL,
    assigned_role   VARCHAR(20) NOT NULL
                    CHECK (assigned_role IN ('contributor', 'moderator', 'admin')),
    institution_id  UUID REFERENCES institutions(id),  -- NULL for moderator invitations
    created_by_user_id UUID REFERENCES users(id),           -- scopes "manage own invitations"
    expires_at      TIMESTAMPTZ NOT NULL,
    used_at         TIMESTAMPTZ,
    created_at      TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- Review locks released when a Moderator is deactivated or changes role
CREATE TABLE review_locks (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    submission_id UUID NOT NULL UNIQUE REFERENCES submissions(id),
    locked_by       UUID NOT NULL REFERENCES users(id),
    acquired_at   TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    expires_at      TIMESTAMPTZ NOT NULL            -- acquired_at + 15 minutes
);

CREATE INDEX idx_users_email               ON users(email);
CREATE INDEX idx_invitation_tokens_created_by ON invitation_tokens(created_by_user_id);
CREATE INDEX idx_review_locks_expiry       ON review_locks(expires_at);
```

2. Module 2

Module 2 delivers the institution-scoped media library (upload, nested albums, AI enrichment, semantic search, lifecycle and deletion), the network-wide Moderator/Administrator approval workflow with failed-publication recovery, and automated brand watermarking at publish time.

2.1. Library Uploads & Albums

2.1.1. User Interface Design

Upload Modal (Media Repository)

Multi-file drag-and-drop accepting JPEG, PNG, WebP, GIF, MP4, MOV, and WebM up to 50 MB each, with per-file progress and inline errors. Before confirming, the actor sets an album (existing, new, nested folder path, or Auto-Match) and at least one tag; a tooltip suggests the event name as a tag. Images show a "Processing…" badge until AI processing finishes; videos are READY immediately.

Album Auto-Match (library upload)

Text-based (tag and filename overlap): confident (≥0.6) auto-fills with a reasons badge, ambiguous (≥0.3) lists up to 3 candidates, otherwise manual entry with a notice.

Album Navigation

Folder-style nested albums (parent/child) with breadcrumbs, album cards (name, asset count), rename, create, move (re-parent, cycle-guarded), and delete. A shared default-institution library is visible to all Contributors.

Move to Album

From the asset menu; an asset always belongs to exactly one album (no "unfiled" option). Cross-institution moves follow role rules.

2.1.2. Front-end Component(s)

MediaRepositoryScreen / MediaToolbar / AlbumCard / AssetCard

- Grid/list browsing and album navigation; `GET /media-assets/albums`, `POST /media-assets/albums`, `POST /media-assets/albums/{id}` (rename), `POST /media-assets/albums/ensure-path`, `PATCH /media-assets/albums/{id}/parent`, `DELETE /media-assets/albums/{id}`.
- React functional components / Signed-in route

UploadModal

- `POST /media-assets/upload-url` → browser PUT to R2 → `POST /media-assets/upload` with album and tags.
- React modal component

AlbumCombobox — shared with UC-1.7 (`suggestions` prop for ranked candidates).

2.1.3. Back-end Component(s)

MediaAssetController

- Upload, album, move (`POST /media-assets/{id}/album`), and tag endpoints (`POST /media-assets/{id}/tags`, `DELETE /media-assets/{id}/tags/{tagId}`). `isAuthenticated()` — all three roles.
- Spring `@RestController`

MediaAssetService

- Validates format/size, requires ≥1 manual tag at upload and keeps ≥1 manual tag on removal (AI tags do not count), computes SHA-256 `content_hash`, generates `asset_code`, enforces album rules (no null album, cycle guard), and applies cross-institution move rules (Contributor: own uploads, own institution or shared library; Moderator/Admin: any institution). Queues image processing; videos are marked READY without classification.
- Spring `@Service`

MediaStorageService — R2 presign, upload verification, proxy URL resolution.

MediaProcessingQueueService / MediaProcessingWorker

- DB-backed job queue (media_processing_jobs) with leases, retries, max_attempts, and dead letters. Job types allowed by the database (V117): `RETRIEVAL_EMBEDDINGS, CLASSIFY_AND_EMBED, EMBED_IMAGE_ONLY, BUILD_SUBMISSION_CONTEXT`. Status values: `PENDING, PROCESSING, RETRY, COMPLETED, DEAD` (default 5 attempts, app.media-processing.max-attempts). Administrators list and retry dead letters through MediaProcessingAdminController (`GET/system-health/media-processing/dead-letters, POST /system-health/media-processing/dead-letters/{jobId}/retry, audited MEDIA_PROCESSING_DEAD_LETTER_RETRIED`).
- Spring `@Service` / `@Scheduled` worker

AIClassificationService

- Google Gemini classification (`GeminiVisionClient`, model gemini-3.8-flash by default) into the 14-category taxonomy and an asset type, with description, up to 30 AI tags, and enrichment for scenes, objects, activities, OCR, people-count range, and event hypotheses. For `CLASSIFY_AND_EMBED` jobs it sets `READY` once classification and both Voyage embeddings succeed and `FAILED` otherwise; for `RETRIEVAL_EMBEDDINGS` jobs Gemini enrichment is optional and does not change readiness.
- Spring `@Service`

MediaImageEmbeddingService / MediaSemanticEmbeddingService

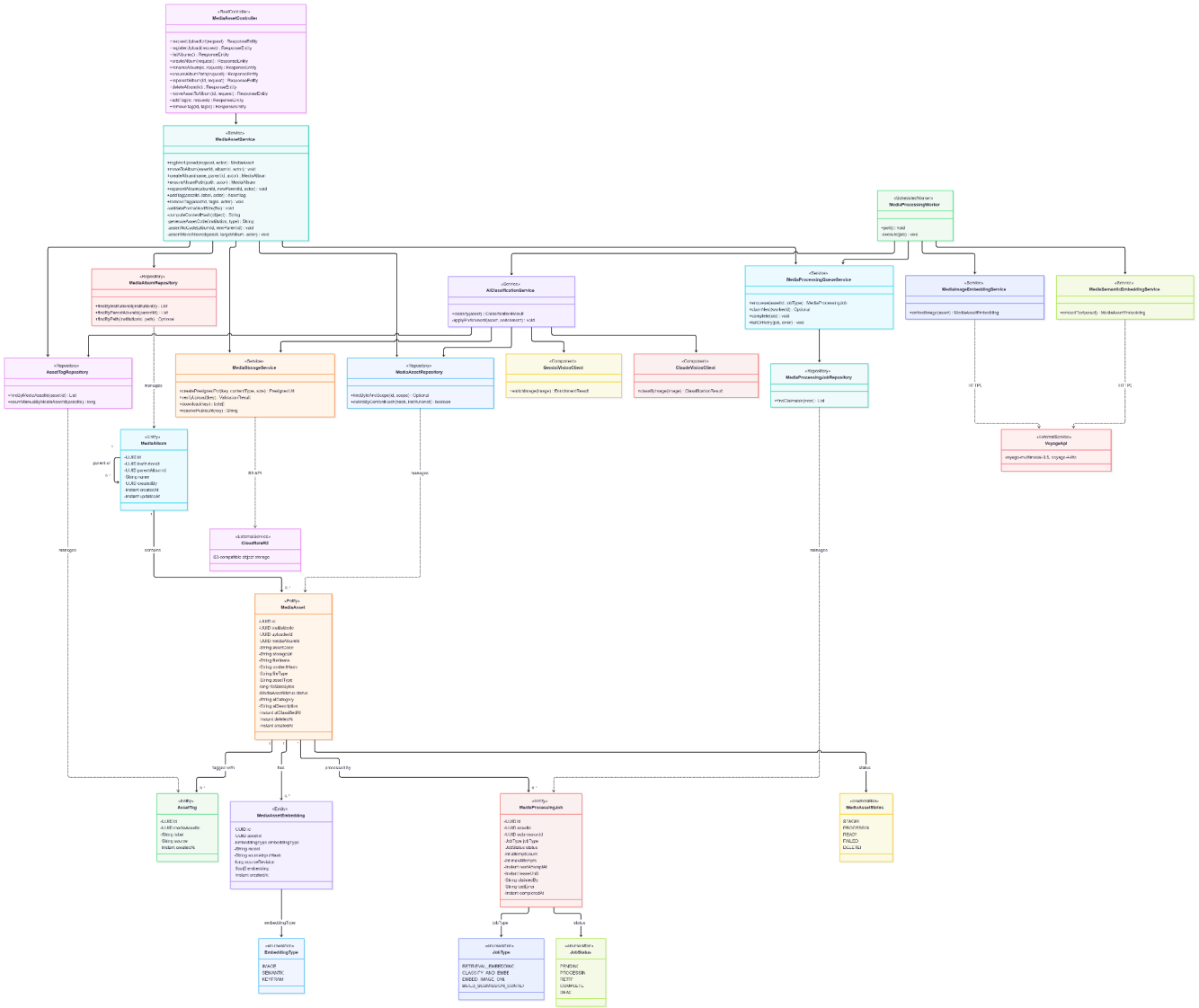
- Voyage `voyage-multimodal-3.5` image embeddings and `voyage-4-lite` semantic text embeddings (1024 dimensions) stored in `media_asset_embeddings`
- Spring `@Service`

MediaAlbumRepository / AssetTagRepository / MediaAssetRepository

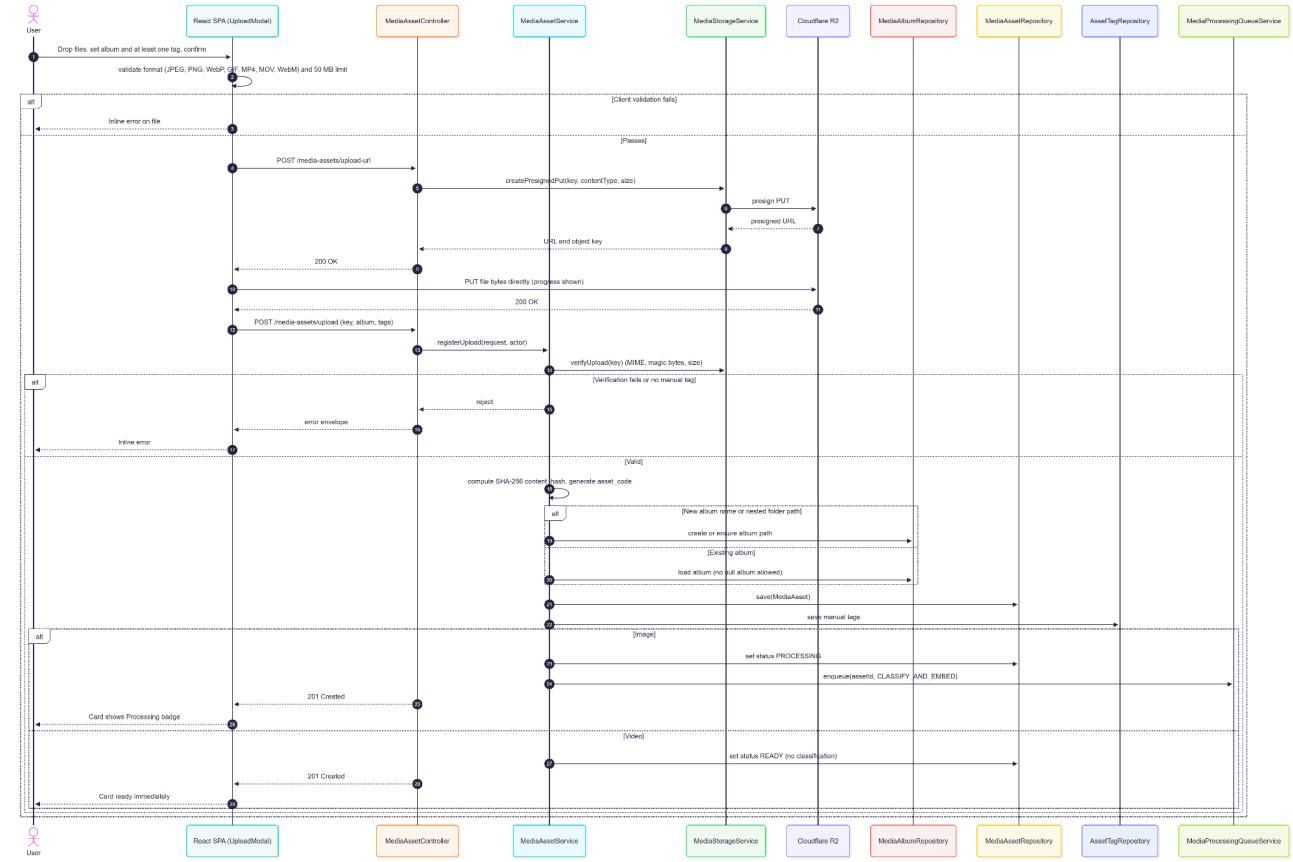
- Spring `@Repository`. Native queries that hydrate `MediaAsset` list every mapped column explicitly.

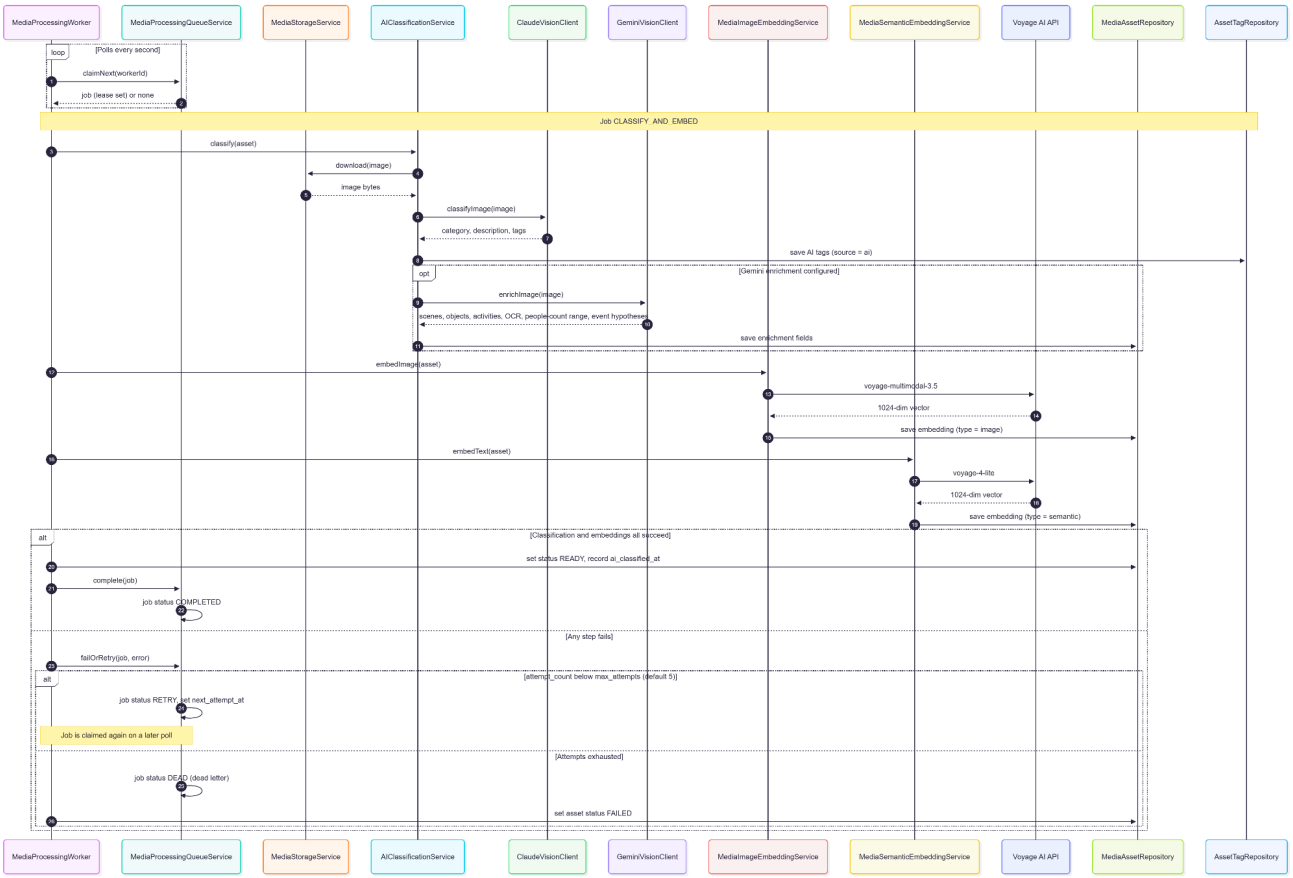
2.1.4. Object-Oriented Component(s)

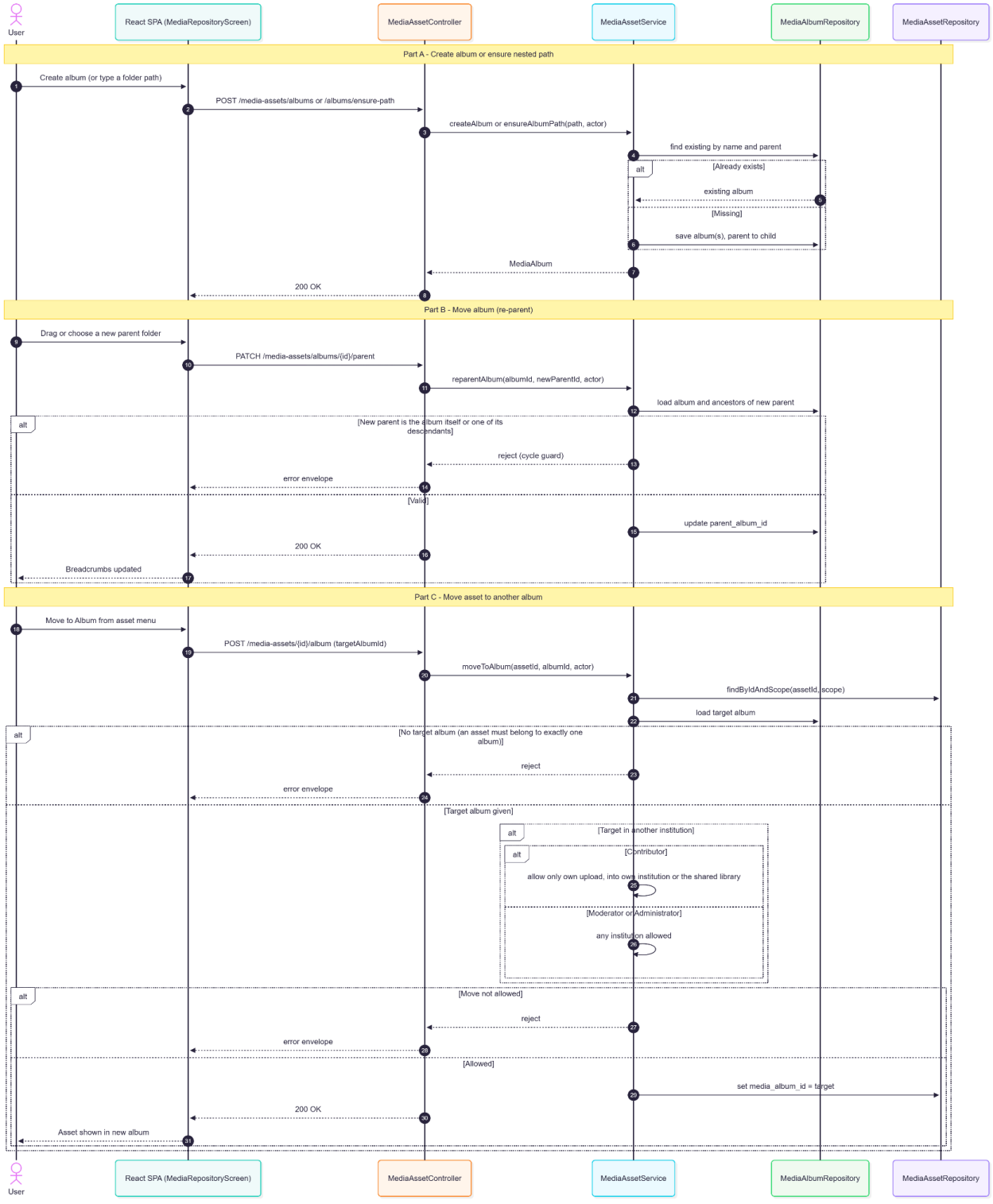
- **Class Diagram**

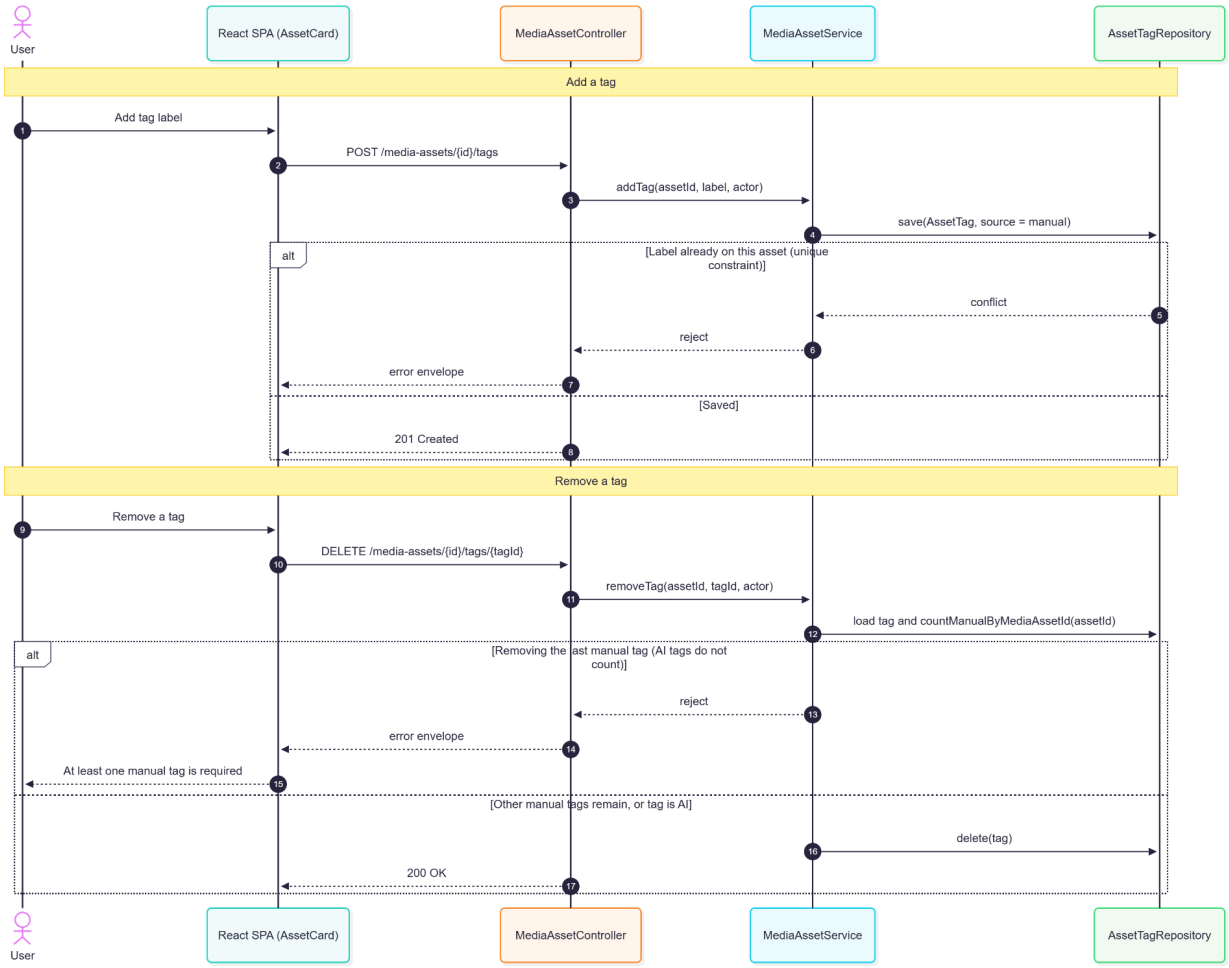


- **Sequence Diagram**


User
React SPA (UploadModal)
MediaAssetController
MediaAssetService
MediaStorageService
Cloudflare R2
MediaAlbumRepository
MediaAssetRepository
AssetTagRepository
MediaProcessingQueueService
Drop files, set album and at least one tag, confirm
validate format (JPEG, PNG, WebP, GIF, MP4, MOV, WebM) and 50 MB limit
alt
[Client validation fails]
Inline error on file
[Passes]
POST /media-assets/upload-url
createPresignedPut(key, contentType, size)
presign PUT
presigned URL
URL and object key
200 OK
PUT file bytes directly (progress shown)
200 OK
POST /media-assets/upload (key, album, tags)
registerUpload(request, actor)
verifyUpload(key) (MIME, magic bytes, size)
alt
[Verification fails or no manual tag]
reject
error envelope
Inline error
[Valid]
compute SHA-256 content hash, generate asset_code
alt
[New album name or nested folder path]
create or ensure album path
[Existing album]
load album (no null album allowed)
save(MediaAsset)
save manual tags
alt
[Image]
set status PROCESSING
enqueue(assetId, CLASSIFY_AND_EMBED)
201 Created
Card shows Processing badge
[Video]
set status READY (no classification)
201 Created
Card ready immediately

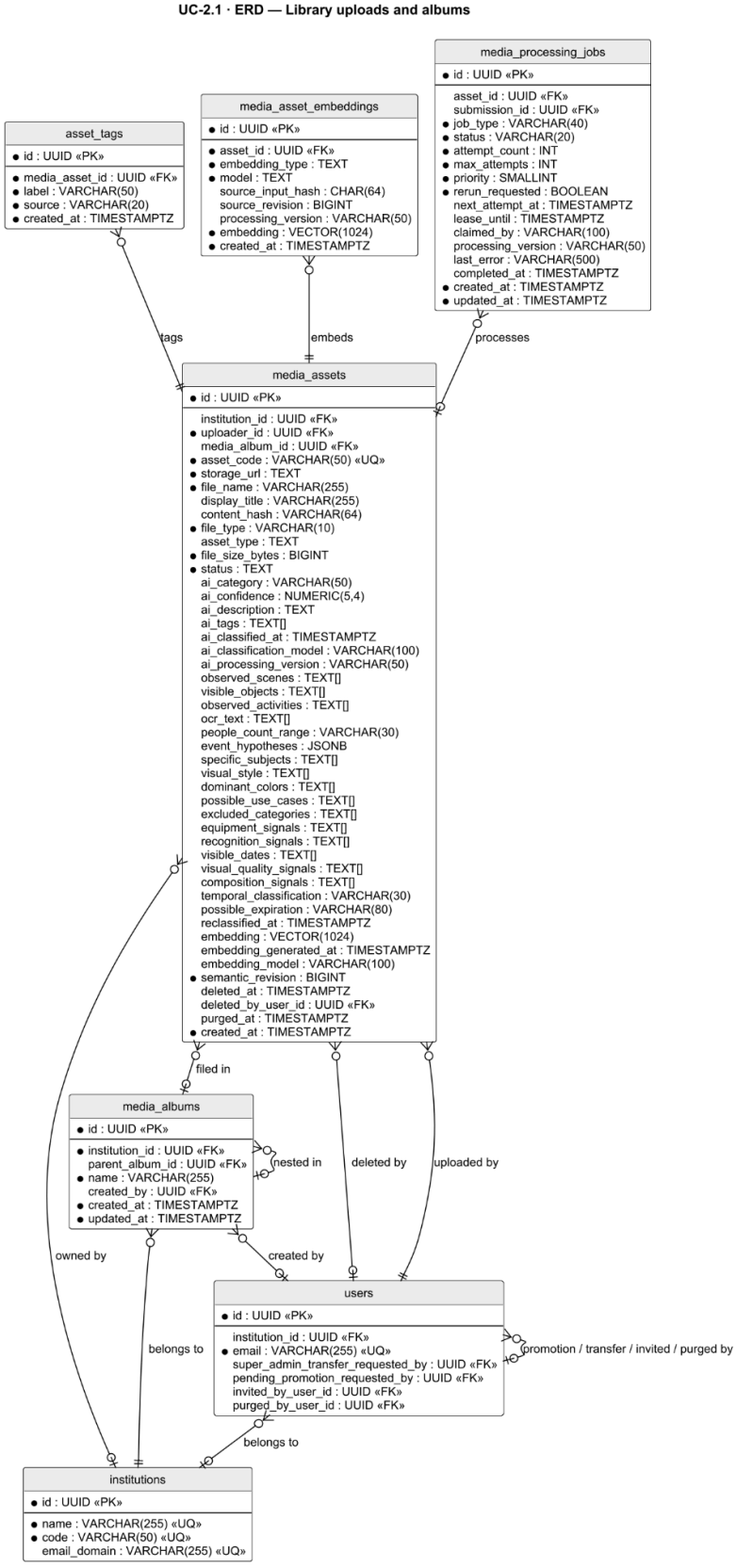

MediaProcessingWorker
MediaProcessingQueueService
MediaStorageService
AIClassificationService
ClaudeVisionClient
GeminiVisionClient
MediaImageEmbeddingService
MediaSemanticEmbeddingService
Voyage AI API
MediaAssetRepository
AssetTagRepository
loop
[Polls every second]
claimNext(workerId)
job (lease set) or none
Job CLASSIFY_AND_EMBED
classify(asset)
download(image)
image bytes
classifyImage(image)
category, description, tags
save AI tags (source = ai)
opt
[Gemini enrichment configured]
enrichImage(image)
scenes, objects, activities, OCR, people-count range, event hypotheses
save enrichment fields
embedImage(asset)
voyage-multimodal-3.5
1024-dim vector
save embedding (type = image)
embedText(asset)
voyage-4-lite
1024-dim vector
save embedding (type = semantic)
alt
[Classification and embeddings all succeed]
set status READY, record ai_classified_at
complete(job)
job status COMPLETED
[Any step fails]
failOrRetry(job, error)
alt
[attempt_count below max_attempts (default 5)]
job status RETRY, set next_attempt_at
Job is claimed again on a later poll
[Attempts exhausted]
job status DEAD (dead letter)
set asset status FAILED


User
React SPA (MediaRepositoryScreen)
MediaAssetController
MediaAssetService
MediaAlbumRepository
MediaAssetRepository
Part A - Create album or ensure nested path
1 Create album (or type a folder path)
2 POST /media-assets/albums or /albums/ensure-path
3 createAlbum or ensureAlbumPath(path, actor)
4 find existing by name and parent
alt
[Already exists]
5 existing album
[Missing]
6 save album(s), parent to child
7 MediaAlbum
8 200 OK
Part B - Move album (re-parent)
9 Drag or choose a new parent folder
10 PATCH /media-assets/albums/{id}/parent
11 reparentAlbum(albumId, newParentId, actor)
12 load album and ancestors of new parent
alt
[New parent is the album itself or one of its descendants]
13 reject (cycle guard)
14 error envelope
[Valid]
15 update parent_album_id
16 200 OK
17 Breadcrumbs updated
Part C - Move asset to another album
18 Move to Album from asset menu
19 POST /media-assets/{id}/album (targetAlbumId)
20 moveToAlbum(assetId, albumId, actor)
21 findByIdAndScope(assetId, scope)
22 load target album
alt
[No target album (an asset must belong to exactly one album)]
23 reject
24 error envelope
[Target album given]
alt
[Target in another institution]
alt
[Contributor]
25 allow only own upload, into own institution or the shared library
[Moderator or Administrator]
26 any institution allowed
alt
[Move not allowed]
27 reject
28 error envelope
[Allowed]
29 set media_album_id = target
30 200 OK
31 Asset shown in new album



2.1.5. Data Design

- **Entity Relationship Diagram**



- **Schema**

```
CREATE TABLE media_albums (
```

```
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    institution_id  UUID NOT NULL REFERENCES institutions(id),
    parent_album_id UUID REFERENCES media_albums(id),   -- nested folders
    name            VARCHAR(255) NOT NULL,
    created_by      UUID REFERENCES users(id),
    created_at      TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at      TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE TABLE media_assets (
    id                      UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    institution_id          UUID REFERENCES institutions(id),       -- NULL while STAGED; bound on submit
(V73)
    uploader_id             UUID NOT NULL REFERENCES users(id),
    media_album_id          UUID REFERENCES media_albums(id),
    asset_code              VARCHAR(50) UNIQUE NOT NULL,    -- e.g. CITU-IMG-0042
    storage_url             TEXT NOT NULL,                  -- backend proxy URL
    file_name               VARCHAR(255) NOT NULL,
    display_title           VARCHAR(255),                   -- editable, display-only (V90)
    content_hash            VARCHAR(64),                    -- SHA-256 duplicate detection (V92)
    file_type               VARCHAR(10) NOT NULL
                            CHECK (file_type IN ('jpeg','png','webp','gif','mp4','mov','webm')),
    asset_type              TEXT,                   -- image | video
    file_size_bytes         BIGINT NOT NULL,
    status                  TEXT NOT NULL,          -- STAGED | PROCESSING | READY | FAILED | DELETED
    ai_category             VARCHAR(50),
    ai_confidence           NUMERIC(5,4),
    ai_description          TEXT,
    ai_tags                 TEXT[],
    ai_classified_at        TIMESTAMPTZ,
    ai_classification_model VARCHAR(100),
    ai_processing_version   VARCHAR(50),
    observed_scenes         TEXT[],                         -- Gemini enrichment fields
    visible_objects         TEXT[],
    observed_activities     TEXT[],
    ocr_text                TEXT[],
    people_count_range      VARCHAR(30),
    event_hypotheses        JSONB,
    specific_subjects       TEXT[],
    visual_style            TEXT[],
    dominant_colors         TEXT[],
    possible_use_cases      TEXT[],
    excluded_categories     TEXT[],
    equipment_signals       TEXT[],
    recognition_signals     TEXT[],
    visible_dates           TEXT[],
    visual_quality_signals  TEXT[],
    composition_signals     TEXT[],
    temporal_classification VARCHAR(30),
    possible_expiration     VARCHAR(80),
    reclassified_at         TIMESTAMPTZ,
    embedding               VECTOR(1024),                   -- legacy; superseded by
media_asset_embeddings
    embedding_generated_at  TIMESTAMPTZ,
    embedding_model         VARCHAR(100),
    semantic_revision       BIGINT NOT NULL DEFAULT 0,
    deleted_at              TIMESTAMPTZ,                    -- soft delete
    deleted_by_user_id      UUID REFERENCES users(id),
    purged_at               TIMESTAMPTZ,
    created_at              TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE TABLE asset_tags (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    media_asset_id UUID NOT NULL REFERENCES media_assets(id) ON DELETE CASCADE,
    label           VARCHAR(50) NOT NULL,
    source          VARCHAR(20) NOT NULL DEFAULT 'manual',   -- manual | ai
    created_at      TIMESTAMPTZ NOT NULL DEFAULT NOW(),
```

```
    UNIQUE (media_asset_id, label)
);

CREATE TABLE media_asset_embeddings (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    asset_id        UUID NOT NULL REFERENCES media_assets(id) ON DELETE CASCADE,
    embedding_type TEXT NOT NULL CHECK (embedding_type IN ('image','semantic','keyframe')),
    model           TEXT NOT NULL,
    source_input_hash       CHAR(64),
    source_revision         BIGINT,
    processing_version      VARCHAR(50),
    embedding       VECTOR(1024) NOT NULL,
    created_at      TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    UNIQUE (asset_id, embedding_type)
);

CREATE TABLE media_processing_jobs (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    asset_id        UUID REFERENCES media_assets(id) ON DELETE CASCADE,
    submission_id   UUID REFERENCES submissions(id) ON DELETE CASCADE,
    job_type        VARCHAR(40) NOT NULL
                    CHECK (job_type IN ('RETRIEVAL_EMBEDDINGS', 'CLASSIFY_AND_EMBED',
                                        'EMBED_IMAGE_ONLY', 'BUILD_SUBMISSION_CONTEXT')),
    status          VARCHAR(20) NOT NULL
                    CHECK (status IN ('PENDING','PROCESSING','RETRY','COMPLETED','DEAD')),
    attempt_count   INT NOT NULL DEFAULT 0,
    max_attempts    INT NOT NULL,
    priority                SMALLINT NOT NULL DEFAULT 10,
    rerun_requested         BOOLEAN NOT NULL DEFAULT FALSE,
    next_attempt_at TIMESTAMPTZ,
    lease_until     TIMESTAMPTZ,
    claimed_by      VARCHAR(100),
    processing_version VARCHAR(50),
    last_error      VARCHAR(500),
    completed_at    TIMESTAMPTZ,
    created_at      TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at      TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_media_assets_institution ON media_assets(institution_id) WHERE deleted_at IS NULL;
CREATE INDEX idx_media_assets_media_album_id ON media_assets(media_album_id);
CREATE INDEX idx_media_asset_embeddings_vector ON media_asset_embeddings
    USING hnsw (embedding vector_cosine_ops) WITH (m = 16, ef_construction = 64);
```

2.2. Semantic Search & Filtering

2.2.1. User Interface Design

Media Repository Screen (`/media-repository`)

A grid/list of assets scoped to the actor's visibility: Contributors see their institution and the shared network-default institution; Moderators and Administrators see the whole network (Network View is always on for them) with an institution filter and institution badges. Each card shows thumbnail or video poster, asset code, editable display title, upload date, size/type, and a Processing badge or AI category. Controls: keyword/semantic search, album filter, institution filter (Moderator/Admin), sort (Newest, Oldest, Name, Size), grid/list toggle, and a result strip with match count and clearable filter chips. An empty state appears when nothing matches.

Asset Detail Panel

Media preview (lightbox zoom or video player), read-only asset code, click-to-edit title (Enter/blur saves, Escape cancels), metadata grid (filename, uploader, institution, upload date, album, size,

resolution/duration, type), AI Tags with confidence, "Your tags" with add/remove, Used In history (UC-2.3), and actions: Use in New Post, Add to Existing Draft (all roles), Download, Delete.

2.2.2. Front-end Component(s)

MediaRepositoryScreen / useMediaAssets

- `GET /media-assets` (keyword, album, institution, sort, pagination) and `GET /media-assets/search` (semantic).
- React functional component / hook

AssetDetailPanel / AssetLightbox

- `GET /media-assets/{id}`, `POST /media-assets/{id}/title`, tag endpoints, history (UC-2.3).
- React functional components

AddToDraftModal

- Lists the actor's drafts and attaches via `POST /submissions/{id}/assets`; with no drafts, offers a new post. Reports partial success at the 10-asset limit.
- React modal component

MediaLibraryTab (composer and review picker)

- Library picker with institution filter for Moderator/Admin (defaults to the submission's institution).
- React functional component

2.2.3. Back-end Component(s)

MediaAssetController

- `GET /media-assets`, `GET /media-assets/search`, `GET /media-assets/{id}`, `POST /media-assets/{id}/title`.
- Spring `@RestController`

MediaAssetService

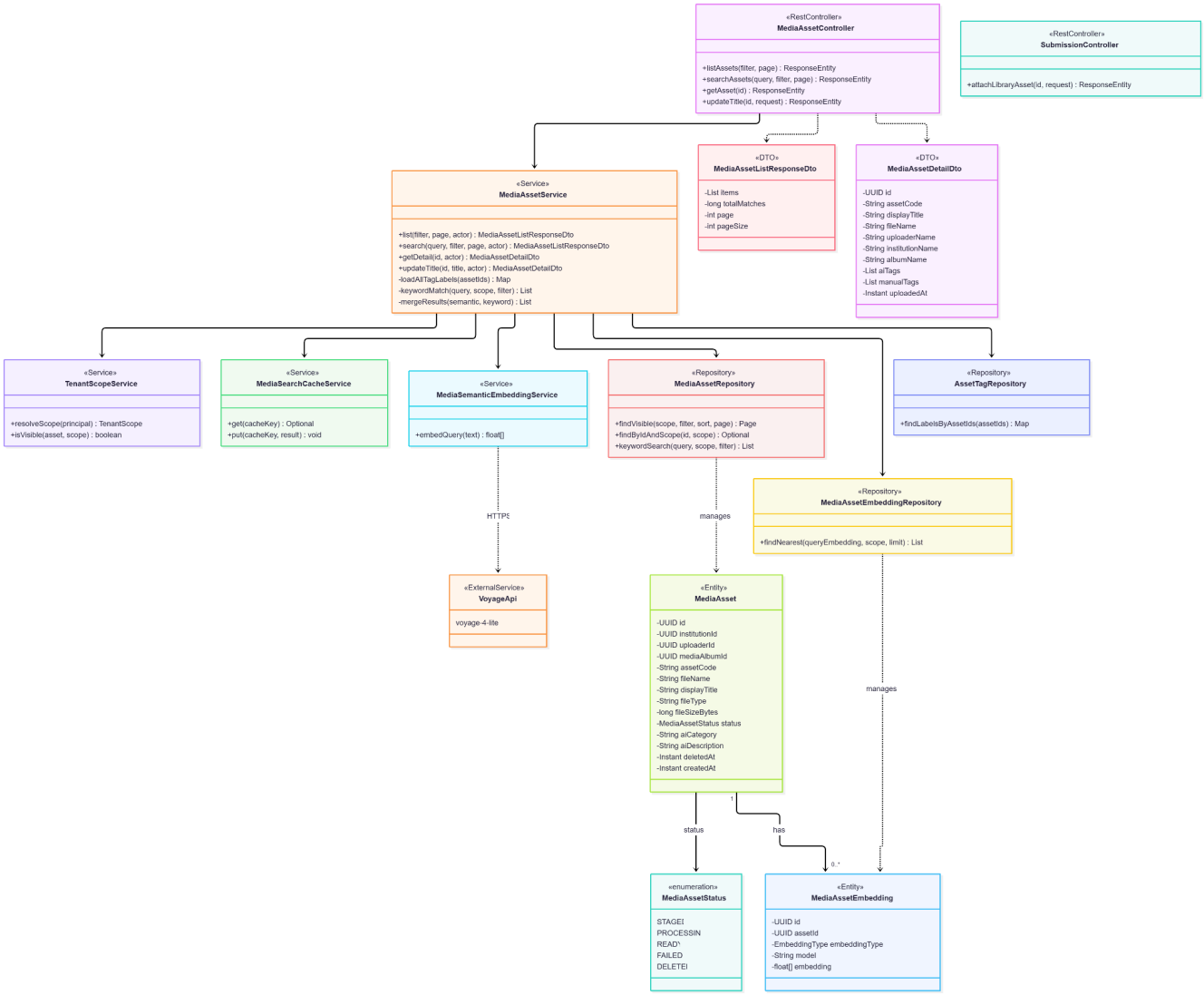
- Visibility scoping via `TenantScopeService`; keyword matching across filename, display title, asset code, uploader email, AI metadata, and all tags (manual and AI, batch-loaded by `loadAllTagLabels`). A cross-institution request outside visibility returns `404`.
- Spring `@Service`

MediaSemanticEmbeddingService / MediaSearchCacheService

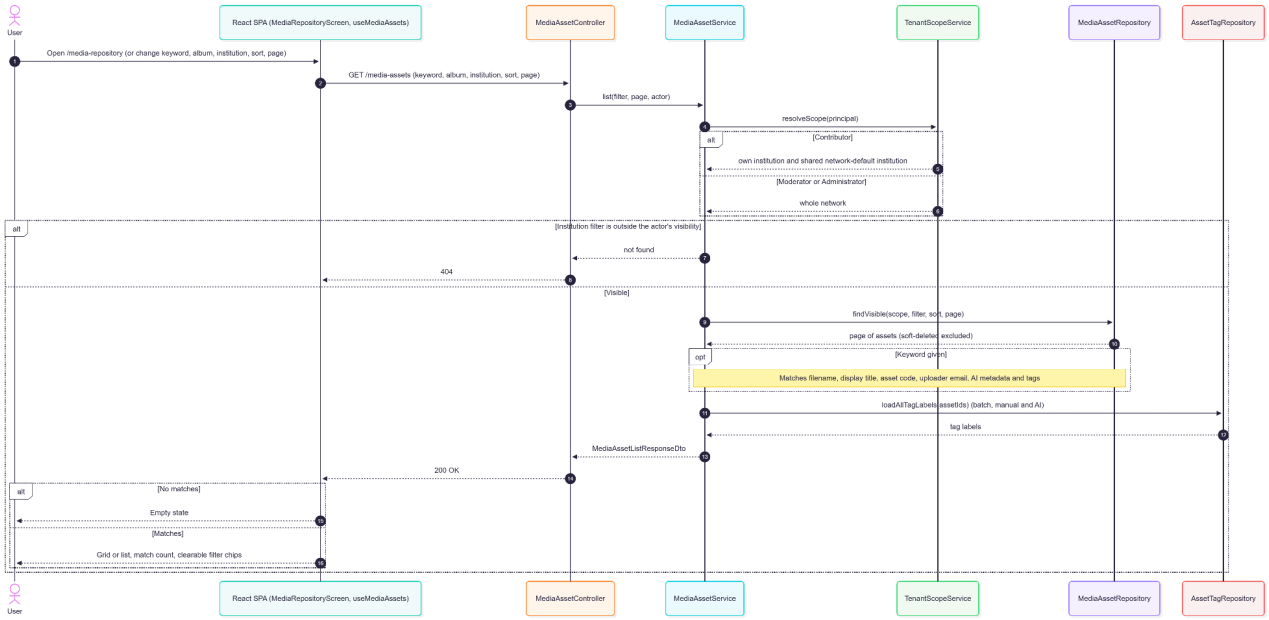
- Voyage query embedding + pgvector ranking, with keyword fallback/supplement; short-lived search result caching. Embeddings are not recomputed on tag or title edits.
- Spring `@Service`

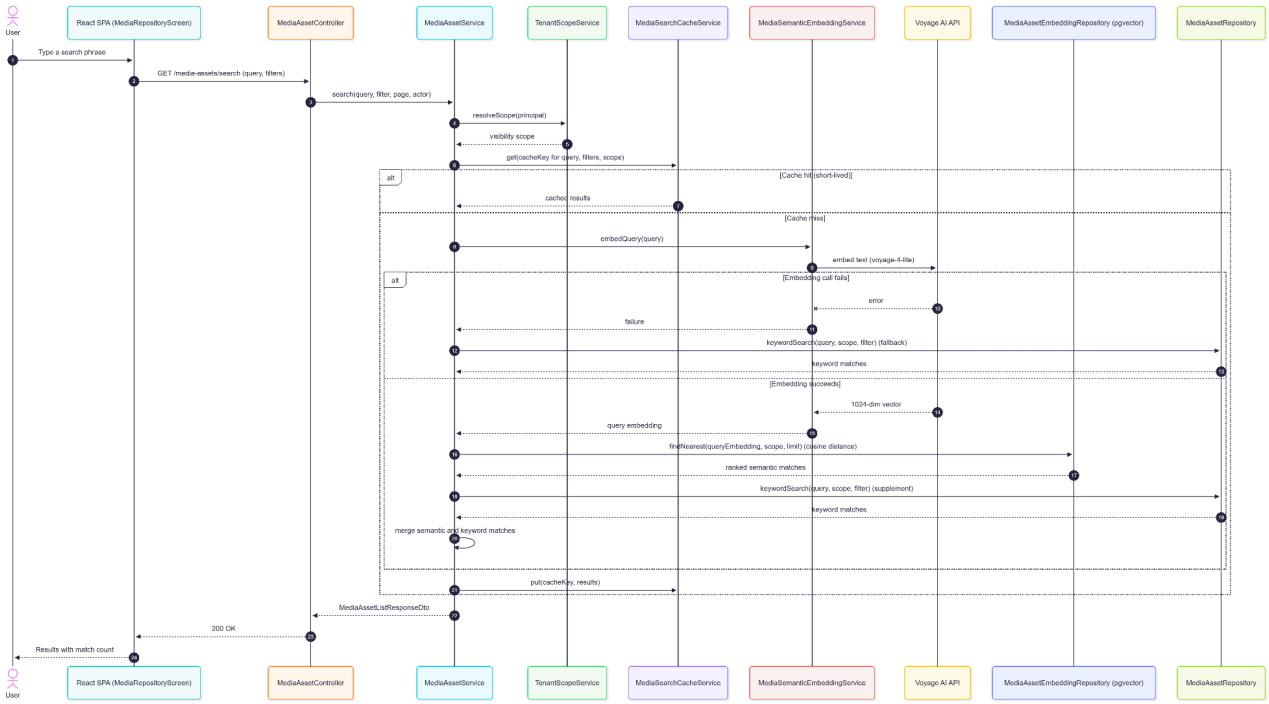
2.2.4. Object-Oriented Component(s)

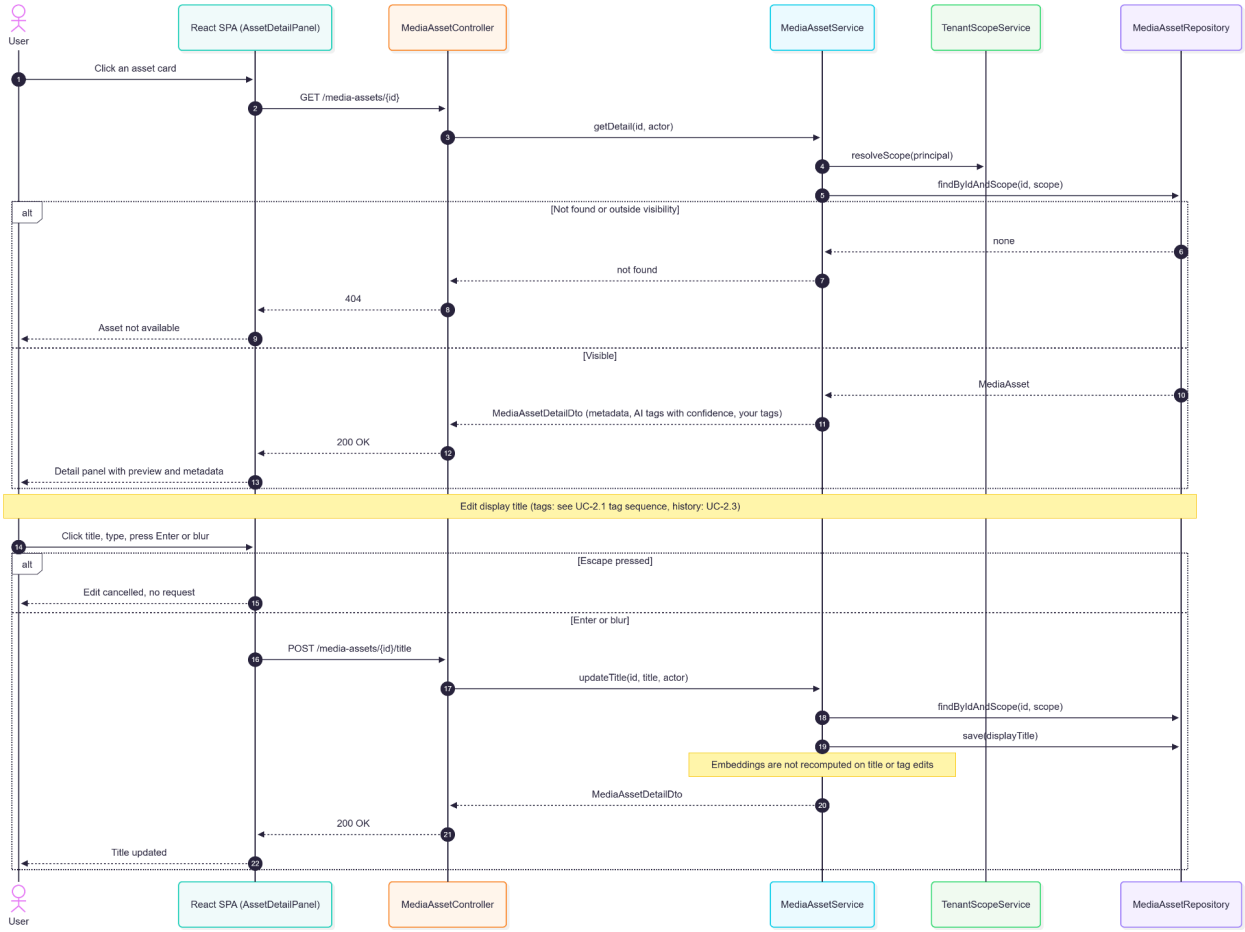
- **Class Diagram**

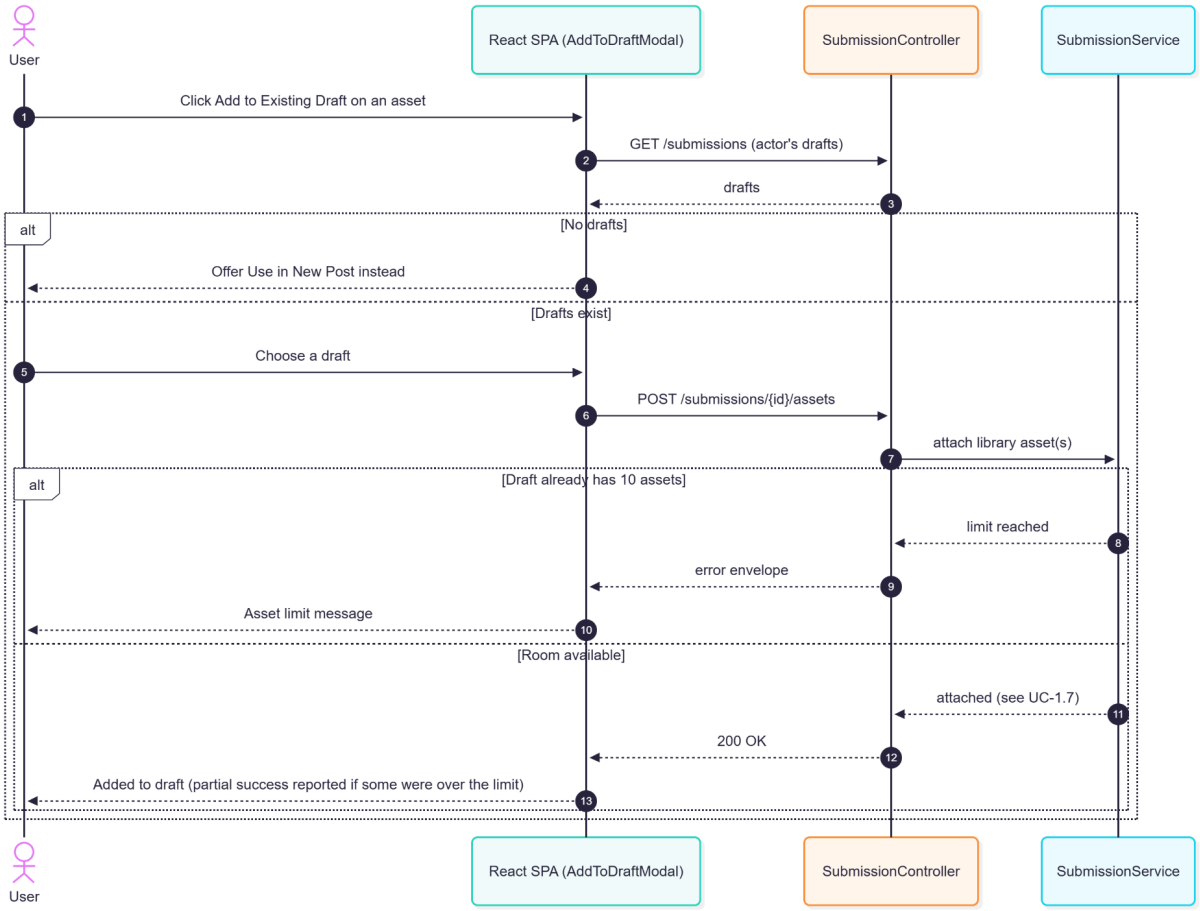


- **Sequence Diagram**


User
React SPA (MediaRepositoryScreen, useMediaAssets)
MediaAssetController
MediaAssetService
TenantScopeService
MediaAssetRepository
AssetTagRepository
Open /media-repository (or change keyword, album, institution, sort, page)
GET /media-assets (keyword, album, institution, sort, page)
list(filter, page, actor)
resolveScope(principal)
alt
[Contributor]
own institution and shared network-default institution
[Moderator or Administrator]
whole network
alt
[institution filter is outside the actor's visibility]
not found
404
[Visible]
findVisible(scope, filter, sort, page)
page of assets (soft-deleted excluded)
opt
[Keyword given]
Matches filename, display title, asset code, uploader email, AI metadata and tags
loadAllTagLabels(assetIds) (batch, manual and AI)
tag labels
MediaAssetListResponseDto
200 OK
alt
[No matches]
Empty state
[Matches]
Grid or list, match count, clearable filter chips


User
React SPA (MediaRepositoryScreen)
MediaAssetController
MediaAssetService
TenantScopeService
MediaSearchCacheService
MediaSemanticEmbeddingService
Voyage AI API
MediaAssetEmbeddingRepository (pgvector)
MediaAssetRepository
Type a search phrase
GET /media-assets/search (query, filters)
search(query, filter, page, actor)
resolveScope(principal)
visibility scope
get(cacheKey for query, filters, scope)
alt
[Cache hit (short-lived)]
cached results
[Cache miss]
embedQuery(query)
embed text (voyage-4-lite)
alt
[Embedding call fails]
error
failure
keywordSearch(query, scope, filter) (fallback)
keyword matches
[Embedding succeeds]
1024-dim vector
query embedding
findNearest(queryEmbedding, scope, limit) (cosine distance)
ranked semantic matches
keywordSearch(query, scope, filter) (supplement)
keyword matches
merge semantic and keyword matches
put(cacheKey, results)
MediaAssetListResponseDto
200 OK
Results with match count

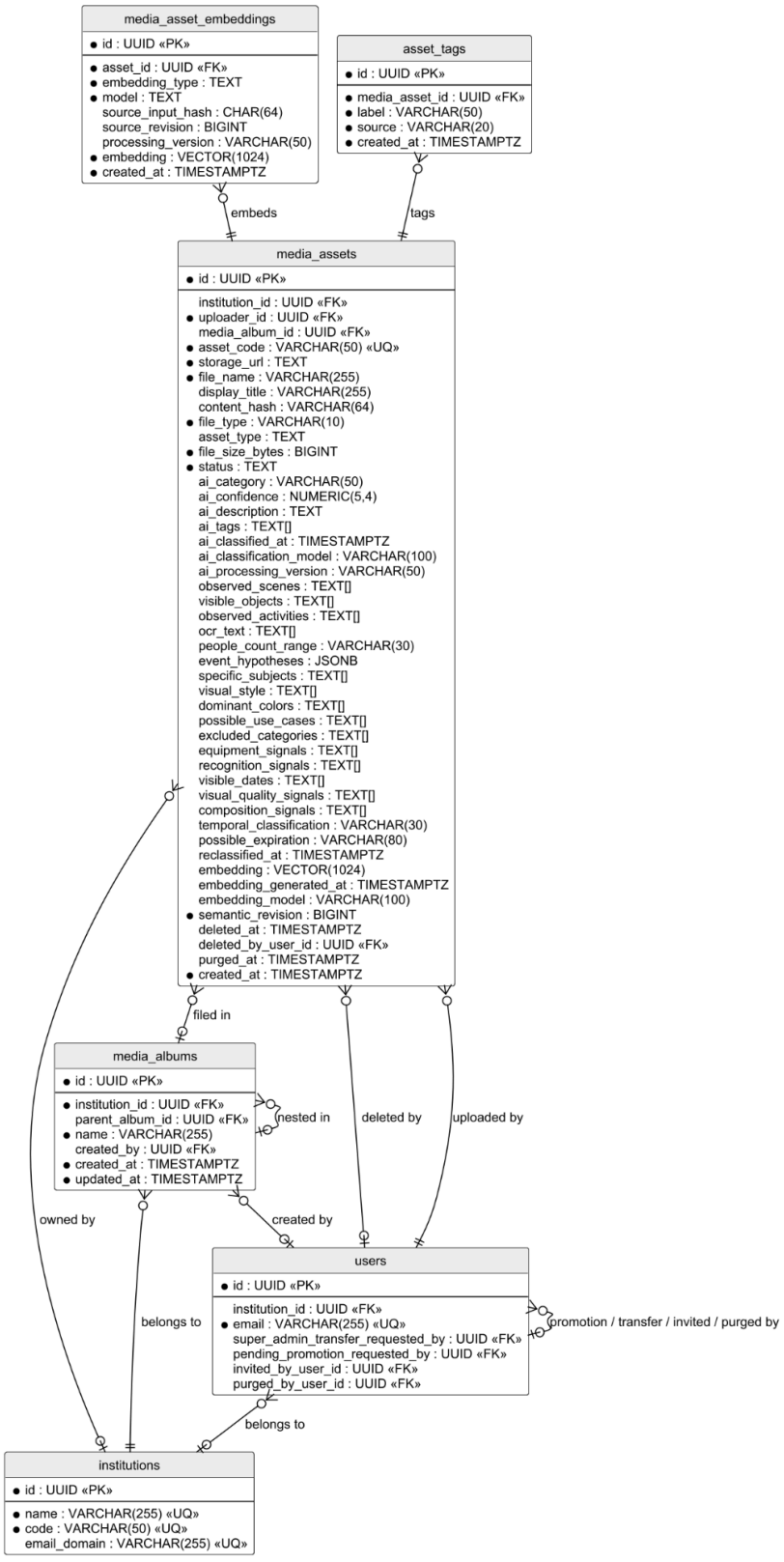

User
React SPA (AssetDetailPanel)
MediaAssetController
MediaAssetService
TenantScopeService
MediaAssetRepository
Click an asset card
GET /media-assets/{id}
getDetail(id, actor)
resolveScope(principal)
findByIdAndScope(id, scope)
alt
[Not found or outside visibility]
none
not found
404
Asset not available
[Visible]
MediaAsset
MediaAssetDetailDto (metadata, AI tags with confidence, your tags)
200 OK
Detail panel with preview and metadata
Edit display title (tags: see UC-2.1 tag sequence, history: UC-2.3)
Click title, type, press Enter or blur
alt
[Escape pressed]
Edit cancelled, no request
[Enter or blur]
POST /media-assets/{id}/title
updateTitle(id, title, actor)
findByIdAndScope(id, scope)
save(displayTitle)
Embeddings are not recomputed on title or tag edits
MediaAssetDetailDto
200 OK
Title updated
User
React SPA (AssetDetailPanel)
MediaAssetController
MediaAssetService
TenantScopeService
MediaAssetRepository



2.2.5. Data Design

- **Entity Relationship Diagram**

UC-2.2 · ERD — Semantic search and filtering



- **Schema**

```
CREATE EXTENSION IF NOT EXISTS vector;      -- pgvector: semantic search
```

```
CREATE EXTENSION IF NOT EXISTS pg_trgm;   -- trigram: ILIKE substring search

-- media_assets (columns searched, filtered, sorted, or shown in the detail panel)
CREATE TABLE media_assets (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    institution_id  UUID REFERENCES institutions(id),  -- visibility scope / institution filter
    uploader_id     UUID NOT NULL REFERENCES users(id),             --       uploader       email       is
keyword-searchable
    media_album_id  UUID REFERENCES media_albums(id),               -- album filter
    asset_code      VARCHAR(50) UNIQUE NOT NULL,                    -- keyword search
    file_name       VARCHAR(255) NOT NULL,                          -- keyword search, sort by name
    display_title   VARCHAR(255),                                   -- editable title, keyword search
    file_type       VARCHAR(10) NOT NULL,
    file_size_bytes   BIGINT NOT NULL,                              -- sort by size
    status          TEXT NOT NULL,                            -- Processing badge until READY
    ai_category     VARCHAR(50),                                    -- AI label shown on card
    ai_description  TEXT,                                           -- keyword search
    ai_tags         TEXT[],                                         --  AI  tags  with  confidence  in  detail
panel
    deleted_at      TIMESTAMPTZ,                                    -- soft-deleted assets excluded
    created_at      TIMESTAMPTZ NOT NULL DEFAULT NOW()              -- sort newest / oldest
);

-- Voyage embeddings used for meaning-based search (one row per asset per type)
CREATE TABLE media_asset_embeddings (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    asset_id        UUID NOT NULL REFERENCES media_assets(id) ON DELETE CASCADE,
    embedding_type TEXT NOT NULL CHECK (embedding_type IN ('image', 'semantic', 'keyframe')),
    embedding       VECTOR(1024) NOT NULL,
    model           TEXT NOT NULL,
    source_input_hash       CHAR(64),
    source_revision         BIGINT,
    processing_version      VARCHAR(50),
    created_at      TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- Manual and AI tags (all are keyword-searchable)
CREATE TABLE asset_tags (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    media_asset_id UUID NOT NULL REFERENCES media_assets(id) ON DELETE CASCADE,
    label           VARCHAR(50) NOT NULL,
    source          VARCHAR(20) NOT NULL DEFAULT 'manual',
    created_at      TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    CONSTRAINT uq_asset_tags_media_asset_label UNIQUE (media_asset_id, label)
);

-- Albums (album filter and folder navigation)
CREATE TABLE media_albums (
    id              UUID PRIMARY KEY,
    institution_id  UUID NOT NULL REFERENCES institutions(id),
    parent_album_id UUID REFERENCES media_albums(id),
    name            VARCHAR(255) NOT NULL,
    created_by      UUID REFERENCES users(id),
    created_at      TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at      TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- Indexes supporting search and filtering
CREATE INDEX idx_media_assets_institution   ON media_assets(institution_id) WHERE deleted_at IS NULL;
CREATE INDEX idx_media_assets_status_ready  ON media_assets(institution_id, status, created_at DESC)
    WHERE deleted_at IS NULL;
CREATE INDEX idx_media_assets_ai_category   ON media_assets(institution_id, ai_category)
```

```
    WHERE deleted_at IS NULL;
CREATE INDEX idx_media_assets_media_album_id  ON media_assets(media_album_id);
CREATE INDEX idx_media_assets_file_name_trgm  ON media_assets USING GIN (file_name gin_trgm_ops)
    WHERE deleted_at IS NULL;
CREATE INDEX idx_media_assets_asset_code_trgm ON media_assets USING GIN (asset_code gin_trgm_ops)
    WHERE deleted_at IS NULL;
CREATE UNIQUE INDEX uq_media_asset_embedding_type ON media_asset_embeddings(asset_id, embedding_type);
CREATE INDEX idx_media_asset_embeddings_vector      ON media_asset_embeddings
    USING hnsw (embedding vector_cosine_ops) WITH (m = 16, ef_construction = 64);
CREATE INDEX idx_asset_tags_media_asset_id ON asset_tags(media_asset_id);
CREATE INDEX idx_asset_tags_source          ON asset_tags(source);
CREATE INDEX idx_media_albums_parent        ON media_albums(parent_album_id);
CREATE UNIQUE INDEX uq_media_albums_inst_parent_name ON media_albums (
    institution_id,
    COALESCE(parent_album_id, '00000000-0000-0000-0000-000000000000'::uuid),
    LOWER(name)
);
```

2.3. Media History Tracking

2.3.1. User Interface Design

Used In Block (Asset Detail Panel)

Lists submissions referencing the asset (title, date, status badge, jump link subject to access), followed by historical reuse records for deleted submissions shown as "[Submission Deleted]". Assets removed from terminal submissions show "[Asset Deleted]".

Activity History

A per-asset, role-scoped slice of audit events (upload, title/album changes, reuse, deletion).

Delete Confirmation Modal (tiered)

- **Blocked** — referenced by a `pending`, `in_review`,`scheduled,` or `publishing` submission; shows the conflicting submissions with links.
- **Warning** — referenced only by `draft`/`needs_revision` submissions; lists affected drafts and requires "Delete Anyway" (`force=true`).
- **Free** — referenced only by terminal submissions or unreferenced; a simple confirmation.

Bulk delete applies the same tiers per asset.

Duplicate Detected

On upload, an exact SHA-256 match prompts the actor to use the existing asset or upload anyway.

Trash View (Administrator only)

Lists soft-deleted assets within the 30-day retention window with Restore, Bulk Restore, Purge, Bulk Purge, and Empty Trash.

2.3.2. Front-end Component(s)

AssetDetailPanel (Used In / history)

- `GET /media-assets/{id}` (usage) and `GET /media-assets/{id}/history`.

DeleteModal

- `DELETE /media-assets/{id}?force=…`, `POST /media-assets/bulk-delete`.
- React modal component

TrashView

- `GET /media-assets/trash`, `POST /media-assets/trash/{id}/restore`, `POST /media-assets/trash/bulk-restore`, `DELETE /media-assets/trash/{id}`, `POST /media-assets/trash/bulk-purge`, `POST /media-assets/trash/empty`.
- React functional component (Admin)

2.3.3. Back-end Component(s)

MediaAssetController — history, delete, bulk-delete, and trash endpoints (trash is `hasRole('ADMIN')`).

MediaAssetService

- Builds Used In (current relationships + historical `MEDIA_ASSET_REUSED` audit rows); tiered deletion authorization (Administrator any asset; Moderator and Contributor only assets they uploaded) and reference tiers; soft delete sets `deleted_at`, `deleted_by_user_id`, status deleted, and removes the asset's embeddings; duplicate detection by `content_hash`.
- Spring `@Service`

MediaAssetRetentionService / MediaAssetRetentionPurgeJob

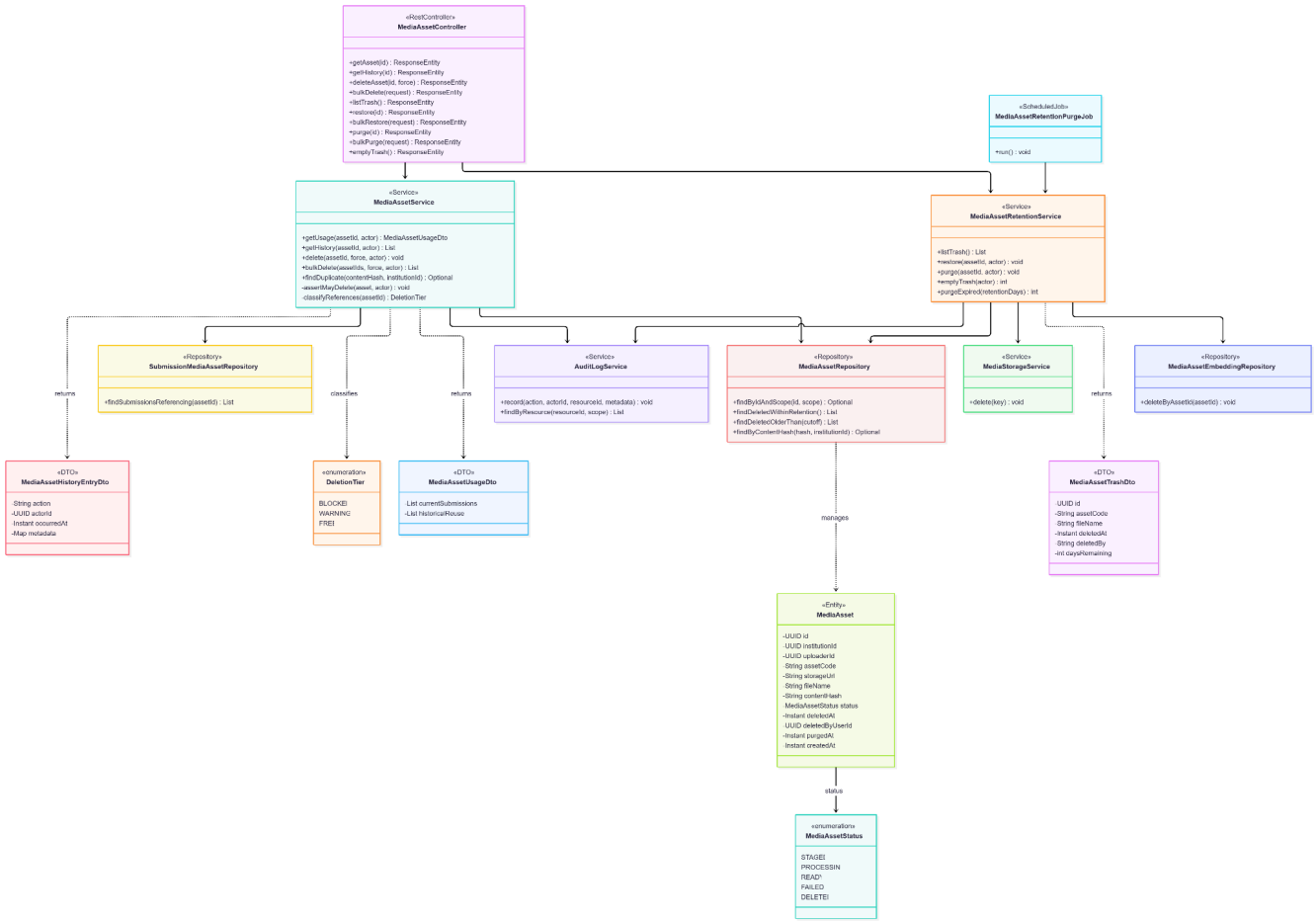
- After `app.media-assets.deleted-retention-days` (default 30), purges R2 objects and embeddings and marks `purged_at`; restore is possible before purge.
- Spring `@Service` / `@Scheduled`

AuditLogService

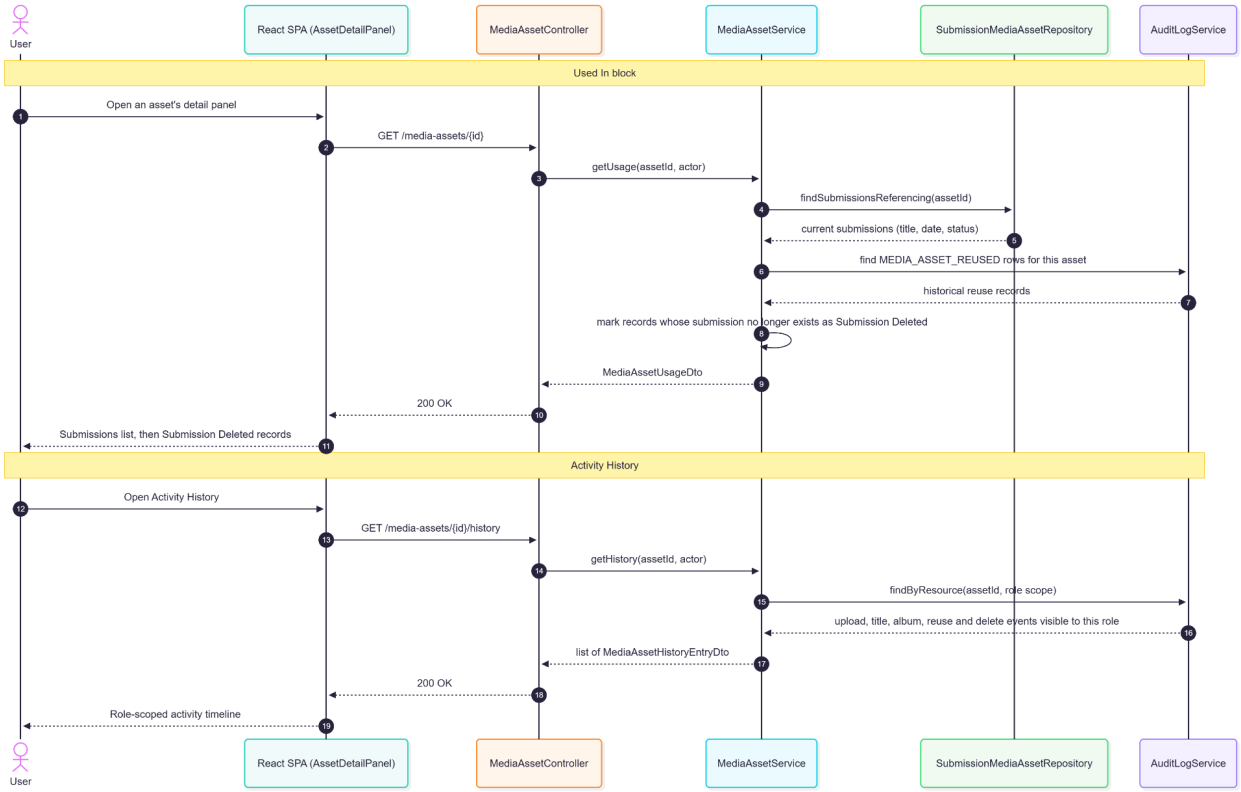
- Media lifecycle events (`MEDIA_ASSET_UPLOADED`, `MEDIA_ASSET_REUSED`, album changes, `MEDIA_ASSET_DELETED`, restore/purge).

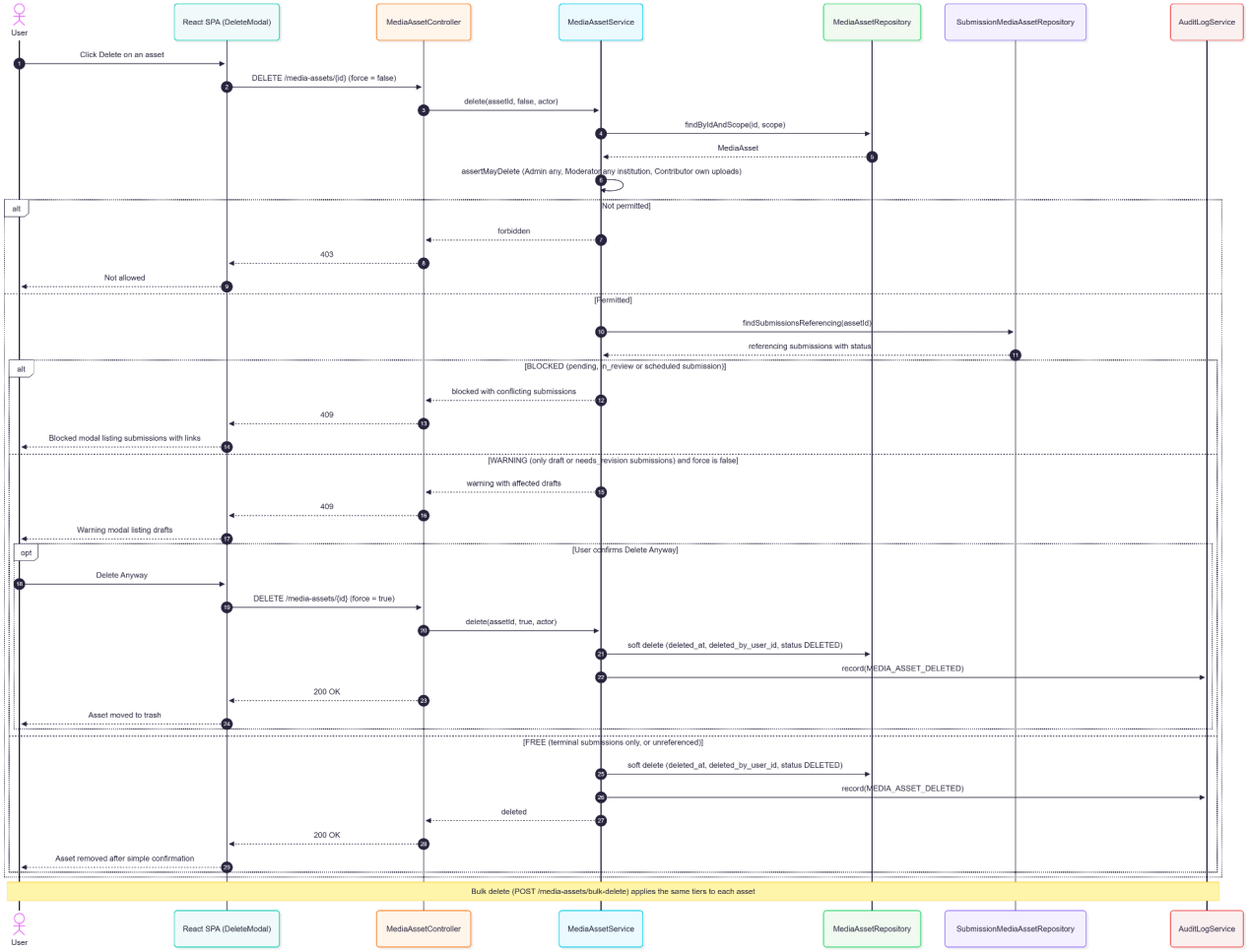
2.3.4. Object-Oriented Component(s)

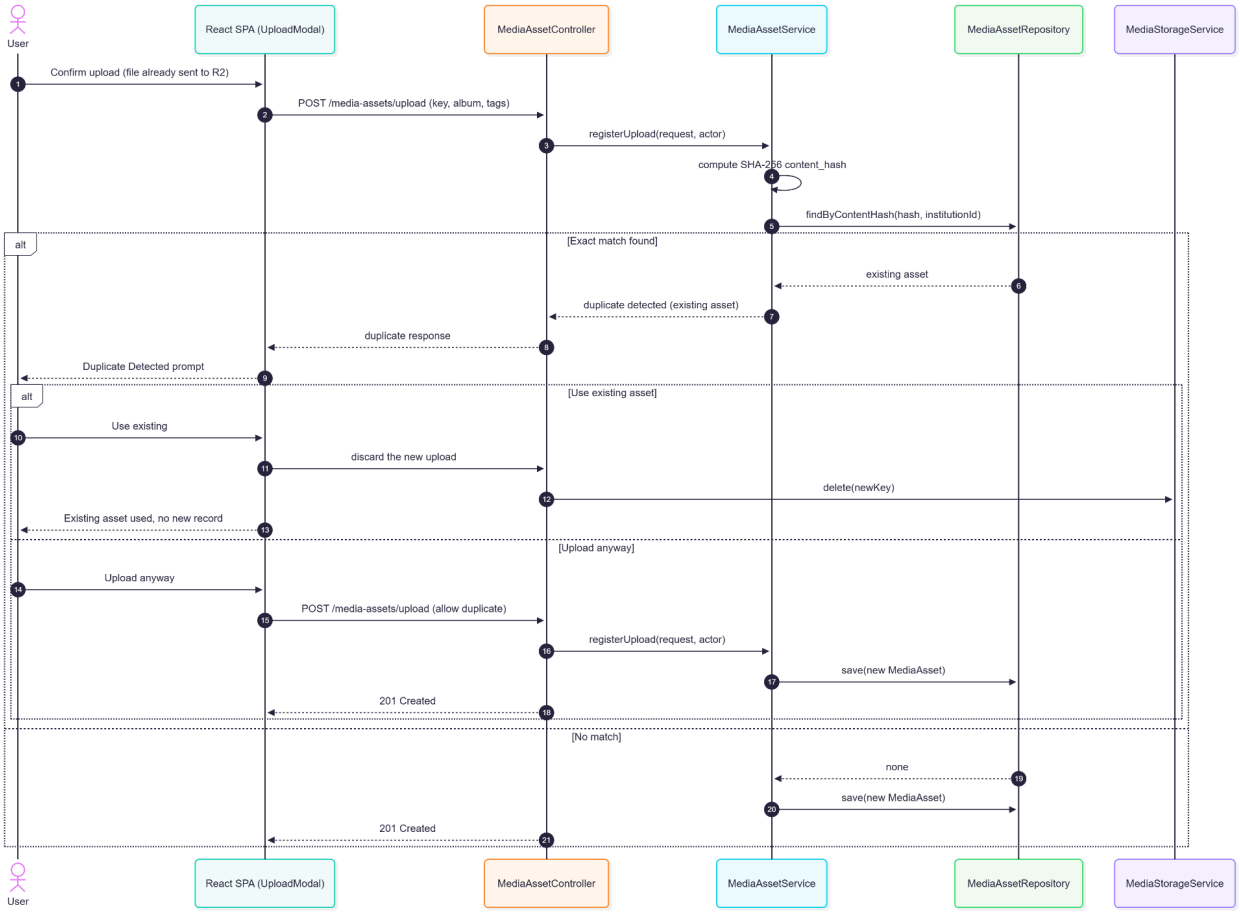
- **Class Diagram**

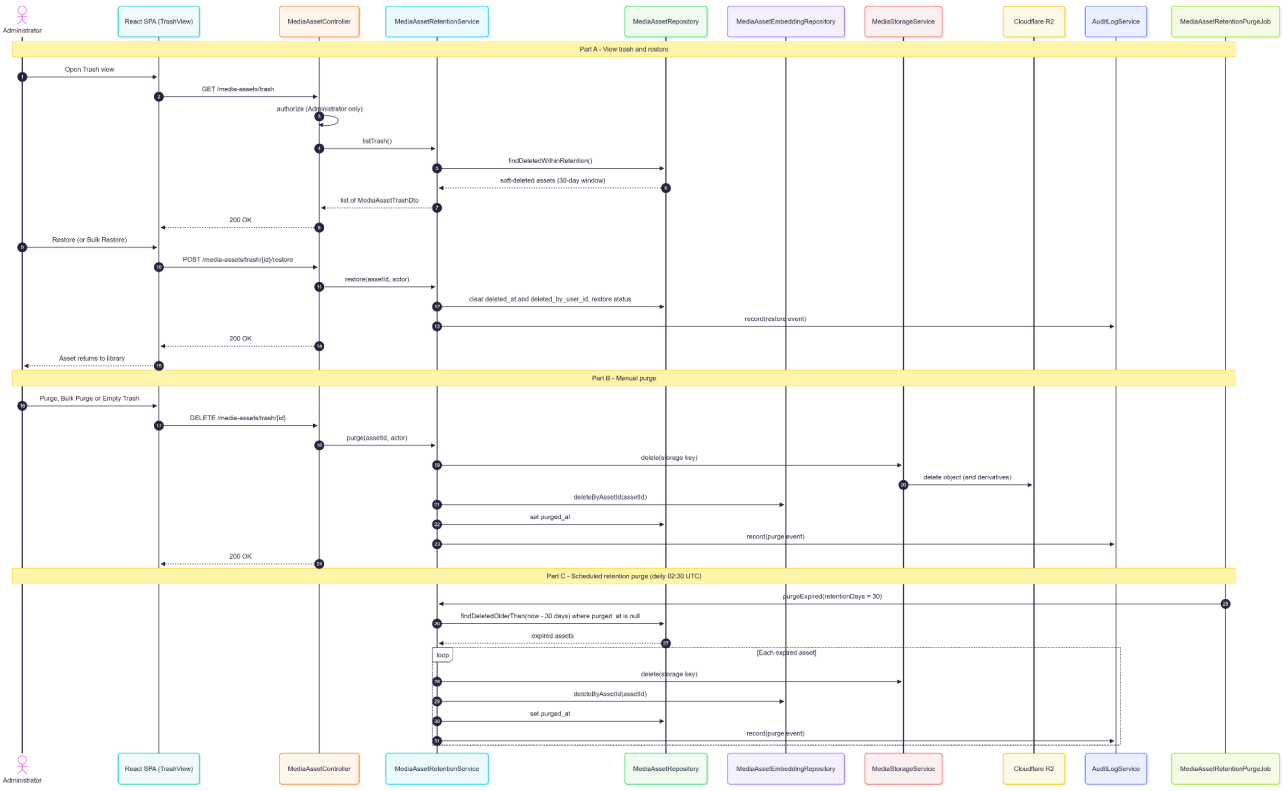


- **Sequence Diagram**


User
React SPA (AssetDetailPanel)
MediaAssetController
MediaAssetService
SubmissionMediaAssetRepository
AuditLogService
Used In block
1 Open an asset's detail panel
2 GET /media-assets/{id}
3 getUsage(assetId, actor)
4 findSubmissionsReferencing(assetId)
5 current submissions (title, date, status)
6 find MEDIA_ASSET_REUSED rows for this asset
7 historical reuse records
8 mark records whose submission no longer exists as Submission Deleted
9 MediaAssetUsageDto
10 200 OK
11 Submissions list, then Submission Deleted records
Activity History
12 Open Activity History
13 GET /media-assets/{id}/history
14 getHistory(assetId, actor)
15 findByResource(assetId, role scope)
16 upload, title, album, reuse and delete events visible to this role
17 list of MediaAssetHistoryEntryDto
18 200 OK
19 Role-scoped activity timeline
User
React SPA (AssetDetailPanel)
MediaAssetController
MediaAssetService
SubmissionMediaAssetRepository
AuditLogService

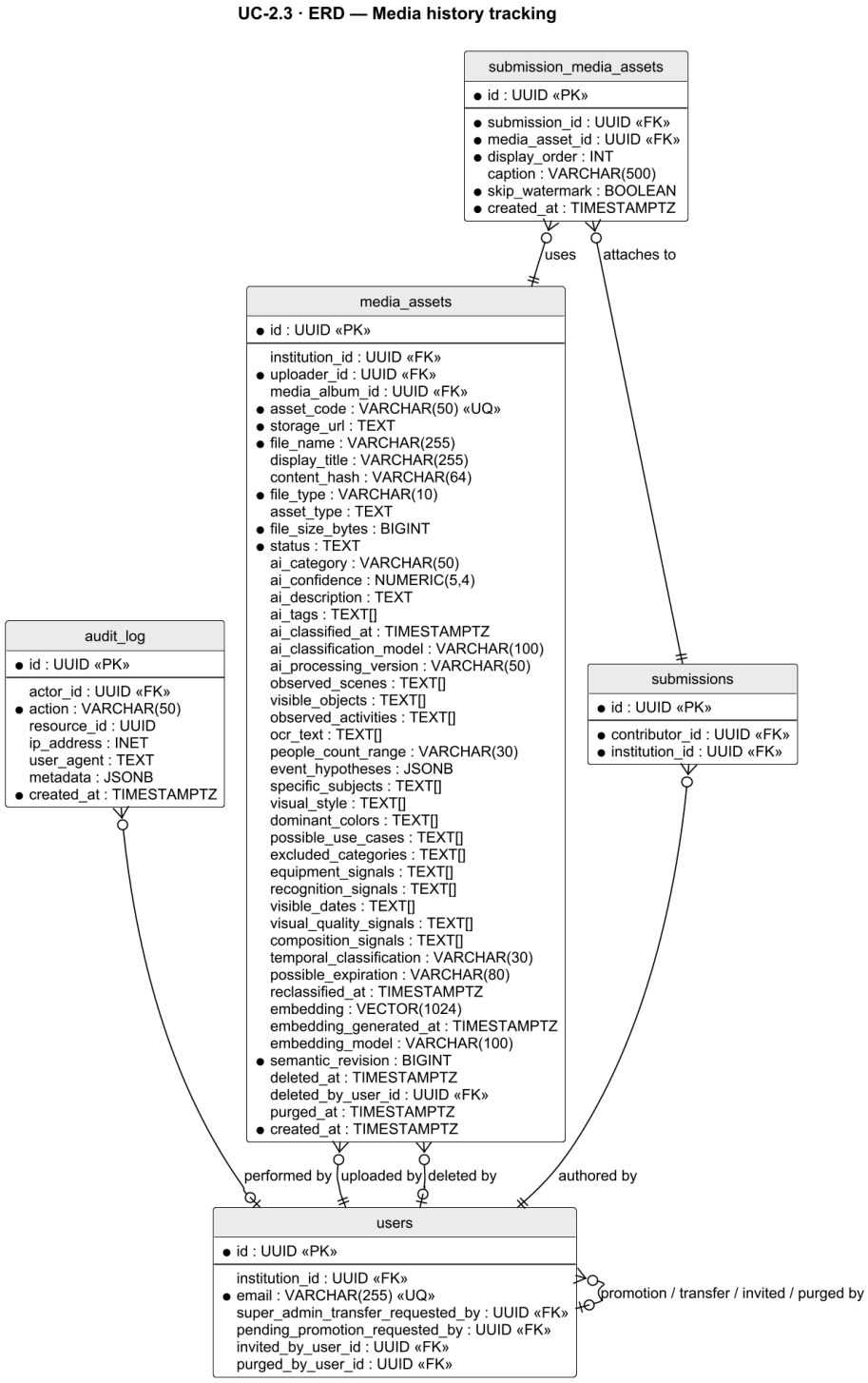

User
React SPA (DeleteModal)
MediaAssetController
MediaAssetService
MediaAssetRepository
SubmissionMediaAssetRepository
AuditLogService
1 Click Delete on an asset
2 DELETE /media-assets/{id} (force = false)
3 delete(assetId, false, actor)
4 findByIdAndScope(id, scope)
5 MediaAsset
6 assertMayDelete (Admin any, Moderator any institution, Contributor own uploads)
alt
[Not permitted]
7 forbidden
8 403
9 Not allowed
[Permitted]
10 findSubmissionsReferencing(assetId)
11 referencing submissions with status
alt
[BLOCKED (pending_review or scheduled submission)]
12 blocked with conflicting submissions
13 409
14 Blocked modal listing submissions with links
[WARNING (only draft or needs_revision submissions) and force is false]
15 warning with affected drafts
16 409
17 Warning modal listing drafts
opt
[User confirms Delete Anyway]
18 Delete Anyway
19 DELETE /media-assets/{id} (force = true)
20 delete(assetId, true, actor)
21 soft delete (deleted_at, deleted_by_user_id, status DELETED)
22 record(MEDIA_ASSET_DELETED)
23 200 OK
24 Asset moved to trash
[FREE (terminal submissions only, or unreferenced)]
25 soft delete (deleted_at, deleted_by_user_id, status DELETED)
26 record(MEDIA_ASSET_DELETED)
27 deleted
28 200 OK
29 Asset removed after simple confirmation
Bulk delete (POST /media-assets/bulk-delete) applies the same tiers to each asset
User
React SPA (DeleteModal)
MediaAssetController
MediaAssetService
MediaAssetRepository
SubmissionMediaAssetRepository
AuditLogService


User
React SPA (UploadModal)
MediaAssetController
MediaAssetService
MediaAssetRepository
MediaStorageService
Confirm upload (file already sent to R2)
POST /media-assets/upload (key, album, tags)
registerUpload(request, actor)
compute SHA-256 content_hash
findByContentHash(hash, institutionId)
alt
[Exact match found]
existing asset
duplicate detected (existing asset)
duplicate response
Duplicate Detected prompt
alt
[Use existing asset]
Use existing
discard the new upload
delete(newKey)
Existing asset used, no new record
[Upload anyway]
Upload anyway
POST /media-assets/upload (allow duplicate)
registerUpload(request, actor)
save(new MediaAsset)
201 Created
[No match]
none
save(new MediaAsset)
201 Created


Administrator
React SPA (TrashView)
MediaAssetController
MediaAssetRetentionService
MediaAssetRepository
MediaAssetEmbeddingRepository
MediaStorageService
Cloudflare R2
AuditLogService
MediaAssetRetentionPurgeJob
Part A - View trash and restore
Open Trash view
GET /media-assets/trash
authorize (Administrator only)
listTrash()
findDeletedWithinRetention()
soft-deleted assets (30-day window)
list of MediaAssetTrashDto
200 OK
Restore (or Bulk Restore)
POST /media-assets/trash/{id}/restore
restore(assetId, actor)
clear deleted_at and deleted_by_user_id, restore status
record(restore event)
200 OK
Asset returns to library
Part B - Manual purge
Purge, Bulk Purge or Empty Trash
DELETE /media-assets/trash/{id}
purge(assetId, actor)
delete(storage key)
delete object (and derivatives)
deleteByAssetId(assetId)
set purged_at
record(purge event)
200 OK
Part C - Scheduled retention purge (daily 02:30 UTC)
purgeExpired(retentionDays = 30)
findDeletedOlderThan(now - 30 days) where purged_at is null
expired assets
loop
[Each expired asset]
delete(storage key)
deleteByAssetId(assetId)
set purged_at
record(purge event)

2.3.5. Data Design

- **Entity Relationship Diagram**



- **Schema**

```
-- media_assets (lifecycle and duplicate-detection columns)
CREATE TABLE media_assets (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    institution_id  UUID REFERENCES institutions(id),       -- NULL while STAGED; bound on submit (V73)
```

```
    uploader_id     UUID NOT NULL REFERENCES users(id),     -- Contributors may delete only their own
uploads
    asset_code      VARCHAR(50) UNIQUE NOT NULL,
    storage_url     TEXT NOT NULL,                          -- R2 object removed at purge
    file_name       VARCHAR(255) NOT NULL,
    status          TEXT NOT NULL,                  -- DELETED once soft-deleted
    content_hash    VARCHAR(64),                            -- SHA-256 exact-duplicate detection
    deleted_at      TIMESTAMPTZ,                            -- soft delete; starts the 30-day window
    deleted_by_user_id UUID REFERENCES users(id),
    purged_at       TIMESTAMPTZ,                            -- set when the retention job purges files
    created_at      TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- Submission ↔ asset references (Used In block; determines blocked / warn / free deletion)
CREATE TABLE submission_media_assets (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    submission_id  UUID NOT NULL REFERENCES submissions(id) ON DELETE CASCADE,
    media_asset_id UUID NOT NULL REFERENCES media_assets(id),
    display_order  INT NOT NULL DEFAULT 0,
    caption         VARCHAR(500),
    skip_watermark BOOLEAN NOT NULL DEFAULT FALSE,
    created_at      TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    CONSTRAINT uq_submission_media_asset UNIQUE (submission_id, media_asset_id)
);

-- Media lifecycle events (upload, reuse, album change, delete, restore, purge)
CREATE TABLE audit_log (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    actor_id        UUID REFERENCES users(id),
    action          VARCHAR(50) NOT NULL,   -- e.g. MEDIA_ASSET_REUSED, MEDIA_ASSET_DELETED
    ip_address  INET,
    user_agent  TEXT,
    resource_id UUID,                       -- the media asset (or submission) affected
    metadata        JSONB,                  -- event details (before/after values, context)
    created_at  TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- Indexes
CREATE INDEX idx_media_assets_institution_content_hash ON media_assets(institution_id, content_hash)
    WHERE deleted_at IS NULL AND content_hash IS NOT NULL;  -- duplicate lookup on upload
CREATE INDEX idx_media_assets_deleted_retention ON media_assets(deleted_at)
    WHERE deleted_at IS NOT NULL AND purged_at IS NULL;     -- trash view and retention purge
CREATE INDEX idx_submission_media_submission ON submission_media_assets(submission_id);
CREATE INDEX idx_submission_media_asset     ON submission_media_assets(media_asset_id);  -- Used In lookup
CREATE INDEX idx_audit_log_resource ON audit_log(resource_id, created_at DESC);             -- asset
history
```

2.4. Admin Approval Workflow

2.4.1. User Interface Design

Review Queue (`/queue`)

A network-wide, server-paginated queue shared by Moderators and Administrators with tabs **All, Pending, In Review, Needs Revision, Scheduled, Published, Rejected, Failed**. Fast-Track items sort to the top of the active tabs, then items follow the selected sort; items due within 30 minutes show a priority flag (GR-T1) but are not moved by the paginated queue. Counts appear on each tab.

Submission Details Panel

The single details surface: media (with a "failed to load" placeholder and Retry), AI tags, submitter, institution, event details, schedule, caption, and a Review History entry point. Clicking **Review** acquires a 15-minute lock (renewed while open); other reviewers see who holds it and a read-only

view. Self-submissions cannot be reviewed. A 30-minute countdown / slot-passed banner appears for Scheduled posts nearing or past their slot.

Edit During Review

Edit caption, title, date, media (add from library, remove, reorder), tags, and schedule (Admin-only Schedule ↔ Live Event mode toggle; a static mode badge for Moderators). **Review & Save** shows every change with Undo before saving (with a Live Event warning); **Restore original** returns a field to the contributor's value; **Check writing** runs on demand and automatically after a caption edit.

Decisions

- **Approve** — `scheduled`, or immediate publish for Fast-Track. A passed slot requires a new slot, or an Administrator "Publish now" override.
- **Request Revision** — remarks of 10–1,000 characters.
- **Reject** — reason code (Inappropriate content, Out of scope, Duplicate event, No longer relevant, Rights or privacy, Wrong institution, Other — notes required for Other). Not final: the contributor may edit and resubmit.

Failed Tab

Lists `publish_failed` items with failure reason, unresolved staged photo IDs (copyable), and actions: **Retry** (same mode), **Retry with new schedule** (a missed_review item returns to pending for review), **Retry as Live** (Admin only), and **Publish Manually** — a 3-step workflow (Copy content → Open Facebook → Record details with optional URL/notes) with a 2-hour countdown and cancel confirmation.

Guides

Four view-only walkthroughs (queue, review/decision, failed publishing, moderator editing) that open what each step describes without mutating state.

2.4.2. Front-end Component(s)

ValidationQueueScreen / useValidationQueue

- `GET /validation/queue/page`, `GET /validation/dashboard-summary`, lock endpoints (`GET/POST/DELETE /validation/{id}/lock`), decisions (`POST /validation/{id}/approve[?publishNow=true]`, `/revise`, `/reject`), edits (`POST /validation/{id}/edit`, `/assets`, `DELETE /validation/{id}/assets/{assetId}`, `PATCH /validation/{id}/media/order`), history (`GET /validation/{id}/log`).
- React functional component / Reviewer route

ReviewChangesDialog / WordDiffText

- Review & Save confirmation with per-change Undo and word-level diff.

ReviewLibraryPickerModal / ReviewAiSuggestionsModal

- Add library media and AI suggestions during review.

ResolutionRetryModal / ManualPublishWorkflowPanel

- `GET /resolution/failures/page`, `GET /resolution/{id}`, `POST /resolution/{id}/retry`, `/retry-with-new-schedule`, `/retry-as-live`, `/manual-publish/start|complete|cancel`.

SpotlightTour — review guides.

2.4.3. Back-end Component(s)

ValidationController

- Queue, lock, edit, media, decision, and log endpoints; `hasAnyRole('MODERATOR','ADMIN')`.
- Spring `@RestController`

ValidationService

- Network-wide queue; unconditional self-review block; approve (snapshot `original_scheduled_at`, lock slot to `locked`, refuse a passed slot with `409` unless Admin `publishNow` → audited `LATE_PUBLISH_OVERRIDE`); request revision (remarks 10–1,000, slot released); reject (`REJECTION_REASON_LABELS`, revision-type reasons return `422`, slot released). Edits validated against guard rails and the 10-media limit; a non-Admin changing `fastTrack` gets `403`. Diffs are classified QUIET / FLAGGED / ADDED_MEDIA and accumulated over the whole review cycle (since the last terminal action). Contributors receive a separate edit notification (email only for FLAGGED/ADDED_MEDIA); FLAGGED/ADDED_MEDIA also notify other Administrators. Fires `SubmissionApprovedEvent`.
- Spring `@Service`

ReviewLockService / ReviewLockCleanupJob

- 15-minute lock (`LOCK_DURATION_MINUTES = 15`), renewal, release; expiry reverts `in_review` → `pending`.
- Spring `@Service` / `@Scheduled`

ValidationDeadlineNotificationJob / StaleSubmissionDetectorJob

- Approaching-deadline reviewer alerts (the GR-T1 flag itself is computed when the queue is read); unreviewed past slot → `missed_review`; the missed-review sweep skips locked submissions.

ProofreadService — advisory writing check (shared with UC-1.6).

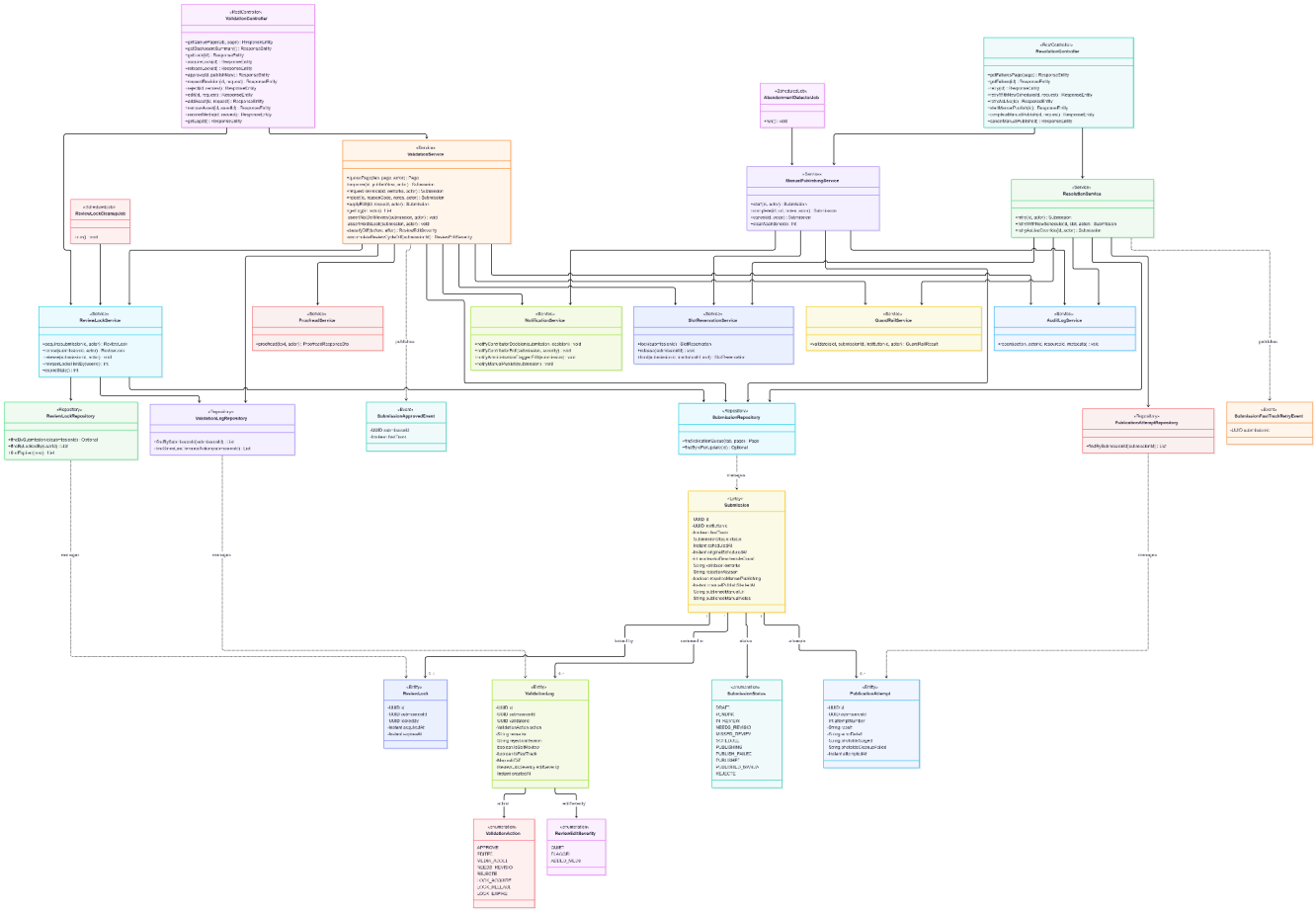
ResolutionController / ResolutionService / ManualPublishingService / AbandonmentDetectorJob

- `retry` (replays the same mode; Fast-Track fires SubmissionFastTrackRetryEvent), retryWithNewSchedule (publish_failed → scheduled, resetting `original_scheduled_at` and `moderator_reschedule_count`; `missed_review` → pending with a new held slot; clears Fast-Track; Admin-gated when the item was Fast-Track; a hard guard-rail block needs an Administrator override reason), retryAsLiveOverride (Admin only), manual publish start/complete/cancel (`published_manual`, releases slot, notifies contributor), 2-hour abandonment.
- Spring `@RestController` / `@Service` / `@Scheduled`

AuditLogService / NotificationService — decision, edit, override, and manual-publish events; T-02, T-03, revision, edit, and T-05 notifications.

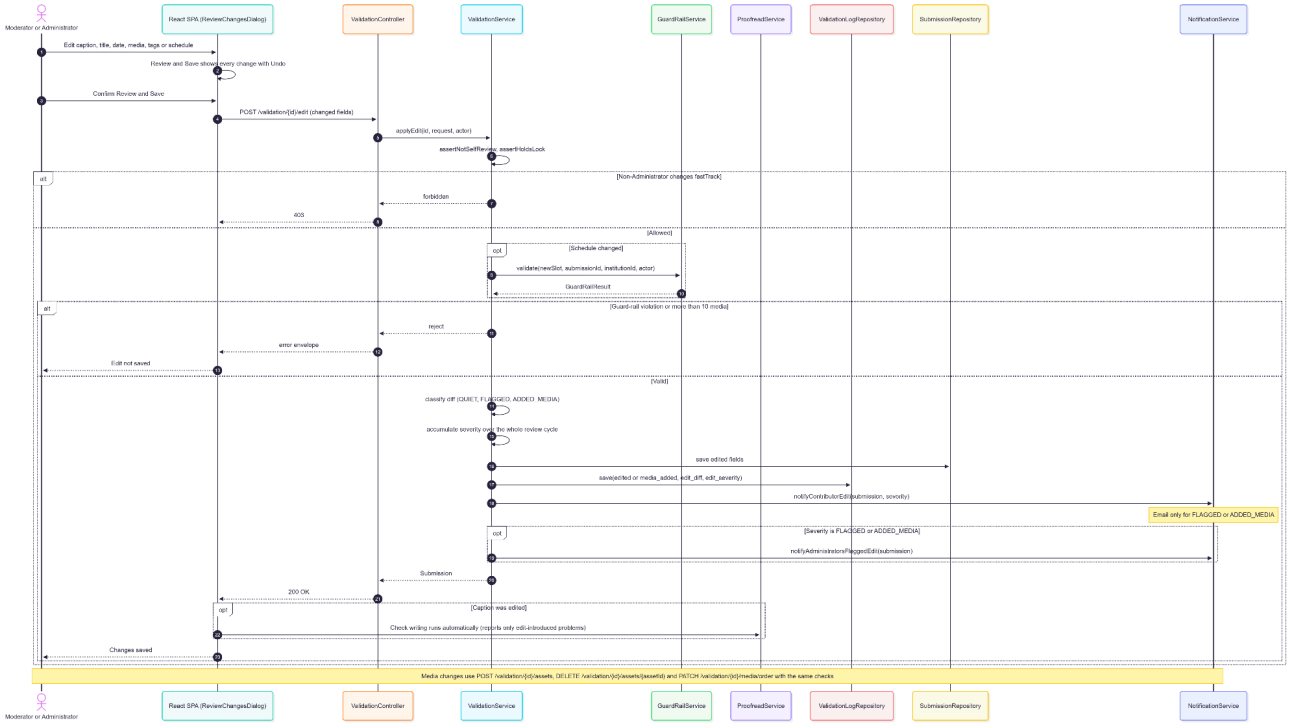
2.4.4. Object-Oriented Component(s)

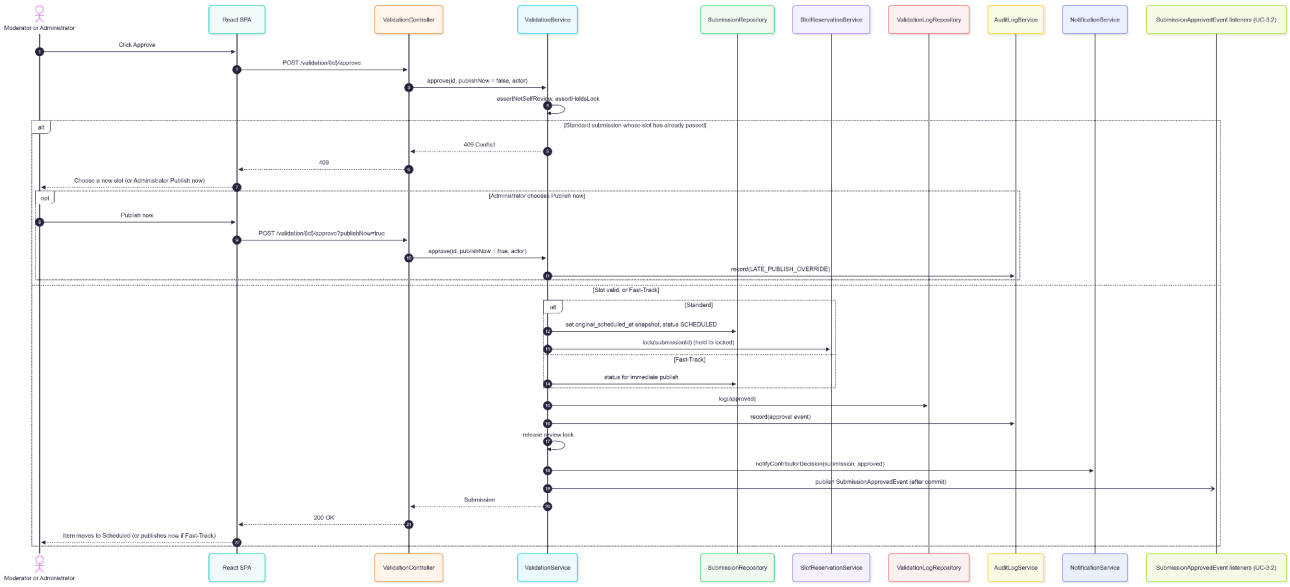
- **Class Diagram**

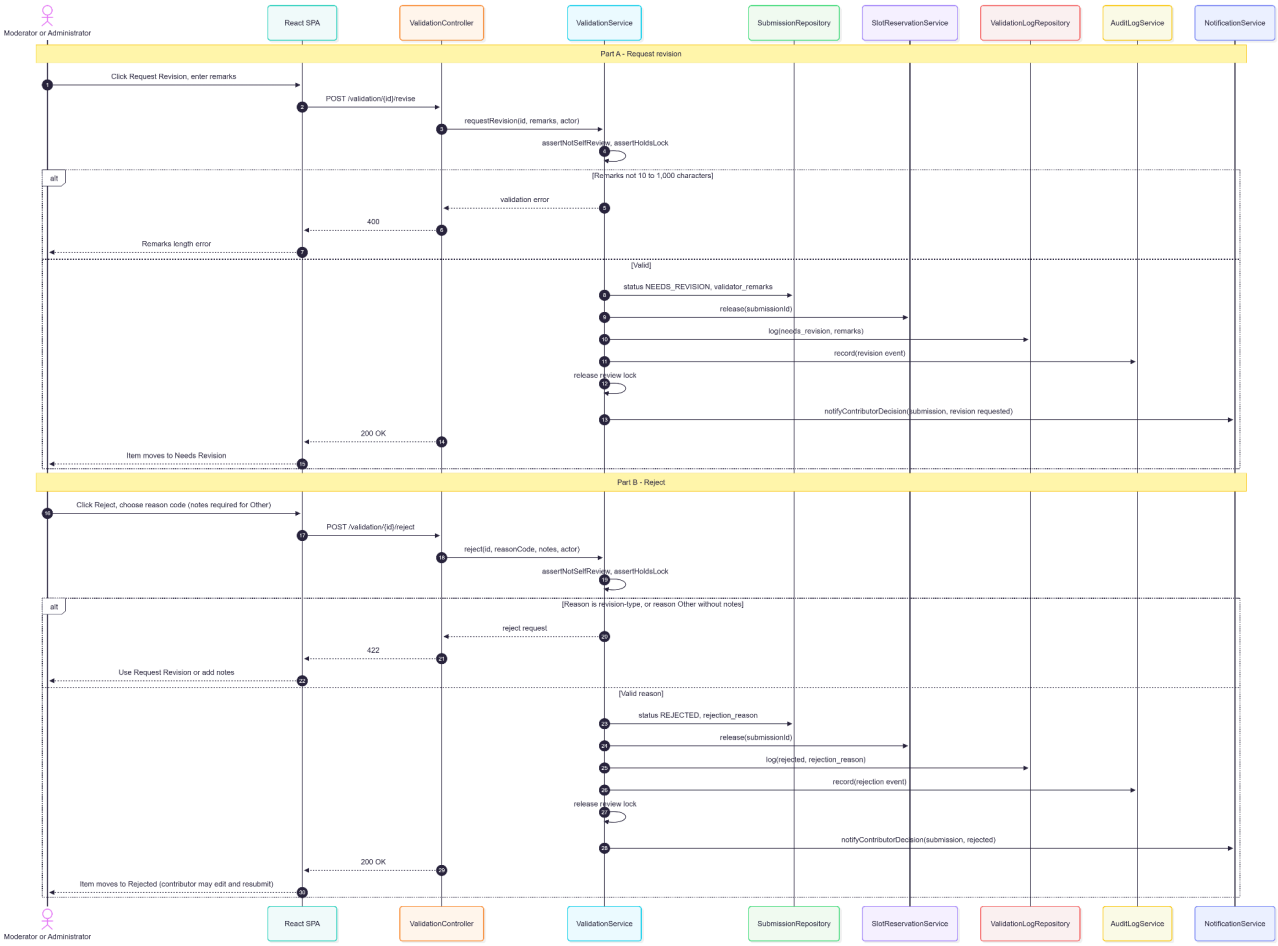


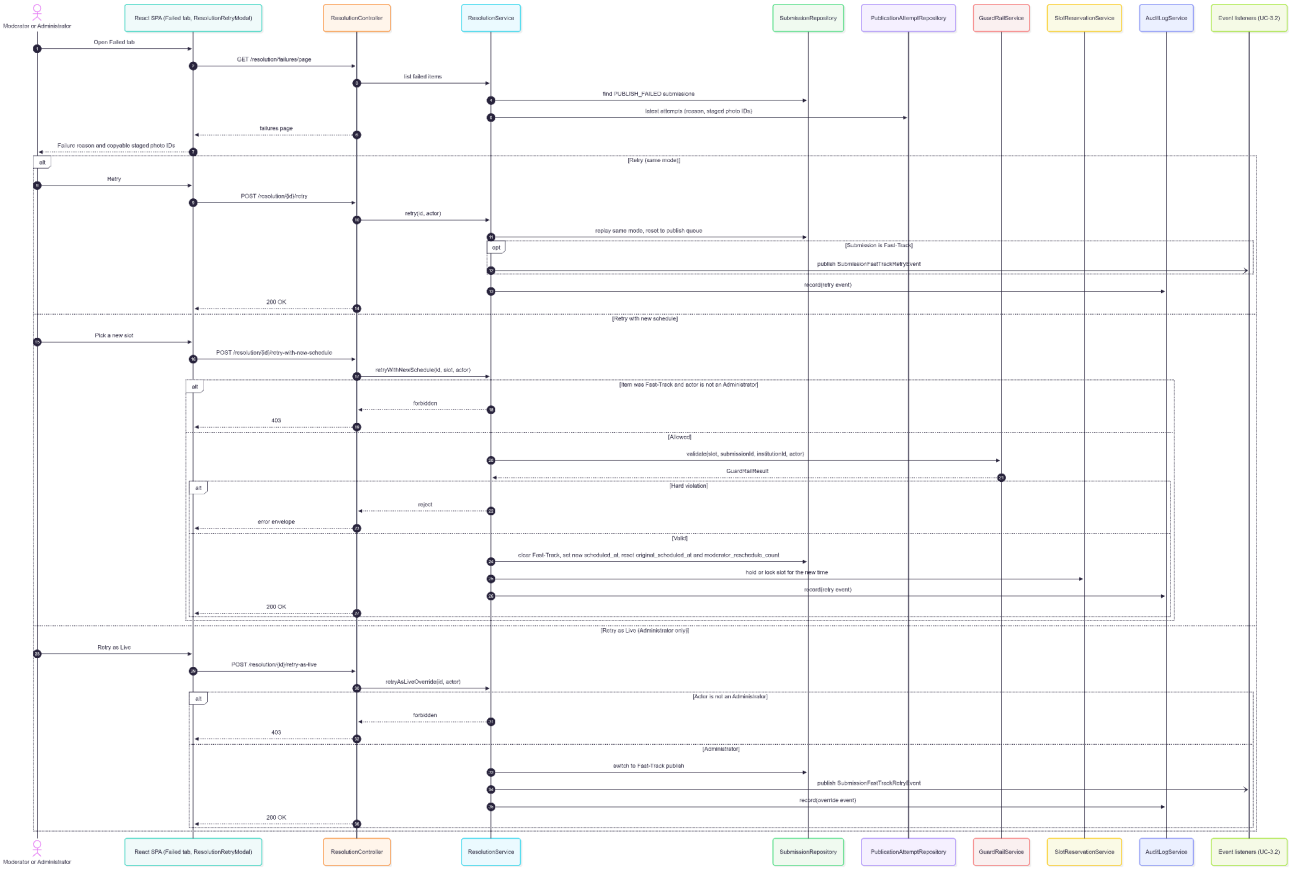
- **Sequence Diagram**

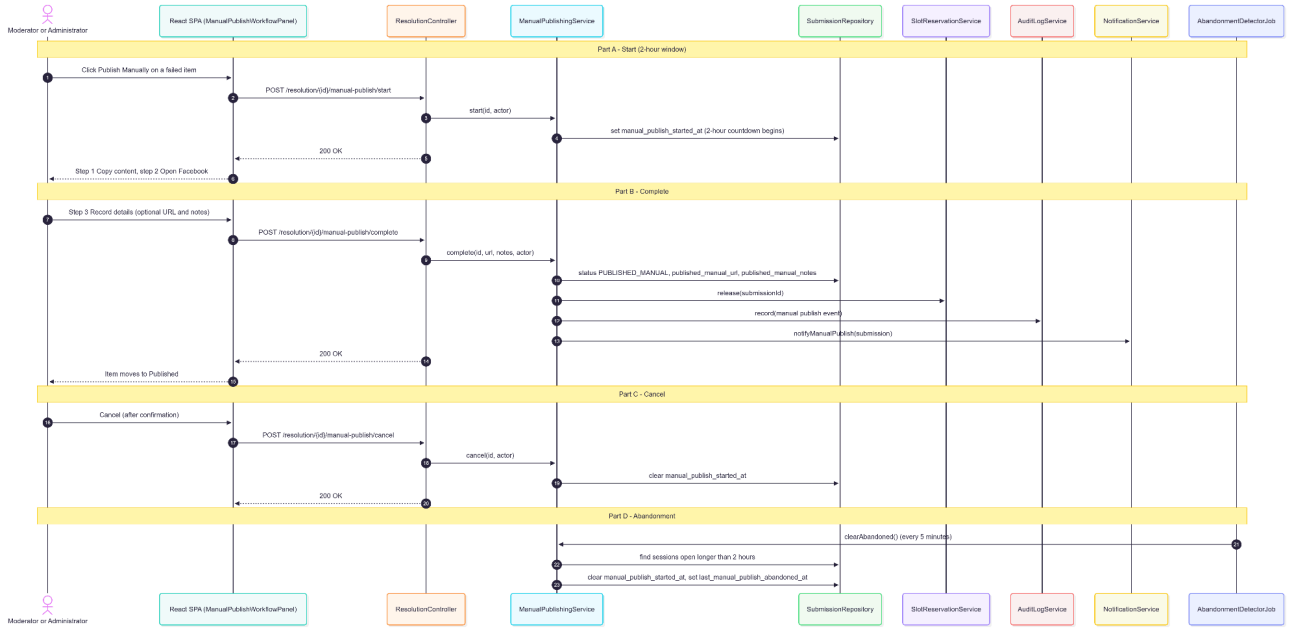
Moderator or Administrator
Another reviewer
React SPA (ValidationQueueScreen)
ValidationController
ValidationService
ReviewLockService
ReviewLockRepository
SubmissionRepository
ValidationLogRepository
ReviewLockCleanupJob
Part A - Open queue
Open /queue
GET /validation/queue/page (tab, page)
queuePage(tab, page, actor)
findValidationQueue (Fast-Track first, then slot, then submitted time)
network-wide page and tab counts
page
200 OK
Queue with priority flags
Part B - Click Review (self-review check, then lock)
Click Review
POST /validation/{id}/lock
acquire(submissionId, actor)
load submission
alt
[Reviewer is the submitter (self-review)]
reject
403
Self-submissions cannot be reviewed
[Not self-review]
findBySubmissionId(submissionId)
alt
[Active lock held by someone else]
lock (holder)
locked
409 with holder
Read-only view showing who holds the lock
[No active lock (or own lock)]
save lock (expires_at = now + 15 min)
status PENDING to IN_REVIEW
log(lock_acquired)
ReviewLock
200 OK
Details panel opens in review mode
Part C - Renew and release
loop
[While the panel stays open]
POST /validation/{id}/lock (renew)
renew(submissionId, actor)
extend expires_at by 15 min
Close panel without deciding
DELETE /validation/{id}/lock
release(submissionId, actor)
delete lock
status IN_REVIEW back to PENDING
log(lock_released)
Part D - Lock expiry
expireStale() (every minute)
findExpired(now)
loop
[Each expired lock]
delete lock
status IN_REVIEW back to PENDING
log(lock_expired)
Review the same item afterwards
POST /validation/{id}/lock
lock acquired (see Part B)


Moderator or Administrator
React SPA (ReviewChangesDialog)
ValidationController
ValidationService
GuardRailService
ProofreadService
ValidationLogRepository
SubmissionRepository
NotificationService
Edit caption, title, date, media, tags or schedule
Review and Save shows every change with Undo
Confirm Review and Save
POST /validation/{id}/edit (changed fields)
applyEdit(id, request, actor)
assertNotSelfReview; assertHoldsLock
alt
[Non-Administrator changes fastTrack]
forbidden
403
[Allowed]
opt
[Schedule changed]
validate(newSlot, submissionId, institutionId, actor)
GuardRailResult
[Guard-rail violation or more than 10 media]
reject
error envelope
Edit not saved
[Valid]
classify diff (QUIET, FLAGGED, ADDED_MEDIA)
accumulate severity over the whole review cycle
save edited fields
save(edited or media_added, edit_diff, edit_severity)
notifyContributorEdit(submission, severity)
Email only for FLAGGED or ADDED_MEDIA
opt
[Severity is FLAGGED or ADDED_MEDIA]
notifyAdministratorsFlaggedEdit(submission)
Submission
200 OK
opt
[Caption was edited]
Check writing runs automatically (reports only edit-introduced problems)
Changes saved
Media changes use POST /validation/{id}/assets, DELETE /validation/{id}/assets/{assetId} and PATCH /validation/{id}/media/order with the same checks


Moderator or Administrator
React SPA
ValidationController
ValidationService
SubmissionRepository
SlotReservationService
ValidationLogRepository
AuditLogService
NotificationService
SubmissionApprovedEvent listeners (UC-3.2)
Click Approve
POST /validation/{id}/approve
approve(id, publishNow = false, actor)
assertNotSelfReview; assertHoldsLock
alt
[Standard submission whose slot has already passed]
409 Conflict
409
Choose a new slot (or Administrator Publish now)
opt
[Administrator chooses Publish now]
Publish now
POST /validation/{id}/approve?publishNow=true
approve(id, publishNow = true, actor)
record(LATE_PUBLISH_OVERRIDE)
[Slot valid, or Fast-Track]
alt
[Standard]
set original_scheduled_at snapshot, status SCHEDULED
lock(submissionId) (held to locked)
[Fast-Track]
status for immediate publish
log(approved)
record(approval event)
release review lock
notifyContributorDecision(submission, approved)
publish SubmissionApprovedEvent (after commit)
Submission
200 OK
Item moves to Scheduled (or publishes now if Fast-Track)


Moderator or Administrator
React SPA
ValidationController
ValidationService
SubmissionRepository
SlotReservationService
ValidationLogRepository
AuditLogService
NotificationService
Part A - Request revision
Click Request Revision, enter remarks
POST /validation/{id}/revise
requestRevision(id, remarks, actor)
assertNotSelfReview, assertHoldsLock
alt
[Remarks not 10 to 1,000 characters]
validation error
400
Remarks length error
[Valid]
status NEEDS_REVISION, validator_remarks
release(submissionId)
log(needs_revision, remarks)
record(revision event)
release review lock
notifyContributorDecision(submission, revision requested)
200 OK
Item moves to Needs Revision
Part B - Reject
Click Reject, choose reason code (notes required for Other)
POST /validation/{id}/reject
reject(id, reasonCode, notes, actor)
assertNotSelfReview, assertHoldsLock
alt
[Reason is revision-type, or reason Other without notes]
reject request
422
Use Request Revision or add notes
[Valid reason]
status REJECTED, rejection_reason
release(submissionId)
log(rejected, rejection_reason)
record(rejection event)
release review lock
notifyContributorDecision(submission, rejected)
200 OK
Item moves to Rejected (contributor may edit and resubmit)

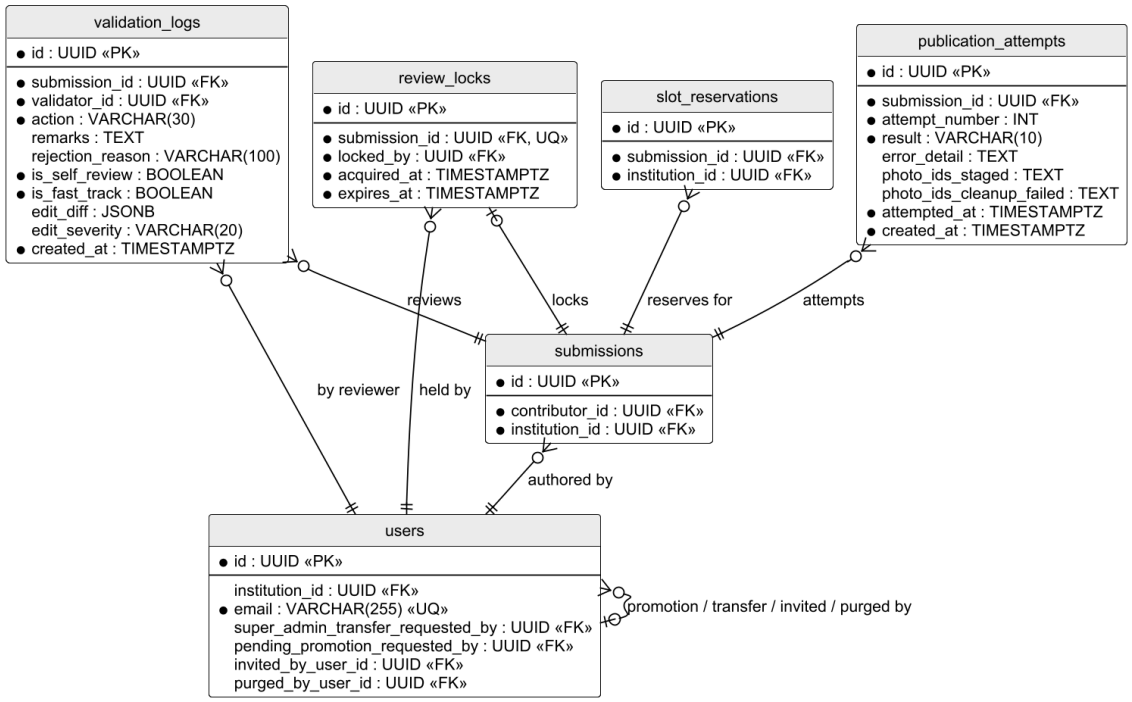




2.4.5. Data Design

- **Entity Relationship Diagram**

UC-2.4 · ERD — Admin approval workflow



- **Schema**

```
CREATE TABLE review_locks (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    submission_id UUID NOT NULL UNIQUE REFERENCES submissions(id) ON DELETE CASCADE,
    locked_by       UUID NOT NULL REFERENCES users(id),
    acquired_at   TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    expires_at      TIMESTAMPTZ NOT NULL            -- acquired_at + 15 minutes, renewed while open
);

CREATE TABLE validation_logs (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    submission_id   UUID NOT NULL REFERENCES submissions(id) ON DELETE CASCADE,
    validator_id    UUID NOT NULL REFERENCES users(id),
    action          VARCHAR(30) NOT NULL,           -- approved | edited | media_added | needs_revision |
                                                    -- rejected | lock_acquired | lock_released |
lock_expired ...
    remarks         TEXT,
    rejection_reason VARCHAR(100),
    is_self_review   BOOLEAN NOT NULL DEFAULT FALSE,
    is_fast_track   BOOLEAN NOT NULL DEFAULT FALSE,
    edit_diff       JSONB,                          -- before/after field diff
    edit_severity   VARCHAR(20) CHECK (edit_severity IN ('QUIET','FLAGGED','ADDED_MEDIA')),
    created_at      TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_review_locks_expiry ON review_locks(expires_at);
CREATE INDEX idx_validation_logs_submission ON validation_logs(submission_id, created_at DESC);
```

Related tables used by this use case:

```
-- publication_attempts (defined in UC-3.2)
CREATE TABLE publication_attempts (
    id                      UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    submission_id           UUID NOT NULL REFERENCES submissions(id) ON DELETE CASCADE,
    attempt_number          INT NOT NULL,
    result                  VARCHAR(10) NOT NULL,
```

```
    error_detail            TEXT,
    photo_ids_staged        TEXT,
    photo_ids_cleanup_failed TEXT,
    attempted_at            TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    created_at              TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
```

2.5. Automated Watermarking

2.5.1. User Interface Design

Watermark Configuration (Settings → Page Settings, Administrator only)

A transparent canvas editor with 1:1, 4:5, and 16:9 preview toggles. The Administrator adds up to 3 elements — image, text, or shape (rectangle, divider line, or gradient scrim) — dragged and resized as percentage-based coordinates, with type-specific properties (image reference, text/font, shape/color). An Enabled switch and Save button persist the network-wide configuration.

2.5.2. Front-end Component(s)

WatermarkCanvasEditor

- `GET /settings/watermark` (any authenticated role — every role needs it to render post previews), `PUT /settings/watermark` (Admin). Element images uploaded through the media presign flow.
- React functional component

2.5.3. Back-end Component(s)

WatermarkConfigurationController / WatermarkConfigurationService

- Reads/writes the single network-wide configuration (institution overrides were removed).
- Spring `@RestController` / `@Service`

WatermarkApplicationService

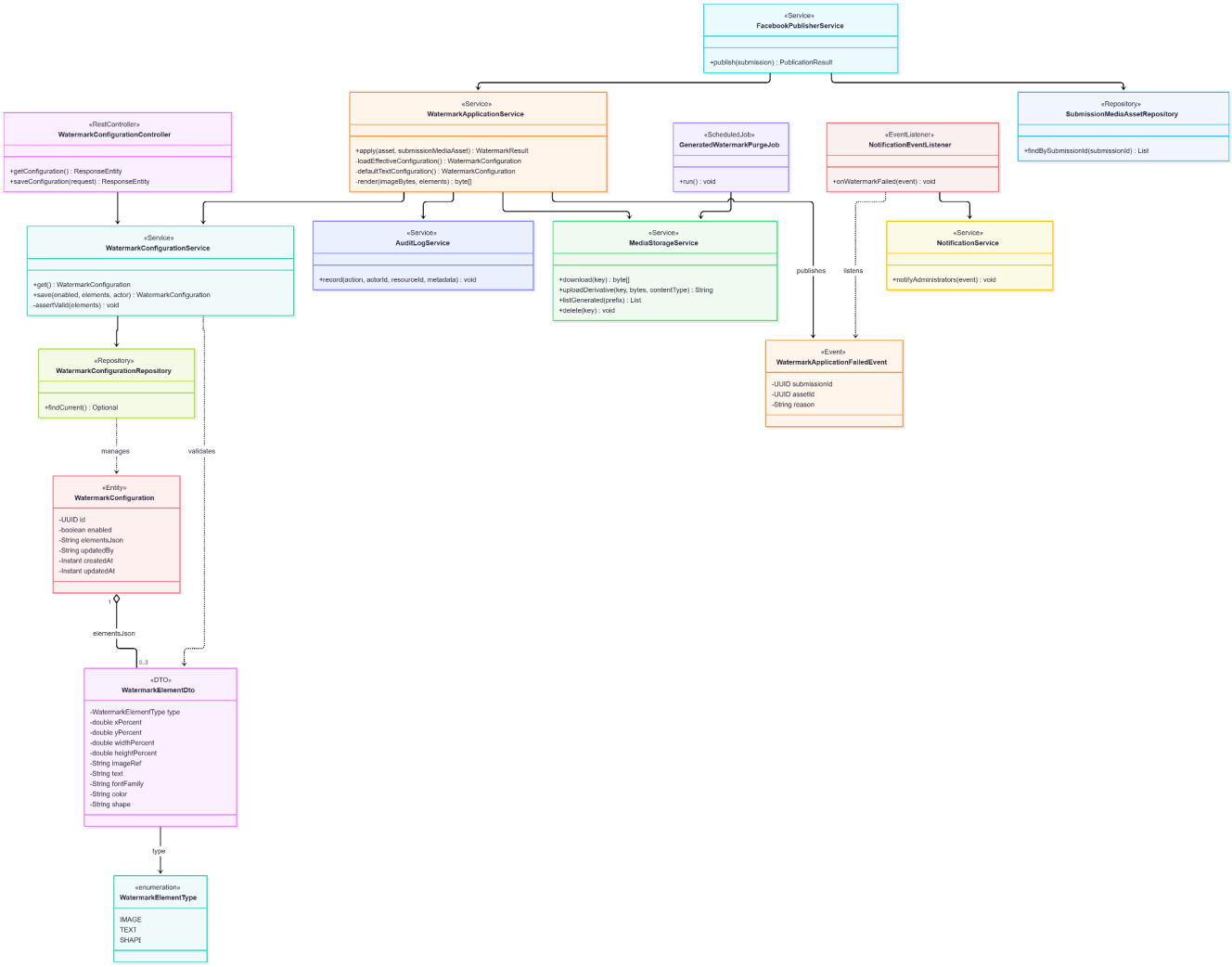
- Called at publish time by `FacebookPublisherService`: honors per-asset `skip_watermark` first, reads the live configuration (a missing configuration falls back to a built-in default text watermark; a disabled one publishes unwatermarked), renders elements scaled to each photo (JPEG/PNG/GIF/WebP), and writes a derivative to `generated/watermarked/` in R2. Videos are never watermarked. A rendering failure publishes the original image, writes `PUBLISH_WATERMARK_FAILED`, and fires `WatermarkApplicationFailedEvent` (in-app + email to every Administrator).
- Spring `@Service`

GeneratedWatermarkPurgeJob / MediaStorageService

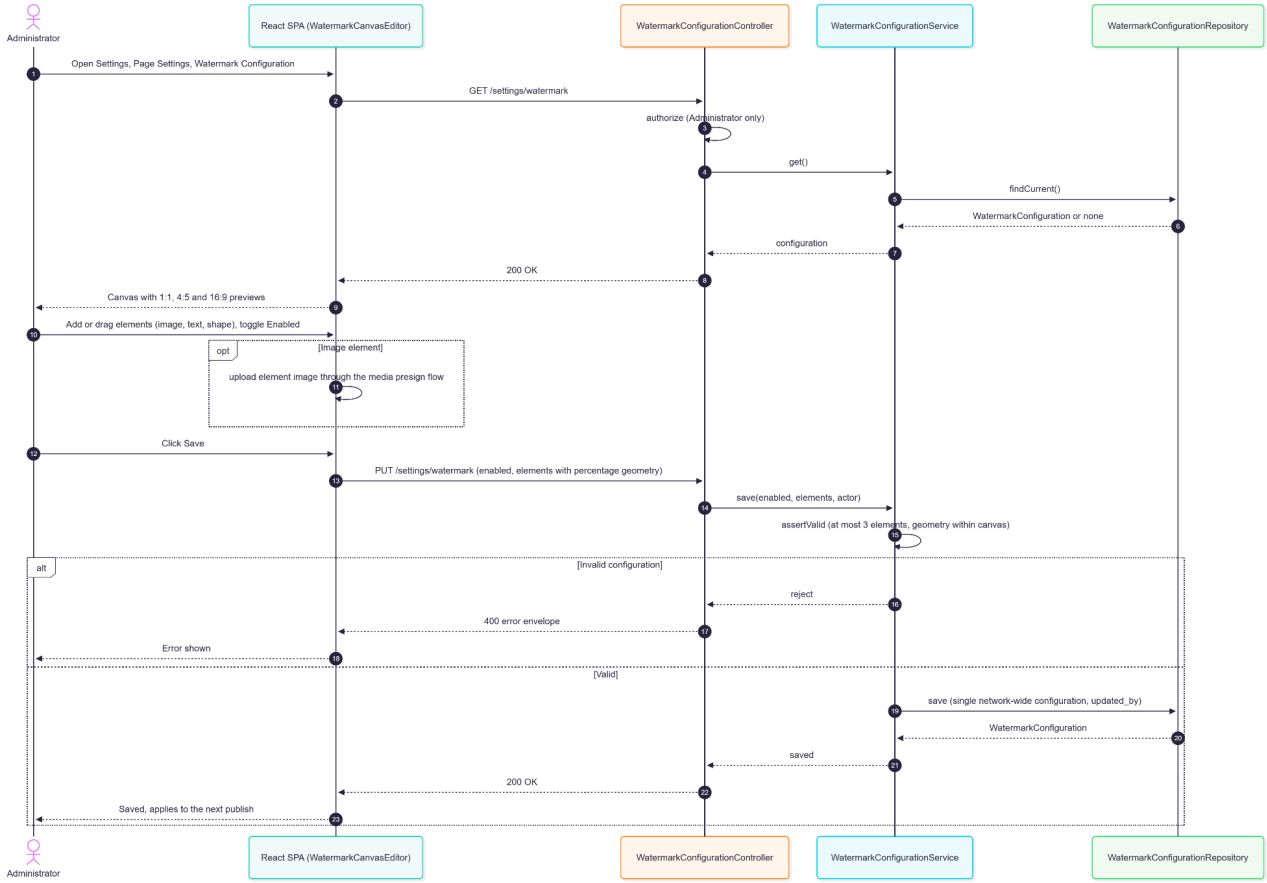
- Daily purge of derivatives older than `app.watermark.generated-retention-days` (default 3).
- Spring `@Scheduled` / `@Service`

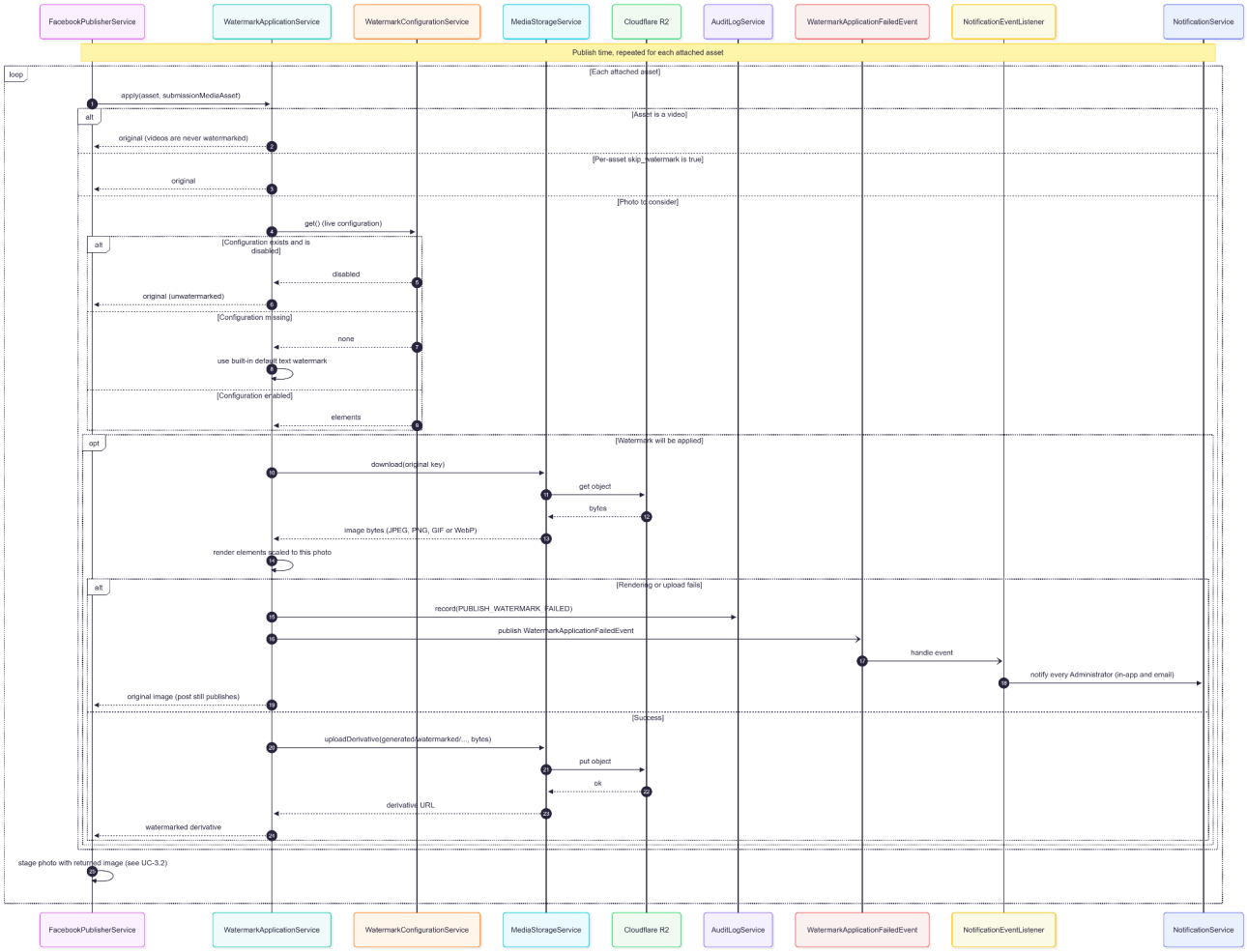
2.5.4. Object-Oriented Component(s)

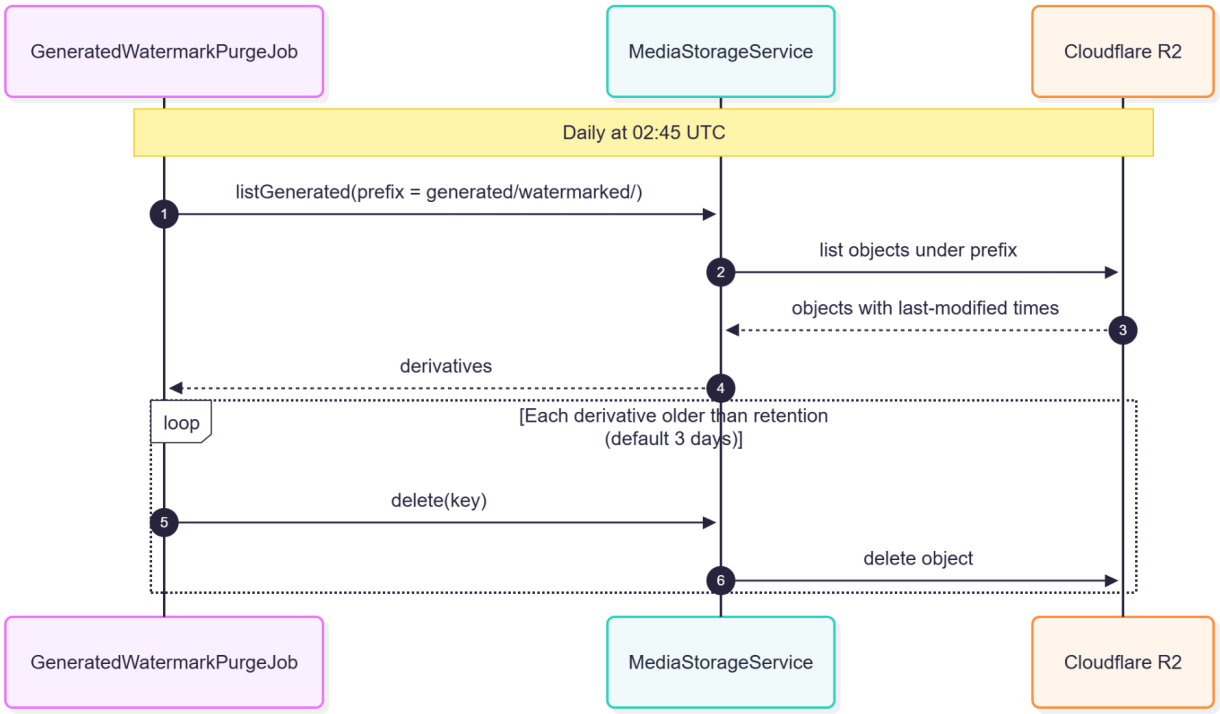
- **Class Diagram**



- **Sequence Diagram**
-

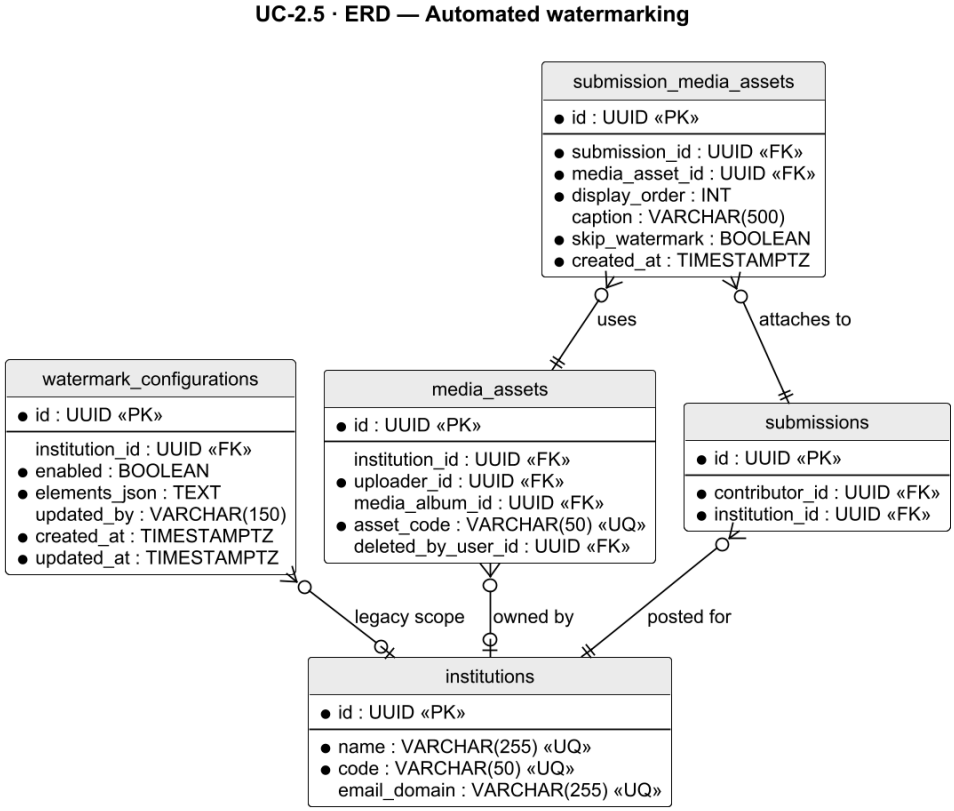

Administrator
React SPA (WatermarkCanvasEditor)
WatermarkConfigurationController
WatermarkConfigurationService
WatermarkConfigurationRepository
Open Settings, Page Settings, Watermark Configuration
GET /settings/watermark
authorize (Administrator only)
get()
findCurrent()
WatermarkConfiguration or none
configuration
200 OK
Canvas with 1:1, 4:5 and 16:9 previews
Add or drag elements (image, text, shape), toggle Enabled
opt
[Image element]
upload element image through the media presign flow
Click Save
PUT /settings/watermark (enabled, elements with percentage geometry)
save(enabled, elements, actor)
assertValid (at most 3 elements, geometry within canvas)
alt
[Invalid configuration]
reject
400 error envelope
Error shown
[Valid]
save (single network-wide configuration, updated_by)
WatermarkConfiguration
saved
200 OK
Saved, applies to the next publish
Administrator
React SPA (WatermarkCanvasEditor)
WatermarkConfigurationController
WatermarkConfigurationService
WatermarkConfigurationRepository


FacebookPublisherService
WatermarkApplicationService
WatermarkConfigurationService
MediaStorageService
Cloudflare R2
AuditLogService
WatermarkApplicationFailedEvent
NotificationEventListener
NotificationService
Publish time, repeated for each attached asset
loop
[Each attached asset]
apply(asset, submissionMediaAsset)
alt
[Asset is a video]
original (videos are never watermarked)
[Per-asset skip_watermark is true]
original
[Photo to consider]
get() (live configuration)
alt
[Configuration exists and is disabled]
disabled
original (unwatermarked)
[Configuration missing]
none
use built-in default text watermark
[Configuration enabled]
elements
opt
[Watermark will be applied]
download(original key)
get object
bytes
image bytes (JPEG, PNG, GIF or WebP)
render elements scaled to this photo
alt
[Rendering or upload fails]
record(PUBLISH_WATERMARK_FAILED)
publish WatermarkApplicationFailedEvent
handle event
notify every Administrator (in-app and email)
original image (post still publishes)
[Success]
uploadDerivative(generated/watermarked/..., bytes)
put object
ok
derivative URL
watermarked derivative
stage photo with returned image (see UC-3.2)


GeneratedWatermarkPurgeJob
MediaStorageService
Cloudflare R2
Daily at 02:45 UTC
listGenerated(prefix = generated/watermarked/)
list objects under prefix
objects with last-modified times
derivatives
loop
[Each derivative older than retention (default 3 days)]
delete(key)
delete object
GeneratedWatermarkPurgeJob
MediaStorageService
Cloudflare R2

2.5.5. Data Design

- **Entity Relationship Diagram**



- **Schema**

```
CREATE TABLE watermark_configurations (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    institution_id UUID REFERENCES institutions(id),   -- unused; kept for FK cleanup on institution
delete
    enabled         BOOLEAN NOT NULL DEFAULT TRUE,
    elements_json  TEXT NOT NULL,                   -- up to 3 elements, percentage geometry
    updated_by      VARCHAR(150),
    created_at      TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at      TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
```

Related tables used by this use case:

```
-- submission_media_assets (defined in UC-1.7)
CREATE TABLE submission_media_assets (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    submission_id  UUID NOT NULL REFERENCES submissions(id) ON DELETE CASCADE,
    media_asset_id UUID NOT NULL REFERENCES media_assets(id),
    display_order  INT NOT NULL DEFAULT 0,
    caption         VARCHAR(500),
    skip_watermark BOOLEAN NOT NULL DEFAULT FALSE,
    created_at      TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    UNIQUE (submission_id, media_asset_id)
);
```

3. Module 3

Module 3 delivers the role-scoped master calendar and rescheduling, the automated Facebook publishing pipeline, the notification system, social engagement analytics, the Administrator System Health dashboard, and the Audit Log.

3.1. Master Calendar Visibility

3.1.1. User Interface Design

Calendar Screen (`/calendar`)

FullCalendar month and week views with a toolbar and legend. Cards are color-coded by state (Pending, In Review, Scheduled, Publishing, Published, Published Manual, Publish Failed, Missed Review — each with its own color and a text label). Moderators and Administrators see every submission network-wide in full detail. Contributors see the network view with full detail for their own institution and DASIG Central Visayas, masked cards (time and institution badge only) for other institutions, and their own in-flight submissions marked as "yours".

Event Detail Modal

A read-only details drawer for the selected card (status, institution, schedule, caption with a Copy action, media). It has no reschedule action.

Reschedule Modal

Triggered only by dragging a `scheduled` card to a new slot (Moderator/Admin); there is currently no keyboard-accessible alternative. Shows the new slot, guard-rail result, and a mandatory justification for every reschedule. Moderators are pre-checked client-side against the 2-reschedule cap and ±1-day window (from the originally approved slot); a hard block tells a Moderator to ask an Administrator, while an Administrator may override with the stated reason.

3.1.2. Front-end Component(s)

CalendarScreen / CalendarView / CalendarToolbar / CalendarLegend / CalendarStates

- `GET /calendar?from=&to=`; `editable` only for Moderator/Admin on `scheduled` events; `handleEventDrop` pre-checks Moderator caps.
- React functional components / Signed-in route

CalendarEventDetailModal

- Event detail and actions.

CalendarRescheduleModal

- `PATCH /submissions/{id}/reschedule` with `scheduledAt`, `reason`, and override flag; surfaces backend messages from `response.data.error`.

3.1.3. Back-end Component(s)

CalendarController

- `GET /calendar` (`isAuthenticated()`), `PATCH /submissions/{id}/reschedule` (`hasAnyRole('ADMIN','MODERATOR')`).
- Spring `@RestController`

CalendarService

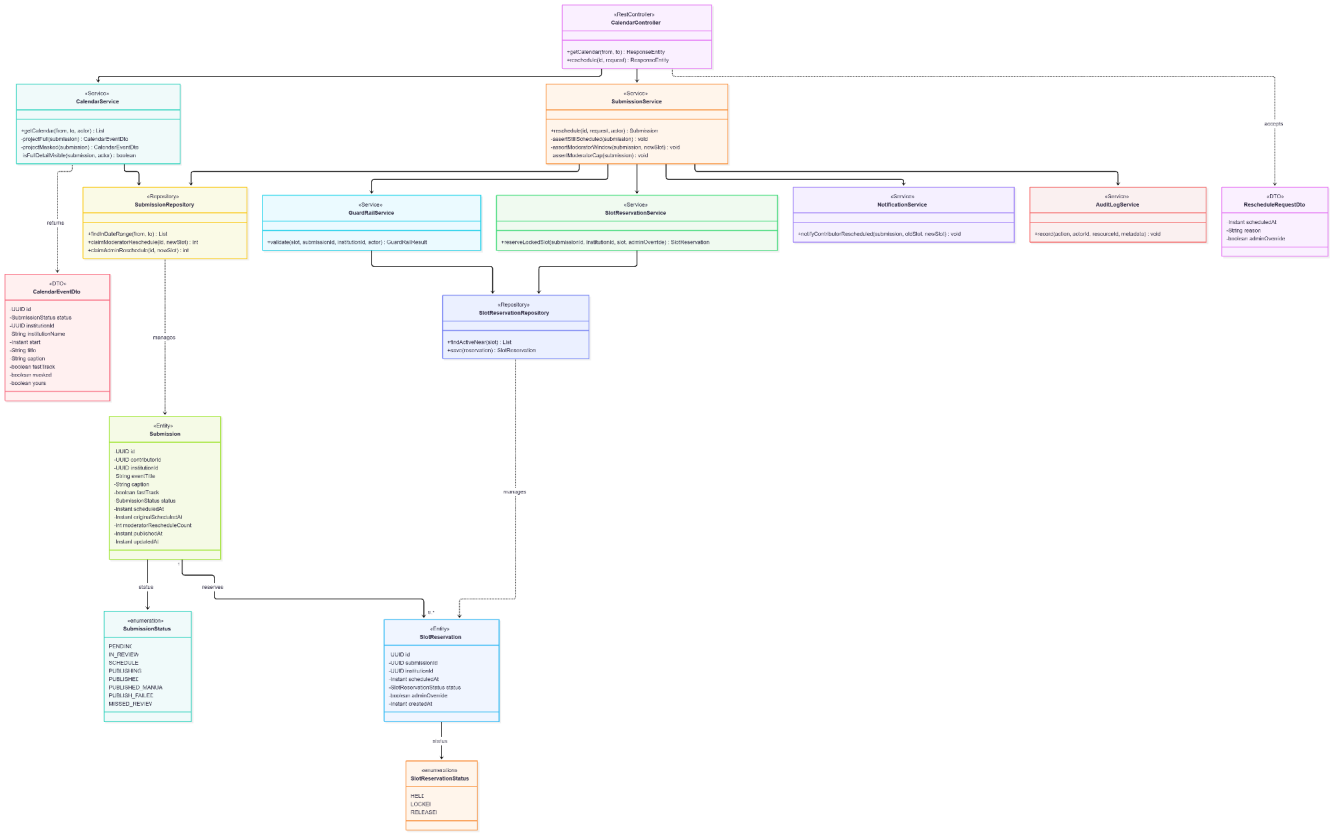
- Role projection: Moderator/Admin → full network calendar; Contributor → masked network view plus own in-flight submissions.
- Spring `@Service`

SubmissionService.reschedule / GuardRailService / SlotReservationService

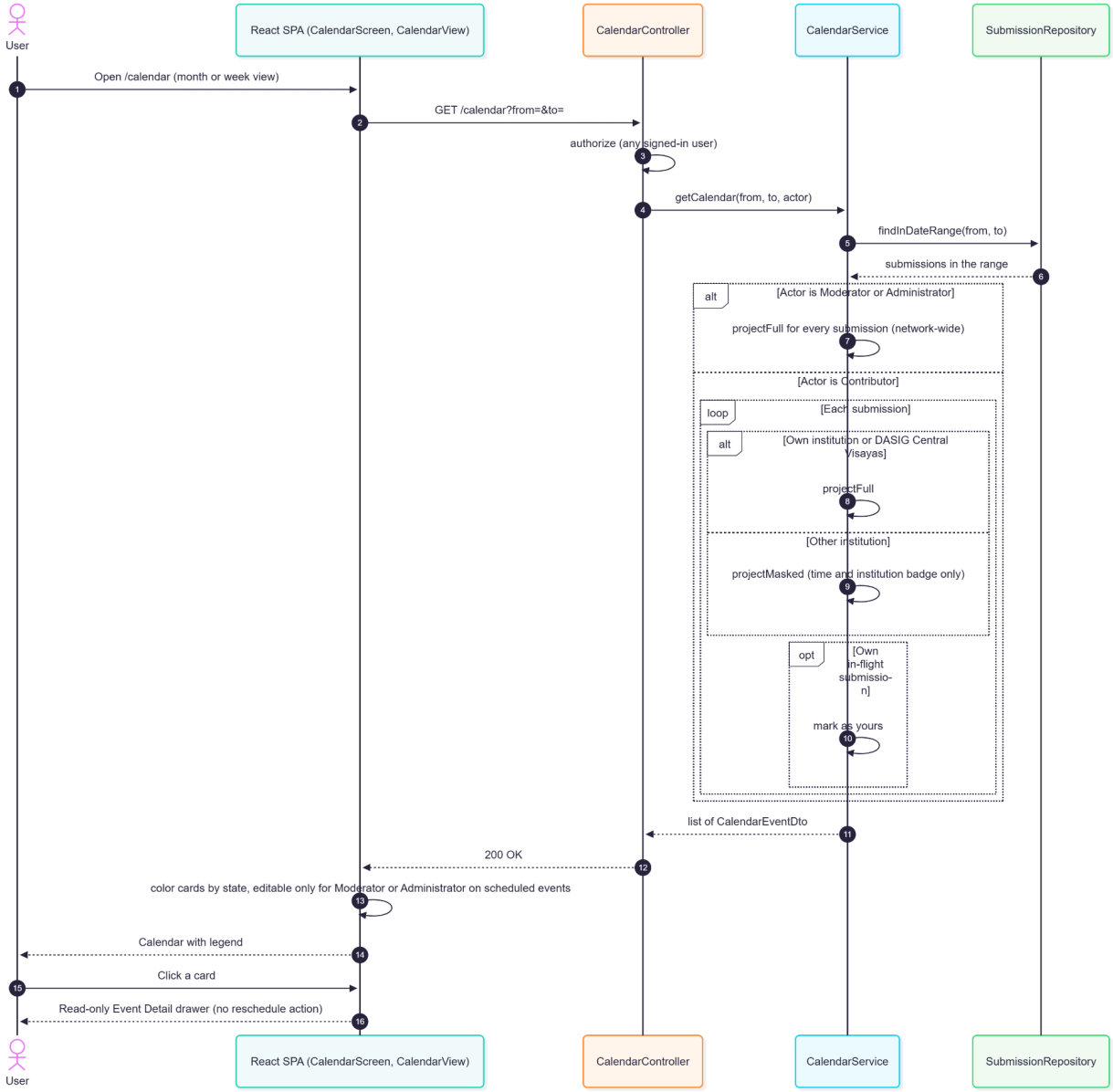
- Server re-validation (must still be `scheduled`; Moderator cap and window; guard rails). Admin override reserves via `reserveLockedSlot(..., adminOverride = true)`. The write is an atomic conditional update (`claimModeratorReschedule` / `claimAdminReschedule`); a claim that affects ≠ 1 row returns `409`. Notifies the contributor (T-10) and audits the reschedule/override.
- Spring `@Service`

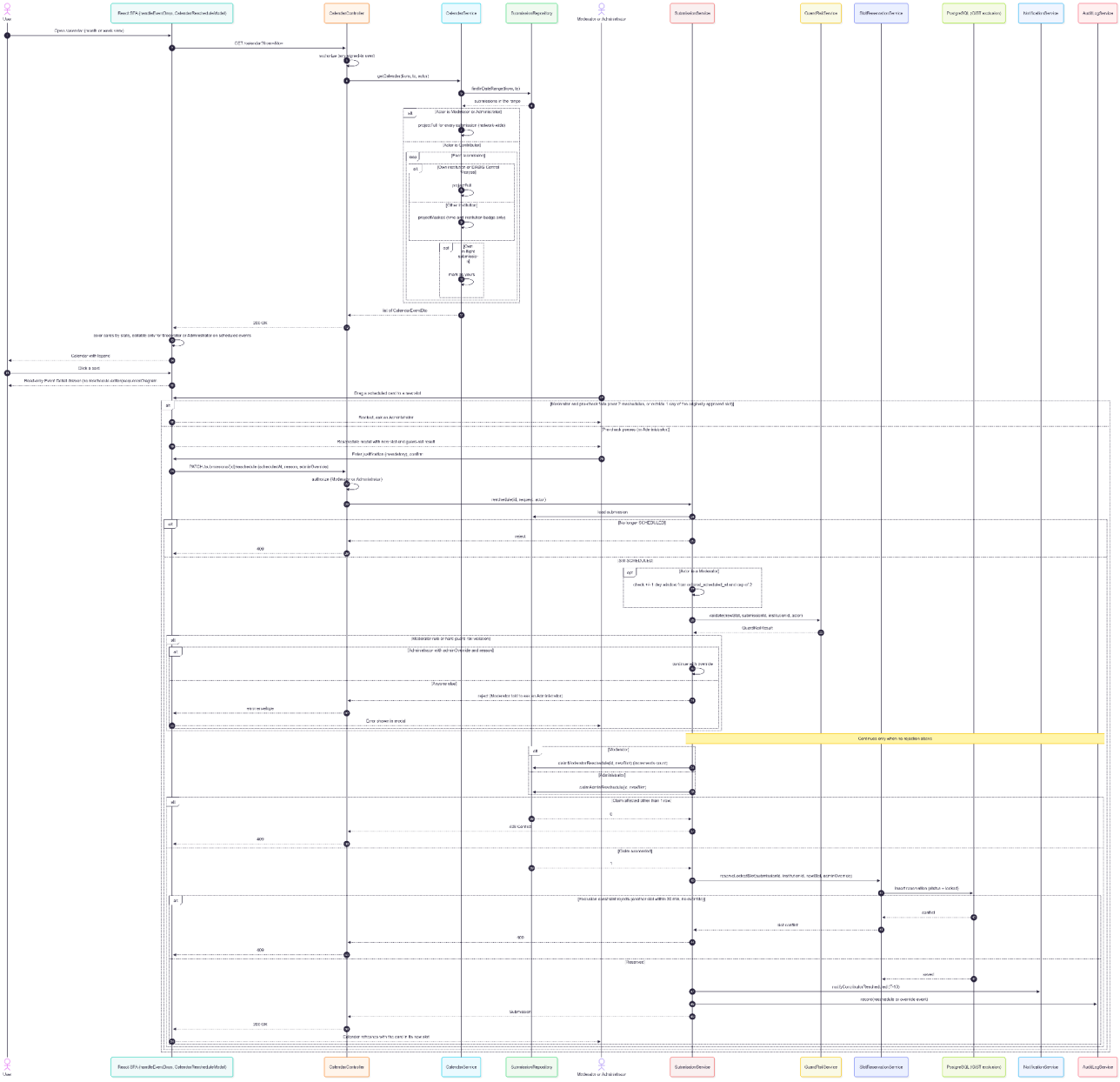
3.1.4. Object-Oriented Component(s)

- **Class Diagram**



- **Sequence Diagram**

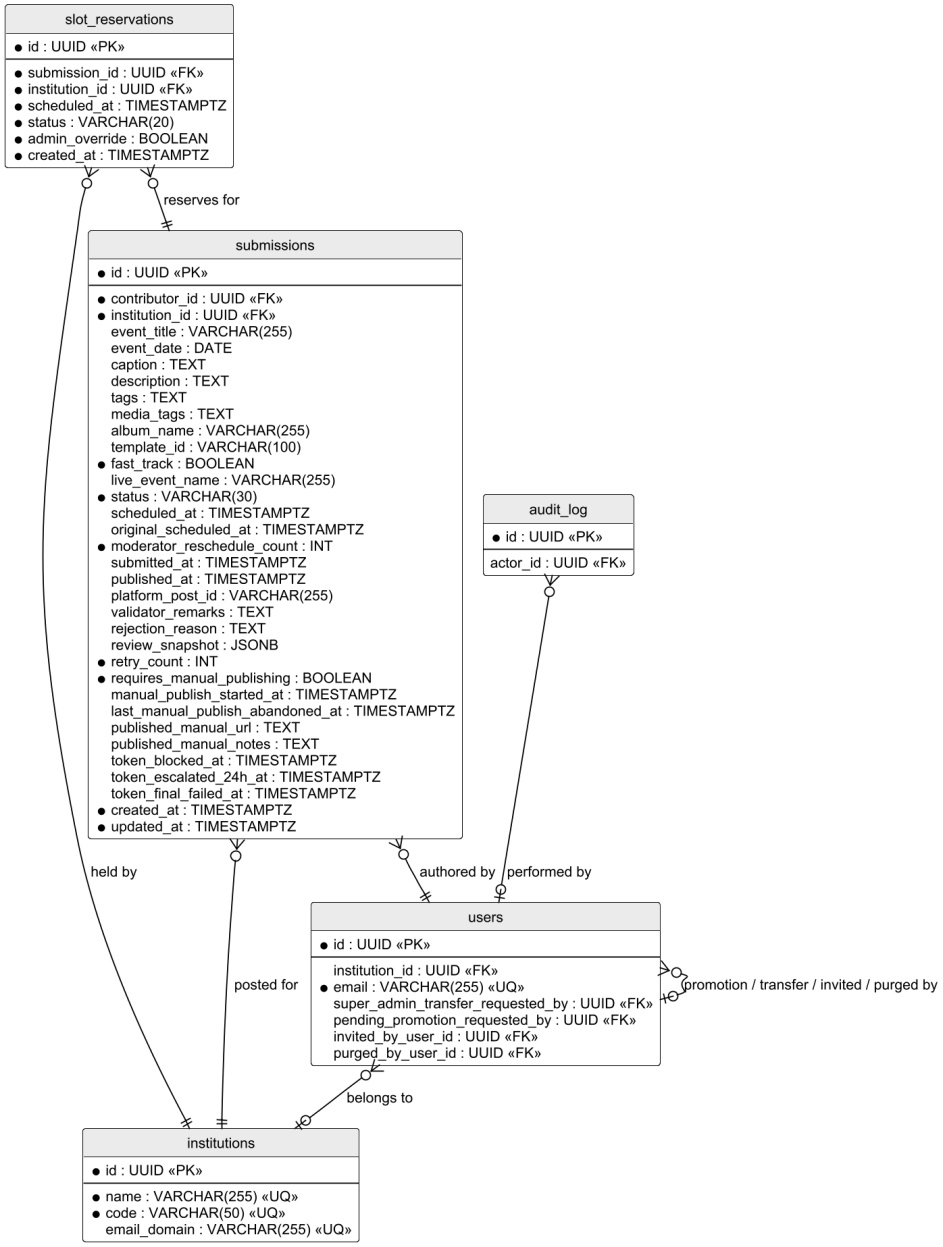

User
React SPA (CalendarScreen, CalendarView)
CalendarController
CalendarService
SubmissionRepository
Open /calendar (month or week view)
GET /calendar?from=&to=
authorize (any signed-in user)
getCalendar(from, to, actor)
findInDateRange(from, to)
submissions in the range
alt
[Actor is Moderator or Administrator]
projectFull for every submission (network-wide)
[Actor is Contributor]
loop
[Each submission]
alt
[Own institution or DASIG Central Visayas]
projectFull
[Other institution]
projectMasked (time and institution badge only)
opt
[Own in-flight submissio-n]
mark as yours
list of CalendarEventDto
200 OK
color cards by state, editable only for Moderator or Administrator on scheduled events
Calendar with legend
Click a card
Read-only Event Detail drawer (no reschedule action)
User
React SPA (CalendarScreen, CalendarView)
CalendarController
CalendarService
SubmissionRepository



3.1.5. Data Design

- **Entity Relationship Diagram**

UC-3.1 · ERD — Master calendar visibility



- **Schema**

```
    -- submissions (columns used by calendar display and rescheduling)
CREATE TABLE submissions (
    id                      UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    contributor_id          UUID NOT NULL REFERENCES users(id),     -- "your submission" marker; hidden on
masked cards
    institution_id          UUID NOT NULL REFERENCES institutions(id), -- card color / masking scope
    event_title             VARCHAR(255),                           -- hidden on masked cards
    caption                 TEXT,                                   -- hidden on masked cards
    fast_track              BOOLEAN NOT NULL DEFAULT FALSE,
    status                  VARCHAR(30) NOT NULL,                   -- card state; only 'scheduled' can be
moved
```

```
    scheduled_at            TIMESTAMPTZ,                            -- calendar position
    original_scheduled_at   TIMESTAMPTZ,                            --  snapshot  at  approval;  anchors  the
±1-day window
    moderator_reschedule_count INTEGER NOT NULL DEFAULT 0,              -- Moderator cap: at most 2
    published_at            TIMESTAMPTZ,                            -- position for published cards
    updated_at              TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- slot_reservations (the slot taken by a scheduled post)
CREATE TABLE slot_reservations (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    submission_id  UUID NOT NULL REFERENCES submissions(id),
    institution_id UUID NOT NULL REFERENCES institutions(id),
    scheduled_at   TIMESTAMPTZ NOT NULL,
    status          VARCHAR(20) NOT NULL DEFAULT 'held'
                    CHECK (status IN ('held', 'locked', 'released')),
    admin_override BOOLEAN NOT NULL DEFAULT FALSE,   -- TRUE only for an Administrator hard-block override
    created_at      TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- GR-H1: no two active reservations within 30 minutes, network-wide (V95/V96)
ALTER TABLE slot_reservations ADD CONSTRAINT excl_slot_reservations_network_buffer
    EXCLUDE USING gist (
            int8range(epoch_seconds_immutable(scheduled_at),
                    epoch_seconds_immutable(scheduled_at) + 1800, '[]') WITH &&
    ) WHERE (status <> 'released' AND NOT admin_override);

-- Indexes
CREATE INDEX idx_submissions_scheduled_at ON submissions(scheduled_at)
    WHERE status IN ('scheduled', 'publishing', 'pending', 'in_review',
                    'direct_post_scheduled', 'direct_post_publishing');   -- calendar date-range query
CREATE INDEX idx_slot_reservations_slot ON slot_reservations(scheduled_at, institution_id)
    WHERE status != 'released';
CREATE INDEX idx_audit_log_resource ON audit_log(resource_id, created_at DESC);  -- reschedule history
```

The reschedule write is an atomic conditional `UPDATE` on `submissions` (`claimModeratorReschedule` / `claimAdminReschedule`) guarded on `status = 'scheduled'` and, for Moderators, `moderator_reschedule_count < 2`; the ±1-day window is checked in `SubmissionService.reschedule()` before the write. A result of zero rows returns `409`.

3.2. Automated Facebook Post Publishing

3.2.1. User Interface Design

This use case is system-driven and has no dedicated screen. Its results are visible on the Calendar (state colors), the Submissions list, notifications, the Review Queue Failed tab (failure reason, unresolved photo IDs), and System Health (token status, job health, publish success rate).

3.2.2. Front-end Component(s)

No dedicated components. Outcomes are rendered by `CalendarScreen`, `SubmissionListScreen`, `ValidationQueueScreen` (Failed tab), `NotificationsScreen`, and `SystemHealthScreen`.

3.2.3. Back-end Component(s)

PublishingSchedulerJob

- Every minute: finds `scheduled` / `direct_post_scheduled` submissions due within a 5-minute lookback and claims each atomically (`scheduled → publishing`) via `PublishingQueryService.claimForPublishing` (conditional `UPDATE` that also bumps `updated_at`) before any Graph API call. No DB transaction is held open across the Graph API call.
- Spring `@Scheduled`

FastTrackPublishingListener

- `@TransactionalEventListener(AFTER_COMMIT)` on `SubmissionApprovedEvent` / `SubmissionFastTrackRetryEvent`; claims and publishes asynchronously without an enclosing transaction.
- Spring component

FacebookPublisherService

- Resolves the active Page token (`findFirstByIsActiveTrue`) and decrypts it; image-only posts stage each photo unpublished with its per-asset caption (watermark applied immediately before staging), then create one `/feed` post with `attached_media`; video-only posts use a single `/videos` call; mixed media fails immediately without retry. Up to 3 attempts with backoff. Tags already in the caption are not appended again. On success: `published`, `platform_post_id`, `published_at`, slot released, T-04. On failure: cleans up staged photos, records unresolved IDs in `publication_attempts.photo_ids_cleanup_failed`, `publish_failed`, T-06. Token failures suspend without consuming retries (`token_blocked_at`). One-time env bootstrap of the token row if the table is empty.
- Spring `@Service`

PublishingQueryService / PublishingStateService — short transactional reads, claims, and state writes around the publish call; `PublishingStateService` performs the stale-detector transitions in their own transactions.

StaleSubmissionDetectorJob

- Every 5 minutes: approved `scheduled` past the window → `publish_failed`; never-reviewed past slot → `missed_review` (slot released; listed in the Failed tab, where Retry with new schedule returns it to `pending` for review; Administrators and the Contributor are notified); stuck `publishing` recovery, including Fast-Track rows (`scheduled_at IS NULL`, `updated_at` older than 5 minutes); token-blocked rows excluded.
- Spring `@Scheduled`

TokenPublishingEscalationJob

- Every 5 minutes: retries token-blocked submissions once `hasUsableActiveToken()`; escalates at 24 hours and fails at 48 hours.
- Spring `@Scheduled`

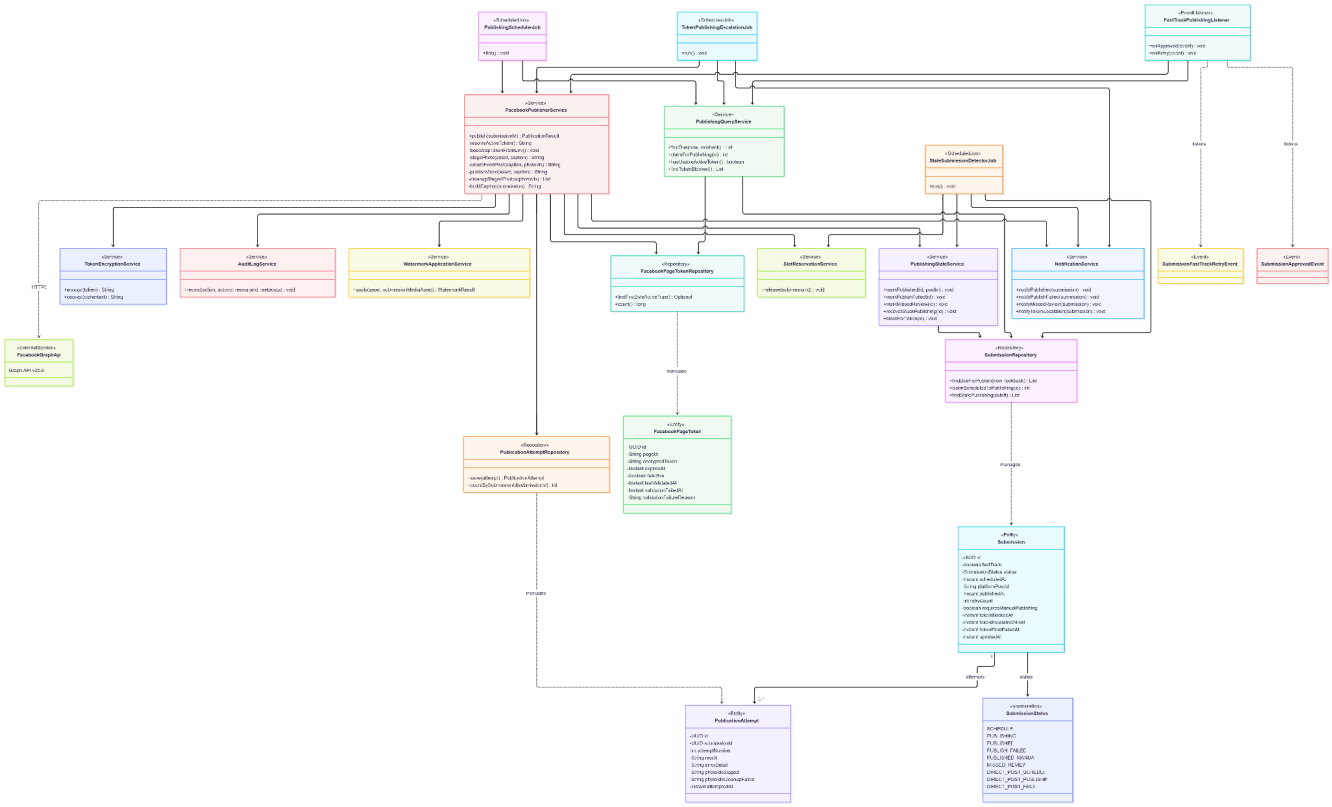
TokenEncryptionService

- AES-256-GCM encryption of Page tokens; the key is the SHA-256 hash of `FACEBOOK_APP_SECRET`
- Spring `@Service`

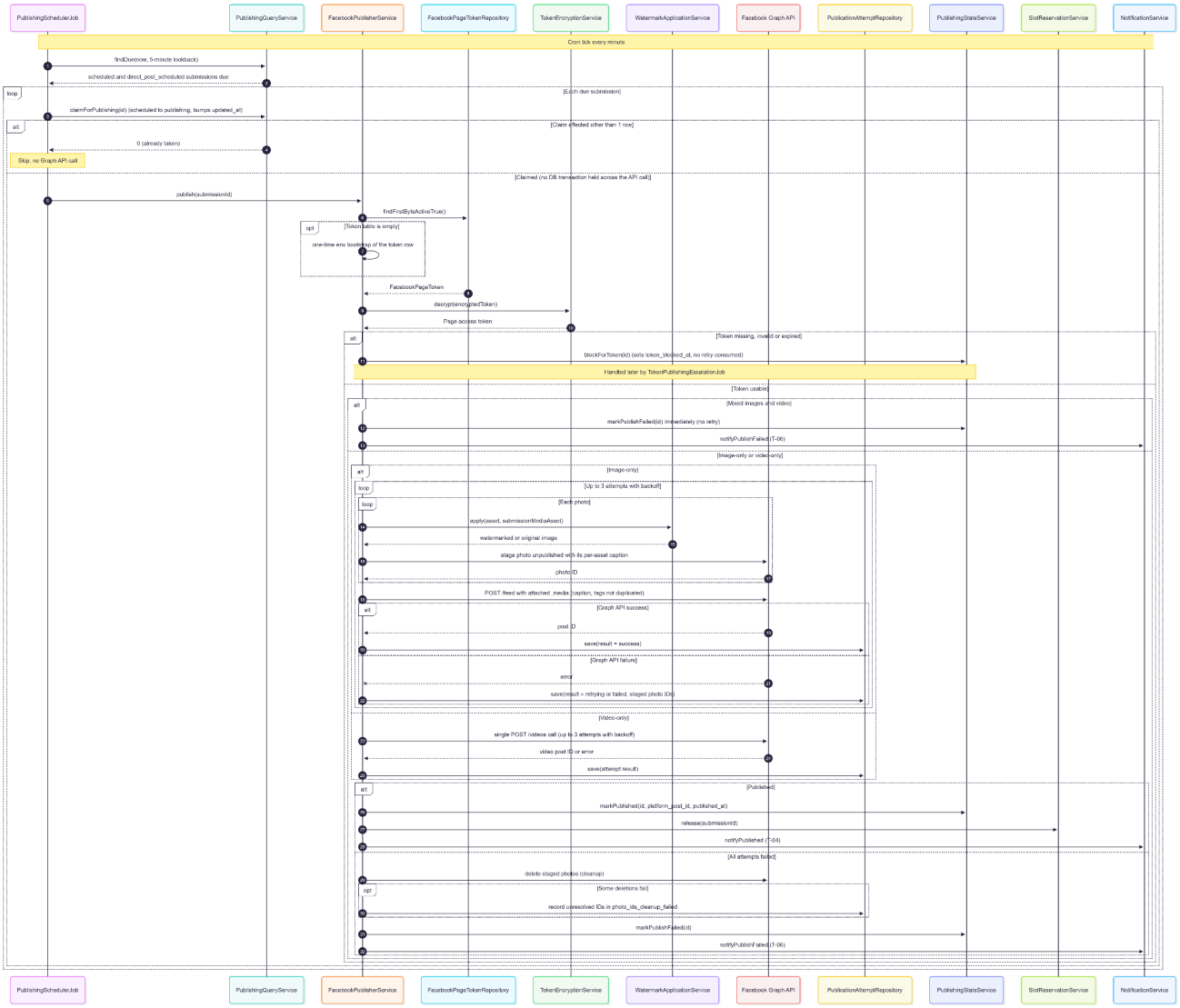
PublicationAttemptRepository / SlotReservationService / NotificationService / AuditLogService

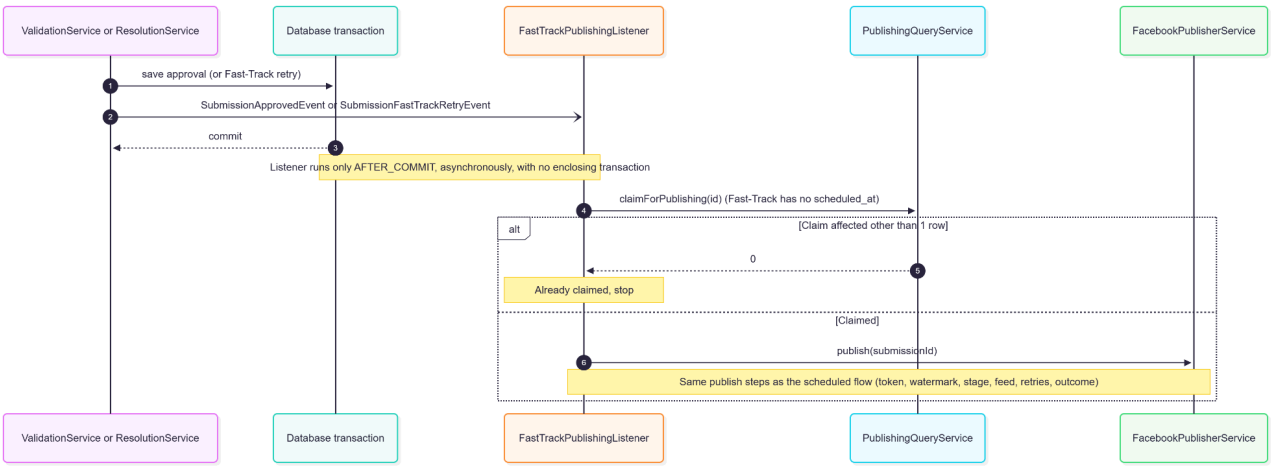
3.2.4. Object-Oriented Component(s)

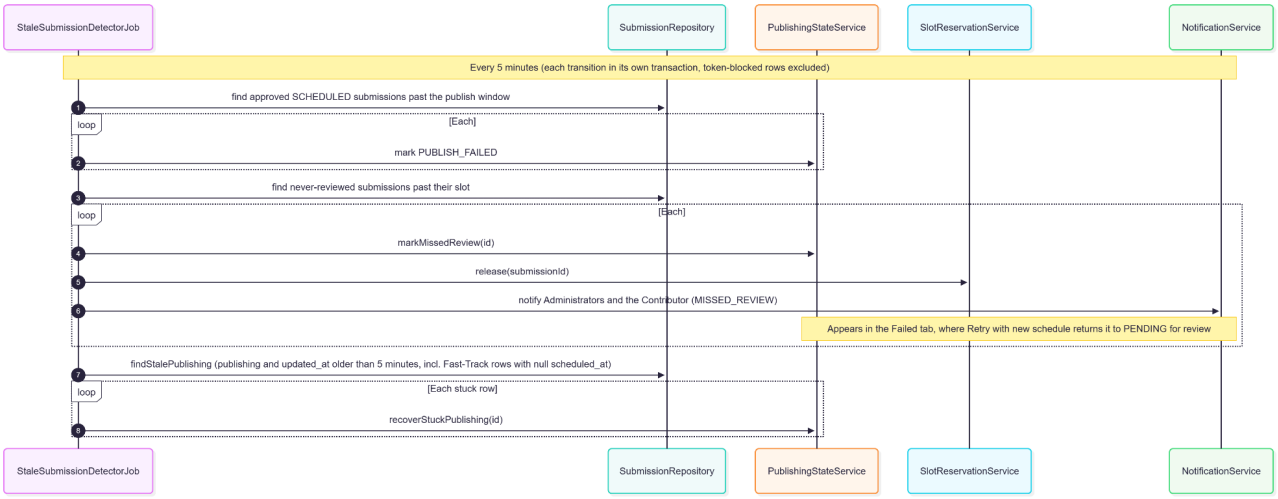
- **Class Diagram**

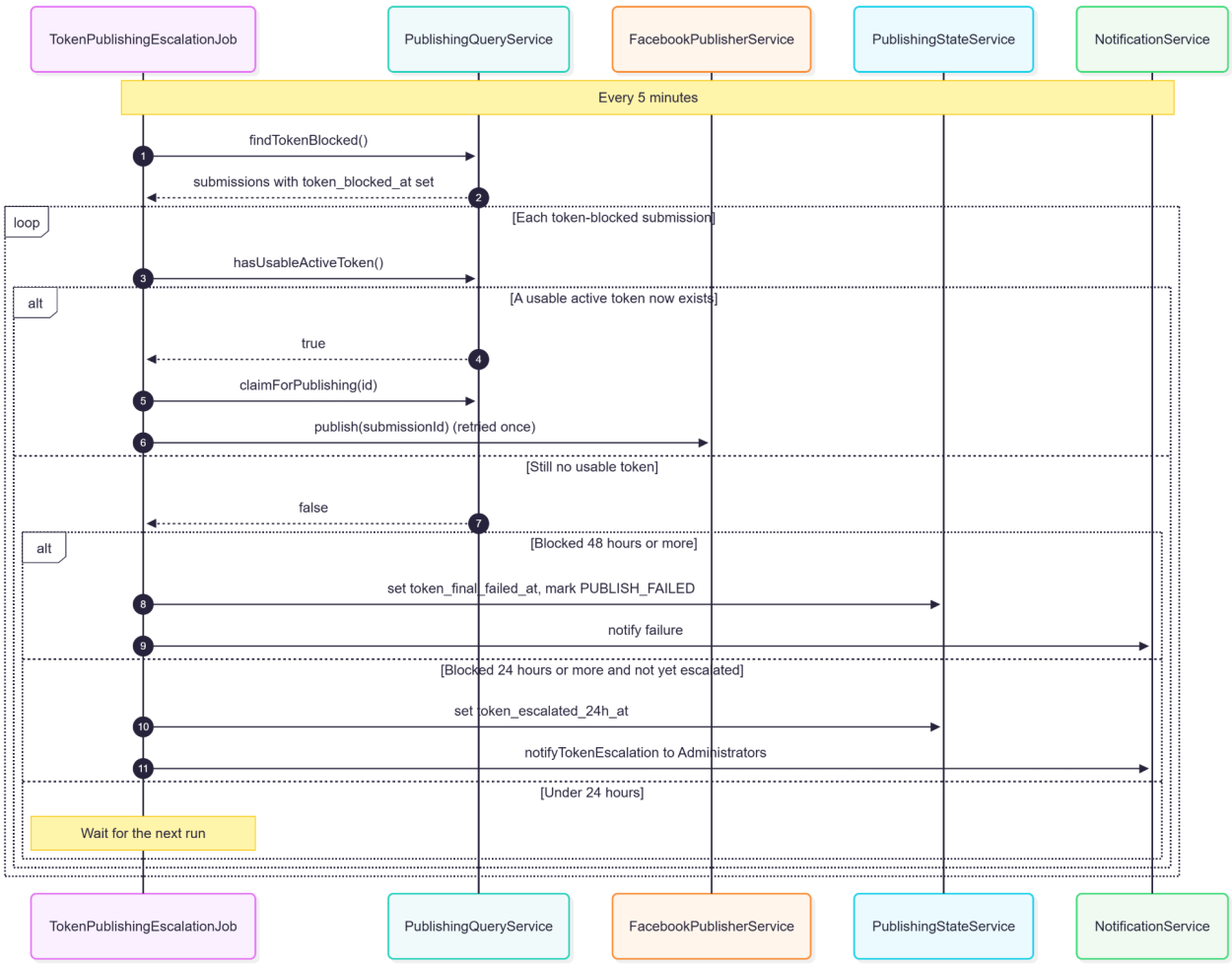


- **Sequence Diagram**


PublishingSchedulerJob
PublishingQueryService
FacebookPublisherService
FacebookPageTokenRepository
TokenEncryptionService
WatermarkApplicationService
Facebook Graph API
PublicationAttemptRepository
PublishingStateService
SlotReservationService
NotificationService
Cron tick every minute
findDue(now, 5-minute lookback)
scheduled and direct_post_scheduled submissions due
loop
[Each due submission]
claimForPublishing(id) (scheduled to publishing, bumps updated_at)
alt
[Claim affected other than 1 row]
0 (already taken)
Skip, no Graph API call
[Claimed (no DB transaction held across the API call)]
publish(submissionId)
findFirstByIsActiveTrue()
opt
[Token table is empty]
one-time env bootstrap of the token row
FacebookPageToken
decrypt(encryptedToken)
Page access token
alt
[Token missing, invalid or expired]
blockForToken(id) (sets token_blocked_at, no retry consumed)
Handled later by TokenPublishingEscalationJob
[Token usable]
alt
[Mixed images and video]
markPublishFailed(id) immediately (no retry)
notifyPublishFailed (T-06)
[Image-only or video-only]
alt
[Image-only]
loop
[Up to 3 attempts with backoff]
loop
[Each photo]
apply(asset, submissionMediaAsset)
watermarked or original image
stage photo unpublished with its per-asset caption
photo ID
POST /feed with attached_media (caption, tags not duplicated)
alt
[Graph API success]
post ID
save(result = success)
[Graph API failure]
error
save(result = retrying or failed, staged photo IDs)
[Video-only]
single POST /videos call (up to 3 attempts with backoff)
video post ID or error
save(attempt result)
alt
[Published]
markPublished(id, platform_post_id, published_at)
release(submissionId)
notifyPublished (T-04)
[All attempts failed]
delete staged photos (cleanup)
opt
[Some deletions fail]
record unresolved IDs in photo_ids_cleanup_failed
markPublishFailed(id)
notifyPublishFailed (T-06)


ValidationService or ResolutionService
Database transaction
FastTrackPublishingListener
PublishingQueryService
FacebookPublisherService
save approval (or Fast-Track retry)
SubmissionApprovedEvent or SubmissionFastTrackRetryEvent
commit
Listener runs only AFTER_COMMIT, asynchronously, with no enclosing transaction
claimForPublishing(id) (Fast-Track has no scheduled_at)
alt
[Claim affected other than 1 row]
0
Already claimed, stop
[Claimed]
publish(submissionId)
Same publish steps as the scheduled flow (token, watermark, stage, feed, retries, outcome)

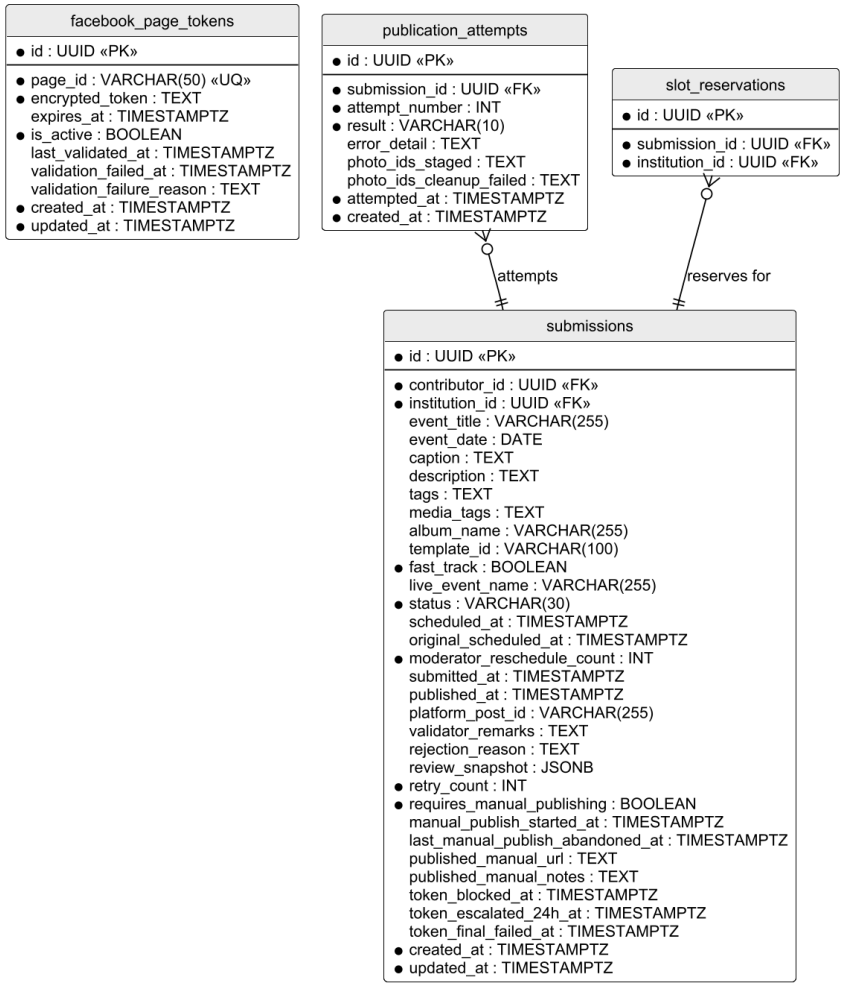




3.2.5. Data Design

- **Entity Relationship Diagram**

UC-3.2 · ERD — Automated Facebook post publishing



- **Schema**

```
-- Connected Facebook Page(s); exactly one row is active (DB-driven, not env-driven)
CREATE TABLE facebook_page_tokens (
    id                      UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    page_id                 VARCHAR(50) UNIQUE NOT NULL,
    encrypted_token         TEXT NOT NULL,          -- AES-256-GCM (IV + tag stored with ciphertext)
    expires_at              TIMESTAMPTZ,
    is_active               BOOLEAN NOT NULL DEFAULT FALSE,
    last_validated_at       TIMESTAMPTZ,
    validation_failed_at      TIMESTAMPTZ,
    validation_failure_reason TEXT,
    created_at              TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at              TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE TABLE publication_attempts (
    id                      UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    submission_id           UUID NOT NULL REFERENCES submissions(id) ON DELETE CASCADE,
    attempt_number          INT NOT NULL,
    result                  VARCHAR(10) NOT NULL,   -- success | failed | retrying
    error_detail            TEXT,
    photo_ids_staged        TEXT,
    photo_ids_cleanup_failed TEXT,                  -- surfaced in the Failed tab (V97)
    attempted_at            TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    created_at              TIMESTAMPTZ NOT NULL DEFAULT NOW()
```

```
);

CREATE INDEX idx_submissions_due_for_publish ON submissions(scheduled_at) WHERE status = 'scheduled';
CREATE INDEX idx_pub_attempts_submission ON publication_attempts(submission_id, attempted_at DESC);
```

Related tables used by this use case:

```
-- submissions (defined in UC-1.5)
CREATE TABLE submissions (
    id                              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    contributor_id                  UUID NOT NULL REFERENCES users(id),
    institution_id                  UUID NOT NULL REFERENCES institutions(id),
    event_title                     VARCHAR(255),
    event_date                      DATE,
    caption                         TEXT,
    description                     TEXT,
    tags                            TEXT,
    media_tags                      TEXT,
    album_name                      VARCHAR(255),
    template_id                     VARCHAR(100),
    fast_track                      BOOLEAN NOT NULL DEFAULT FALSE,
    live_event_name                 VARCHAR(255),
    status                          VARCHAR(30) NOT NULL DEFAULT 'draft'
                                    CHECK (status IN ('draft', 'pending', 'in_review', 'needs_revision',
                                           'missed_review', 'scheduled', 'publishing', 'publish_failed',
                                           'published', 'published_manual', 'admin_direct_post',
                                           'direct_post_scheduled', 'direct_post_publishing',
                                           'direct_post_failed', 'rejected')),
    scheduled_at                    TIMESTAMPTZ,
    original_scheduled_at           TIMESTAMPTZ,
    moderator_reschedule_count      INT NOT NULL DEFAULT 0,
    submitted_at                    TIMESTAMPTZ,
    published_at                    TIMESTAMPTZ,
    platform_post_id                VARCHAR(255),
    validator_remarks               TEXT,
    rejection_reason                TEXT,
    review_snapshot                 JSONB,
    retry_count                     INT NOT NULL DEFAULT 0,
    requires_manual_publishing      BOOLEAN NOT NULL DEFAULT FALSE,
    manual_publish_started_at       TIMESTAMPTZ,
    last_manual_publish_abandoned_at TIMESTAMPTZ,
    published_manual_url            TEXT,
    published_manual_notes          TEXT,
    token_blocked_at                TIMESTAMPTZ,
    token_escalated_24h_at          TIMESTAMPTZ,
    token_final_failed_at           TIMESTAMPTZ,
    created_at                      TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at                      TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
```

3.3. Automated Reminders & Alerts

3.3.1. User Interface Design

Notification Bell (global header)

An unread badge kept current through its own SSE subscription (refetches the true count on `notification`, `read`, and `read-all` events), with a 3-minute poll only as a dropped-connection fallback.

Notifications Screen (`/notifications`)

A paginated list (newest first) with event icon, message, relative time, and unread indicator; clicking navigates to the item's deep link and marks it read; "Mark All as Read" in the header; a history view for older items. Read-state changes are pushed to all of the user's open sessions.

Email Notification

A branded HTML email with a CTA deep link, sent for triggers configured with an email channel and respecting the user's channel preferences.

Messenger Notification

A Page message to Moderators/Administrators who linked Messenger (UC-1.4 A8), for T-01, T-06, and T-11.

3.3.2. Front-end Component(s)

NotificationsScreen / useNotifications

- `GET /notifications`, `GET /notifications/history`, `PATCH /notifications/{id}/read`, `PATCH /notifications/read-all`, `GET /notifications/stream` (SSE); applies `read`/`read-all` events idempotently.
- React functional component / hook

useNotificationUnreadCount (navbar bell)

- `GET /notifications/unread-count` + its own SSE subscription.
- React hook

3.3.3. Back-end Component(s)

NotificationController

- List, history, unread count, mark read, mark all read, and SSE stream.
- Spring `@RestController`

NotificationService / NotificationEventListener

- Resolves recipients by trigger category (review triggers → active Moderators and Administrators; content outcomes → the contributor; system/infrastructure → Administrators), persists `notifications`, pushes SSE to every live emitter of the recipient, dispatches email and Messenger per trigger configuration, and pushes `read`/`read-all` events after `markRead`/`markAllRead`. Each event creates its own notification, except publish failures: when more than 5 `PUBLISH_FAILED` audit entries occur within an hour, individual T-06 alerts are suppressed and Administrators receive one consolidated high-volume alert (GR-T6). Covers the 27 `NotificationEventType` values, including the named T-01–T-12 triggers. Both channels honour the recipient's in-app and email preferences (users.notify_in_app, users.notify_email).
- Spring `@Service` / event listener

EmailService / EmailDeliveryService

- Resend API dispatch, one attempt per email with no automatic retry; each attempt is recorded in `email_delivery_log` (queued, then sent or failed); email failure never blocks in-app delivery. Invitations and password resets go through EmailService and ignore the notification preference.

- Spring `@Service`

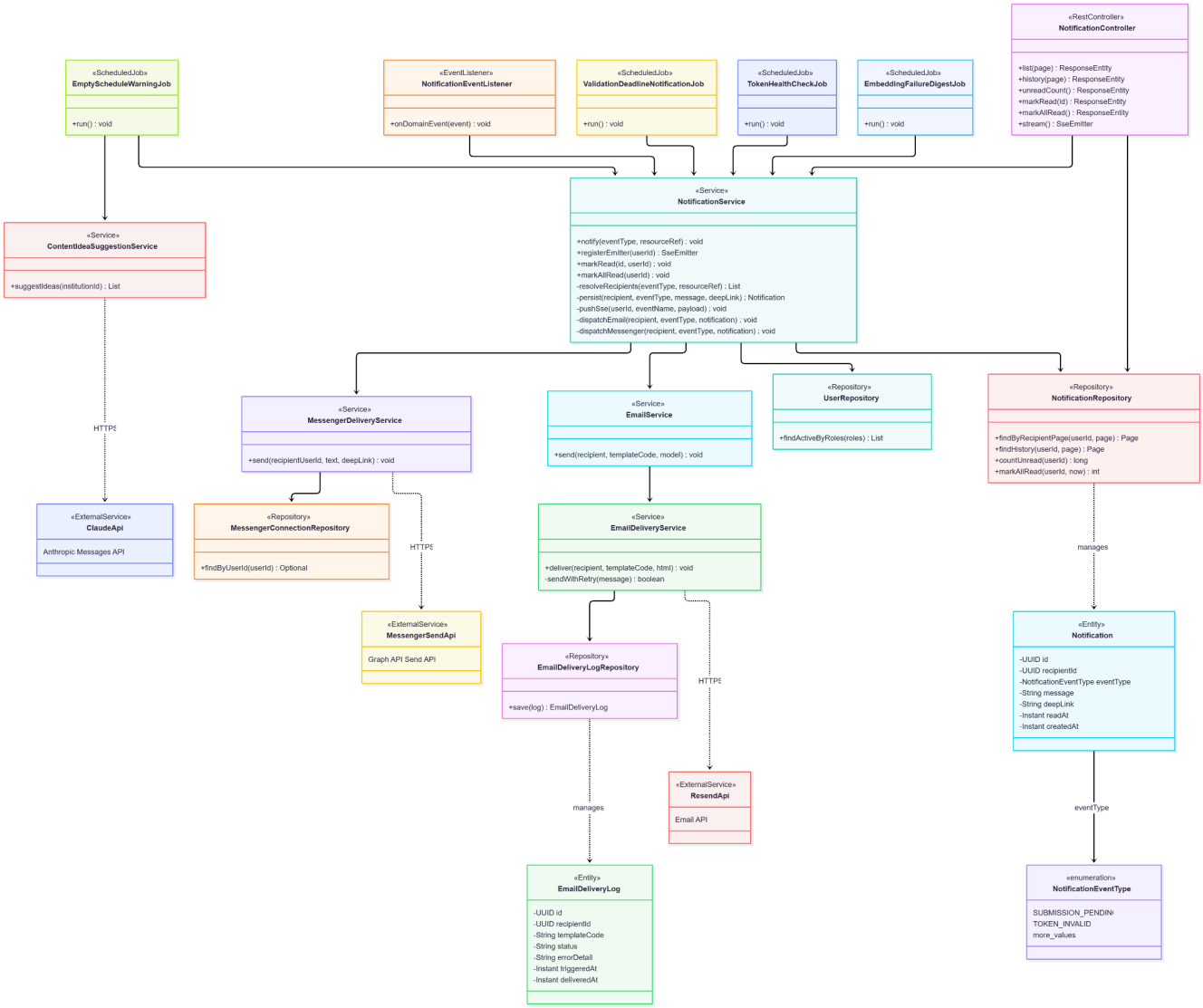
MessengerDeliveryService — Graph API Send API to linked recipients; unlinked recipients are silently skipped.

Scheduled triggers

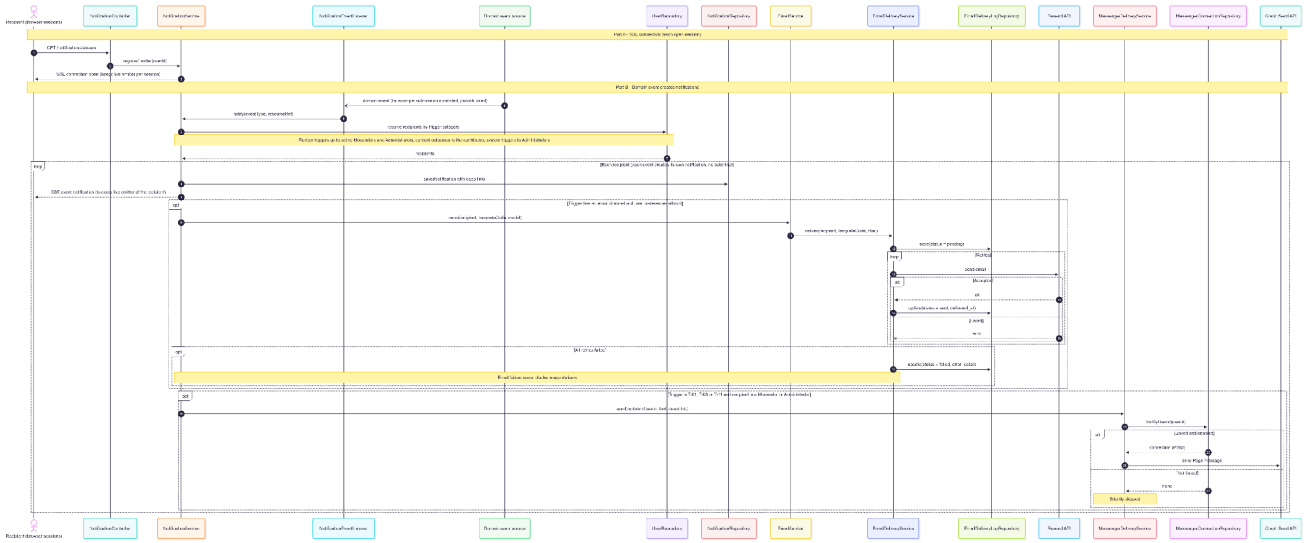
- `ValidationDeadlineNotificationJob` (deadline alerts), `TokenHealthCheckJob` (T-08, T-09), `EmptyScheduleWarningJob` + `ContentIdeaSuggestionService` (T-07 with Claude-generated content ideas from cross-institution signals and the institution's 10 most recently published titles), `EmbeddingFailureDigestJob` (T-12).

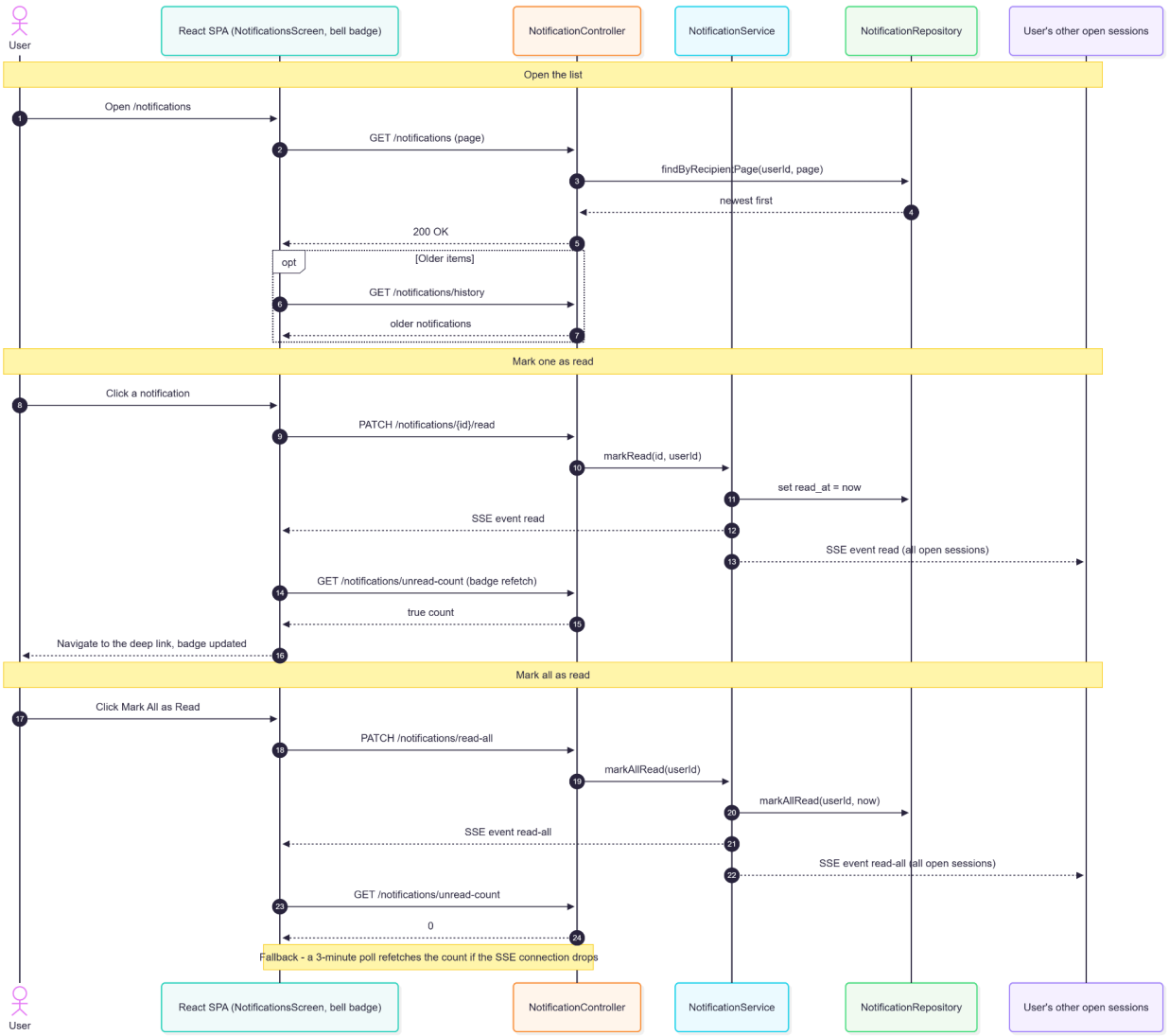
3.3.4. Object-Oriented Component(s)

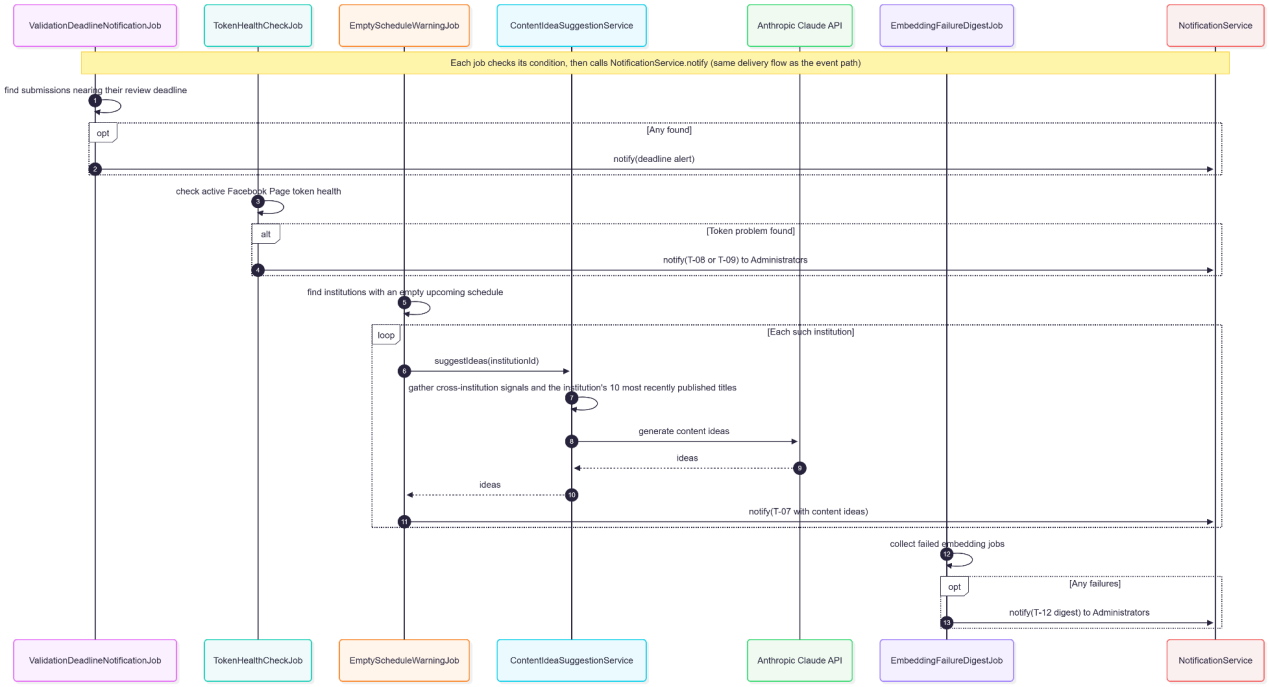
- **Class Diagram**



- **Sequence Diagram**



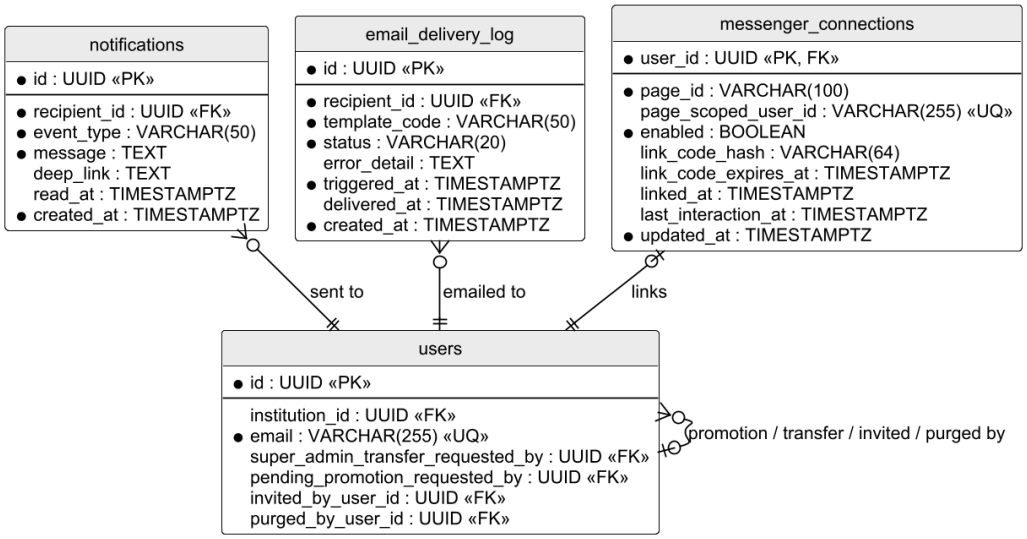

User
React SPA (NotificationsScreen, bell badge)
NotificationController
NotificationService
NotificationRepository
User's other open sessions
Open the list
1 Open /notifications
2 GET /notifications (page)
3 findByRecipientPage(userId, page)
4 newest first
5 200 OK
opt
[Older items]
6 GET /notifications/history
7 older notifications
Mark one as read
8 Click a notification
9 PATCH /notifications/{id}/read
10 markRead(id, userId)
11 set read_at = now
12 SSE event read
13 SSE event read (all open sessions)
14 GET /notifications/unread-count (badge refetch)
15 true count
16 Navigate to the deep link, badge updated
Mark all as read
17 Click Mark All as Read
18 PATCH /notifications/read-all
19 markAllRead(userId)
20 markAllRead(userId, now)
21 SSE event read-all
22 SSE event read-all (all open sessions)
23 GET /notifications/unread-count
24 0
Fallback - a 3-minute poll refetches the count if the SSE connection drops
User
React SPA (NotificationsScreen, bell badge)
NotificationController
NotificationService
NotificationRepository
User's other open sessions



3.3.5. Data Design

- **Entity Relationship Diagram**

UC-3.3 · ERD — Automated reminders and alerts



- **Schema**

```
CREATE TABLE notifications (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    recipient_id UUID NOT NULL REFERENCES users(id),
    event_type    VARCHAR(50) NOT NULL,      -- NotificationEventType (e.g. submission_pending,
token_invalid)
    message         TEXT NOT NULL,
    deep_link       TEXT,                           -- in-app route of the affected resource
    read_at         TIMESTAMPTZ,                    -- NULL = unread
    created_at    TIMESTAMPTZ NOT NULL DEFAULT NOW()
```

```
);

CREATE TABLE email_delivery_log (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    recipient_id    UUID NOT NULL REFERENCES users(id),
    template_code   VARCHAR(50) NOT NULL,
    status          VARCHAR(20) NOT NULL,   -- pending | sent | failed
    error_detail    TEXT,
    triggered_at    TIMESTAMPTZ NOT NULL,
    delivered_at    TIMESTAMPTZ,
    created_at      TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_notifications_recipient_created_at ON notifications(recipient_id, created_at DESC);
CREATE INDEX idx_notifications_recipient_read_at    ON notifications(recipient_id, read_at);

        Related tables used by this use case:

-- messenger_connections (defined in UC-1.4)
CREATE TABLE messenger_connections (
    user_id                 UUID PRIMARY KEY REFERENCES users(id) ON DELETE CASCADE,
    page_id                 VARCHAR(100) NOT NULL,
    page_scoped_user_id     VARCHAR(255) UNIQUE,       -- Messenger PSID once linked
    enabled                 BOOLEAN NOT NULL DEFAULT FALSE,
    link_code_hash          VARCHAR(64),
    link_code_expires_at    TIMESTAMPTZ,               -- 10 minutes
    linked_at               TIMESTAMPTZ,
    last_interaction_at     TIMESTAMPTZ,
    updated_at              TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
```

3.4. Social Engagement Analytics

3.4.1. User Interface Design

Analytics Dashboard (`/analytics`)

Available to all three roles with role-scoped data. A date-range picker (7d, 30d, 90d, YTD, custom) and, for Administrators only, a multi-select institution filter. The dashboard auto-refreshes every 5 minutes while visible and has a manual Refresh button.

- **Executive summary and KPI tiles** — Average Posting Delay, Content Completeness (95% target), Total Posts Published, each with a period-over-period delta and sparkline; clicking opens the Full Report.
- **Publishing trend and status breakdown** charts.
- **Facebook Engagement** — reach, reactions, comments, shares from the cached engagement sync, with a "pending updated figures" count and last-refresh time.
- **Posts by Institution** chart (Moderator/Admin, network-wide).
- **Administrator only** — Operational Health (including On-Time Publication with its 95% target badge), AI Feature Adoption (insufficient-data notice below 20 events), Page Performance (live Page insights, degrades gracefully), and — after drilling into one institution — Review Workload and Contributor Breakdown.

Full Report Modal

Daily-breakdown and detail-row tabs for a metric, with Export CSV; export failures show an inline error with "Retry Download".

3.4.2. Front-end Component(s)

AnalyticsDashboardPage

- `GET /analytics/summary?from=&to=&institutionId=`; role-aware panels.
- React functional component / Signed-in route

KpiTileGroup / ExecutiveSummaryStrip / PublishingTrendChart / StatusBreakdownPanel / StatusDonutChart / ContentIssuesPanel / RoleMetricPanel / ContributorAnalyticsView

- Shared dashboard panels.

FacebookEngagementPanel / SocialEngagementCard / PagePerformanceCard / PostsByInstitutionChart / OperationsAndEngagementCard

- Engagement and network panels.

AdminAnalyticsPanel / OperationalHealthPanel / ContributorBreakdownTable

- Administrator panels.

FullReportModal / AnalyticsDateRangePicker

- `GET /analytics/report/{metric}`, `GET /analytics/export/{metric}` (CSV).

3.4.3. Back-end Component(s)

AnalyticsController

- `GET /analytics/summary`, `GET /analytics/report/{metric}`, `GET /analytics/export/{metric}`; `isAuthenticated()`.
- Spring `@RestController`

MetricsAggregatorService

- `scopeFor()`: Contributor → own institution (any `institutionId` param → `403`); Moderator → whole network (institution filter → `403`); Admin → network with multi-institution filter. `summary` is `@Cacheable` (60 s). `operational-health` and `ai-performance` metrics are Admin-only (`403` otherwise); `posting-delay`, `content-completeness`, `posts-by-institution`, and `facebook-engagement` are open to all roles within scope. `escapeCsv` neutralizes leading `= + - @`, tab, and CR.
- Spring `@Service`

AnalyticsRepository / PublishSuccessRateRepository

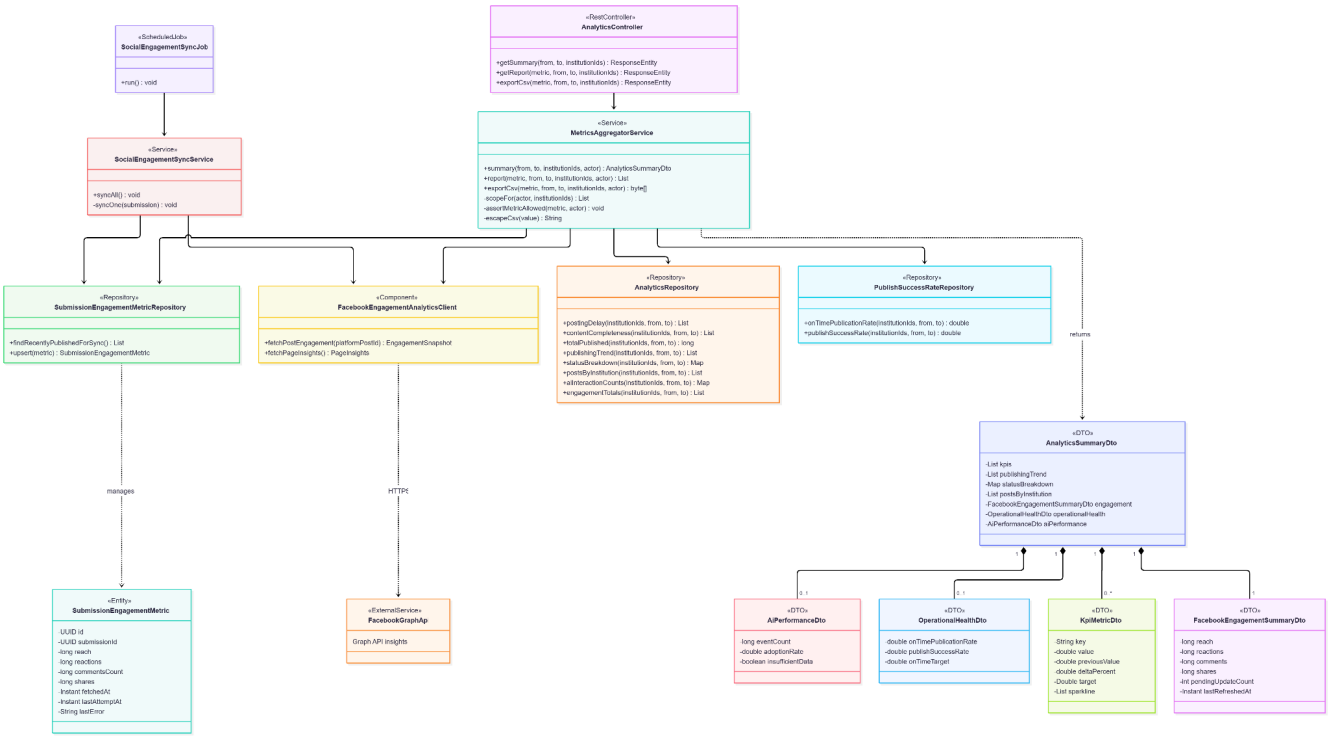
- Native aggregation queries filtered by `institution_id IN (:institutionIds)`; on-time publication computed as `|published_at - scheduled_at| ≤ 300 s`.
- Spring `@Repository`

SocialEngagementSyncService / SocialEngagementSyncJob / FacebookEngagementAnalyticsClient

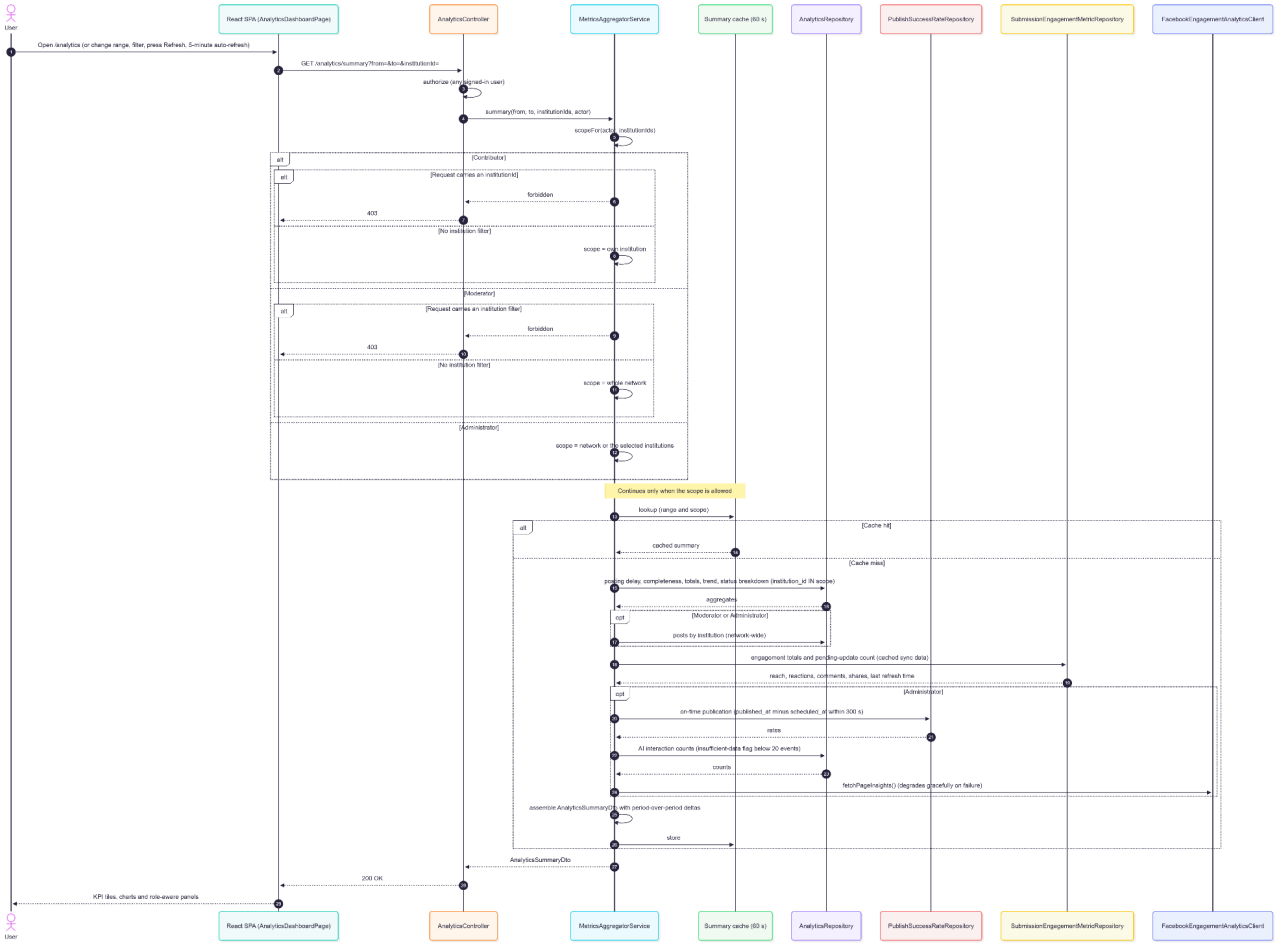
- Every 15 minutes, refreshes per-post reach, reactions, comments, and shares into `submission_engagement_metrics`.
- Spring `@Service` / `@Scheduled` / `@Component`

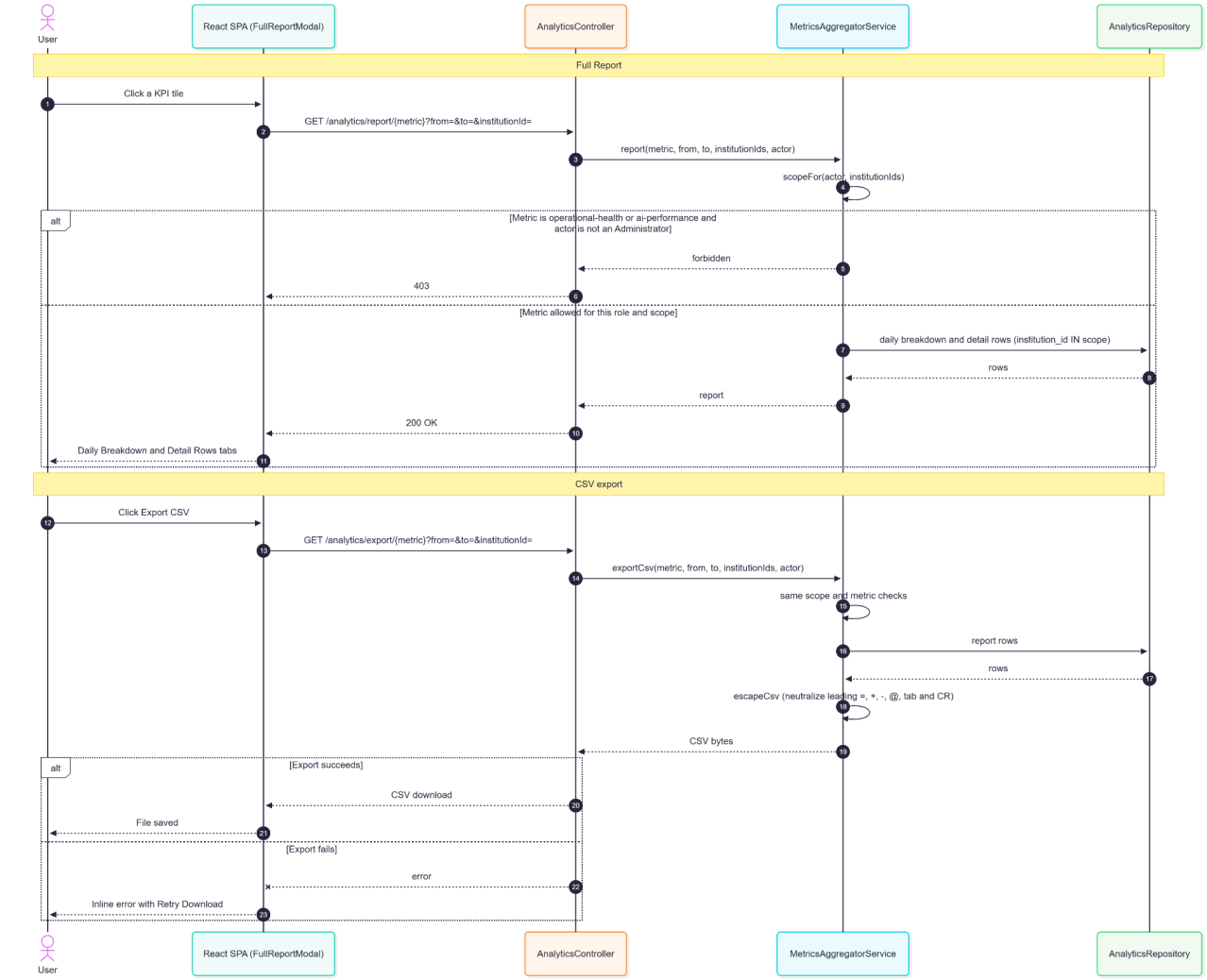
3.4.4. Object-Oriented Component(s)

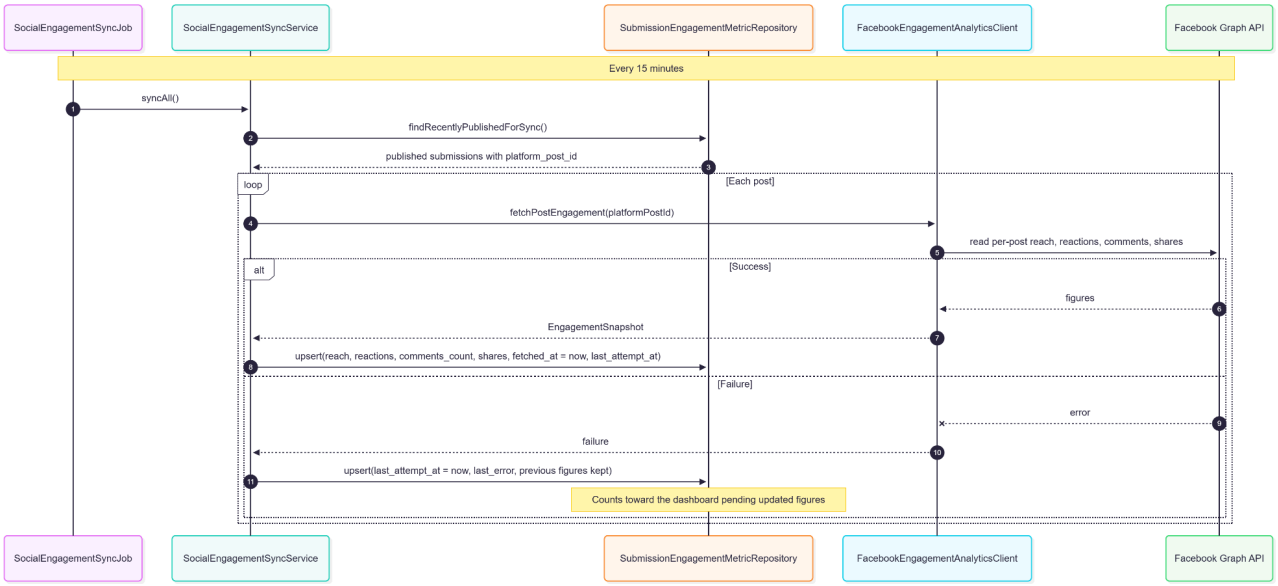
- **Class Diagram**



- **Sequence Diagram**

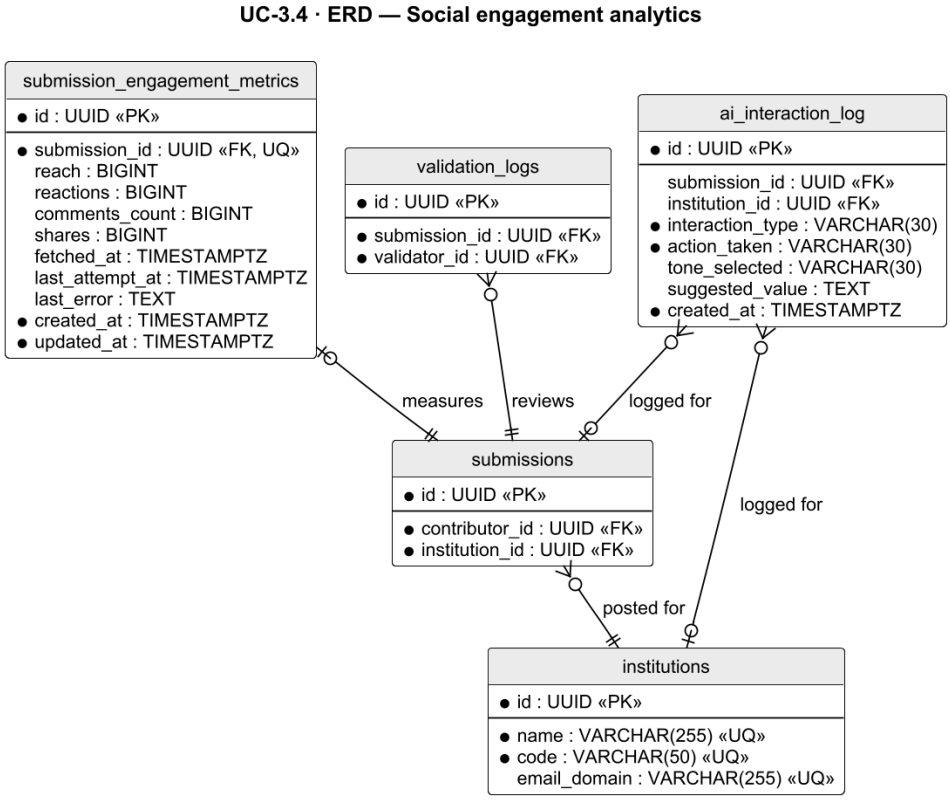

User
React SPA (AnalyticsDashboardPage)
AnalyticsController
MetricsAggregatorService
Summary cache (60 s)
AnalyticsRepository
PublishSuccessRateRepository
SubmissionEngagementMetricRepository
FacebookEngagementAnalyticsClient
Open /analytics (or change range, filter, press Refresh, 5-minute auto-refresh)
GET /analytics/summary?from=&to=&institutionId=
authorize (any signed-in user)
summary(from, to, institutionIds, actor)
scopeFor(actor, institutionIds)
alt
[Contributor]
alt
[Request carries an institutionId]
forbidden
403
[No institution filter]
scope = own institution
[Moderator]
alt
[Request carries an institution filter]
forbidden
403
[No institution filter]
scope = entire network
[Administrator]
scope = network or the selected institutions
Continues only when the scope is allowed
lookup (range and scope)
alt
[Cache hit]
cached summary
[Cache miss]
posting delay, completeness, totals, trend, status breakdown (institution_id IN scope)
aggregates
opt
[Moderator or Administrator]
posts by institution (network-wide)
engagement totals and pending-update count (cached sync data)
reach, reactions, comments, shares, last refresh time
opt
[Administrator]
on-time publication (published_at minus scheduled_at within 300 s)
rates
AI interaction counts (insufficient-data flag below 20 events)
counts
fetchPageInsights() (degrades gracefully on failure)
assemble AnalyticsSummaryDto with period-over-period deltas
store
AnalyticsSummaryDto
200 OK
KPI tiles, charts and role-aware panels





3.4.5. Data Design

- **Entity Relationship Diagram**



- **Schema**

```
CREATE TABLE submission_engagement_metrics (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    submission_id   UUID UNIQUE NOT NULL REFERENCES submissions(id) ON DELETE CASCADE,
    reach           BIGINT,
    reactions       BIGINT,
    comments_count  BIGINT,
    shares          BIGINT,
    fetched_at      TIMESTAMPTZ,            -- last successful refresh
    last_attempt_at TIMESTAMPTZ,
    last_error      TEXT,
    created_at      TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at      TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
-- Workflow KPIs are computed at query time from submissions, validation_logs,
-- publication_attempts, and ai_interaction_log; no KPI columns are stored.
```

Related tables used by this use case:

```
-- ai_interaction_log (defined in UC-1.6)
CREATE TABLE ai_interaction_log (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    submission_id   UUID REFERENCES submissions(id),
    institution_id   UUID REFERENCES institutions(id),
    interaction_type VARCHAR(30) NOT NULL,
    action_taken    VARCHAR(30) NOT NULL,
    tone_selected   VARCHAR(30),
    suggested_value  TEXT,
    created_at      TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
```

3.5. System Health Analytics

3.5.1. User Interface Design

System Health Screen (`/admin/system-health`, Administrator only)

An overall status (worst status across all rows with WARNING / UNHEALTHY / UNAVAILABLE counts) above four sections, each row carrying a HEALTHY, WARNING, UNHEALTHY, UNAVAILABLE, or SCHEDULED indicator:

- **Storage** — database size and R2 bucket usage against configurable tier limits (80% warning, 95% critical).
- **External Services** — Supabase PostgreSQL, Cloudflare R2, Facebook Graph API (token status and expiry countdown), Anthropic, Voyage AI, and the email provider. Google Gemini is not probed; its health shows through the Media AI metrics.
- **Background Jobs** — 16 tracked jobs with last run, duration, error, and a stale flag (last run > 2× expected interval), each with a **Re-run** action.
- **Operational Metrics** (rolling 30 days) — approval turnaround, Edit & Approve rate, manual fallback resolution rate, publish success rate (same calculation as Analytics), Fast-Track volume, and missed-review rate (WARNING above 10%).

Additional panels: Media AI metrics and embedding coverage, media-processing dead letters with Retry, Facebook token management (Reauthorize via OAuth, paste a token manually, re-check token, and — Admin Owner only — Connect a Different Page), and Export Snapshot (CSV).

3.5.2. Front-end Component(s)

SystemHealthScreen

- `GET /system-health/summary`, `/storage`, `/external-services`, `/jobs`, `/operational-metrics`, `/media-ai-metrics`, `/media-ai-coverage`, `/tokens`; `POST /system-health/jobs/{jobKey}/run`; `POST /system-health/tokens/recheck`; `GET /system-health/tokens/{tokenId}/oauth-init`; `PUT /system-health/tokens/{tokenId}/manual`; `PUT /system-health/tokens/connect`; `GET /system-health/export`; `GET /system-health/media-processing/dead-letters`, `POST …/dead-letters/{jobId}/retry`. Adjusts toast copy for the two asynchronous jobs.
- React functional component / Admin route

3.5.3. Back-end Component(s)

SystemHealthController / MediaProcessingAdminController

- Class-level `hasRole('ADMIN')`.
- Spring `@RestController`

SystemHealthService

- Storage (`pg_database_size` + live R2 scan with DB-estimate fallback), external service probes, job health from `scheduled_job_runs` against `EXPECTED_JOBS`, operational metrics including `missedReviewRate()`, status roll-up, and snapshot CSV export (audit-logged; any cell beginning with = + - @, tab, or CR is prefixed with an apostrophe against formula injection — PR #409)..
- Spring `@Service`

ScheduledJobHealthService / ManualJobRunner

- Records job runs; on-demand re-runs (audit `BACKGROUND_JOB_RUN`); `PublishingSchedulerJob` and `TokenPublishingEscalationJob` run asynchronously (`ASYNC_JOB_KEYS`), the rest synchronously.
- Spring `@Service` / `@Component`

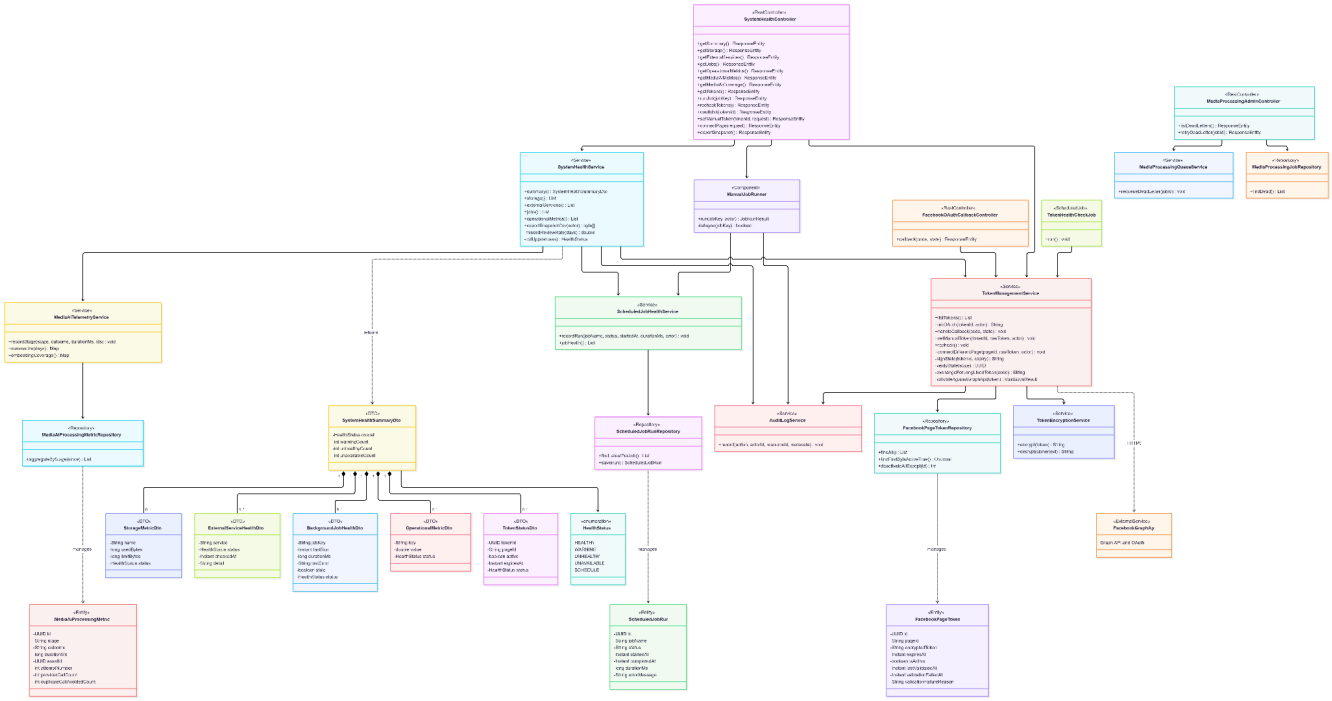
TokenManagementService / FacebookOAuthCallbackController

- `initOAuth` with an HMAC-signed stateless `state` (token ID + expiry + signature); `GET /facebook/oauth-callback` exchanges the code for a long-lived Page token, validates it against the Graph API, encrypts and stores it (`TOKEN_REAUTHORIZED`). Manual paste (`TOKEN_MANUALLY_SET`) is validated the same way. Connect a Different Page (`FACEBOOK_PAGE_CONNECTED`) is Admin Owner only and deactivates all other rows. Suspended publishing resumes when `TokenPublishingEscalationJob` next finds a usable token. Redirect URI defaults from `BACKEND_PUBLIC_BASE_URL`.
- Spring `@Service` / `@RestController`

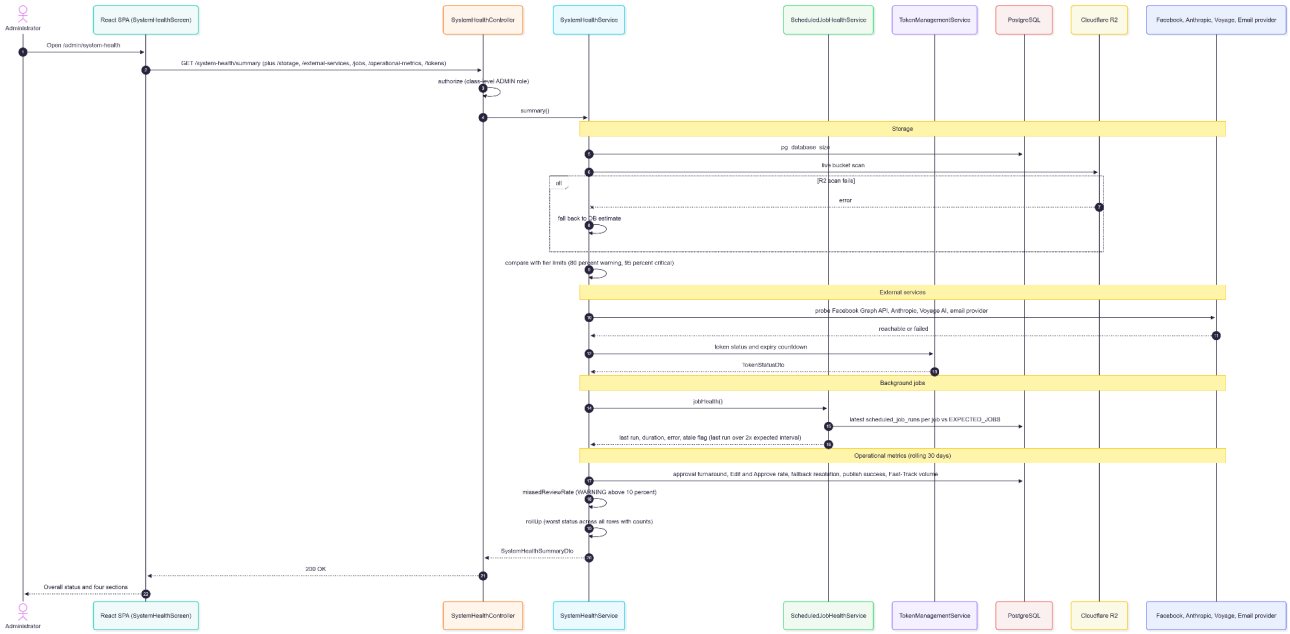
TokenHealthCheckJob / MediaAiTelemetryService — daily token validation; media AI stage metrics.

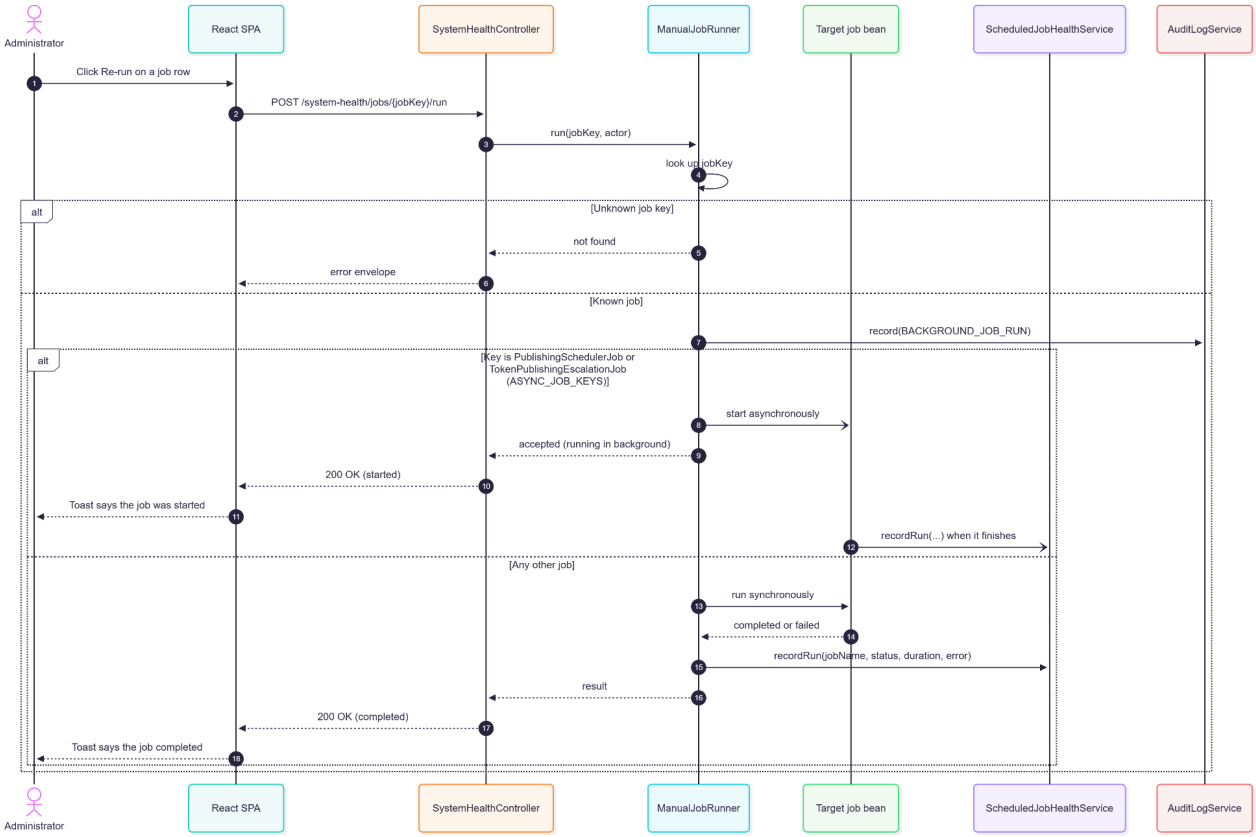
3.5.4. Object-Oriented Component(s)

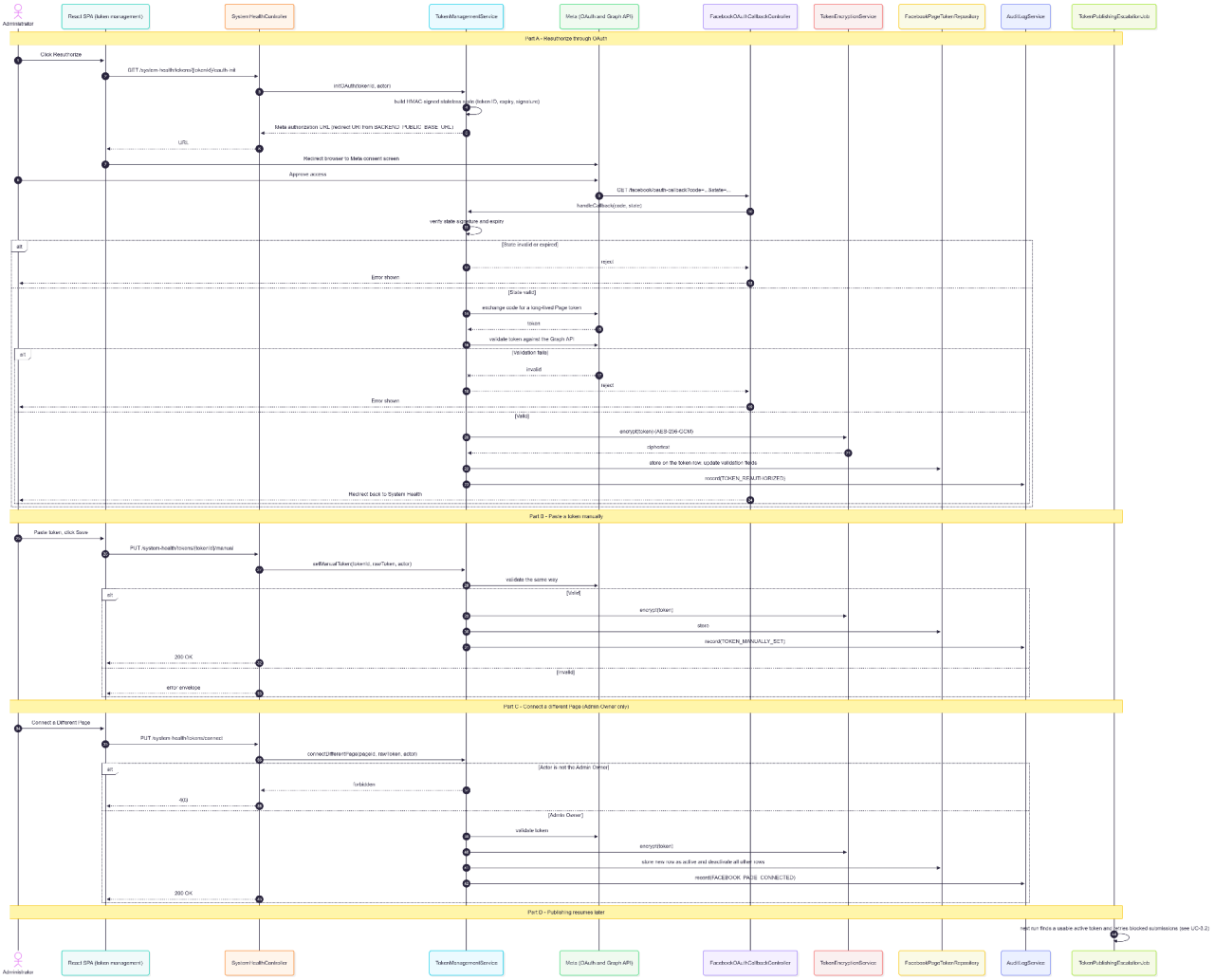
- **Class Diagram**

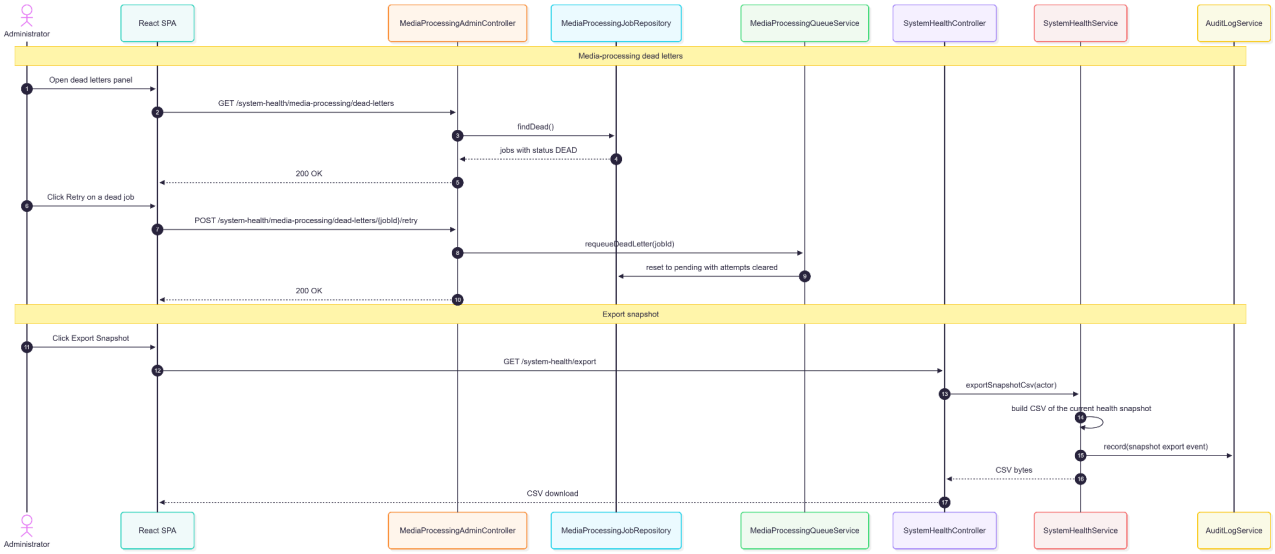


- **Sequence Diagram**



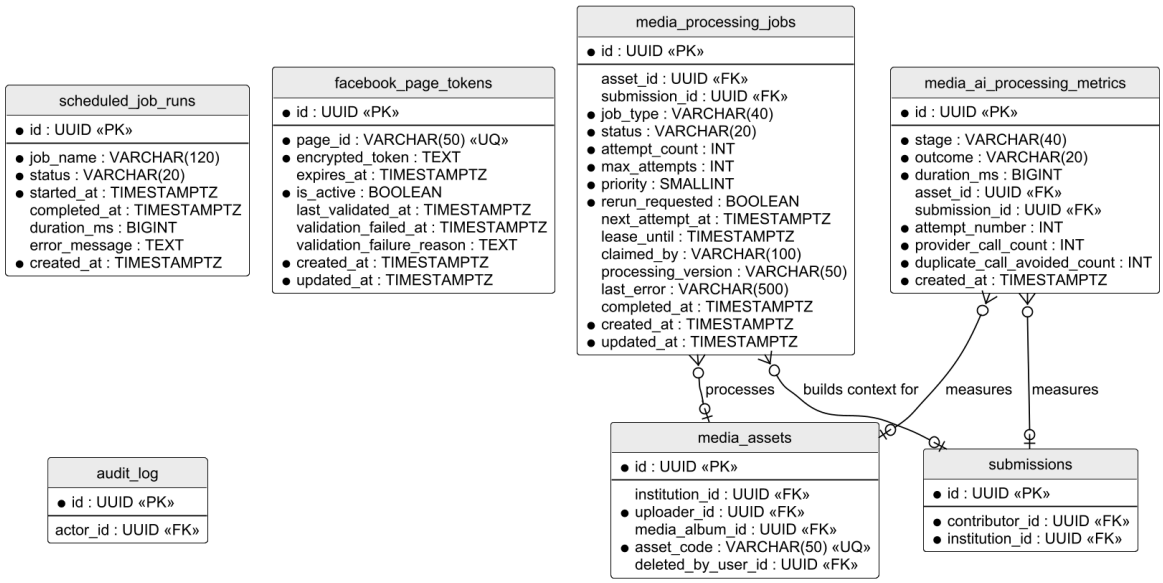

Administrator
React SPA
SystemHealthController
ManualJobRunner
Target job bean
ScheduledJobHealthService
AuditLogService
Click Re-run on a job row
POST /system-health/jobs/{jobKey}/run
run(jobKey, actor)
look up jobKey
alt
[Unknown job key]
not found
error envelope
[Known job]
record(BACKGROUND_JOB_RUN)
alt
[Key is PublishingSchedulerJob or TokenPublishingEscalationJob (ASYNC_JOB_KEYS)]
start asynchronously
accepted (running in background)
200 OK (started)
Toast says the job was started
recordRun(...) when it finishes
[Any other job]
run synchronously
completed or failed
recordRun(jobName, status, duration, error)
result
200 OK (completed)
Toast says the job completed
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18





3.5.5. Data Design

- **Entity Relationship Diagram**



- **Schema**

```
CREATE TABLE scheduled_job_runs (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    job_name        VARCHAR(120) NOT NULL,
    status          VARCHAR(20) NOT NULL CHECK (status IN ('SUCCESS', 'FAILED')),
    started_at      TIMESTAMPTZ NOT NULL,
    completed_at  TIMESTAMPTZ,
    duration_ms   BIGINT,
    error_message TEXT,
    created_at      TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- Media AI telemetry (per-stage latency/outcome)
CREATE TABLE media_ai_processing_metrics (
    id                              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    stage                           VARCHAR(40) NOT NULL
                                  CHECK (stage IN ('R2_UPLOAD','DB_REGISTRATION','QUEUE_DELAY',
                                        'VOYAGE_IMAGE_EMBEDDING','CLAUDE_CLASSIFICATION',

'VOYAGE_SEMANTIC_EMBEDDING','READY_LATENCY','AI_SUGGESTION_QUERY')),
    outcome                         VARCHAR(20) NOT NULL CHECK (outcome IN
('SUCCESS','FAILURE','REUSED')),
    duration_ms                     BIGINT NOT NULL DEFAULT 0,
    asset_id                        UUID REFERENCES media_assets(id) ON DELETE SET NULL,
    submission_id                   UUID REFERENCES submissions(id) ON DELETE SET NULL,
    attempt_number                  INT NOT NULL DEFAULT 1,
    provider_call_count             INT NOT NULL DEFAULT 0,
    duplicate_call_avoided_count INT NOT NULL DEFAULT 0,
    created_at                      TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_scheduled_job_runs_job ON scheduled_job_runs(job_name, started_at DESC);
```

Related tables used by this use case:

```
-- facebook_page_tokens (defined in UC-3.2)
CREATE TABLE facebook_page_tokens (
    id                      UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    page_id                 VARCHAR(50) UNIQUE NOT NULL,
    encrypted_token         TEXT NOT NULL,
    expires_at              TIMESTAMPTZ,
```

```
    is_active               BOOLEAN NOT NULL DEFAULT FALSE,
    last_validated_at       TIMESTAMPTZ,
    validation_failed_at    TIMESTAMPTZ,
    validation_failure_reason TEXT,
    created_at              TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at              TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- media_processing_jobs (defined in UC-2.1)
CREATE TABLE media_processing_jobs (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    asset_id        UUID REFERENCES media_assets(id) ON DELETE CASCADE,
    submission_id   UUID REFERENCES submissions(id) ON DELETE CASCADE,
    job_type        VARCHAR(40) NOT NULL
                    CHECK (job_type IN ('RETRIEVAL_EMBEDDINGS', 'CLASSIFY_AND_EMBED',
                                        'EMBED_IMAGE_ONLY', 'BUILD_SUBMISSION_CONTEXT')),
    status          VARCHAR(20) NOT NULL
                    CHECK (status IN ('PENDING','PROCESSING','RETRY','COMPLETED','DEAD')),
    attempt_count   INT NOT NULL DEFAULT 0,
    max_attempts    INT NOT NULL,
    priority                SMALLINT NOT NULL DEFAULT 10,
    rerun_requested         BOOLEAN NOT NULL DEFAULT FALSE,
    next_attempt_at TIMESTAMPTZ,
    lease_until     TIMESTAMPTZ,
    claimed_by      VARCHAR(100),
    processing_version VARCHAR(50),
    last_error      VARCHAR(500),
    completed_at    TIMESTAMPTZ,
    created_at      TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at      TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
```

3.6. Audit Log Review

3.6.1. User Interface Design

Audit Log Screen (`/admin/audit-log`, Administrator only)

A server-paginated table, newest first, with Actor, Timestamp (Asia/Manila), Category/Action badge, Affected Entity, and a one-line Summary. Filters: date range, actor (ID or name/email), category (11), action code, entity type (8), resource ID, and free-text search — combinable. Export CSV downloads the filtered result (capped at 5,000 rows; failures surface as a toast with filters preserved).

Audit Detail Modal

Actor & Execution (name, email, role, institution, Admin Owner flag, action code, IP, user agent), Target Entity (type, jump link or "Deleted/Unavailable" badge, ID), generated summary including any override/justification reason, a structured Field / Previous / Updated diff for edits, schedule changes, status transitions, and role changes, and a raw-detail inspector.

3.6.2. Front-end Component(s)

AuditLogScreen

- `GET /audit-log` (filters + pagination), `GET /audit-log/categories`, `GET /audit-log/export-csv`.
- React functional component / Admin route

AuditDetailModal

- Structured diff and raw metadata view.

- React modal component

3.6.3. Back-end Component(s)

AuditLogController

- Class-level `hasRole('ADMIN')`.
- Spring `@RestController`

AuditLogService

- Single write path for audit entries; derives category and entity type from the action code at read time (`AuditLogCategory`, `AuditEntityType`); server-side filtering; CSV export (itself audited) with formula-injection-safe `escapeCsv`. Routine page views and self-service preference changes are intentionally not audited.
- Spring `@Service`

AuditLogRetentionJob

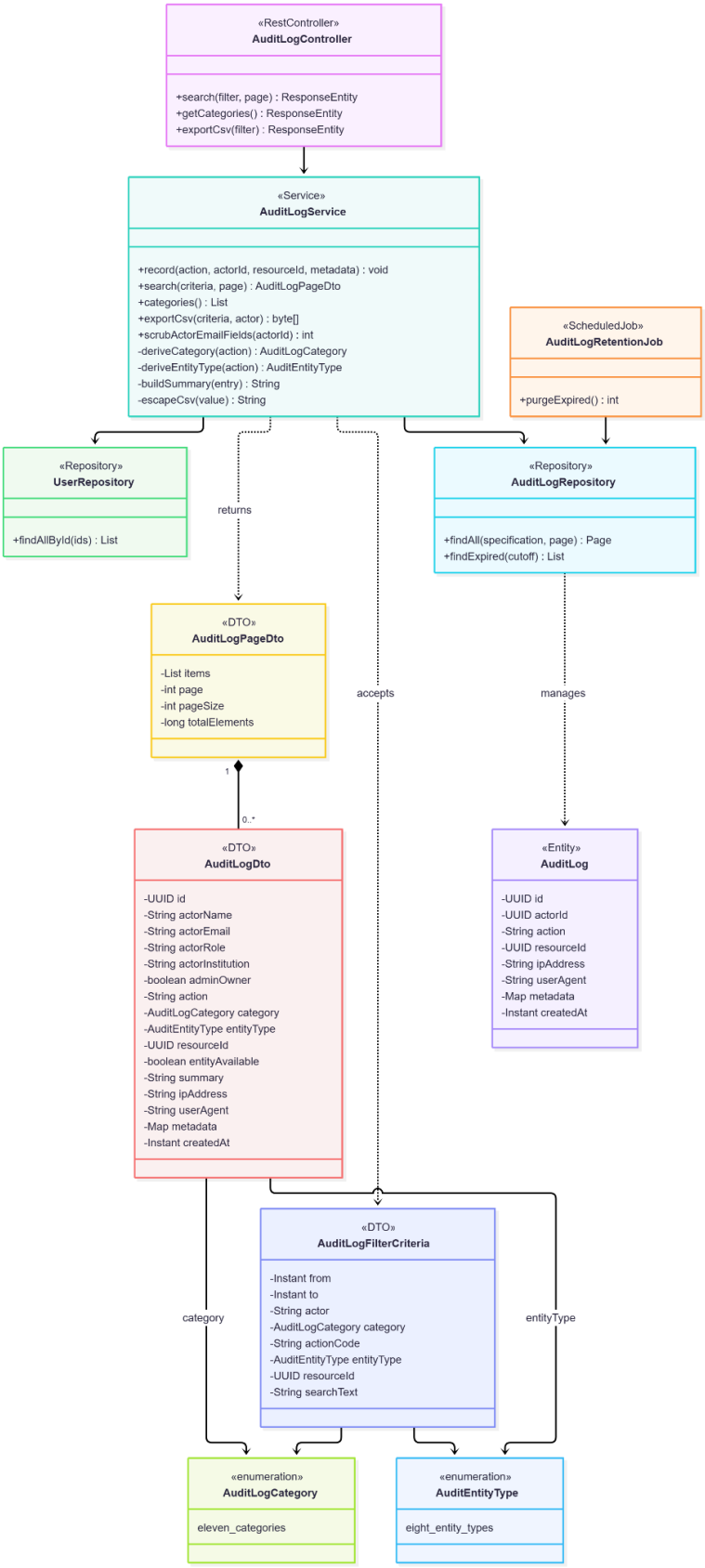
- Scheduled retention purge — the only sanctioned deletion path besides the right-to-be-forgotten scrub of actor email fields.
- Spring `@Scheduled`

AuditLogRepository

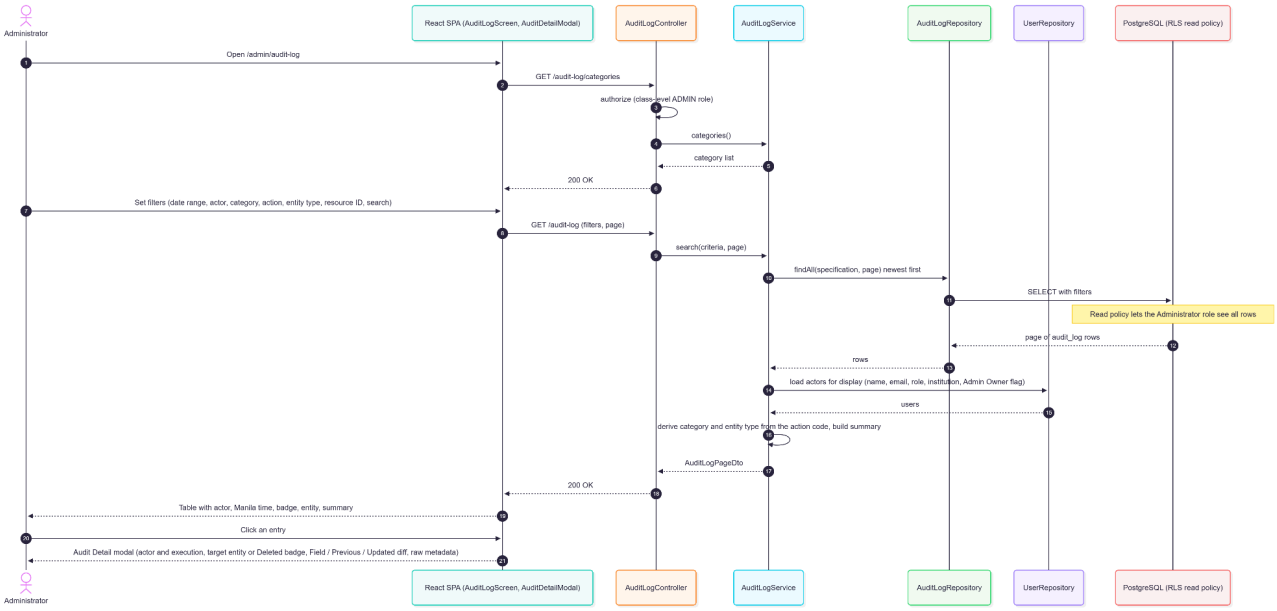
- Spring `@Repository` extending `JpaRepository<AuditLog, UUID>` with specification-based filtering.

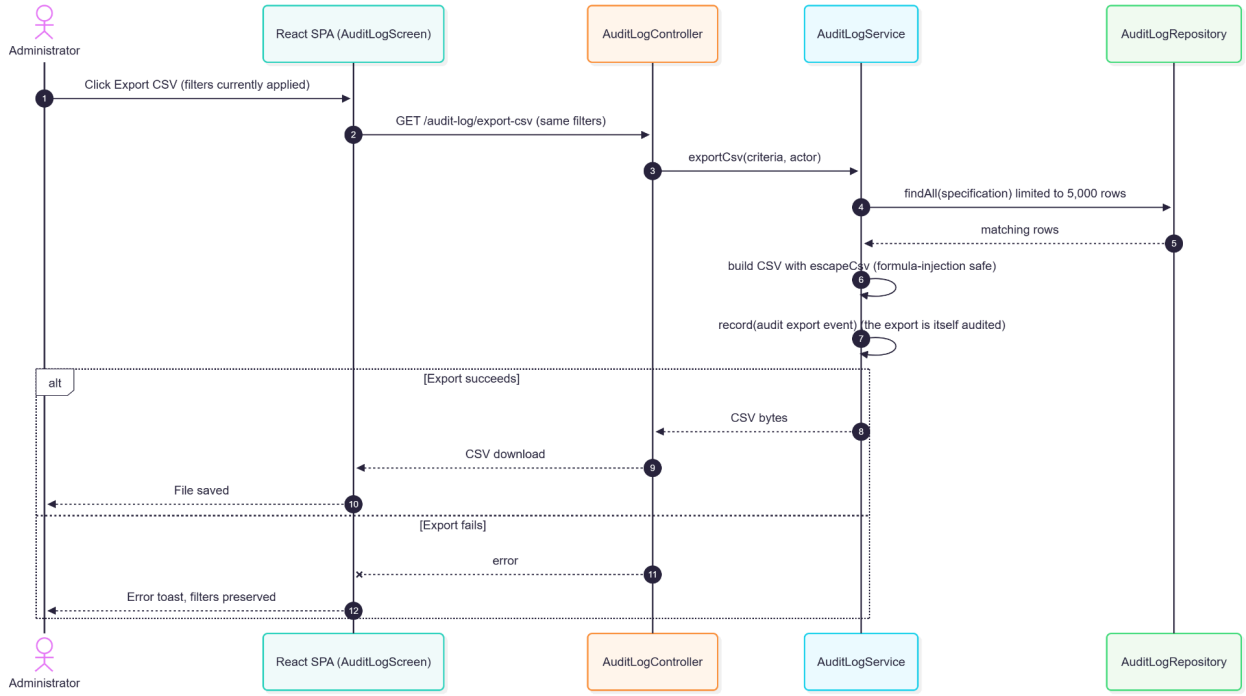
3.6.4. Object-Oriented Component(s)

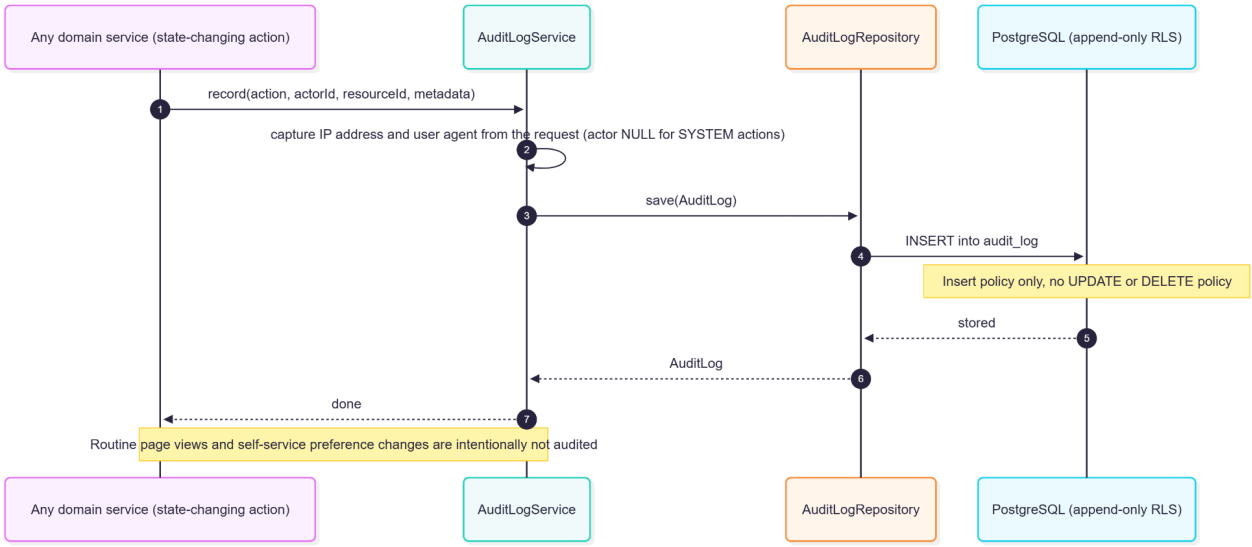
- **Class Diagram**

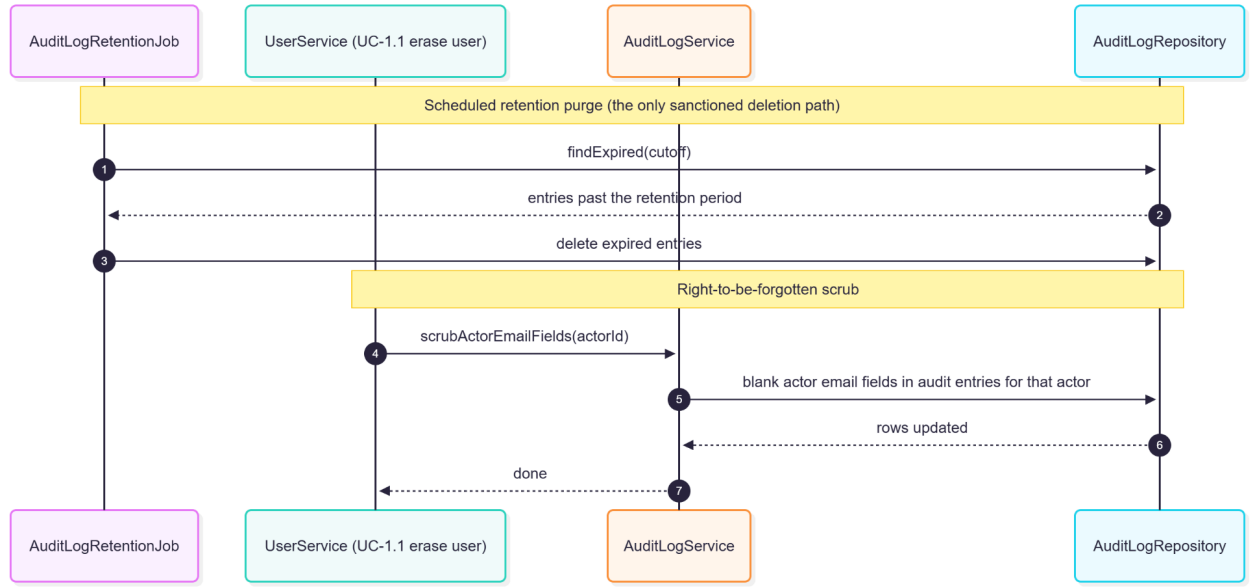


- **Sequence Diagram**





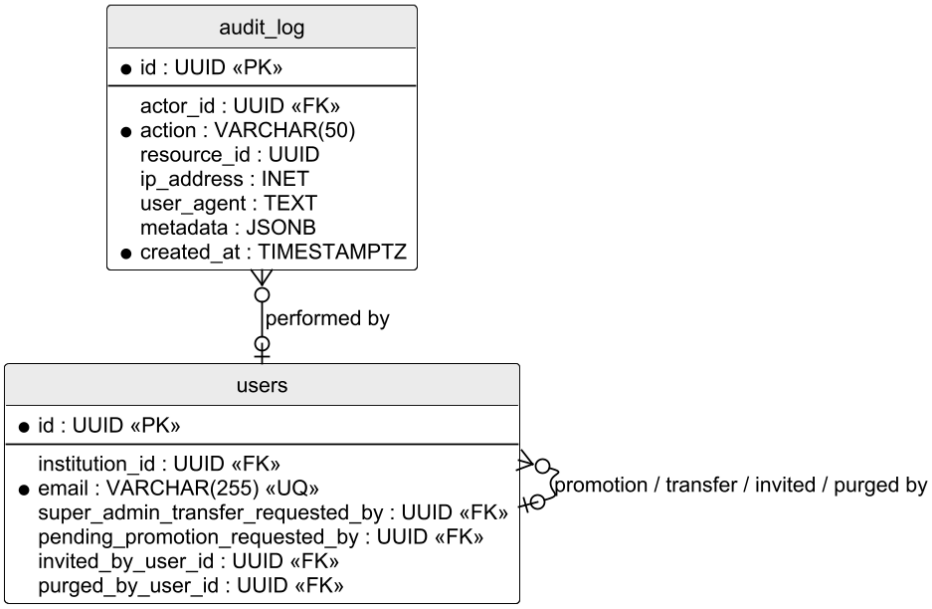




3.6.5. Data Design

- **Entity Relationship Diagram**

UC-3.6 · ERD — Audit log review



- **Schema**

```
CREATE TABLE audit_log (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    actor_id        UUID REFERENCES users(id),   -- NULL for SYSTEM actions
    action          VARCHAR(50) NOT NULL,
    resource_id UUID,
    ip_address  INET,
    user_agent  TEXT,
    metadata        JSONB,                          -- before/after diff, reasons, context
    created_at  TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_audit_log_actor    ON audit_log(actor_id, created_at DESC);
CREATE INDEX idx_audit_log_resource ON audit_log(resource_id, created_at DESC);

-- Append-only guarantee (V78): insert/select policies only; no UPDATE/DELETE policy
ALTER TABLE audit_log ENABLE ROW LEVEL SECURITY;
CREATE POLICY audit_log_append ON audit_log FOR INSERT WITH CHECK (true);
CREATE POLICY audit_log_read ON audit_log FOR SELECT
    USING (
            current_setting('app.current_role', true) = 'administrator'
            OR EXISTS (
            SELECT 1 FROM users u
            WHERE u.id = audit_log.actor_id
            AND u.institution_id = nullif(current_setting('app.current_institution_id', true), '')::UUID
            )
    );
```

4. Planned Design for Validation-Driven Refactoring (RP-01–RP-05)

These items come from the MVP Validation Findings & Refactoring Priorities (12 Oct 2026, VF-01–VF-05) and implement SRS v7.0 Section 3.6.

> **Status: Planned — not implemented in release v3.0.2.** Each item lists the components to add or change, the data and API design, and the STD acceptance test that will verify it. Scheduling is in SPMP v2.1 §5.2.5.

4.1. RP-01 — Submission gate separated from quality tips (VF-01; REQ-124)

- **Current design:** `features/submission/utils.ts` computes one score `round(required/7 × 75 + recommended/5 × 25)`; `readinessTone` colours it.
- **Target design:** replace the score with two derived values computed in the same hook: `gate = { passed, total: 7, nextMissing }` and `tips = { applied, total: 5 }`. The readiness panel shows "Required: 7 of 7 — Ready to submit" or names `nextMissing` with a jump link; tips render separately as "Tips: 3 of 5 applied" with no percentage. Submit becomes the primary action whenever `gate.passed === gate.total`, on every screen size (shared with §3.4.4's mobile action bar). Server-side `SubmissionService.assertContentComplete` is unchanged (it already checks only the required items).
- **Data:** none. Optional metric: drafts whose gate passed but that were not submitted within 24 h (query over `submissions` and the task-event log of §4.2).
- **Test:** STD Case-30a.

4.2. RP-02 — Next action, plain-language status, task-event log (VF-02; REQ-125, REQ-126)

- **Status:** Implemented (refactor/p2-first-time-independence)
- **Next-action card:** GET /api/v1/me/next-action (TaskTelemetryController → TaskTelemetryService.getNextAction) returns { kind, label, count, deepLink } computed per role — Contributor: needs_revision draft first, else oldest draft with missing required items; Moderator: count of pending (Fast-Track prioritized) with link to the oldest; Administrator: count of publish_failed, else pending reviews. Rendered at the top of DashboardScreen.tsx.
- **Status wording:** shared statusCopy[status] = { label, meaning, whoActsNext }, statusLabels, and statusGuidance in features/submission/constants.ts, mirrored in NotificationEventListener.java (verified by NotificationEventListenerTest), so submission lists, the review queue, and in-app/email/Messenger notifications display identical plain-language status explanations.
- **Task-event log:** migration V127__task_completion_telemetry.sql creates task_events and task_completion_logs, written via POST /api/v1/task-events and POST /api/v1/telemetry/tasks/{start,complete} (useTaskTelemetry.ts across SubmissionScreen, ValidationQueueScreen, InvitationComposer, InviteScreen) and summarized via GET /api/v1/telemetry/tasks/summary. No free text is stored.
- **Test:** STD Case-30b (PASS).

4.3. RP-03 — Caption on upload, media-first picker, log-based GO3 (VF-03; REQ-320, REQ-321)

- **Status: Implemented** (refactor/p3-ai-caption-on-upload).
- **Background caption:** after the first image is attached to a saved draft that has an event title, useAiCaptionAssist triggers background caption generation (POST /api/v1/ai/caption / /api/v1/ai/caption/background, same 30/h limit and 30 s timeout); CaptionGenerationService and AiCaptionSuggestion offer the result inline as "**AI draft ready — Use / Edit / Dismiss**", never inserting it automatically. Failure falls back silently to manual entry.
- **Measurement:** coverage = background suggestions (auto_generated) ÷ submissions with images; acceptance = (use + use_then_edited) ÷ generated (auto_generated + re_generate); quality = mean

quality_rating (one-click 1–5 at decision in AiCaptionSuggestion); drafting time = drafting_duration_ms from opening Post Details to caption decision, recorded via POST /api/v1/ai/caption/log (V130__ai_caption_suggestions_telemetry.sql).

- **Media-first:** `MediaAssetsPicker` initial tab defaults to ai (**AI Suggestions**) when event context (≥ 10 characters or a selected image) is present; otherwise upload.
- **Test:** STD Case-30c. (**PASS**).

4.4. RP-04 — Mobile operability (VF-04; REQ-218)

- **Status: Implemented** (refactor/p4-mobile-operability).
- **Persistent primary-action bar:** each role-critical screen keeps its own sticky bottom action bar at phone/tablet widths — composer (Preview / Next / Submit for Approval), review panel (Approve / Request Revision / Reject / Edit once the lock is held), Failed publications (Retry / Start Manual Publish), Master Calendar (drag a scheduled post → Confirm Reschedule dialog), and invitation activation (Activate). No horizontal scrolling at 360 × 800 or 768 × 1024.
- **Deep links:** NotificationEventListener sends reviewers /queue? submissionId={id}&mode=review for T-01 (new submission) and T-11 (Fast-Track) in-app, by email and by Messenger; ValidationQueueScreen opens that submission, acquires the review lock for pending/in_review items (self-review and locks held by another Moderator are still rejected by ReviewLockService), then removes submissionId and mode from the URL. Publish-failed alerts send Administrators/Moderators /queue?tab=failed&submissionId={id} (in-app, email, Messenger) and the Contributor /submissions/{id}.
- **Tests:** NotificationEventListenerTest asserts the exact links; Playwright e2e/mobileoperability.spec.ts runs five role scenarios (submit, review decision with lock, reschedule, resolve a failed publication, accept an invitation) at 360 × 800 and 768 × 1024 (STD Case-29, **PASS**).

4.5. RP-05 — Bulk invitations with preview and batch audit (VF-05; REQ-127)

- **API:** `POST /api/v1/invitations/batch/preview` (Moderator, Administrator) accepts `[{ email, role, institutionId }]` (≤ 200 rows, from CSV or paste) and returns a per-row status: `valid`, `duplicate_in_batch`, `invalid_email`, `inactive_institution`, `active_account_exists`, `deactivated_use_reactivation`, `pending_will_be_superseded`, `admin_cap_exceeded`, `role_not_permitted`. `POST /api/v1/invitations/batch` re-validates server-side, creates invitations for valid rows only through the existing `InvitationService.create` path, and returns per-row results.
- **Audit:** new `invitation_batches (id, created_by, row_count, sent_count, rejected_count, created_at)`; each invitation's audit metadata carries `batchId`; one `INVITATION_BATCH_SENT` summary row is written.
- **Rules:** Moderators may batch-invite Contributors only; the 3-Admin cap is checked across the whole batch; self-registration remains out of scope (SO 1.4).
- **Test:** STD Case-30d.
- **Status:** Implemented (refactor/p5-bulk-invitations) and verified by STD Case-30d.

Appendix

Overview

DASIGConnect uses a single PostgreSQL relational database hosted on Supabase. The schema is owned by Flyway and built from versioned migrations V1–V122 (`backend/src/main/resources/db/migration`); Hibernate DDL generation is not used. It consists of 31 tables introduced progressively across the three development modules. All tables use UUID primary keys generated by `gen_random_uuid()`. Tenant isolation is enforced through Row-Level Security (RLS) policies scoped to `institution_id`. Media assets use soft deletes (`deleted_at`) with a 30-day retention window before purge. The pgvector extension provides `VECTOR(1024)` columns (`media_asset_embeddings.embedding` and the legacy `media_assets.embedding`) for cosine similarity search. Media files themselves are stored in Cloudflare R2, not in the database. Enum values are stored in lowercase.

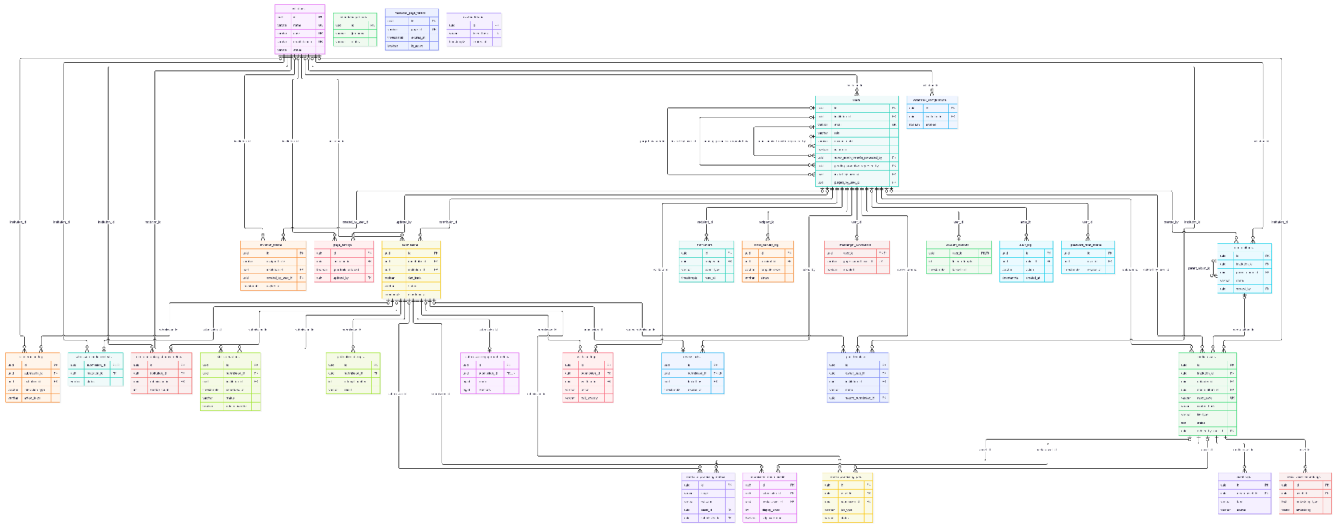
Table Inventory

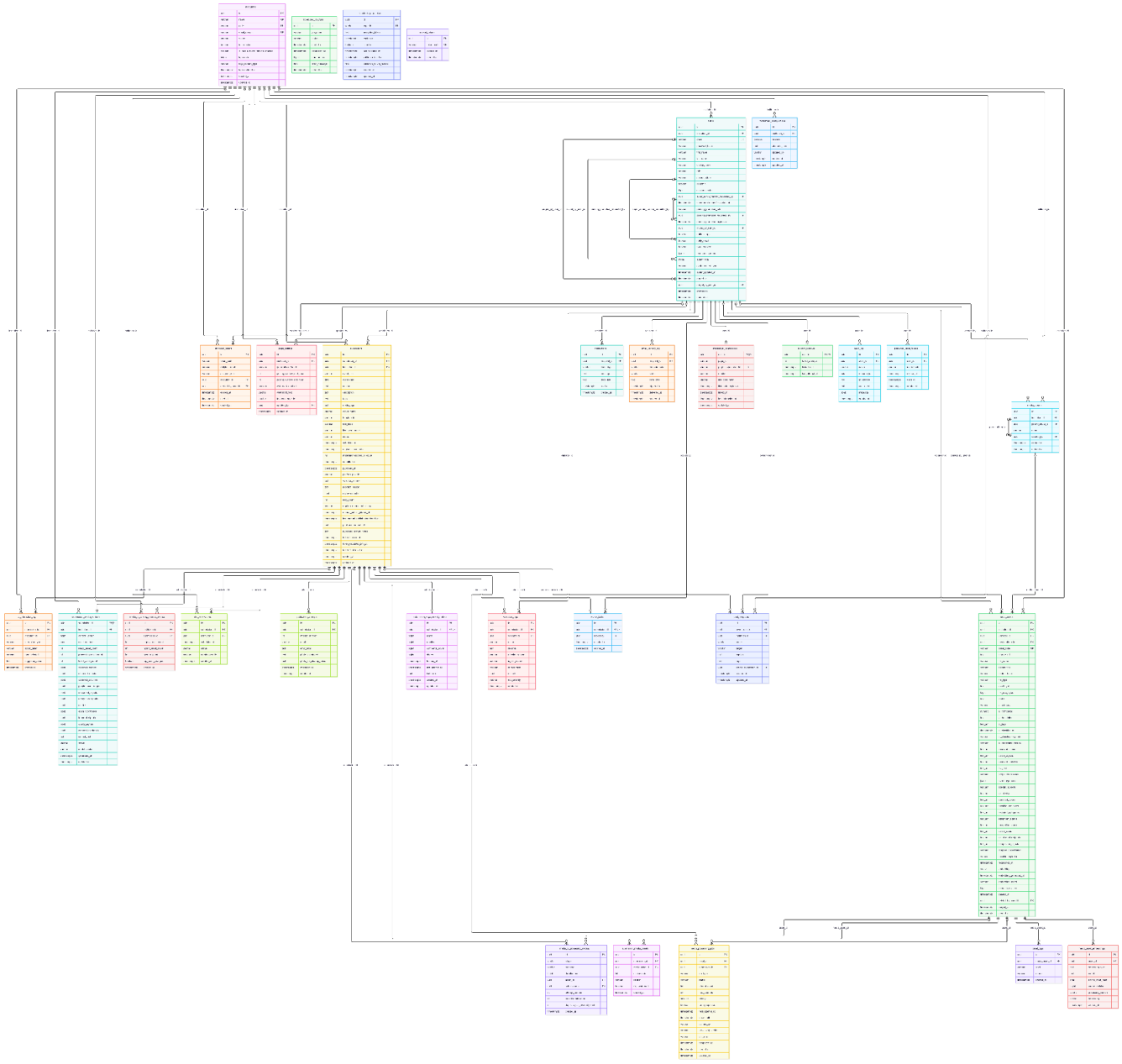
#	Table	Introduced By	Purpose
1	institutions	UC-1.2	Institution workspace records, status, protection flag, logo
2	users	UC-1.1	All accounts (Contributor, Moderator, Administrator), Admin Owner flag, session version, pending promotions/transfers, preferences
3	invitation_tokens	UC-1.1	One-time activation tokens for all roles
4	password_reset_tokens	UC-1.4	One-hour password reset tokens
5	account_lockouts	UC-1.4	Failed-login tracking and lockout
6	revoked_tokens	UC-1.4	Single-token logout revocation
7	messenger_connections	UC-1.4	Messenger link codes and linked Page-scoped IDs
8	page_settings	UC-1.4	Network-wide guard-rail switch and thresholds (plus legacy columns)
9	audit_log	UC-3.6	Append-only record of state-changing, destructive, and security actions
10	submissions	UC-1.5	Content submissions across all workflow states
11	post_templates	UC-1.5	Personal post templates
12	ai_interaction_log	UC-1.6	AI caption, proofreading, and recommendation interaction events
13	submission_media_assets	UC-1.7	Submission ↔ media junction with order, per-asset caption, skip-watermark
14	submission_media_contexts	UC-1.7	Aggregated AI media context per submission
15	slot_reservations	UC-1.8	Slot hold/lock/release with DB-level conflict exclusion
16	media_albums	UC-2.1	Nested, institution-scoped albums
17	media_assets	UC-2.1	Media metadata, AI classification/enrichment, hash, lifecycle
18	asset_tags	UC-2.1	Manual and AI tags per asset

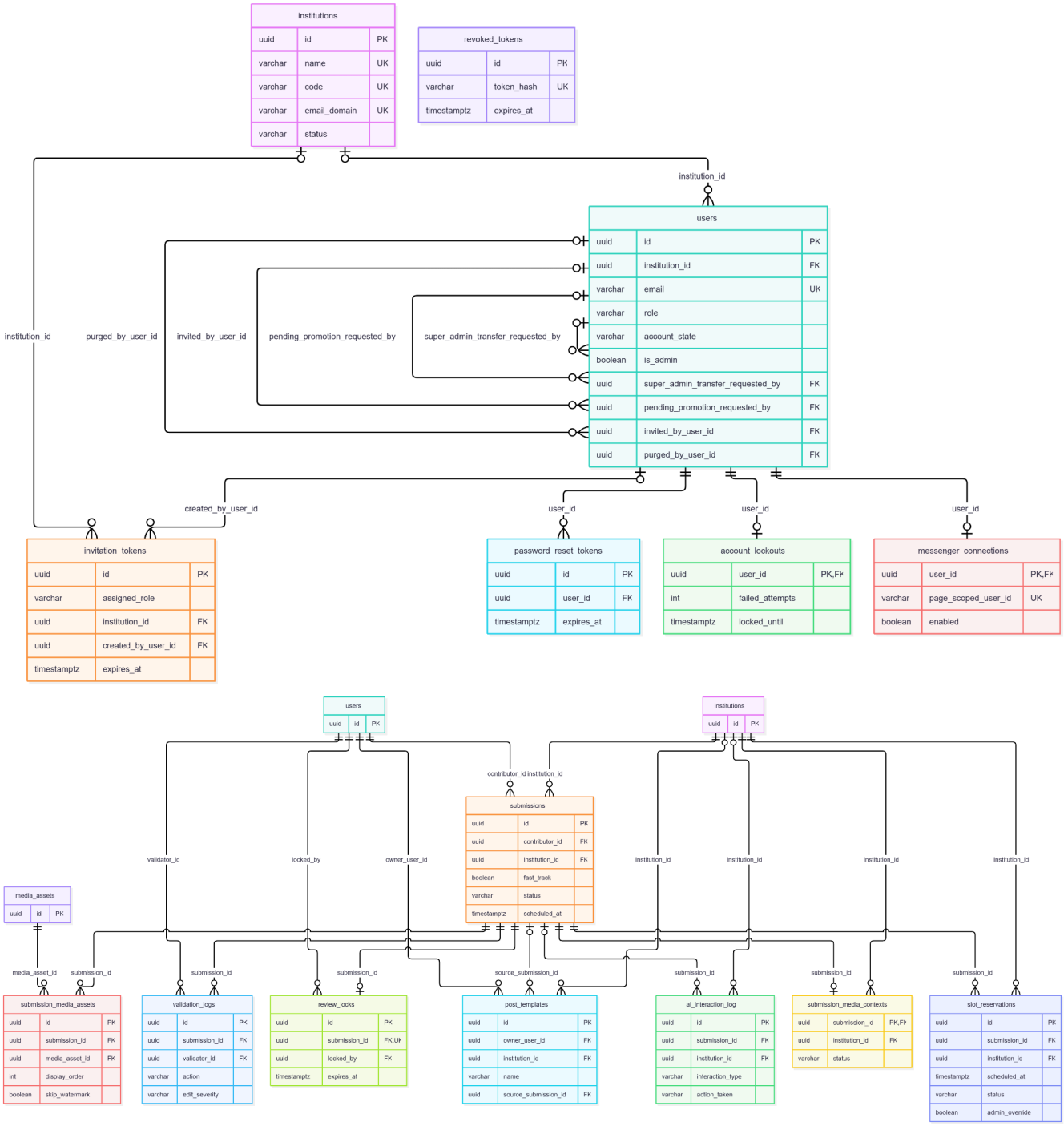
#	Table	Introduced By	Purpose
19	media_asset_embeddings	UC-2.1	Image and semantic Voyage embeddings
20	media_processing_jobs	UC-2.1	DB-backed AI processing queue with retries and dead letters
21	review_locks	UC-2.4	15-minute concurrent review locks
22	validation_logs	UC-2.4	Review actions with edit diff and severity
23	watermark_configurations	UC-2.5	Network-wide watermark design
24	facebook_page_tokens	UC-3.2	Encrypted Page tokens; one active page
25	publication_attempts	UC-3.2	Per-attempt publishing trail and staged/unresolved photo IDs
26	notifications	UC-3.3	In-app notifications
27	email_delivery_log	UC-3.3	Email delivery tracking
28	submission_engagement_metrics	UC-3.4	Cached Facebook engagement per published post
29	scheduled_job_runs	UC-3.5	Background job run history
30	media_ai_processing_metrics	UC-3.5	Media AI stage telemetry
31	media_ai_ranking_shadow_metrics	UC-3.5	Shadow-mode hybrid ranking evaluation

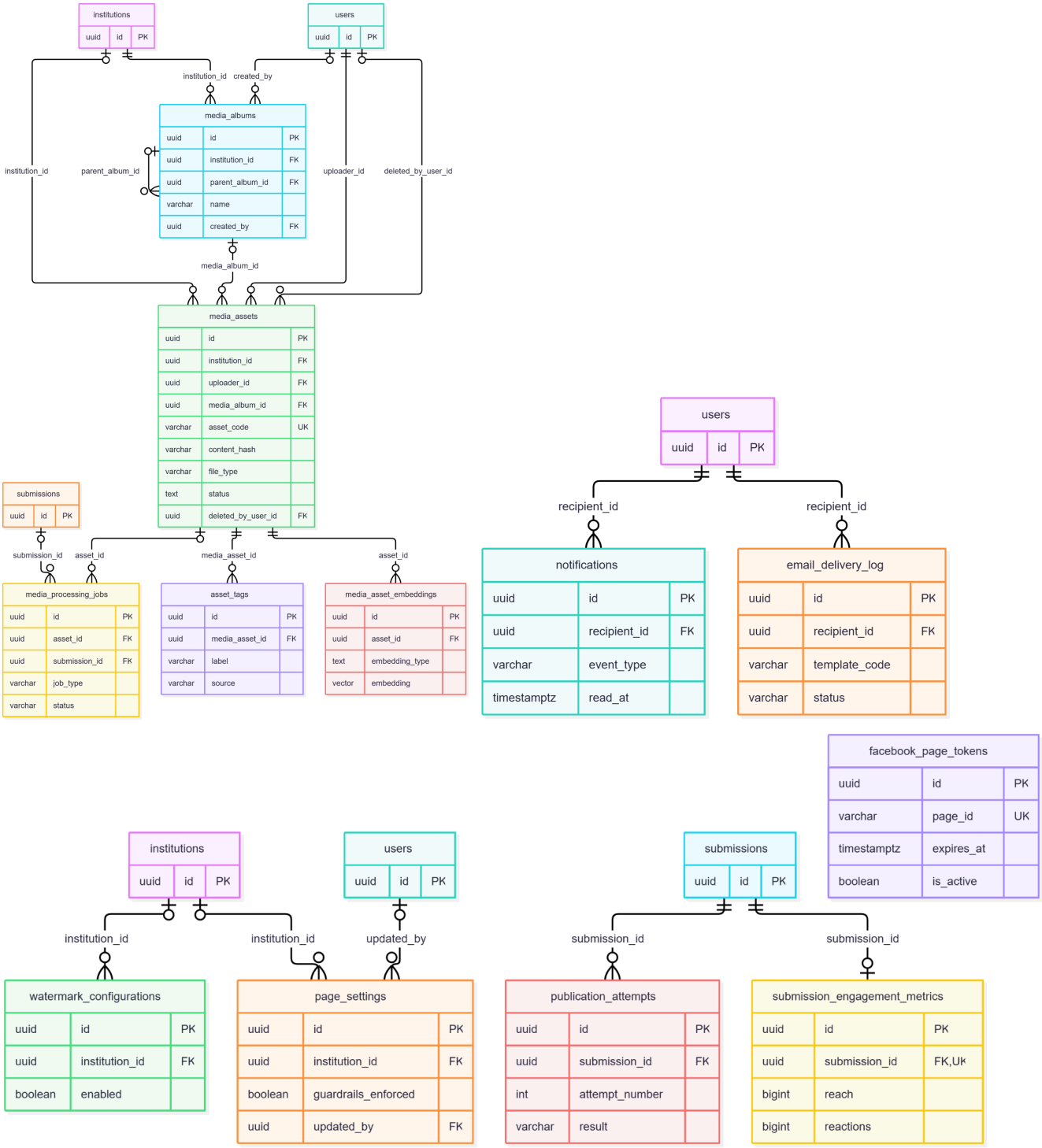
Removed in later migrations and not part of the current schema: `override_requests` (Contributor override requests were descoped) and `media_asset_search_documents`.

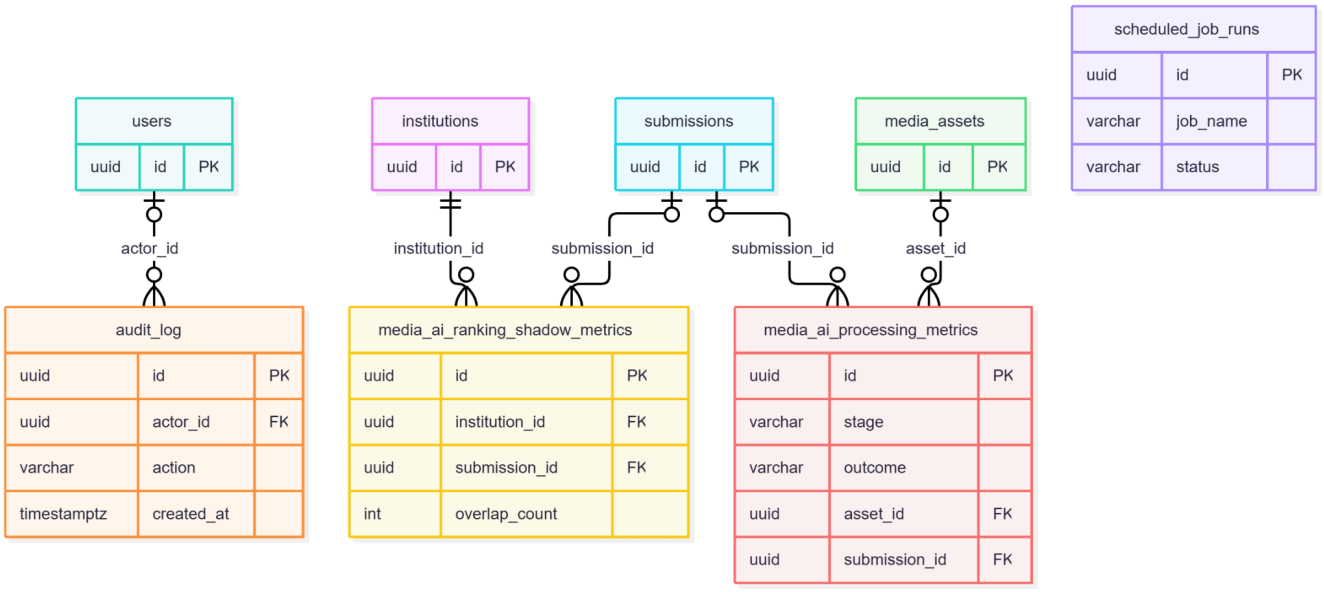
Master Entity Relationship Diagram






institutions
uuid id PK
varchar name UK
varchar code UK
varchar email_domain UK
varchar status
revoked_tokens
uuid id PK
varchar token_hash UK
timestamptz expires_at
institution_id
users
uuid id PK
uuid institution_id FK
varchar email UK
varchar role
varchar account_state
boolean is_admin
uuid super_admin_transfer_requested_by FK
uuid pending_promotion_requested_by FK
uuid invited_by_user_id FK
uuid purged_by_user_id FK
institution_id
purged_by_user_id
invited_by_user_id
pending_promotion_requested_by
super_admin_transfer_requested_by
created_by_user_id
user_id
user_id
user_id
invitation_tokens
uuid id PK
varchar assigned_role
uuid institution_id FK
uuid created_by_user_id FK
timestamptz expires_at
password_reset_tokens
uuid id PK
uuid user_id FK
timestamptz expires_at
account_lockouts
uuid user_id PK,FK
int failed_attempts
timestamptz locked_until
messenger_connections
uuid user_id PK,FK
varchar page_scoped_user_id UK
boolean enabled
users
uuid id PK
institutions
uuid id PK
contributor_id institution_id
submissions
uuid id PK
uuid contributor_id FK
uuid institution_id FK
boolean fast_track
varchar status
timestamptz scheduled_at
validator_id
locked_by
owner_user_id
institution_id
institution_id
institution_id
institution_id
media_assets
uuid id PK
media_asset_id
submission_id
submission_id
submission_id
submission_id
source_submission_id
submission_id
submission_id
submission_id
submission_media_assets
uuid id PK
uuid submission_id FK
uuid media_asset_id FK
int display_order
boolean skip_watermark
validation_logs
uuid id PK
uuid submission_id FK
uuid validator_id FK
varchar action
varchar edit_severity
review_locks
uuid id PK
uuid submission_id FK,UK
uuid locked_by FK
timestamptz expires_at
post_templates
uuid id PK
uuid owner_user_id FK
uuid institution_id FK
varchar name
uuid source_submission_id FK
ai_interaction_log
uuid id PK
uuid submission_id FK
uuid institution_id FK
varchar interaction_type
varchar action_taken
submission_media_contexts
uuid submission_id PK,FK
uuid institution_id FK
varchar status
slot_reservations
uuid id PK
uuid submission_id FK
uuid institution_id FK
timestamptz scheduled_at
varchar status
boolean admin_override


institutions
uuid id PK
users
uuid id PK
institution_id
created_by
media_albums
uuid id PK
uuid institution_id FK
uuid parent_album_id FK
varchar name
uuid created_by FK
institution_id
parent_album_id
uploader_id
deleted_by_user_id
media_album_id
media_assets
uuid id PK
uuid institution_id FK
uuid uploader_id FK
uuid media_album_id FK
varchar asset_code UK
varchar content_hash
varchar file_type
text status
uuid deleted_by_user_id FK
submissions
uuid id PK
submission_id
asset_id
media_asset_id
asset_id
media_processing_jobs
uuid id PK
uuid asset_id FK
uuid submission_id FK
varchar job_type
varchar status
asset_tags
uuid id PK
uuid media_asset_id FK
varchar label
varchar source
media_asset_embeddings
uuid id PK
uuid asset_id FK
text embedding_type
vector embedding
users
uuid id PK
recipient_id
recipient_id
notifications
uuid id PK
uuid recipient_id FK
varchar event_type
timestamptz read_at
email_delivery_log
uuid id PK
uuid recipient_id FK
varchar template_code
varchar status
facebook_page_tokens
uuid id PK
varchar page_id UK
timestamptz expires_at
boolean is_active
institutions
uuid id PK
users
uuid id PK
institution_id
institution_id
updated_by
watermark_configurations
uuid id PK
uuid institution_id FK
boolean enabled
page_settings
uuid id PK
uuid institution_id FK
boolean guardrails_enforced
uuid updated_by FK
submissions
uuid id PK
submission_id
submission_id
publication_attempts
uuid id PK
uuid submission_id FK
int attempt_number
varchar result
submission_engagement_metrics
uuid id PK
uuid submission_id FK,UK
bigint reach
bigint reactions



Complete SQL DDL

The authoritative DDL is the ordered set of Flyway migrations V1–V122. The consolidated listing below reflects the resulting schema; column types are simplified where migrations use equivalent forms.

```
-- ============================================
-- EXTENSIONS
-- ============================================
CREATE EXTENSION IF NOT EXISTS vector;
CREATE EXTENSION IF NOT EXISTS btree_gist;

-- ============================================
-- 1. INSTITUTIONS
-- ============================================
CREATE TABLE institutions (
        id                              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
        name                            VARCHAR(255) UNIQUE NOT NULL,
        code                            VARCHAR(50)  UNIQUE NOT NULL,
        email_domain                    VARCHAR(255) UNIQUE,
        status                          VARCHAR(30) NOT NULL DEFAULT 'active'
                                        CHECK (status IN ('pending', 'active', 'inactive')),
        is_protected                    BOOLEAN NOT NULL DEFAULT FALSE,
        ai_media_hybrid_ranking_enabled BOOLEAN NOT NULL DEFAULT FALSE,
        logo_data                       BYTEA,
        logo_content_type               VARCHAR(40),
        logo_updated_at                 TIMESTAMPTZ,
        created_at                      TIMESTAMPTZ NOT NULL DEFAULT NOW(),
        updated_at                      TIMESTAMPTZ NOT NULL DEFAULT NOW()
);


-- ============================================
-- 2. USERS
-- ============================================
CREATE TABLE users (
        id                              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
        institution_id                  UUID REFERENCES institutions(id),
        email                           VARCHAR(255) UNIQUE NOT NULL,
        password_hash                   VARCHAR(255),
        first_name                      VARCHAR(100),
        last_name                       VARCHAR(100),
        display_name                    VARCHAR(150),
```

```
        role                            VARCHAR(20) NOT NULL
                                        CHECK (role IN ('contributor', 'moderator', 'admin')),
        account_state                   VARCHAR(30) NOT NULL DEFAULT 'pending'
                                        CHECK (account_state IN ('pending', 'pending_email_undelivered',
                                             'active', 'expired', 'inactive', 'cancelled')),
        is_admin                        BOOLEAN NOT NULL DEFAULT FALSE,
        session_version                 BIGINT NOT NULL DEFAULT 0,
        super_admin_transfer_requested_by UUID REFERENCES users(id),
        super_admin_transfer_expires_at   TIMESTAMPTZ,
        pending_promotion_role           VARCHAR(20)                        -- 'admin' | 'moderator'
(V118/V119)
                                        CHECK (pending_promotion_role IN ('admin', 'moderator')),
        pending_promotion_requested_by   UUID REFERENCES users(id),         -- proposing Administrator
        pending_promotion_expires_at     TIMESTAMPTZ,                       -- 72-hour confirmation window
        invited_by_user_id                UUID REFERENCES users(id),
        notify_in_app                   BOOLEAN NOT NULL DEFAULT TRUE,
        notify_email                    BOOLEAN NOT NULL DEFAULT TRUE,
        tours_enabled                   BOOLEAN NOT NULL DEFAULT TRUE,
        tour_seen_screens               JSONB,
        avatar_data                     BYTEA,
        avatar_content_type             VARCHAR(40),
        avatar_updated_at               TIMESTAMPTZ,
        purged_at                       TIMESTAMPTZ,
        purged_by_user_id                 UUID REFERENCES users(id),
        created_at                      TIMESTAMPTZ NOT NULL DEFAULT NOW(),
        updated_at                      TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
CREATE INDEX idx_users_institution ON users(institution_id);
CREATE UNIQUE INDEX idx_users_single_active_admin_owner ON users ((role))
        WHERE role = 'admin' AND account_state = 'active' AND is_admin = TRUE;   -- exactly one active Admin
Owner


-- ═══════════════════════════════════════════
-- 3. INVITATION TOKENS
-- ═══════════════════════════════════════════
CREATE TABLE invitation_tokens (
    id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    token_hash        VARCHAR(64) NOT NULL,
    recipient_email   VARCHAR(255) NOT NULL,
    assigned_role     VARCHAR(20) NOT NULL
                      CHECK (assigned_role IN ('contributor', 'moderator', 'admin')),
    institution_id    UUID REFERENCES institutions(id),
    created_by_user_id UUID REFERENCES users(id),
    expires_at        TIMESTAMPTZ NOT NULL,
    used_at           TIMESTAMPTZ,
    created_at        TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
CREATE INDEX idx_invitation_tokens_hash ON invitation_tokens(token_hash);

-- ═══════════════════════════════════════════
-- 4–7. AUTH SUPPORT
-- ═══════════════════════════════════════════
CREATE TABLE password_reset_tokens (
    id         UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id    UUID NOT NULL REFERENCES users(id),
    token_hash VARCHAR(64) NOT NULL,
    expires_at TIMESTAMPTZ NOT NULL,
    used_at    TIMESTAMPTZ,
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE TABLE account_lockouts (
    user_id         UUID PRIMARY KEY REFERENCES users(id),
    failed_attempts INT NOT NULL DEFAULT 0,
    locked_until    TIMESTAMPTZ,
    last_attempt_at TIMESTAMPTZ
);
```

```
CREATE TABLE revoked_tokens (
    id         UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    token_hash VARCHAR(64) UNIQUE NOT NULL,
    expires_at TIMESTAMPTZ NOT NULL,
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE TABLE messenger_connections (
    user_id              UUID PRIMARY KEY REFERENCES users(id) ON DELETE CASCADE,
    page_id              VARCHAR(100) NOT NULL,
    page_scoped_user_id  VARCHAR(255) UNIQUE,      -- Messenger PSID once linked
    enabled              BOOLEAN NOT NULL DEFAULT FALSE,
    link_code_hash       VARCHAR(64),
    link_code_expires_at TIMESTAMPTZ,               -- 10 minutes
    linked_at            TIMESTAMPTZ,
    last_interaction_at  TIMESTAMPTZ,
    updated_at           TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- ============================================
-- 8. PAGE SETTINGS
-- ============================================
CREATE TABLE page_settings (
        id                      UUID PRIMARY KEY DEFAULT gen_random_uuid(),
        institution_id          UUID REFERENCES institutions(id),
        guardrails_enforced BOOLEAN NOT NULL DEFAULT TRUE,
        posting_window_start_hour INT NOT NULL DEFAULT 8,       -- GR-H4 window start (Asia/Manila, V116)
        posting_window_end_hour   INT NOT NULL DEFAULT 20,      -- GR-H4 window end (exclusive)
        conflict_buffer_minutes   INT NOT NULL DEFAULT 30,      -- GR-H1 (V121)
        minimum_lead_time_hours   INT NOT NULL DEFAULT 2,       -- GR-H2 (V121)
        maximum_lead_time_days  INT NOT NULL DEFAULT 30,        -- GR-H3 (V121)
        per_institution_active_quota INT NOT NULL DEFAULT 3,  -- GR-S1 (V122)
        daily_volume_cap        INT NOT NULL DEFAULT 6,         -- GR-S2 (V122)
        watermark_enabled   BOOLEAN,
        watermark_text          VARCHAR(150),
        facebook_page_id        VARCHAR(255),
        updated_by              UUID REFERENCES users(id),
        updated_at              TIMESTAMPTZ NOT NULL DEFAULT NOW()
);


-- ============================================
-- 9. AUDIT LOG
-- ============================================
CREATE TABLE audit_log (
    id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
        actor_id    UUID REFERENCES users(id),
        action      VARCHAR(50) NOT NULL,
        resource_id UUID,
        ip_address  INET,
        user_agent  TEXT,
        metadata    JSONB,
        created_at  TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
CREATE INDEX idx_audit_log_actor    ON audit_log(actor_id, created_at DESC);
CREATE INDEX idx_audit_log_resource ON audit_log(resource_id, created_at DESC);

-- ============================================
-- 10. SUBMISSIONS
-- ============================================
CREATE TABLE submissions (
        id                              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
        contributor_id                  UUID NOT NULL REFERENCES users(id),
        institution_id                  UUID NOT NULL REFERENCES institutions(id),
        event_title                     VARCHAR(255),
        event_date                      DATE,
        caption                         TEXT,
```

```
        description                   TEXT,
        tags                          TEXT,
        media_tags                    TEXT,
        album_name                    VARCHAR(255),
        template_id                   VARCHAR(100),
        fast_track                    BOOLEAN NOT NULL DEFAULT FALSE,
        live_event_name               VARCHAR(255),
        status                        VARCHAR(30) NOT NULL DEFAULT 'draft'
                                      CHECK (status IN ('draft', 'pending', 'in_review', 'needs_revision',
                                          'missed_review', 'scheduled', 'publishing', 'publish_failed',
                                          'published', 'published_manual', 'admin_direct_post',
                                          'direct_post_scheduled', 'direct_post_publishing',
                                          'direct_post_failed', 'rejected')),
        scheduled_at                  TIMESTAMPTZ,
        original_scheduled_at         TIMESTAMPTZ,
        moderator_reschedule_count    INT NOT NULL DEFAULT 0,
        submitted_at                  TIMESTAMPTZ,
        published_at                  TIMESTAMPTZ,
        platform_post_id              VARCHAR(255),
        validator_remarks             TEXT,
        rejection_reason              TEXT,
        review_snapshot               JSONB,
        retry_count                   INT NOT NULL DEFAULT 0,
        requires_manual_publishing    BOOLEAN NOT NULL DEFAULT FALSE,
        manual_publish_started_at     TIMESTAMPTZ,
        last_manual_publish_abandoned_at TIMESTAMPTZ,
        published_manual_url          TEXT,
        published_manual_notes        TEXT,
        token_blocked_at              TIMESTAMPTZ,
        token_escalated_24h_at        TIMESTAMPTZ,
        token_final_failed_at         TIMESTAMPTZ,
        created_at                    TIMESTAMPTZ NOT NULL DEFAULT NOW(),
        updated_at                    TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
CREATE INDEX idx_submissions_contributor ON submissions(contributor_id);
CREATE INDEX idx_submissions_institution ON submissions(institution_id);
CREATE INDEX idx_submissions_status     ON submissions(status);
CREATE INDEX idx_submissions_due_for_publish ON submissions(scheduled_at) WHERE status = 'scheduled';

-- ═══════════════════════════════════════════
-- 11–12. TEMPLATES & AI INTERACTIONS
-- ═══════════════════════════════════════════
CREATE TABLE post_templates (
        id                      UUID PRIMARY KEY DEFAULT gen_random_uuid(),
        owner_user_id           UUID NOT NULL REFERENCES users(id),
        institution_id          UUID REFERENCES institutions(id),
        name                    VARCHAR(100) NOT NULL,
        target                  VARCHAR(255),
        caption                 TEXT NOT NULL,
        tags                    TEXT,
        source_submission_id UUID REFERENCES submissions(id),
        created_at              TIMESTAMPTZ NOT NULL DEFAULT NOW(),
        updated_at              TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE TABLE ai_interaction_log (
    id               UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    submission_id    UUID REFERENCES submissions(id),
    institution_id   UUID REFERENCES institutions(id),
    interaction_type VARCHAR(30) NOT NULL,
    action_taken     VARCHAR(30) NOT NULL,
    tone_selected    VARCHAR(30),
    suggested_value  TEXT,
    created_at       TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
CREATE INDEX idx_ai_interaction_institution ON ai_interaction_log(institution_id, created_at DESC);

-- ═══════════════════════════════════════════
```

```
-- 13–20. MEDIA
-- ═══════════════════════════════════════════
CREATE TABLE media_albums (
        id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
        institution_id  UUID NOT NULL REFERENCES institutions(id),
        parent_album_id UUID REFERENCES media_albums(id),
        name            VARCHAR(255) NOT NULL,
        created_by      UUID REFERENCES users(id),
        created_at      TIMESTAMPTZ NOT NULL DEFAULT NOW(),
        updated_at      TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE TABLE media_assets (
        id                      UUID PRIMARY KEY DEFAULT gen_random_uuid(),
        institution_id          UUID REFERENCES institutions(id),       -- NULL while STAGED; bound on submit
(V73)
        uploader_id             UUID NOT NULL REFERENCES users(id),
        media_album_id          UUID REFERENCES media_albums(id),
        asset_code              VARCHAR(50) UNIQUE NOT NULL,
        storage_url             TEXT NOT NULL,
        file_name               VARCHAR(255) NOT NULL,
        display_title           VARCHAR(255),
        content_hash            VARCHAR(64),
        file_type               VARCHAR(10) NOT NULL
                                CHECK (file_type IN ('jpeg','png','webp','gif','mp4','mov','webm')),
        asset_type              TEXT,
        file_size_bytes         BIGINT NOT NULL,
        status                  TEXT NOT NULL,          -- STAGED | PROCESSING | READY | FAILED | DELETED
        ai_category             VARCHAR(50),
        ai_confidence           NUMERIC(5,4),
        ai_description          TEXT,
        ai_tags                 TEXT[],
        ai_classified_at        TIMESTAMPTZ,
        ai_classification_model VARCHAR(100),
        ai_processing_version   VARCHAR(50),
        observed_scenes         TEXT[],
        visible_objects         TEXT[],
        observed_activities     TEXT[],
        ocr_text                TEXT[],
        people_count_range      VARCHAR(30),
        event_hypotheses        JSONB,
        specific_subjects       TEXT[],
        visual_style            TEXT[],
        dominant_colors         TEXT[],
        possible_use_cases      TEXT[],
        excluded_categories     TEXT[],
        equipment_signals       TEXT[],
        recognition_signals     TEXT[],
        visible_dates           TEXT[],
        visual_quality_signals  TEXT[],
        composition_signals     TEXT[],
        temporal_classification VARCHAR(30),
        possible_expiration     VARCHAR(80),
        reclassified_at         TIMESTAMPTZ,
        embedding               VECTOR(1024),
        embedding_generated_at  TIMESTAMPTZ,
        embedding_model         VARCHAR(100),
        semantic_revision       BIGINT NOT NULL DEFAULT 0,
        deleted_at              TIMESTAMPTZ,
        deleted_by_user_id      UUID REFERENCES users(id),
        purged_at               TIMESTAMPTZ,
        created_at              TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
CREATE INDEX idx_media_assets_institution ON media_assets(institution_id) WHERE deleted_at IS NULL;
CREATE INDEX idx_media_assets_media_album_id ON media_assets(media_album_id);

CREATE TABLE asset_tags (
        id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
```

```
        media_asset_id UUID NOT NULL REFERENCES media_assets(id) ON DELETE CASCADE,
        label          VARCHAR(50) NOT NULL,
        source         VARCHAR(20) NOT NULL DEFAULT 'manual',
        created_at     TIMESTAMPTZ NOT NULL DEFAULT NOW(),
        UNIQUE (media_asset_id, label)
);

CREATE TABLE media_asset_embeddings (
        id             UUID PRIMARY KEY DEFAULT gen_random_uuid(),
        asset_id       UUID NOT NULL REFERENCES media_assets(id) ON DELETE CASCADE,
        embedding_type TEXT NOT NULL CHECK (embedding_type IN ('image','semantic','keyframe')),
        model          TEXT NOT NULL,
        source_input_hash       CHAR(64),
        source_revision         BIGINT,
        processing_version      VARCHAR(50),
        embedding      VECTOR(1024) NOT NULL,
        created_at     TIMESTAMPTZ NOT NULL DEFAULT NOW(),
        UNIQUE (asset_id, embedding_type)
);
CREATE INDEX idx_media_asset_embeddings_vector ON media_asset_embeddings
        USING hnsw (embedding vector_cosine_ops) WITH (m = 16, ef_construction = 64);

CREATE TABLE media_processing_jobs (
        id             UUID PRIMARY KEY DEFAULT gen_random_uuid(),
        asset_id       UUID REFERENCES media_assets(id) ON DELETE CASCADE,
        submission_id  UUID REFERENCES submissions(id) ON DELETE CASCADE,
        job_type       VARCHAR(40) NOT NULL
                       CHECK (job_type IN ('RETRIEVAL_EMBEDDINGS', 'CLASSIFY_AND_EMBED',
                                           'EMBED_IMAGE_ONLY', 'BUILD_SUBMISSION_CONTEXT')),
        status         VARCHAR(20) NOT NULL
                       CHECK (status IN ('PENDING','PROCESSING','RETRY','COMPLETED','DEAD')),
        attempt_count  INT NOT NULL DEFAULT 0,
        max_attempts   INT NOT NULL,
        priority               SMALLINT NOT NULL DEFAULT 10,
        rerun_requested        BOOLEAN NOT NULL DEFAULT FALSE,
        next_attempt_at TIMESTAMPTZ,
        lease_until    TIMESTAMPTZ,
        claimed_by     VARCHAR(100),
        processing_version VARCHAR(50),
        last_error     VARCHAR(500),
        completed_at   TIMESTAMPTZ,
        created_at     TIMESTAMPTZ NOT NULL DEFAULT NOW(),
        updated_at     TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE TABLE submission_media_assets (
        id             UUID PRIMARY KEY DEFAULT gen_random_uuid(),
        submission_id  UUID NOT NULL REFERENCES submissions(id) ON DELETE CASCADE,
        media_asset_id UUID NOT NULL REFERENCES media_assets(id),
        display_order  INT NOT NULL DEFAULT 0,
        caption        VARCHAR(500),
        skip_watermark BOOLEAN NOT NULL DEFAULT FALSE,
        created_at     TIMESTAMPTZ NOT NULL DEFAULT NOW(),
        UNIQUE (submission_id, media_asset_id)
);

CREATE TABLE submission_media_contexts (
        submission_id          UUID PRIMARY KEY REFERENCES submissions(id) ON DELETE CASCADE,
        institution_id         UUID NOT NULL REFERENCES institutions(id),
        context_version        BIGINT NOT NULL DEFAULT 1,
        asset_set_hash         CHAR(64) NOT NULL,              -- detects a changed set of attached assets
        ready_asset_count      INT NOT NULL DEFAULT 0,
        processing_asset_count INT NOT NULL DEFAULT 0,
        failed_asset_count     INT NOT NULL DEFAULT 0,
        observed_scenes        JSONB NOT NULL DEFAULT '[]'::jsonb,
        observed_objects       JSONB NOT NULL DEFAULT '[]'::jsonb,
        observed_activities    JSONB NOT NULL DEFAULT '[]'::jsonb,
        people_count_ranges    JSONB NOT NULL DEFAULT '[]'::jsonb,
```

```
        equipment_signals       JSONB NOT NULL DEFAULT '[]'::jsonb,
        recognition_signals     JSONB NOT NULL DEFAULT '[]'::jsonb,
        ocr_text                JSONB NOT NULL DEFAULT '[]'::jsonb,
        event_hypotheses        JSONB NOT NULL DEFAULT '[]'::jsonb,
        temporal_signals        JSONB NOT NULL DEFAULT '{}'::jsonb,
        quality_signals         JSONB NOT NULL DEFAULT '[]'::jsonb,
        composition_signals     JSONB NOT NULL DEFAULT '[]'::jsonb,
        context_text            TEXT NOT NULL DEFAULT '',       -- text used for suggestion queries
        status                 VARCHAR(20) NOT NULL CHECK (status IN ('EMPTY', 'PARTIAL', 'READY')),
        model_version          VARCHAR(50) NOT NULL,
        generated_at           TIMESTAMPTZ NOT NULL DEFAULT NOW(),
        updated_at              TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- ═══════════════════════════════════════════
-- 21. SLOT RESERVATIONS (GR-H1 enforced in DB)
-- ═══════════════════════════════════════════
CREATE TABLE slot_reservations (
        id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
        submission_id  UUID NOT NULL REFERENCES submissions(id),
        institution_id UUID NOT NULL REFERENCES institutions(id),
        scheduled_at   TIMESTAMPTZ NOT NULL,
        status          VARCHAR(20) NOT NULL DEFAULT 'held'
                CHECK (status IN ('held', 'locked', 'released')),
        admin_override BOOLEAN NOT NULL DEFAULT FALSE,
        created_at      TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
CREATE OR REPLACE FUNCTION epoch_seconds_immutable(ts timestamptz)
RETURNS bigint LANGUAGE sql IMMUTABLE PARALLEL SAFE
AS $$ SELECT(EXTRACT(EPOCHFROMts))::bigint; $$;
ALTER TABLE slot_reservations ADD CONSTRAINT excl_slot_reservations_network_buffer
        EXCLUDE USING gist (
        int8range(epoch_seconds_immutable(scheduled_at),
                  epoch_seconds_immutable(scheduled_at) + 1800, '[]') WITH &&
        ) WHERE (status <> 'released' AND NOT admin_override);
-- Each reservation covers [t, t + 30 min]; two ranges overlap exactly when the
-- slots are within 30 minutes of each other.

-- ═══════════════════════════════════════════
-- 22–23. REVIEW
-- ═══════════════════════════════════════════
CREATE TABLE review_locks (
    id            UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    submission_id UUID NOT NULL UNIQUE REFERENCES submissions(id) ON DELETE CASCADE,
    locked_by     UUID NOT NULL REFERENCES users(id),
    acquired_at   TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    expires_at    TIMESTAMPTZ NOT NULL
);

CREATE TABLE validation_logs (
    id               UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    submission_id    UUID NOT NULL REFERENCES submissions(id) ON DELETE CASCADE,
    validator_id     UUID NOT NULL REFERENCES users(id),
    action           VARCHAR(30) NOT NULL,
    remarks          TEXT,
    rejection_reason VARCHAR(100),
    is_self_review   BOOLEAN NOT NULL DEFAULT FALSE,
    is_fast_track    BOOLEAN NOT NULL DEFAULT FALSE,
    edit_diff        JSONB,
    edit_severity    VARCHAR(20) CHECK (edit_severity IN ('QUIET','FLAGGED','ADDED_MEDIA')),
    created_at       TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
CREATE INDEX idx_validation_logs_submission ON validation_logs(submission_id, created_at DESC);

-- ═══════════════════════════════════════════
-- 24–27. PUBLISHING
-- ═══════════════════════════════════════════
CREATE TABLE watermark_configurations (
```

```
        id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
        institution_id UUID REFERENCES institutions(id),
        enabled         BOOLEAN NOT NULL DEFAULT TRUE,
        elements_json  TEXT NOT NULL,
        updated_by      VARCHAR(150),
        created_at      TIMESTAMPTZ NOT NULL DEFAULT NOW(),
        updated_at      TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE TABLE facebook_page_tokens (
        id                      UUID PRIMARY KEY DEFAULT gen_random_uuid(),
        page_id                 VARCHAR(50) UNIQUE NOT NULL,
        encrypted_token         TEXT NOT NULL,
        expires_at              TIMESTAMPTZ,
        is_active               BOOLEAN NOT NULL DEFAULT FALSE,
        last_validated_at       TIMESTAMPTZ,
        validation_failed_at    TIMESTAMPTZ,
        validation_failure_reason TEXT,
        created_at              TIMESTAMPTZ NOT NULL DEFAULT NOW(),
        updated_at              TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE TABLE publication_attempts (
        id                      UUID PRIMARY KEY DEFAULT gen_random_uuid(),
        submission_id           UUID NOT NULL REFERENCES submissions(id) ON DELETE CASCADE,
        attempt_number          INT NOT NULL,
        result                  VARCHAR(10) NOT NULL,
        error_detail            TEXT,
        photo_ids_staged        TEXT,
        photo_ids_cleanup_failed TEXT,
        attempted_at            TIMESTAMPTZ NOT NULL DEFAULT NOW(),
        created_at              TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE TABLE submission_engagement_metrics (
        id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
        submission_id   UUID UNIQUE NOT NULL REFERENCES submissions(id) ON DELETE CASCADE,
        reach           BIGINT,
        reactions       BIGINT,
        comments_count  BIGINT,
        shares          BIGINT,
        fetched_at      TIMESTAMPTZ,
        last_attempt_at TIMESTAMPTZ,
        last_error      TEXT,
        created_at      TIMESTAMPTZ NOT NULL DEFAULT NOW(),
        updated_at      TIMESTAMPTZ NOT NULL DEFAULT NOW()
);


-- ═══════════════════════════════════════════
-- 28–29. NOTIFICATIONS
-- ═══════════════════════════════════════════
CREATE TABLE notifications (
        id           UUID PRIMARY KEY DEFAULT gen_random_uuid(),
        recipient_id UUID NOT NULL REFERENCES users(id),
        event_type   VARCHAR(50) NOT NULL,
        message      TEXT NOT NULL,
        deep_link    TEXT,
        read_at      TIMESTAMPTZ,
        created_at   TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
CREATE INDEX idx_notifications_recipient_created_at ON notifications(recipient_id, created_at DESC);
CREATE INDEX idx_notifications_recipient_read_at        ON notifications(recipient_id, read_at);

CREATE TABLE email_delivery_log (
        id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
        recipient_id  UUID NOT NULL REFERENCES users(id),
        template_code VARCHAR(50) NOT NULL,
```

```
        status          VARCHAR(20) NOT NULL,
        error_detail    TEXT,
        triggered_at    TIMESTAMPTZ NOT NULL,
        delivered_at    TIMESTAMPTZ,
        created_at      TIMESTAMPTZ NOT NULL DEFAULT NOW()
);


-- ═══════════════════════════════════════════
-- 30–31. OPERATIONS
-- ═══════════════════════════════════════════
CREATE TABLE scheduled_job_runs (
    id            UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    job_name      VARCHAR(120) NOT NULL,
    status        VARCHAR(20) NOT NULL CHECK (status IN ('SUCCESS', 'FAILED')),
    started_at    TIMESTAMPTZ NOT NULL,
    completed_at  TIMESTAMPTZ,
    duration_ms   BIGINT,
    error_message TEXT,
    created_at    TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
CREATE INDEX idx_scheduled_job_runs_job ON scheduled_job_runs(job_name, started_at DESC);

CREATE TABLE media_ai_processing_metrics (
    id                           UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    stage                        VARCHAR(40) NOT NULL
                                 CHECK (stage IN ('R2_UPLOAD','DB_REGISTRATION','QUEUE_DELAY',
                                        'VOYAGE_IMAGE_EMBEDDING','CLAUDE_CLASSIFICATION',
                                        'VOYAGE_SEMANTIC_EMBEDDING','READY_LATENCY','AI_SUGGESTION_QUERY')),
    outcome                      VARCHAR(20) NOT NULL CHECK (outcome IN ('SUCCESS','FAILURE','REUSED')),
    duration_ms                  BIGINT NOT NULL DEFAULT 0,
    asset_id                     UUID REFERENCES media_assets(id) ON DELETE SET NULL,
    submission_id                UUID REFERENCES submissions(id) ON DELETE SET NULL,
    attempt_number               INT NOT NULL DEFAULT 1,
    provider_call_count          INT NOT NULL DEFAULT 0,
    duplicate_call_avoided_count INT NOT NULL DEFAULT 0,
    created_at                   TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE TABLE media_ai_ranking_shadow_metrics (
    id                  UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    institution_id      UUID NOT NULL REFERENCES institutions(id) ON DELETE CASCADE,
    submission_id       UUID REFERENCES submissions(id) ON DELETE SET NULL,
    legacy_result_count INT NOT NULL,
    hybrid_result_count INT NOT NULL,
    overlap_count       INT NOT NULL,
    top_result_changed  BOOLEAN NOT NULL,
    created_at          TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- ═══════════════════════════════════════════
-- ROW-LEVEL SECURITY
-- ═══════════════════════════════════════════
-- Enabled in Flyway on institution-scoped tables (submissions, slot_reservations,
-- media_assets, media_albums, asset_tags, submission_media_assets, review_locks,
-- validation_logs, ai_interaction_log, ...). Pattern:
ALTER TABLE submissions ENABLE ROW LEVEL SECURITY;
CREATE POLICY institution_isolation ON submissions
    USING (
        institution_id = current_setting('app.current_institution_id', true)::UUID
        OR current_setting('app.current_role', true) IN ('moderator', 'admin')
    );

-- audit_log is append-only (V78): INSERT and SELECT policies only.
ALTER TABLE audit_log ENABLE ROW LEVEL SECURITY;
CREATE POLICY audit_log_append ON audit_log FOR INSERT WITH CHECK (true);
CREATE POLICY audit_log_read ON audit_log FOR SELECT
        USING (
```

```
current_setting('app.current_role', true) = 'administrator'
OR EXISTS (
        SELECT 1 FROM users u
        WHERE u.id = audit_log.actor_id
        AND u.institution_id = nullif(current_setting('app.current_institution_id', true), '')::UUID
)
);
```

Appendix — REST API Endpoint Table

All 168 endpoints exposed by the backend controllers in release v3.0.2, listed by controller. "Access" is the effective rule from the controller's `@PreAuthorize` annotations and `SecurityConfig`: **Public** needs no token; **Any signed-in** needs a valid JWT; **All roles** means Contributor, Moderator, and Administrator; **Moderator, Admin** means either reviewer role. Role checks inside a service (for example Admin Owner–only actions or Moderator limits) are described in the use-case sections above.

Controller	Method	Path	Handler	Access
AIRecommendation	GET	/api/v1/ai/submissions/{id}/similar-media	getSimilarMedia	All roles
AIRecommendation	POST	/api/v1/ai/submissions/{id}/suggest-media	suggestMedia	All roles
AIRecommendation	POST	/api/v1/ai/submissions/{id}/suggest-media-status	getSuggestMediaStatus	All roles
AIRecommendation	POST	/api/v1/ai/submissions/{id}/suggest-album	suggestAlbum	All roles
AIRecommendation	POST	/api/v1/ai/submissions/{id}/log-interaction	logInteraction	All roles
AiTemplate	POST	/api/v1/ai/templates/from-top-posts	fromTopPosts	All roles
Analytics	GET	/api/v1/analytics/summary	summary	Any signed-in
Analytics	GET	/api/v1/analytics/export/{metric}	export	Any signed-in
Analytics	GET	/api/v1/analytics/report/{metric}	report	Any signed-in
AuditLog	GET	/api/v1/audit-log	getAuditLogs	Admin
AuditLog	GET	/api/v1/audit-log/categories	getCategories	Admin
AuditLog	GET	/api/v1/audit-log/export-csv	exportCsv	Admin
Auth	POST	/api/v1/auth/login	login	Public
Auth	POST	/api/v1/auth/logout	logout	Public
Auth	POST	/api/v1/auth/refresh	refresh	Any signed-in
Calendar	GET	/api/v1/calendar	getCalendar	Any signed-in
Calendar	PATCH	/api/v1/submissions/{id}/reschedule	reschedule	Moderator, Admin
Caption	POST	/api/v1/ai/caption	generateCaption	All roles
Caption	POST	/api/v1/ai/caption/log	logCaptionInteraction	All roles
EngagementRecommendation	GET	/api/v1/engagement-recommendations	recommendations	All roles
FacebookOAuthCallback	GET	/api/v1/facebook/oauth-callback	oauthCallback	Public
GuardRail	POST	/api/v1/guardrails/validate	validate	All roles
Health	GET	/health	health	Public
Institution	POST	/api/v1/institutions	createInstitution	Admin
Institution	GET	/api/v1/institutions/{institutionId}	getInstitution	Moderator, Admin

Controller	Method	Path	Handler	Access
Institution	GET	/api/v1/institutions	listInstitutions	Moderator, Admin
Institution	GET	/api/v1/institutions/summary-counts	listSummaryCounts	Moderator, Admin
Institution	GET	/api/v1/institutions/public	listPublicInstitutions	Public
Institution	PUT	/api/v1/institutions/{institutionId}/logo	updateLogo	Admin
Institution	GET	/api/v1/institutions/{institutionId}/logo	getLogo	Public
Institution	PUT	/api/v1/institutions/{institutionId}	updateInstitution	Admin
Institution	PATCH	/api/v1/institutions/{institutionId}/ai-media-hybrid-ranking	setAiMediaHybridRankingEnabled	Admin
Institution	PATCH	/api/v1/institutions/{institutionId}/deactivate	deactivateInstitution	Admin
Institution	PATCH	/api/v1/institutions/{institutionId}/reactivate	reactivateInstitution	Admin
Institution	DELETE	/api/v1/institutions/{institutionId}	deleteInstitution	Admin
Invitation	POST	/api/v1/invitations	create	Moderator, Admin
Invitation	GET	/api/v1/invitations/validate	validate	Public
Invitation	POST	/api/v1/invitations/accept	accept	Public
Invitation	POST	/api/v1/invitations/{id}/resend	resend	Moderator, Admin
Invitation	DELETE	/api/v1/invitations/{id}	cancel	Moderator, Admin
Invitation	DELETE	/api/v1/invitations/by-user/{userId}	cancelByUser	Admin
Invitation	GET	/api/v1/invitations/pending	pending	Moderator, Admin
Invitation	GET	/api/v1/invitations/pending/admins	pendingAdmins	Admin
Invitation	GET	/api/v1/invitations/pending/network	pendingNetwork	Admin
Invitation	GET	/api/v1/invitations/pending/count	pendingCount	Moderator, Admin
MediaAsset	GET	/api/v1/media-assets	list	Any signed-in
MediaAsset	GET	/api/v1/media-assets/search	semanticSearch	Any signed-in
MediaAsset	GET	/api/v1/media-assets/{id}	get	Any signed-in
MediaAsset	GET	/api/v1/media-assets/{id}/history	history	Any signed-in
MediaAsset	POST	/api/v1/media-assets/upload-url	getUploadUrl	Any signed-in
MediaAsset	POST	/api/v1/media-assets/upload	upload	Any signed-in
MediaAsset	GET	/api/v1/media-assets/albums	listAlbums	Any signed-in
MediaAsset	POST	/api/v1/media-assets/albums	createAlbum	Any signed-in
MediaAsset	POST	/api/v1/media-assets/albums/{id}	renameAlbum	Any signed-in
MediaAsset	POST	/api/v1/media-assets/albums/ensure-path	ensureAlbumPath	Any signed-in
MediaAsset	PATCH	/api/v1/media-assets/albums/{id}/parent	moveAlbum	Any signed-in

Controller	Method	Path	Handler	Access
MediaAsset	DELETE	/api/v1/media-assets/albums/{id}	deleteAlbum	Any signed-in
MediaAsset	POST	/api/v1/media-assets/{id}/album	updateAlbum	Any signed-in
MediaAsset	POST	/api/v1/media-assets/{id}/title	renameAsset	Any signed-in
MediaAsset	POST	/api/v1/media-assets/{id}/tags	addTag	Any signed-in
MediaAsset	DELETE	/api/v1/media-assets/{id}/tags/{tagId}	removeTag	Any signed-in
MediaAsset	DELETE	/api/v1/media-assets/{id}	delete	Any signed-in
MediaAsset	POST	/api/v1/media-assets/bulk-delete	bulkDelete	Any signed-in
MediaAsset	GET	/api/v1/media-assets/trash	listTrash	Admin
MediaAsset	POST	/api/v1/media-assets/trash/{id}/restore	restore	Admin
MediaAsset	POST	/api/v1/media-assets/trash/bulk-restore	bulkRestore	Admin
MediaAsset	DELETE	/api/v1/media-assets/trash/{id}	purgeTrash	Admin
MediaAsset	POST	/api/v1/media-assets/trash/bulk-purge	bulkPurgeTrash	Admin
MediaAsset	POST	/api/v1/media-assets/trash/empty	emptyTrash	Admin
MediaProcessingAdmin	GET	/api/v1/system-health/media-processing/dead-letters	deadLetters	Admin
MediaProcessingAdmin	POST	/api/v1/system-health/media-processing/dead-letters/{jobId}/retry	retryDeadLetter	Admin
MediaProxy	GET	/api/v1/media-files/{*objectPath}	serve	Public
MessengerConnection	GET	/api/v1/integrations/messenger/connection	status	Any signed-in
MessengerConnection	POST	/api/v1/integrations/messenger/connection/link-code	createCode	Any signed-in
MessengerConnection	DELETE	/api/v1/integrations/messenger/connection	disconnect	Any signed-in
MessengerWebhook	GET	/api/v1/integrations/messenger/webhook	verify	Public
MessengerWebhook	POST	/api/v1/integrations/messenger/webhook	receive	Public
Notification	GET	/api/v1/notifications	list	Any signed-in
Notification	GET	/api/v1/notifications/unread-count	unreadCount	Any signed-in
Notification	PATCH	/api/v1/notifications/{id}/read	markRead	Any signed-in
Notification	PATCH	/api/v1/notifications/read-all	markAllRead	Any signed-in
Notification	GET	/api/v1/notifications/history	history	Any signed-in
Notification	GET	/api/v1/notifications/stream	stream	Any signed-in
PageSettings	GET	/api/v1/settings/page	get	Admin
PageSettings	PUT	/api/v1/settings/page	update	Admin
Password	POST	/api/v1/auth/forgot-password	forgotPassword	Public

Controller	Method	Path	Handler	Access
Password	POST	/api/v1/auth/reset-password	resetPassword	Public
Password	POST	/api/v1/auth/change-password	changePassword	Any signed-in
PostTemplate	GET	/api/v1/post-templates	list	Any signed-in
PostTemplate	POST	/api/v1/post-templates	create	Any signed-in
PostTemplate	DELETE	/api/v1/post-templates/{id}	delete	Any signed-in
Proofread	POST	/api/v1/ai/proofread	proofread	All roles
Proofread	POST	/api/v1/ai/proofread/applied	fixApplied	All roles
Resolution	GET	/api/v1/resolution/failures	listFailures	Moderator, Admin
Resolution	GET	/api/v1/resolution/failures/page	listFailurePage	Moderator, Admin
Resolution	GET	/api/v1/resolution/{id}	getDetail	Moderator, Admin
Resolution	POST	/api/v1/resolution/{id}/retry	retry	Moderator, Admin
Resolution	POST	/api/v1/resolution/{id}/retry-with-new-schedule	retryWithNewSchedule	Moderator, Admin
Resolution	POST	/api/v1/resolution/{id}/retry-as-live	retryAsLive	Moderator, Admin
Resolution	POST	/api/v1/resolution/{id}/manual-publish/start	startManualPublish	Moderator, Admin
Resolution	POST	/api/v1/resolution/{id}/manual-publish/complete	completeManualPublish	Moderator, Admin
Resolution	POST	/api/v1/resolution/{id}/manual-publish/cancel	cancelManualPublish	Moderator, Admin
Submission	GET	/api/v1/submissions/lookups	lookups	Any signed-in
Submission	GET	/api/v1/submissions	list	Any signed-in
Submission	GET	/api/v1/submissions/page	listPage	Any signed-in
Submission	POST	/api/v1/submissions	create	All roles
Submission	GET	/api/v1/submissions/{id}	get	Any signed-in
Submission	PATCH	/api/v1/submissions/{id}	update	All roles
Submission	DELETE	/api/v1/submissions/{id}	delete	All roles
Submission	POST	/api/v1/submissions/{id}/submit	submit	All roles
Submission	POST	/api/v1/submissions/{id}/withdraw	withdraw	All roles
Submission	POST	/api/v1/submissions/{id}/evaluate-slot	evaluateSlot	All roles
Submission	POST	/api/v1/submissions/{id}/media/upload-url	getSignedUploadUrl	All roles
Submission	POST	/api/v1/submissions/{id}/media	attachMedia	All roles
Submission	PATCH	/api/v1/submissions/{id}/media/order	reorderMedia	All roles
Submission	POST	/api/v1/submissions/{id}/assets	attachAsset	All roles
Submission	DELETE	/api/v1/submissions/{id}/assets/{assetId}	detachAsset	All roles
SystemHealth	GET	/api/v1/system-health/summary	summary	Admin
SystemHealth	GET	/api/v1/system-health/storage	storage	Admin
SystemHealth	GET	/api/v1/system-health/external-services	externalServices	Admin

Controller	Method	Path	Handler	Access
SystemHealth	GET	/api/v1/system-health/jobs	jobs	Admin
SystemHealth	GET	/api/v1/system-health/operational-metrics	operationalMetrics	Admin
SystemHealth	GET	/api/v1/system-health/media-ai-metrics	mediaAiMetrics	Admin
SystemHealth	GET	/api/v1/system-health/media-ai-coverage	mediaAiCoverage	Admin
SystemHealth	GET	/api/v1/system-health/tokens	tokens	Admin
SystemHealth	GET	/api/v1/system-health/tokens/{tokenId}/oauth-init	oauthInit	Admin
SystemHealth	PUT	/api/v1/system-health/tokens/{tokenId}/manual	setManualToken	Admin
SystemHealth	PUT	/api/v1/system-health/tokens/connect	connectPage	Admin
SystemHealth	POST	/api/v1/system-health/jobs/{jobKey}/run	runJob	Admin
SystemHealth	POST	/api/v1/system-health/tokens/recheck	recheckTokenHealth	Admin
SystemHealth	GET	/api/v1/system-health/export	exportSnapshot	Admin
User	GET	/api/v1/me	me	Any signed-in
User	PATCH	/api/v1/me/settings	updateMySettings	Any signed-in
User	PATCH	/api/v1/me/tour-preferences	updateMyTourPreferences	Any signed-in
User	GET	/api/v1/users	listUsers	Moderator, Admin
User	GET	/api/v1/users/admins	listAdmins	Admin
User	GET	/api/v1/users/moderators	listModerators	Admin
User	GET	/api/v1/users/network	listNetworkUsers	Admin
User	GET	/api/v1/users/{id}	getUser	Moderator, Admin
User	PATCH	/api/v1/users/{id}/status	updateStatus	Admin
User	PATCH	/api/v1/users/{id}/role	changeRole	Admin
User	PATCH	/api/v1/users/{id}/institution	reassignInstitution	Admin
User	PUT	/api/v1/users/{id}/avatar	updateAvatar	Any signed-in
User	GET	/api/v1/users/{id}/avatar	getAvatar	Public
User	DELETE	/api/v1/users/{id}	removeUser	Moderator, Admin
User	POST	/api/v1/users/{id}/erase	erasePersonalData	Admin
User	POST	/api/v1/users/{id}/admin-transfer	requestAdminTransfer	Admin
User	POST	/api/v1/users/admin-transfer/confirm	confirmAdminTransfer	Admin
User	POST	/api/v1/users/promotion/confirm	confirmPromotion	Any signed-in
User	DELETE	/api/v1/users/{id}/promotion	cancelPromotion	Admin
User	POST	/api/v1/users/promotion/decline	declinePromotion	Any signed-in
User	GET	/api/v1/users/counts	userCounts	Moderator, Admin
Validation	GET	/api/v1/validation/queue	getQueue	Moderator, Admin
Validation	GET	/api/v1/validation/queue/page	getQueuePage	Moderator, Admin

Controller	Method	Path	Handler	Access
Validation	GET	/api/v1/validation/dashboard-summary	getDashboardSummary	Moderator, Admin
Validation	GET	/api/v1/validation/{id}/lock	getLockStatus	Moderator, Admin
Validation	POST	/api/v1/validation/{id}/lock	acquireLock	Moderator, Admin
Validation	DELETE	/api/v1/validation/{id}/lock	releaseLock	Moderator, Admin
Validation	POST	/api/v1/validation/{id}/approve	approve	Moderator, Admin
Validation	POST	/api/v1/validation/{id}/edit	edit	Moderator, Admin
Validation	POST	/api/v1/validation/{id}/assets	attachReviewLibraryAsset	Moderator, Admin
Validation	DELETE	/api/v1/validation/{id}/assets/{assetId}	detachReviewMedia	Moderator, Admin
Validation	PATCH	/api/v1/validation/{id}/media/order	reorderReviewMedia	Moderator, Admin
Validation	POST	/api/v1/validation/{id}/revise	requestRevision	Moderator, Admin
Validation	POST	/api/v1/validation/{id}/reject	reject	Moderator, Admin
Validation	GET	/api/v1/validation/{id}/log	getLog	Moderator, Admin
WatermarkConfiguration	GET	/api/v1/settings/watermark	get	Any signed-in
WatermarkConfiguration	PUT	/api/v1/settings/watermark	save	Admin